

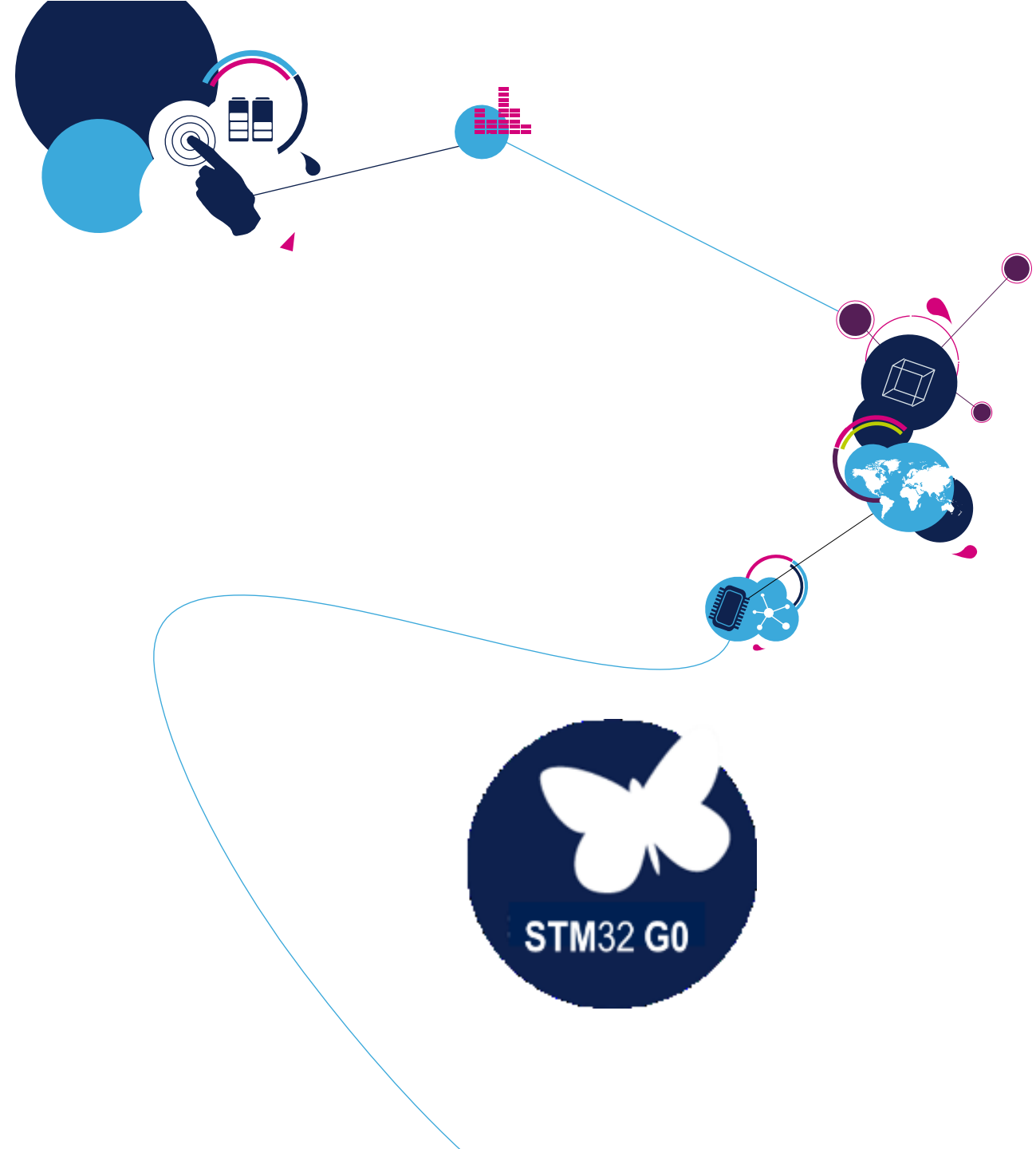
STM32G0 and STM32CubeMX 5.0 Workshop



8:00 AM – 9:00 AM	Registration and system check for pre-installed tools
9:00 AM – 12:00 PM	STM32G0 Overview
	Overview of the STM32CubeProgrammer
	Lab: Upload and save a binary firmware image to a file
	STM32CubeMX 5.0 Overview
	STM32Cube Library
	Lab: Blink an LED by software
	Lab: Use hardware (PWM timer) to blink an LED
	Lab: External Interrupt
	Lab: Low power
	Lab: Power Consumption Estimation
12:00 PM – 1:00 PM	Lunch
1:00 PM – 3:00 PM	Lab: Utilize a printf for console output
	Lab: Low Layer Library usage
	DMA presentation
	Lab: DMA
	Lab: Restore original demo code firmware to the board using 3 methods

STM32G0 MCU series

Efficiency at its best



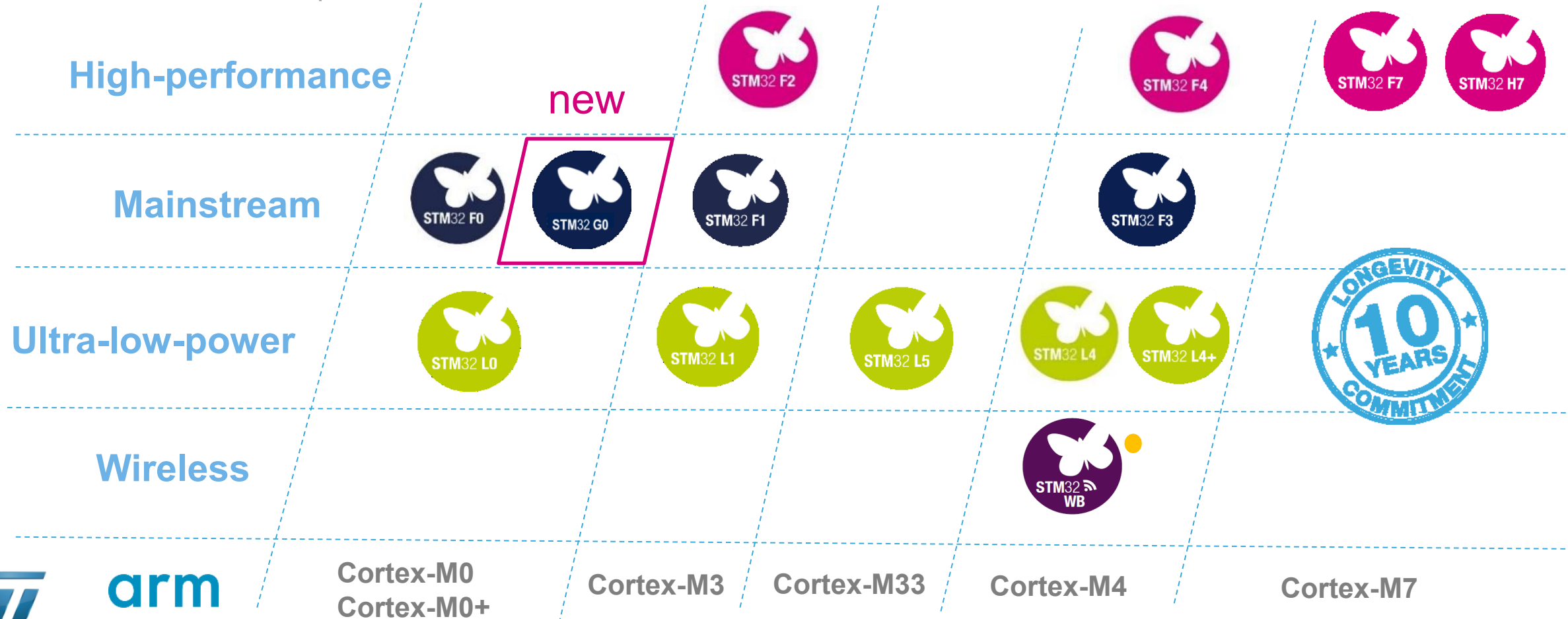


STM32G0: great investment

4

Keep releasing your growing creativity

Note ●: Cortex-M0+ Radio Co-processor



Cortex-M Seamless scalability

Extended Performance

Cortex-M4 / M7

Floating Point Unit (FPU)
DSP (SIMD, fast MAC)

Foundation

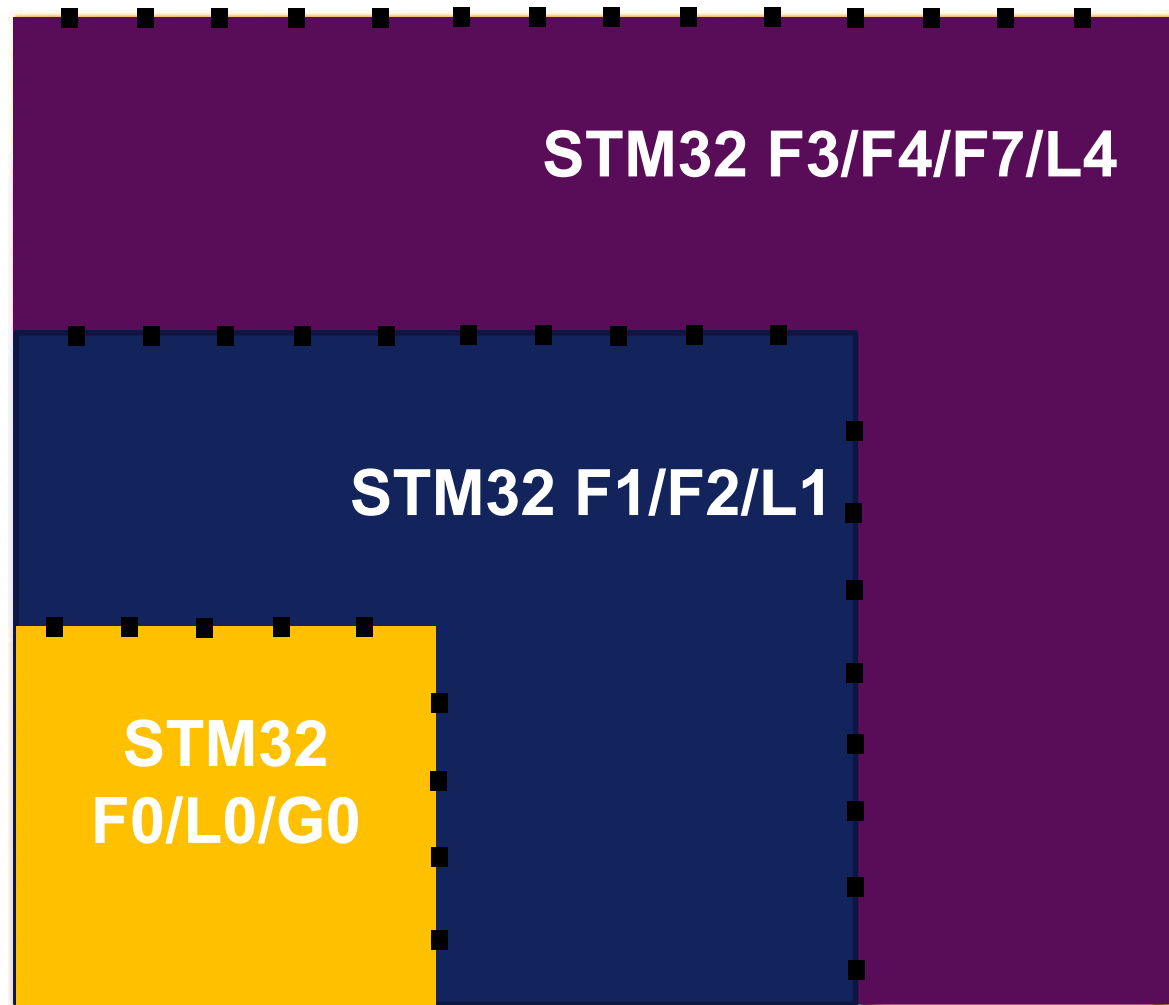
Cortex-M3

Advanced data
processing
Bit field
manipulations

Budget price

Cortex-M0/M0+

General data
processing
I/O control tasks





Key messages of STM32G0 series

6

1

Efficient

- ARM Cortex M0+ at 64MHz
- Compact cost: maximum I/Os count
- Best RAM/Flash Ratio
- Smallest possible package down to 8-pin
- Very low power consumption (3µA in stop, <100µA/MHZ in Run)
- Accurate internal high-speed clock 1% RC
- Best optimization, down to each and every detail
- Offers the best value for money

2

Robust

- Low electromagnetic susceptibility, EMC
- Clock Monitoring and 2 Watchdogs
- Error correction on Flash
- IoT ready with embedded security
- Hardware AES-256 encryption
- New Securable Memory Area
- Safe Firmware upgrade / Install

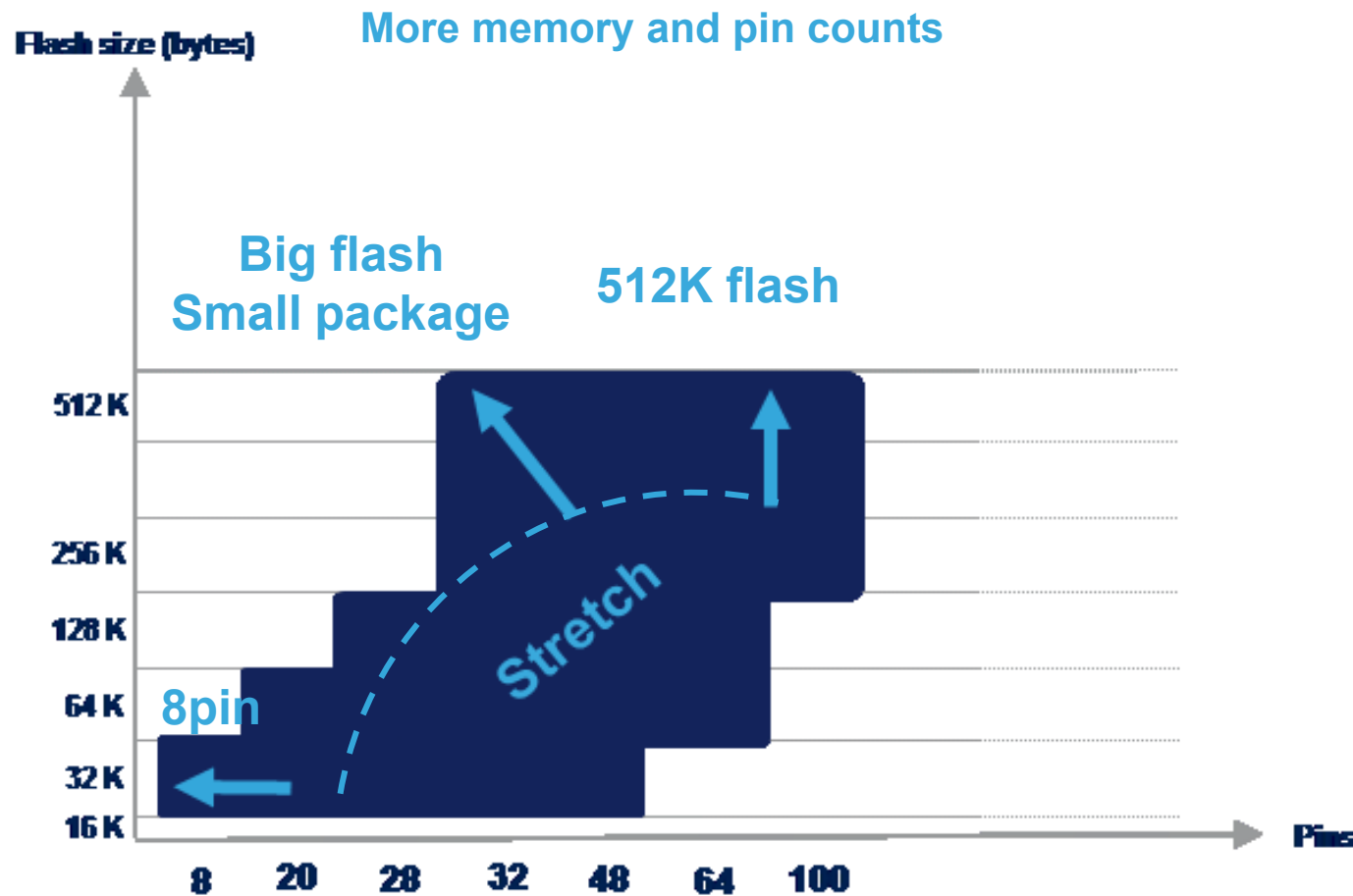
3

Simple

- Easy to configure thanks to the intuitive and graphical STM32CubeMX configuration tool
- Easy to develop based on the Hardware Abstraction Layer library (HAL) or the low-layer library (LL) allowing maximum re-use and faster time-to-market



Portfolio streeeeeeeetched for efficient budget applications



More packages

SU / ISSOP

WLCSP

BGA

QFN

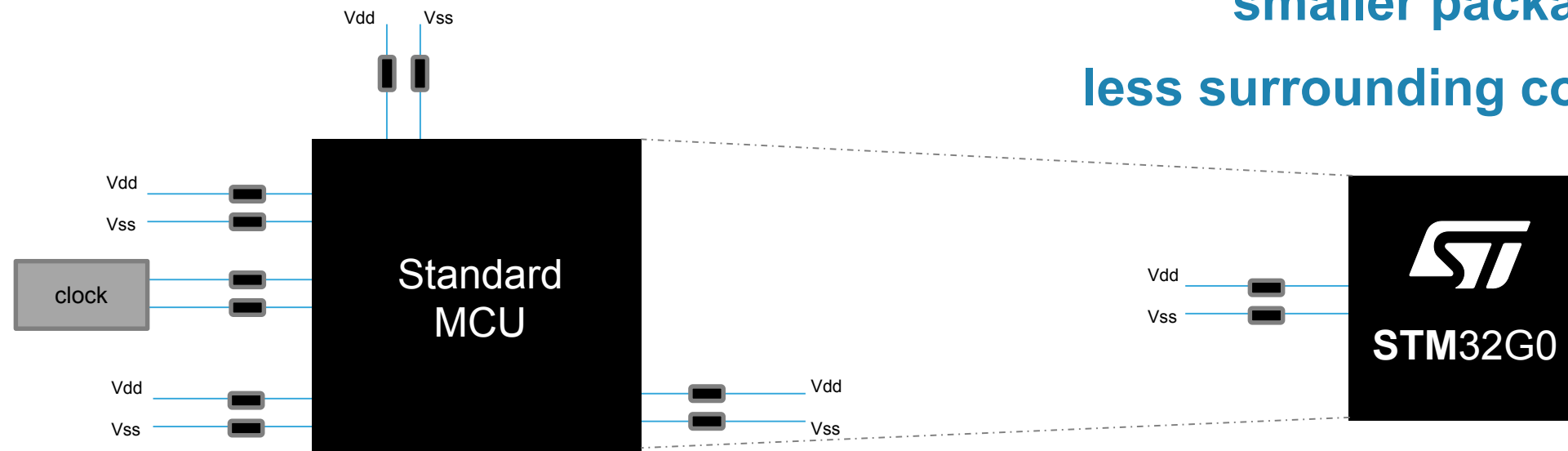
LQFP



Reducing BOM cost

8

New platform optimized with 1 power supply pair only up to 64pin packages



smaller package

less surrounding components



Innovations for your benefit

9

- **No external clock** **-10cts**
Accurate internal high speed clock +/-1% for 0 / 90°C
- **No decoupling capacitances** **-4cts**
Remove up to 6 decoupling capacitors for supply and clocks
- **Smaller PCB** **-1cts**
Smaller package, less components: save on PCB area

Additional benefits for your convenience:

- **USB-C power delivery** **-15cts**
Integrated transceivers, pull-up/down resistors and digital
- **Secure programming** **-25cts**
In house or at 3rd parties



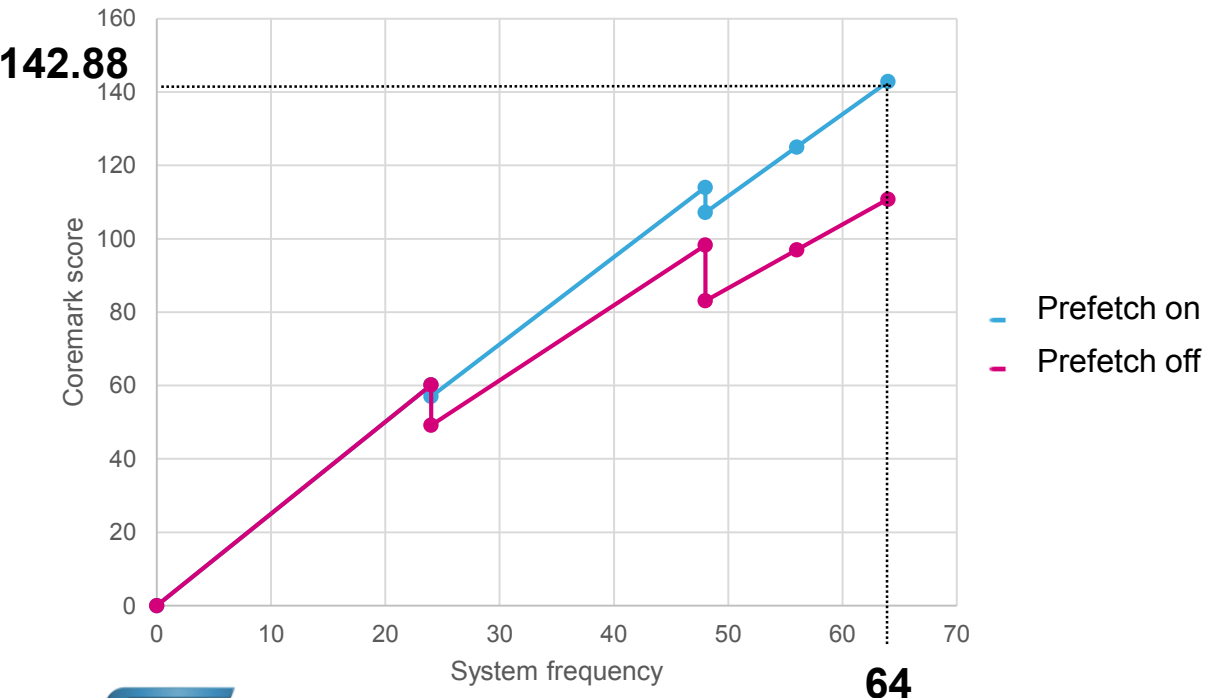


Providing more performance

10

Do not compromise on performance with STM32G0

Performance with 0, 1 and 2 wait state



- Up to 64 MHz/ 59 DMIPS
- Up to >142 CoreMark Result
- ARM Cortex-M0+ with Memory Protection Unit (MPU)
- Flexible **DMA** up to 12 channels



Low-power modes efficiency

11

When Mainstream MCU Series meets low-power requirements

Wake-up time

	VBAT	10 nA / 400 nA*	Tamper: few I/Os, RTC
250 μs	SHUTDOWN	40 nA / 500 nA*	Wake-up sources: reset pin, few I/Os, RTC
14 μs	STANDBY	200 nA / 500 nA*	Wake-up sources: + BOR, IWDG
5 μs	STOP	Flash-RTC off-off/off-on/on-off 3.0μA / 5μA / 8μA	Wake-up sources: + all I/Os, PVD, COMPs, LPUART, LPTIM, I ² C, UART, USB-PD
6 cycles	SLEEP	24MHz, Vdd=3V, PLL=on 800 μA	Wake-up sources: any interrupt or event
	RUN at 64 MHz	<100 μA / MHz	

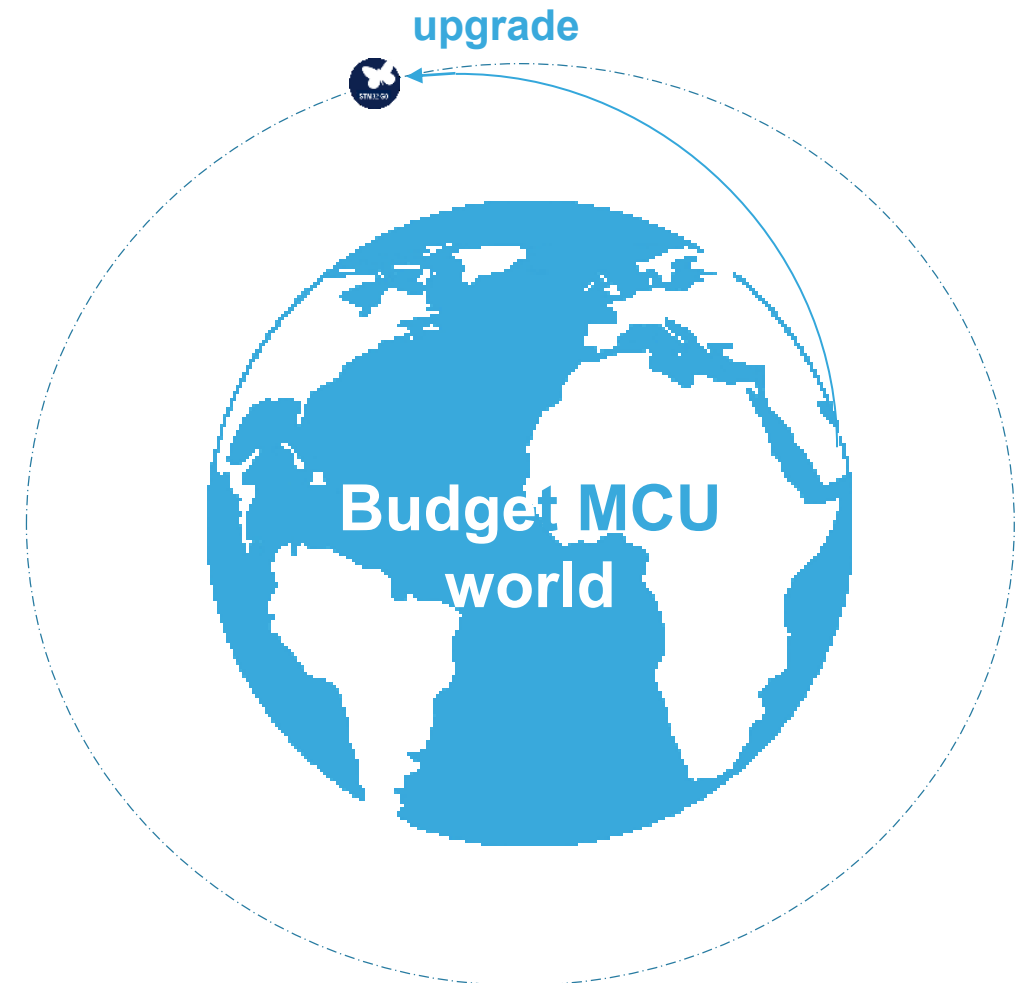
Conditions: 25°C, Vdd = 3V

Note : * without RTC / with RTC



Faster, more accurate analog and digital functions

- More **RAM** for Flash
 - Up to 36KB SRAM for 128KB and 64KB Flash
- **Timers** frequency up to **128MHz** resolution (<8ns)
 - **Advanced control** capabilities
- **12-bit ADC** up to **2.5MSPS** (0.4µs) conversion time
 - **16-bit** oversampling by hardware
- **32Mbit/s SPI**, 7 Mbaud/s USART, 1Mbit/s I²C communication





FD CAN

Up to 2 instances



Industrial

V_{BAT} with RTC

for battery backup

400 nA in V_{BAT} mode
for RTC and
20x 32-bit backup registers



TRNG & AES

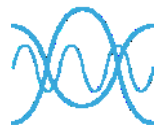
for Security

128-/256-bit AES
key encryption hardware
accelerator



Comparators

2 instances
Down to 30ns propagation delay



DAC

2x 12-bit DAC,

ADC

16x12-bit, 16-bit oversampling
2.5MSPS (0.4μs)

Timers

8ns PWM resolution
Advanced control
16- and 32-bit



Smart peripherals

13



USB-C Power Delivery

Up to 2 ports with dead-battery management



USB

USB 2.0
Full speed
Device / Host



SPI / UART / I²C

4x SPIs
8 USARTs (ISO 7816, LIN, IrDA, modem)
3 I²C

I/Os Up to 92 fast I/Os



Smart integration

14

Save on battery life

Low consumption process and design
Low-Power UART: wake-up on frame
Low-Power Timer: counts and generate signals
I²C wake-up on address



Save on BOM cost



+/-1% high speed clock internal from 0 to 90°C
+/-2% high speed clock internal from -40 to 125°C
IO maximization: smaller package footprint

More flexibility



More RAM or **more safety** with parity enable/disable
Dynamic DMA assignment on **DMAMUX**
All IOs with external interrupt capability



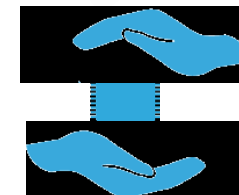
Always keep control Diagnose, react

Main Clock monitoring
Backup clock and interrupts
Voltage monitoring: programmable interrupts and reset
Window watchdog on CPU clock
Independent watchdog on independent clock
Checksum by hardware
ECC on Flash, **Parity** on RAM



High temperature

from -40°C
up to + 125°C



High robustness

Highly immune to fast-transients
Robust IOs against negative injections



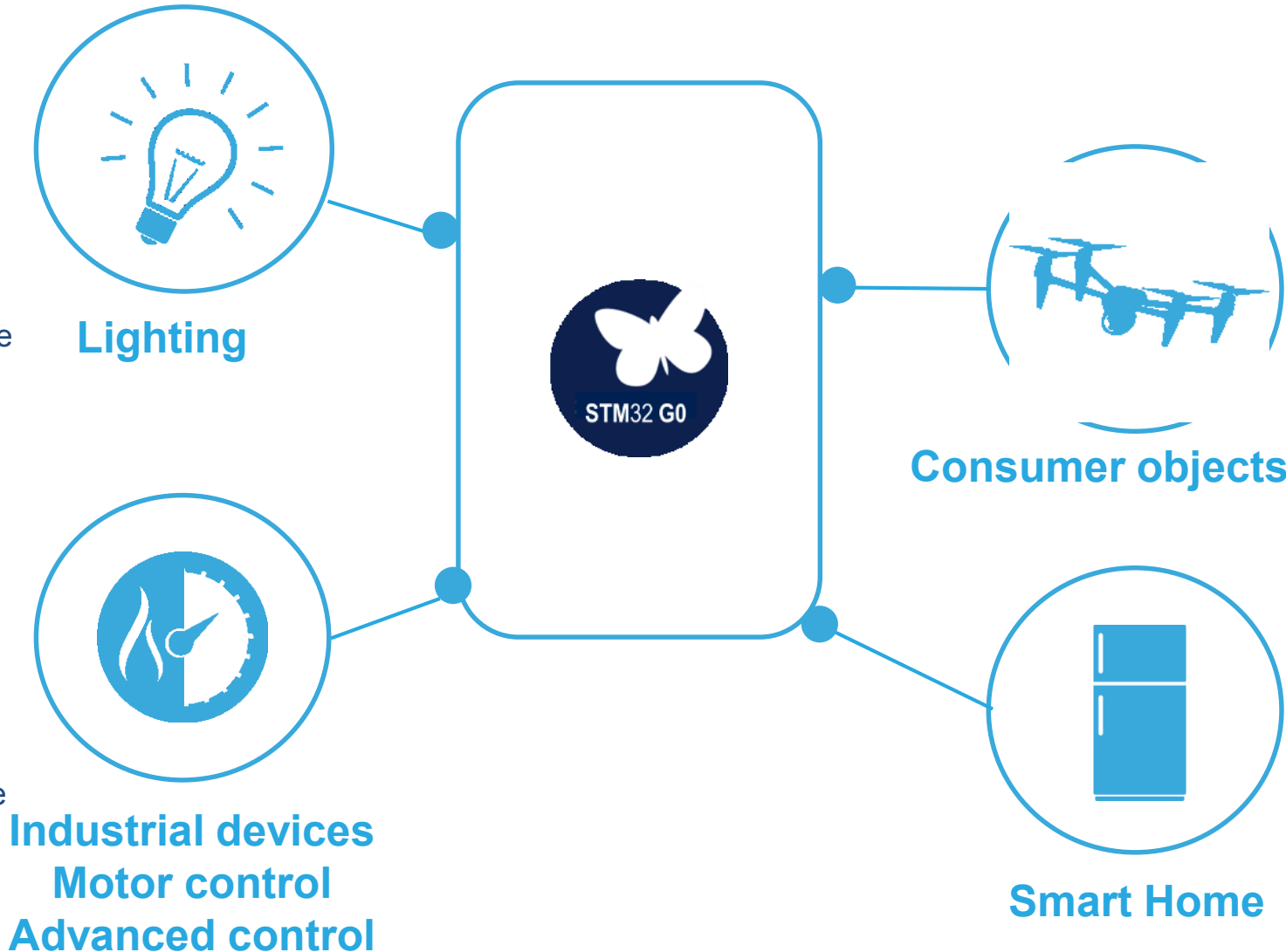
Smart applications

15

- High temperature 125°C
- Fast CPU 64MHz
- Advanced timers with high-resolution 7.8ns
- Fast comparators
- ADC-12bit, DAC-12bit
- Low-thickness packages
- AES & security for secure upgrades

Air conditioning, e-bikes, industrial equipments

- High temperature 125°C
- CANFD support
- SPI, USART, I²C
- Advanced timers with high-resolution 7.8ns
- Real Time Clock with backup registers
- AES & security for secure upgrades



Lighting

Industrial devices Motor control Advanced control

Consumer objects

Smart Home

Smartphones, IoT devices, rechargeable connected devices, drones, toys

- Low-thickness, small form-factor
- 64MHz CPU with DMA
- Low consumption in run and low-power, fast wake-up
- USB type-C Power Delivery 3.0
- USB FS 2.0 dev/host crystal-less

Home appliances, alarms and safety, advanced user interfaces

- High temperature 125°C
- Safety monitoring features
- More RAM for flash
- Low consumption <100µA/MHz in run

STM32 G0 product lines

16

Common peripherals and architecture:

ARM Cortex-M0+
64MHz
0.93 DMIPS/MHz

MPU

Communication Peripheral:
USART, SPI, I²C

Multiple general-purpose
16-bit timers

Integrated reset and brown-
out warning

DMA channels

2x watchdogs
Real-time clock (RTC)

Integrated regulator
PLL and clock circuit

Main oscillator and
32 kHz oscillator

Internal RC oscillators
32kHz , 16 MHz

-40 to +85 °C

Low voltage 1.65 to 3.6 V
(Value Line: 2.0 to 3.6V)

Temperature sensor

Up to +125°C

STM32G0x0 – Value Line (ex: STM32G070)

Up to
512KB
Flash

Up to 80KB
SRAM

12-bit
ADC
2.5MSPS

STM32G0x1 – Access Line (ex: STM32G071)

Up to
512KB
Flash

Up to
80KB
SRAM

12-bit
ADC
2.5MSPS

2x Comp
2x 12-bit
DAC

1x 32-bit
Timer

1x 16-bit
MC Timer
>100MHz

1x16-bit
Timer
>100MHz

USB-PD

USB
OTG
2.0 FS

CAN-FD

Securable
Memory
Area

STM32G0+11 – Access Line & Encryption (ex: STM32G081)

Up to
512KB
Flash

Up to
80KB
SRAM

12-bit
ADC
2.5MSPS

2x Comp
2x 12-bit
DAC

1x 32-bit
Timer

1x 16-bit
MC Timer
>100MHz

1x16-bit
Timer
>100MHz

USB-PD

USB
OTG
2.0 FS

CAN-FD

Securable
Memory
Area

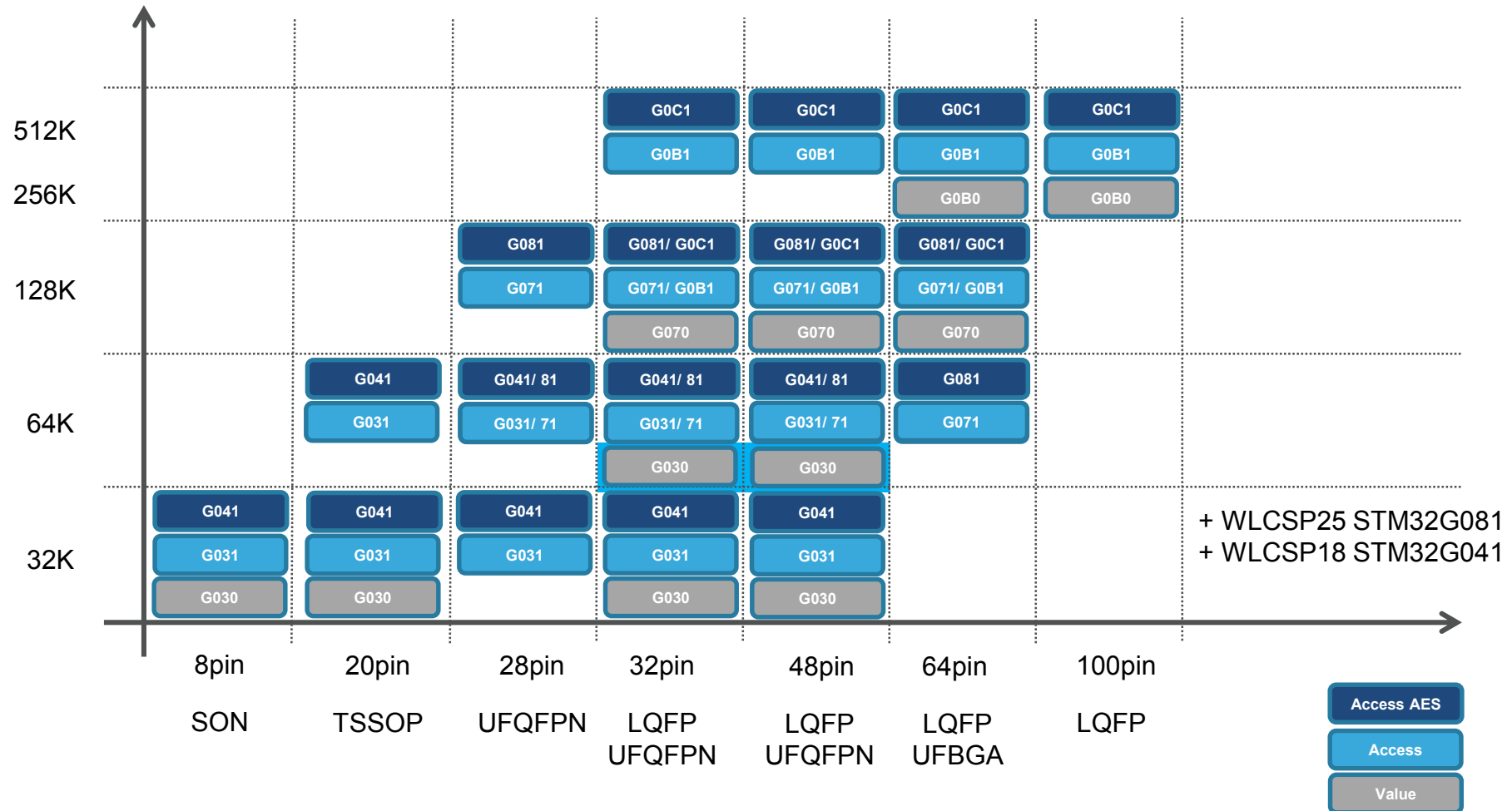
AES
TRNG



On some part-numbers



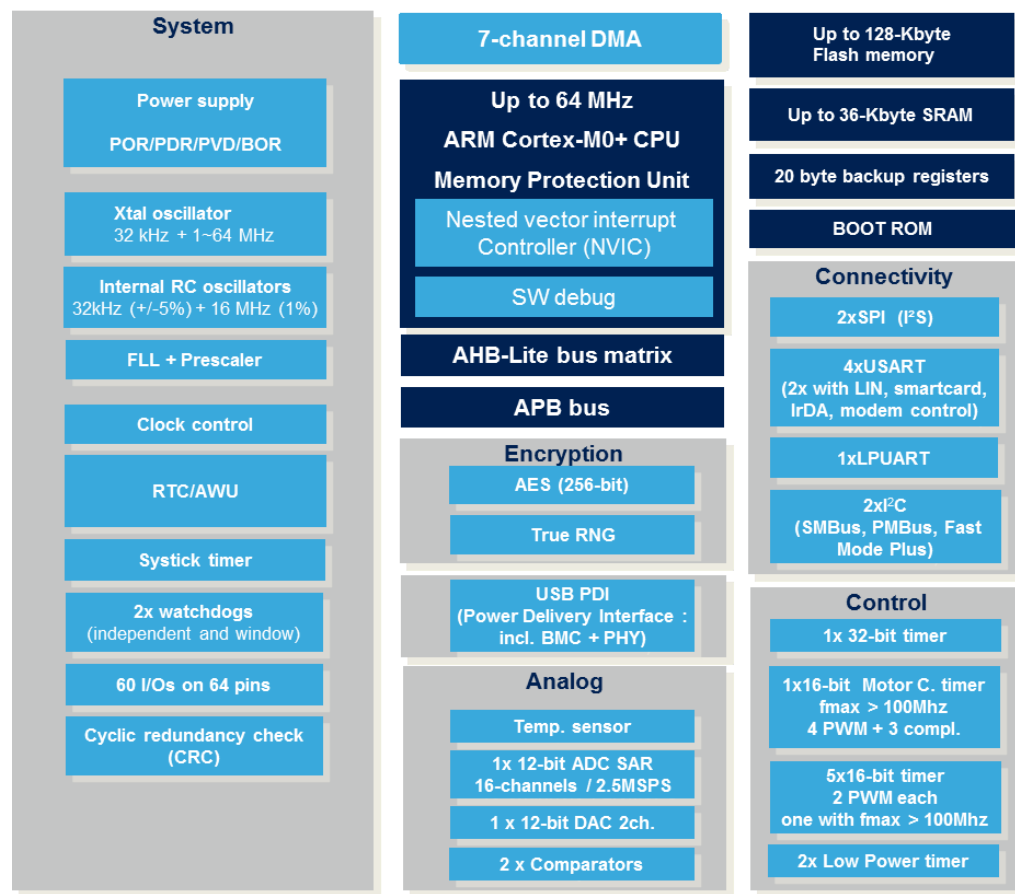
STM32G0 Portfolio





Advanced features and solutions

- Arm 32-bit Cortex-M0+ core
- 1.7 to 3.6V power supply
- RAM maximization
- 1% internal clock
- Direct Memory Access (DMA)
- Communication peripherals
- USB-C Power Delivery

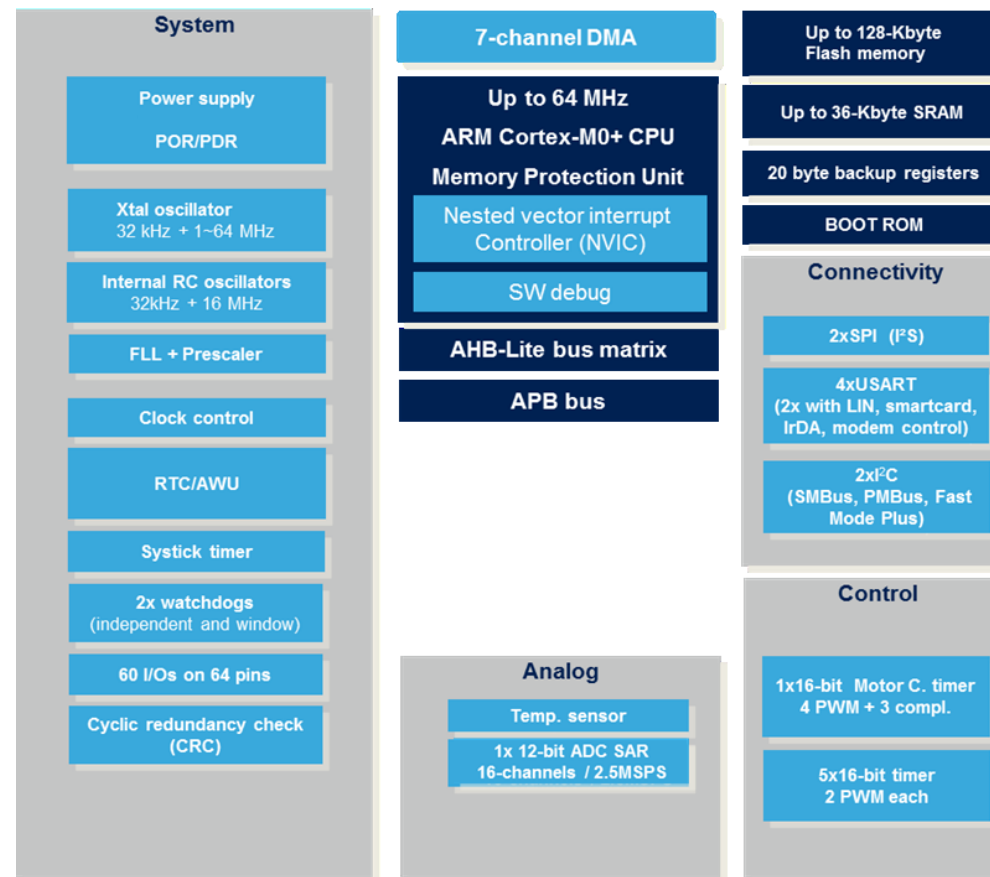


- Timers up to 2xfcpu resolution
- Real Time Clock
- I/O ports maximization
- ADC 12-bit Ultra-fast
- DAC 12-bit
- Comparators
- Safety features
- Advanced Security features



No compromise on what matters

- Arm 32-bit Cortex-M0+ core
- 2.0 to 3.6V power supply
- RAM maximization
- 1% internal clock
- Direct Memory Access (DMA)
- Communication peripherals



- Timers
- Real Time Clock
- I/O ports maximization
- ADC 12-bit Ultra-fast
- Safety features



Integrated security features, ready for tomorrow's needs

Firmware IP protection

Mutual distrustful

Secret key storage

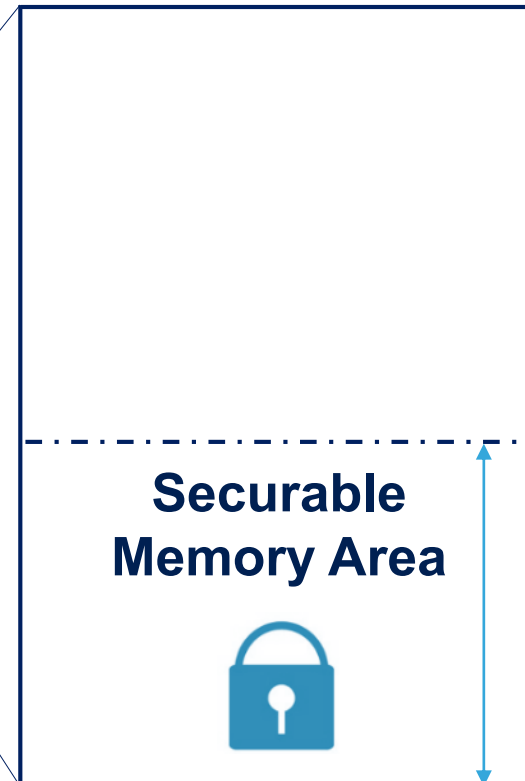
Authentication

Secure firmware upgrade



Securable Memory Area
Execute-only Protection
Read-out Protection
Write Protection
Memory Protection Unit (MPU)
AES-256 / SHA-256 Encryption
True Random Number Generator
Unique ID

User flash



Standard user flash by default

Can be secured once exiting
No more access nor debug

Configurable size

Good fit to store critical data

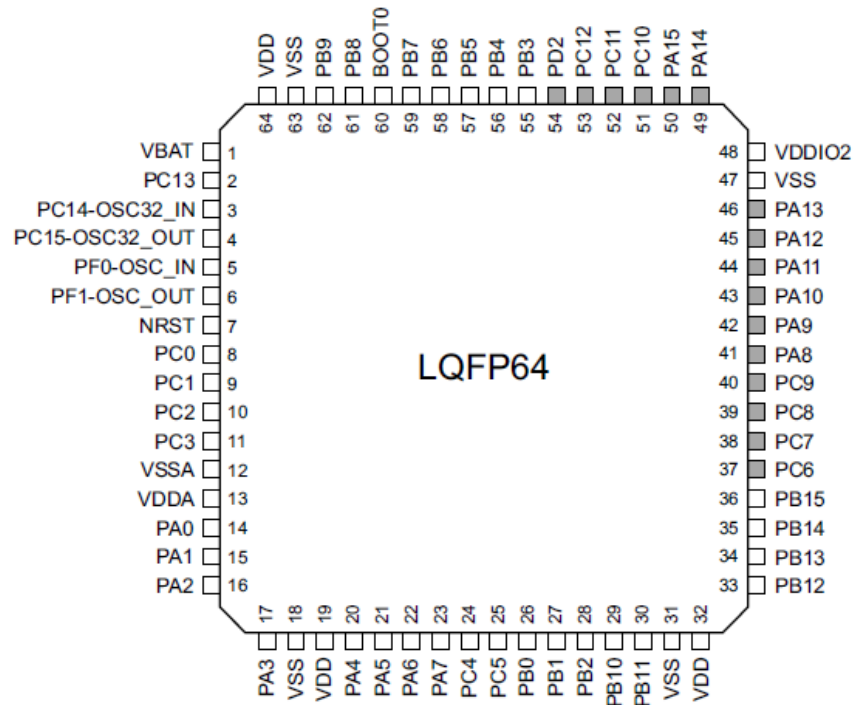
- Critical routines
- Keys



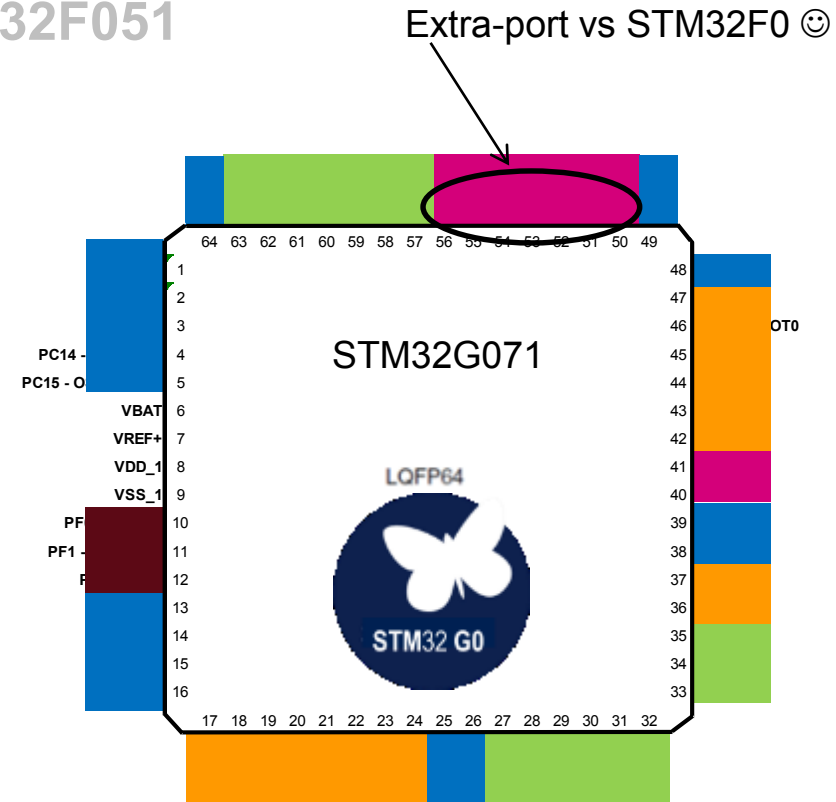
LQFP64 pin-to-pin comparison

21

- 9 IOs more on STM32G071 vs STM32F071
- 5 IOs more on STM32G071 vs STM32F051



51 I/Os



60 I/Os

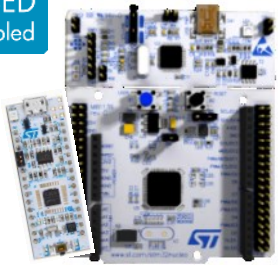


Go fast, be first

HARDWARE TOOLS

STM32 Nucleo

arm
MBED
Enabled



Flexible
prototyping

Discovery kit



Key feature
prototyping

Evaluation board

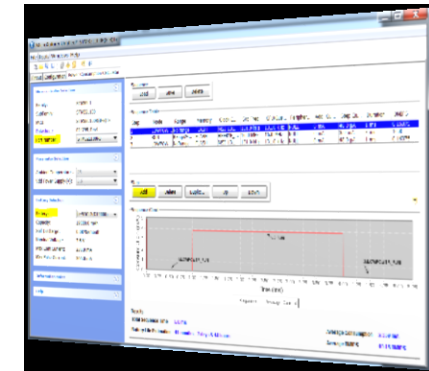
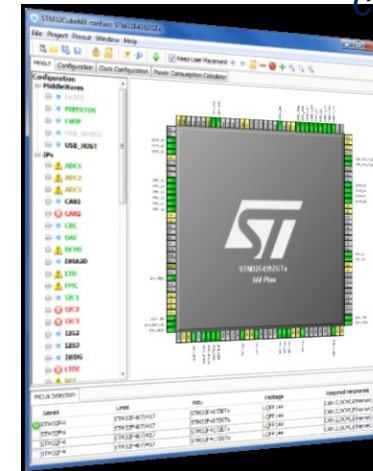


Full feature
evaluation

SOFTWARE TOOLS

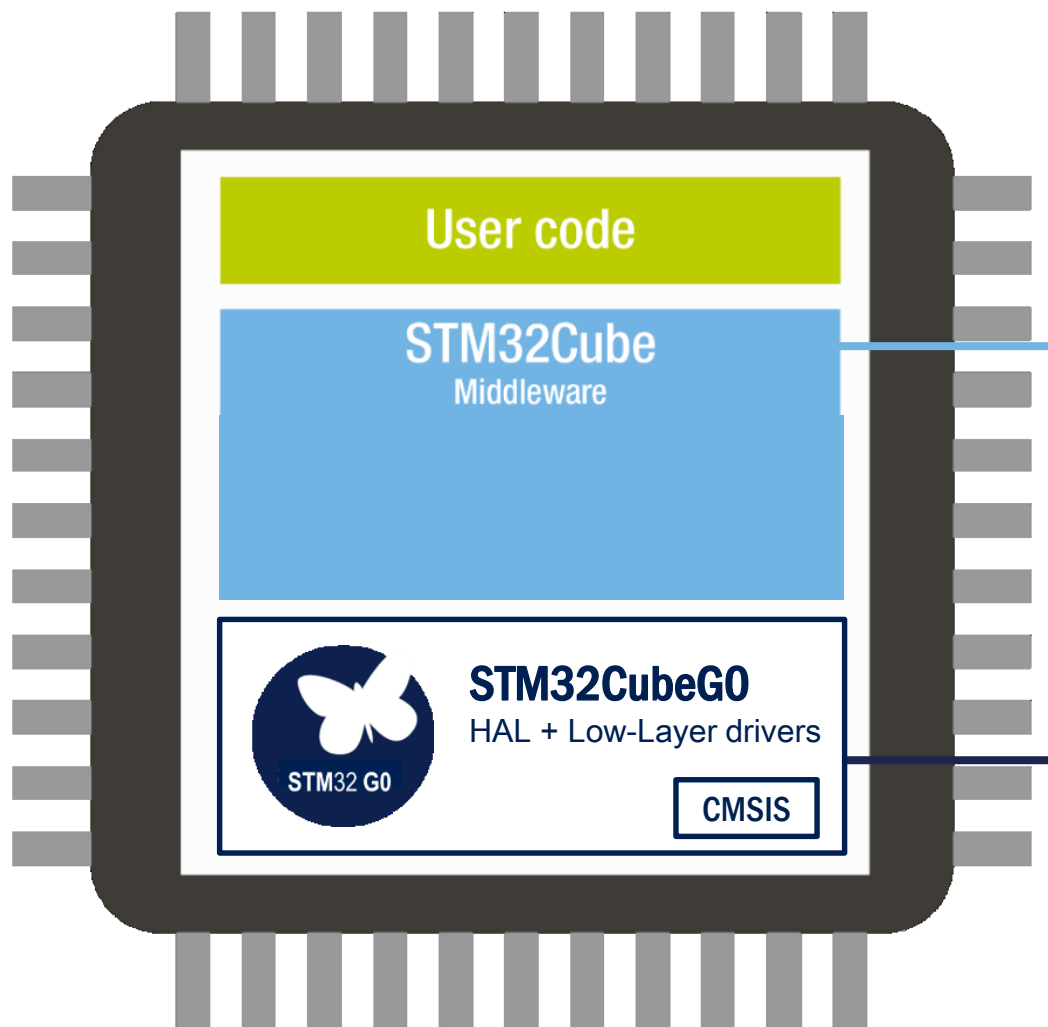


STM32CubeMX featuring intuitive pin selection, clock tree configuration, code generation and power consumption calculation





Platform approach or custom code: you choose



- Open-source FAT file system (FatFs)
- Open-source real-time OS (FreeRTOS)
- Dozens of examples



- STM32G0 Hardware Abstraction Layer (HAL) portable APIs
- **High-performance, light-weight low-layer (LL) APIs**
- High coverage for most STM32 peripherals
- Production-ready and fully qualified
- Dozens of usage examples
- Open-source BSD license



SUMMARY

3 Keys of STM32G0 series

24

1 Efficient

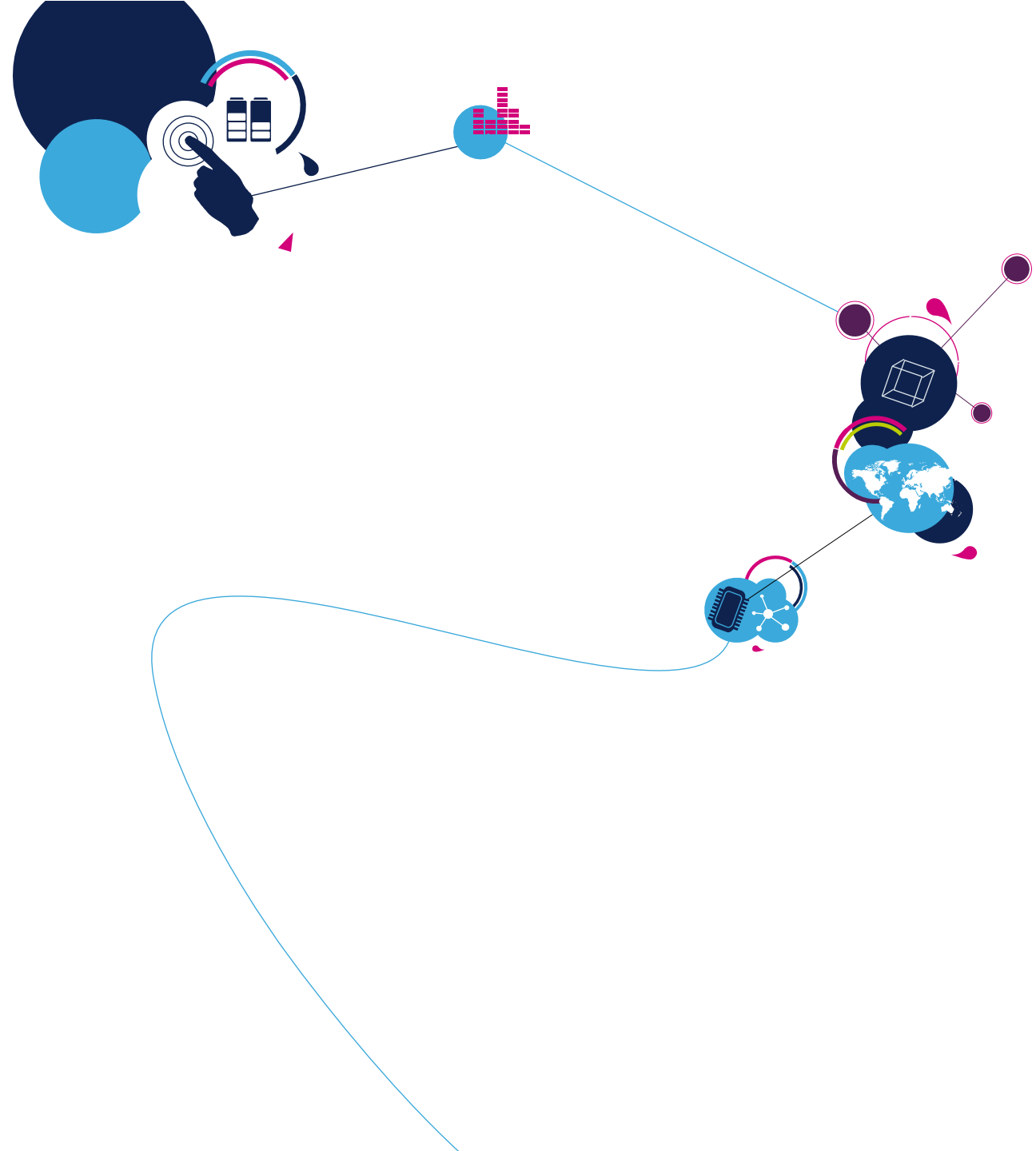
2 Robust

3 Simple

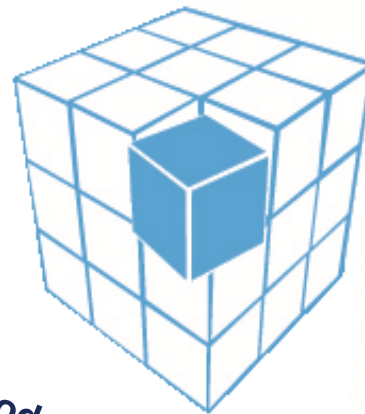


STM32CubeMX 5.0

STM32CubeMX graphical software configuration tool



STM32CubeMX



Initialization Code generation based on user choices



STM32CubeL0



STM32CubeL1



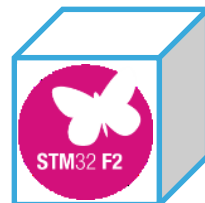
STM32CubeL4



STM32CubeF0



STM32CubeF1



STM32CubeF2



STM32CubeF3



STM32CubeF4



STM32CubeF7



STM32CubeH7

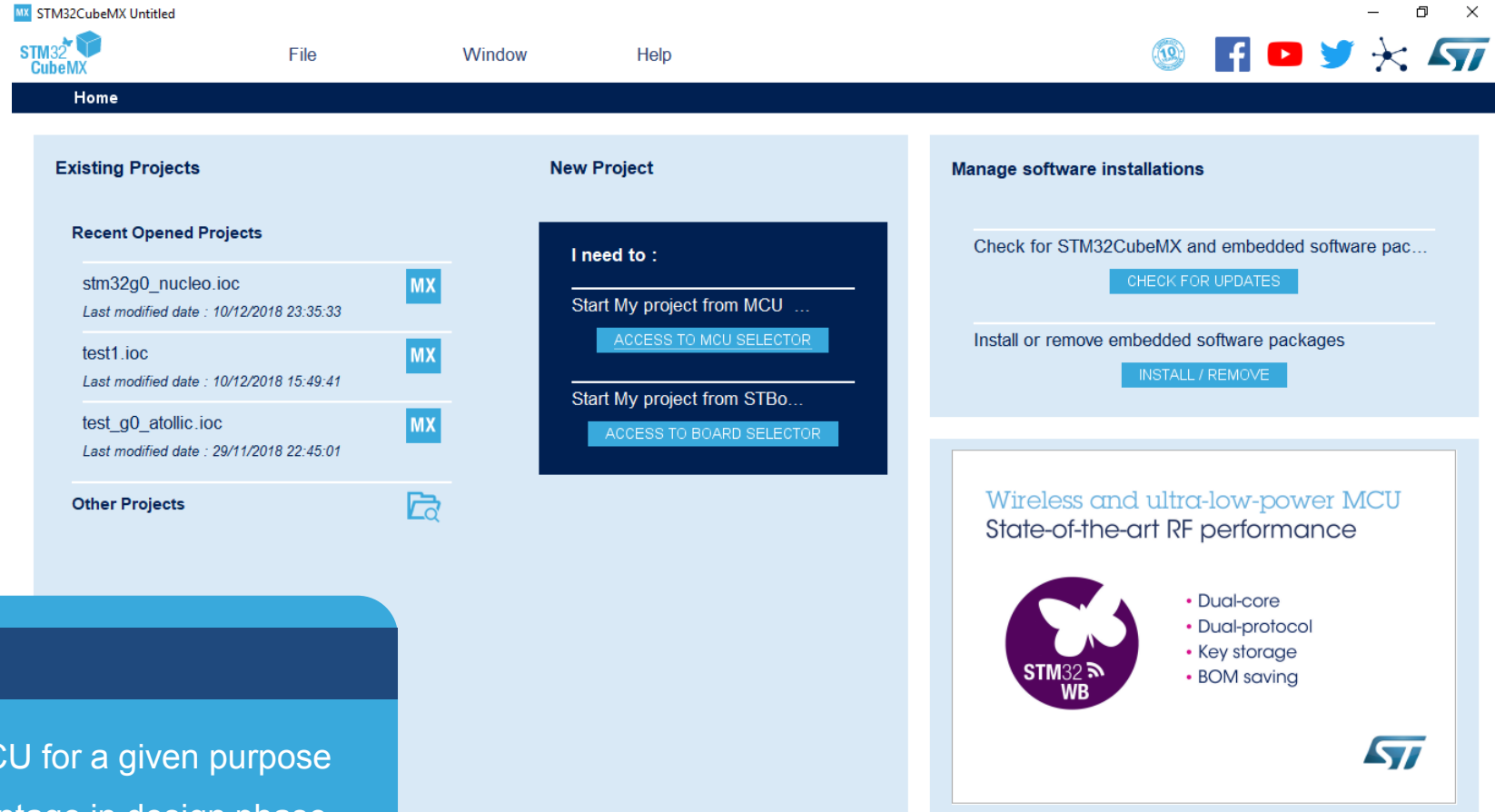


STM32CubeG0

STM32Cube HAL: Portable API within all series - Middleware stacks when applicable: RTOS, USB, TCP/IP, Graphics, ...

- Choose ideal MCU and simply configure

- Pinouts
- Clocks and oscillators
- Peripherals
- Low-power modes
- Middleware



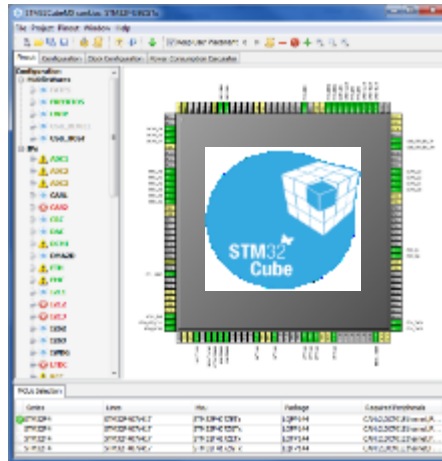
Application benefits

- Helps choose the correct MCU for a given purpose
- Simulation provides an advantage in design phase
- Boosts development speed with a headstart

- Peripheral and middleware parameters
- Power consumption calculator
- Code generation
 - Possible to re-generate code while keeping user code intact.
- Option of command-line and batch operation
- Expandable by plugins
- MCU selector
 - Filter by family, package, peripherals or memory sizes.
 - Search for similar product.
- Pinout configuration
 - Choose peripherals to use and assign GPIO and alternate functions to pins.
- Configure NVIC and DMA
- Clock tree initialization
 - Choose oscillator and set PLL and clock dividers.

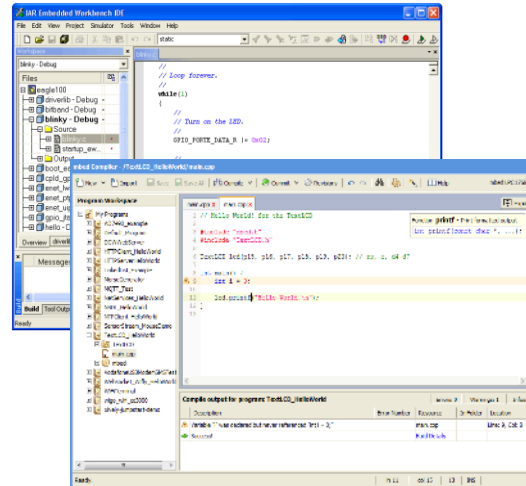
Comprehensive choice of IDEs

30



STM32CubeMX

Generate Code



Partners IDEs

Compile & Debug



STMStudio

Monitor



Free
IDE

STM32G0 – System Architecture

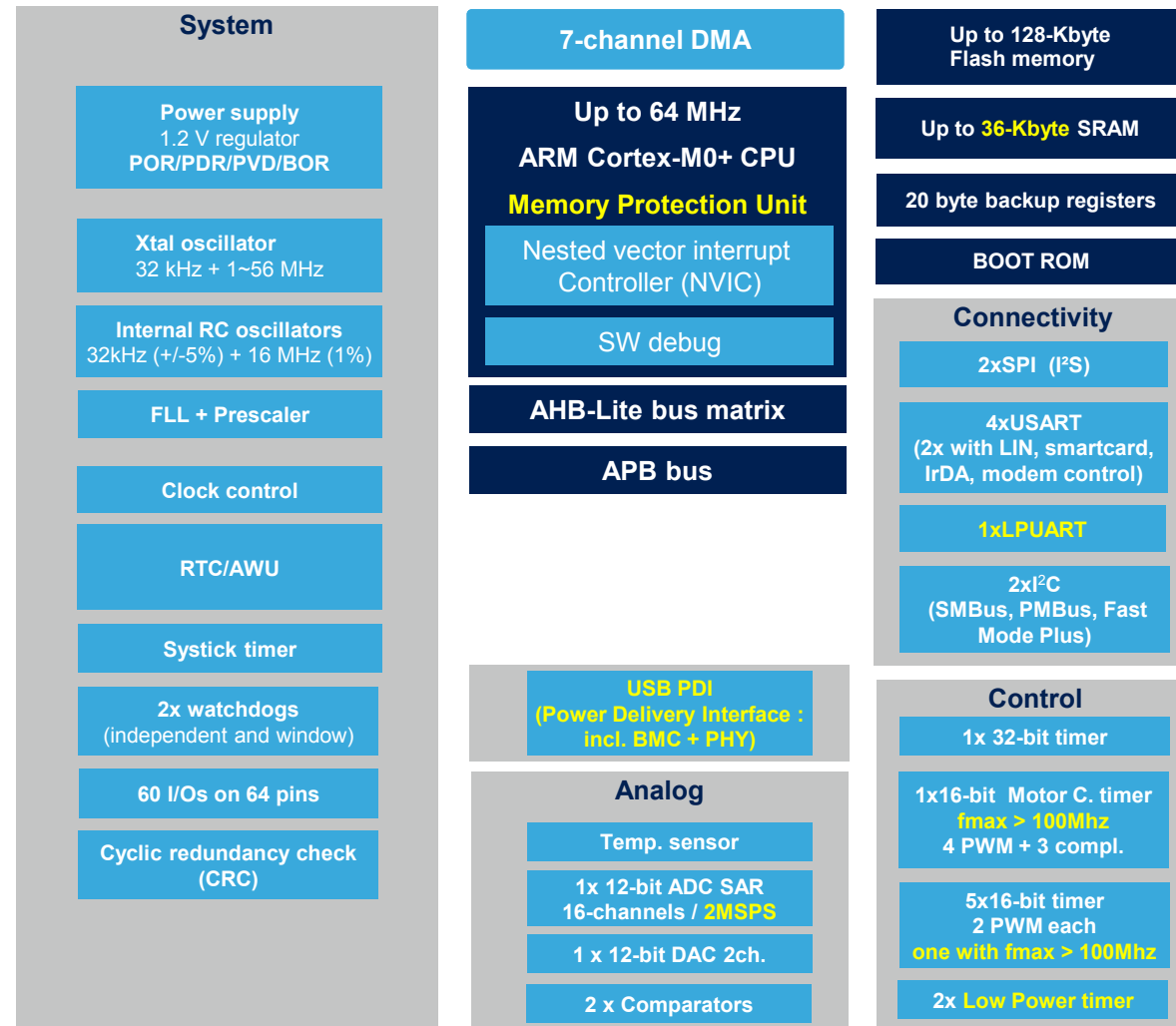
STM32G071 Block diagram

32

Main specification

- 1.65V to 3.6V
- 0.93 DMIPS/MHz
- **Vbat** supply
- Vref+ pin
- **Max ambient temp 125°C**
- One Supply pair
- Securable Memory Area
- High sink I/Os
- <100µA/MHz run mode
- **Stand-by** <1µA @ room temperature
- Stop 5 µA @ room temperature
- **Shutdown mode**
- Low EMI SAE (2.5@24MHz)
- Robust EMC/ESD/EMS
- 28/32/48 and 64 pins

features highlight



Cortex M0+ vs Cortex M0

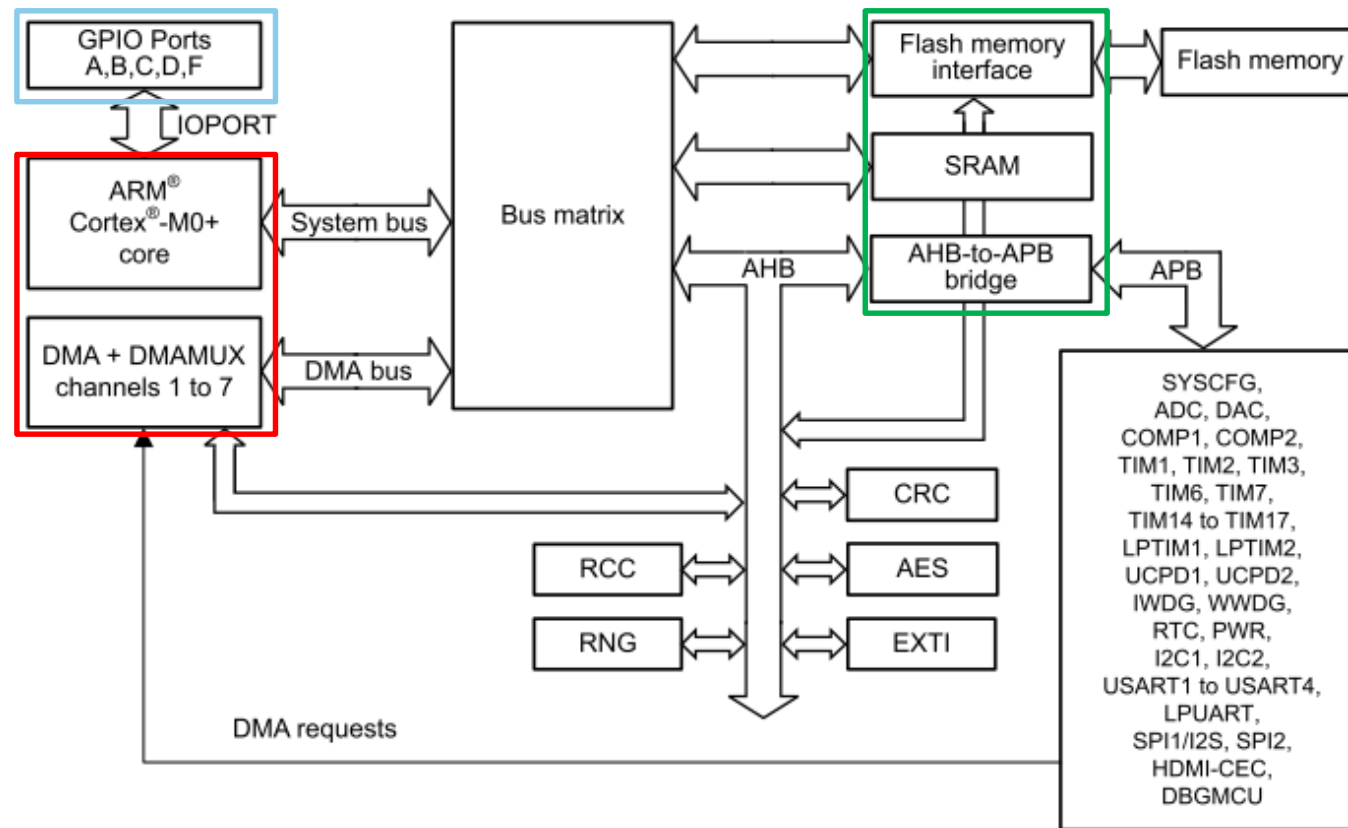
33

- STM32G0 has been upgraded with a Cortex M0+ core bringing more security and performances

	STM32F0	STM32G0
Relocatable vector table	No	Yes
Pipeline	3 Stages	2 Stages
Performance	2.20 Coremark/MHz (48MHz – 1WS)	2.23 Coremark/MHz (64MHz – 2WS)
MPU	No	Yes
Breakpoints	4	4
Single cycle 32bits x 32bits Multiplier	Yes	Yes

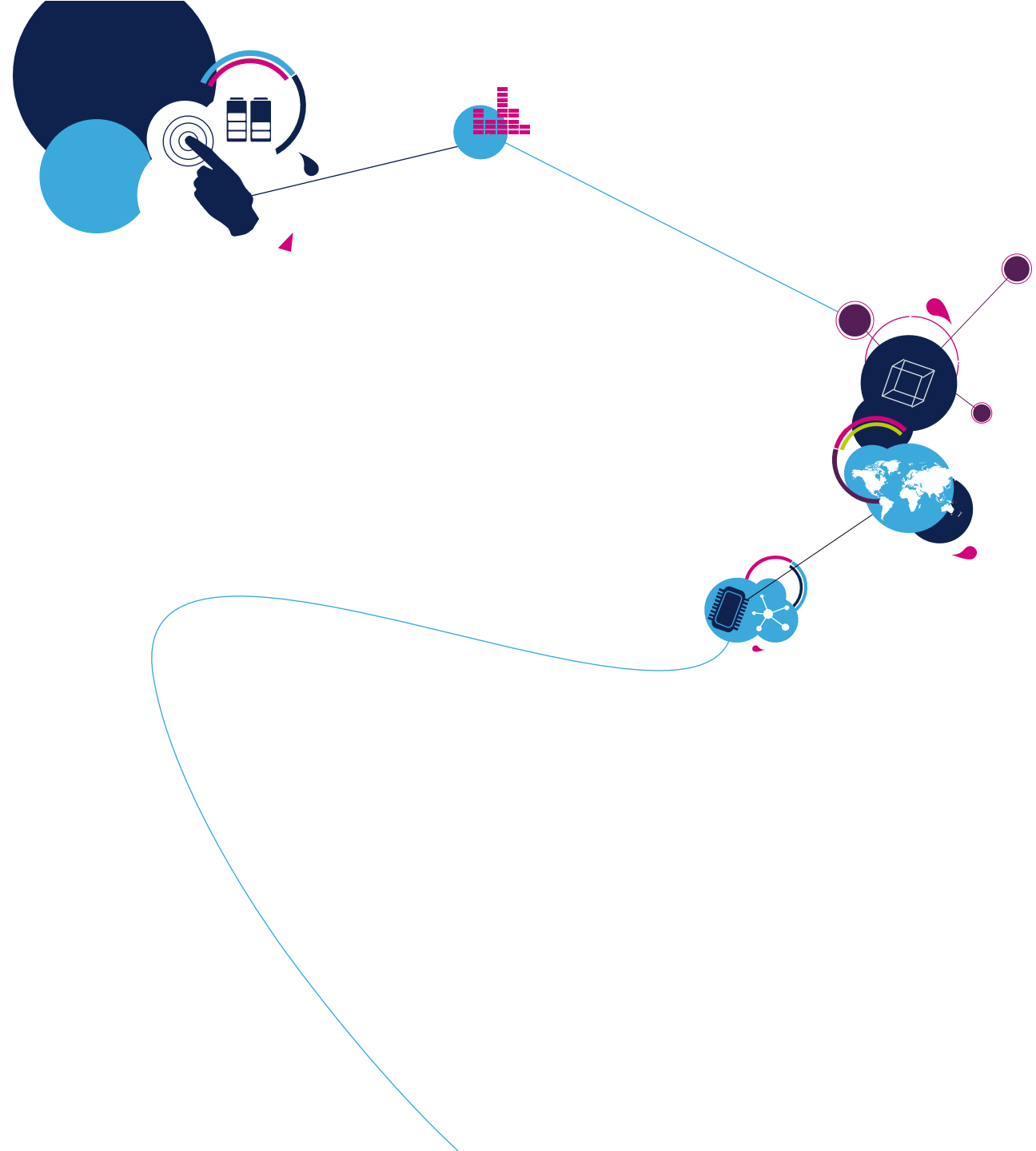
System architecture overview

34



- Two masters:
 - Cortex®-M0+ core
 - General-purpose DMA
- Three slaves:
 - Internal SRAM
 - Internal Flash memory
 - AHB with AHB-to-APB bridge that connects all the APB peripherals
- Dedicated IOPORT for accessing the GPIOs

STM32G0 - RCC



Main Differences with STM32F0

36

- The Reset and Clock Controller is similar to the one implemented in the STM32F0 family with some enhancements

	STM32F0	STM32G0
NRST	Input & output	GPIO, Input, Input & output
Reset Holder	No	Yes
PLL	One output	Three outputs
CSS on LSE + LSCO *	No	Yes
HSI divider to SYSCLK	No	Yes
Timer 1 & 15 running at 2xSYSCLK	No	Yes

* CSS on HSE was already available for both F0 and G0 products

37



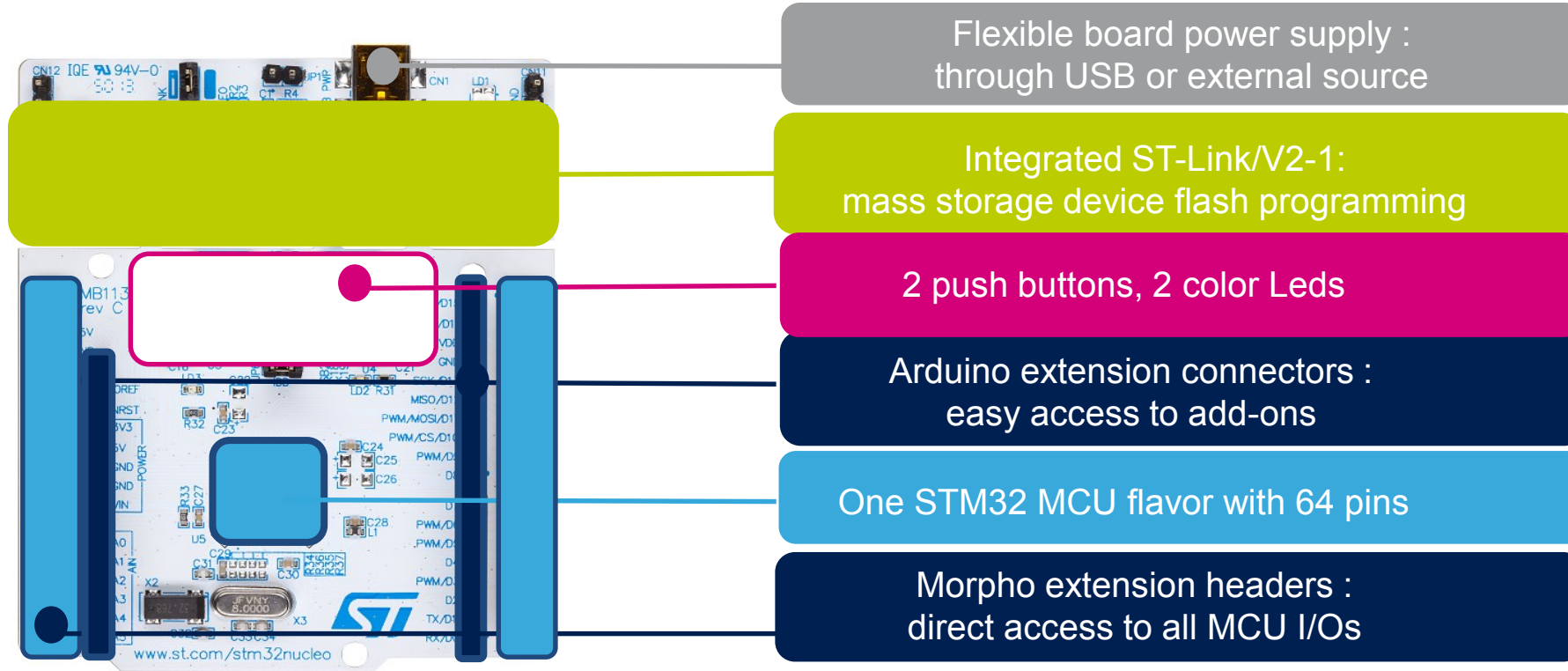
- Selected between HSI16, HSE, PLL, LSI, and LSE
- System clock, AHB and APB maximum frequency: 64 MHz

Voltage range	SYSCLK	HSI16	HSE	PLL
Range 1	64 MHz max.	16 MHz	48 MHz	VCO max = 344 MHz
Range 2	16 MHz max.	16 MHz	16 MHz	VCO max = 128 MHz
Low-power run/sleep	2 MHz max.	Allowed with divider	Not allowed	Not allowed



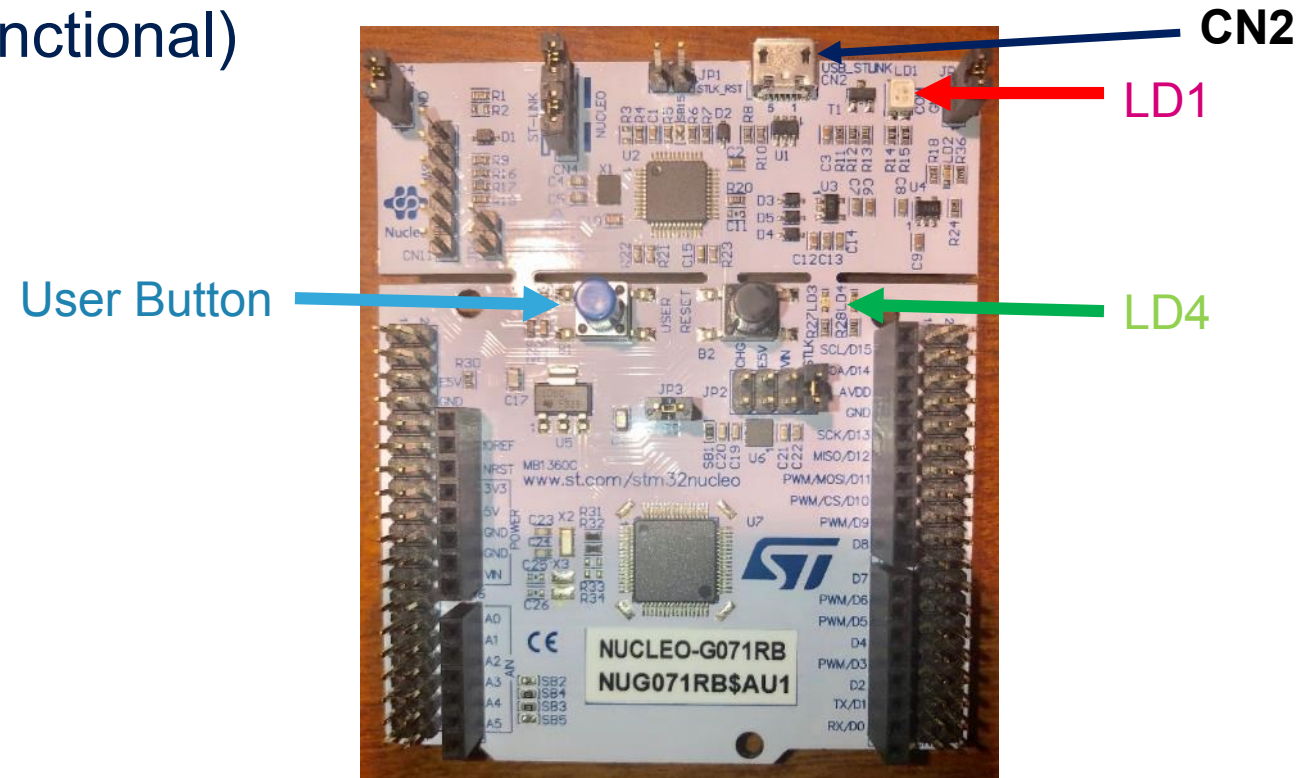
STM32 NUCLEO features

39



We are now going to provide you with the STM32G0 Nucleo board.

- Connect USB ST-LINK (**CN2**) to your PC
 - ST-LINK driver may be installed if this is the first time the board is plugged in.
- **LD1** should be **ON** and solid **RED** (indicating board power available and ST-Link is functional)



Lab: Saving the original code

Objective:

- The objective of this lab is to save the “out-of-the-box” firmware demonstration code using our stand-alone programmer: STM32CubeProgrammer

• Main Features:

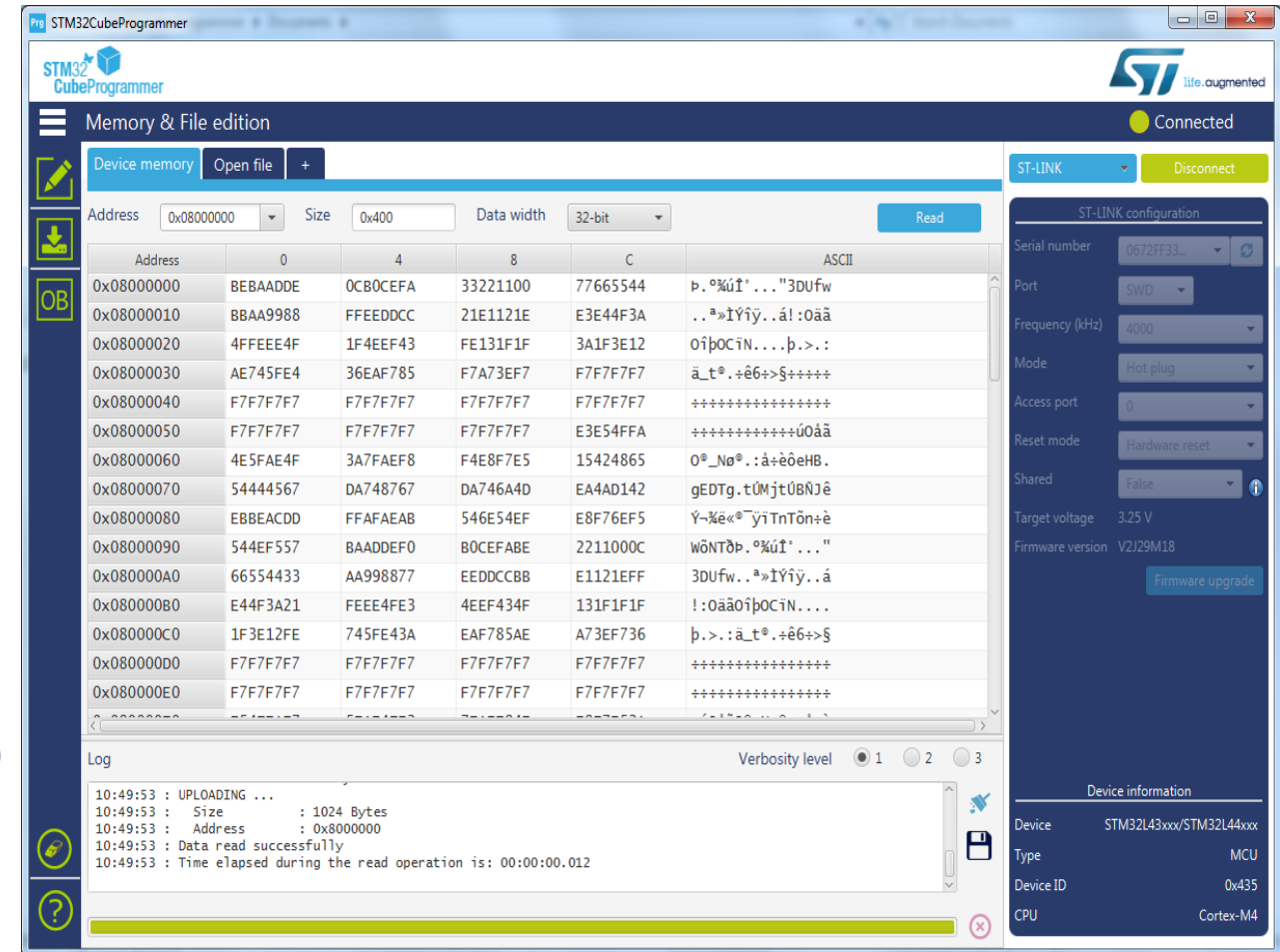
- Unify existing programming software tools : Merge STVP, ST-Link Utility and Bootloader softwares tools in one solution.

- Multiplatform
(Windows, Linux and macOS)



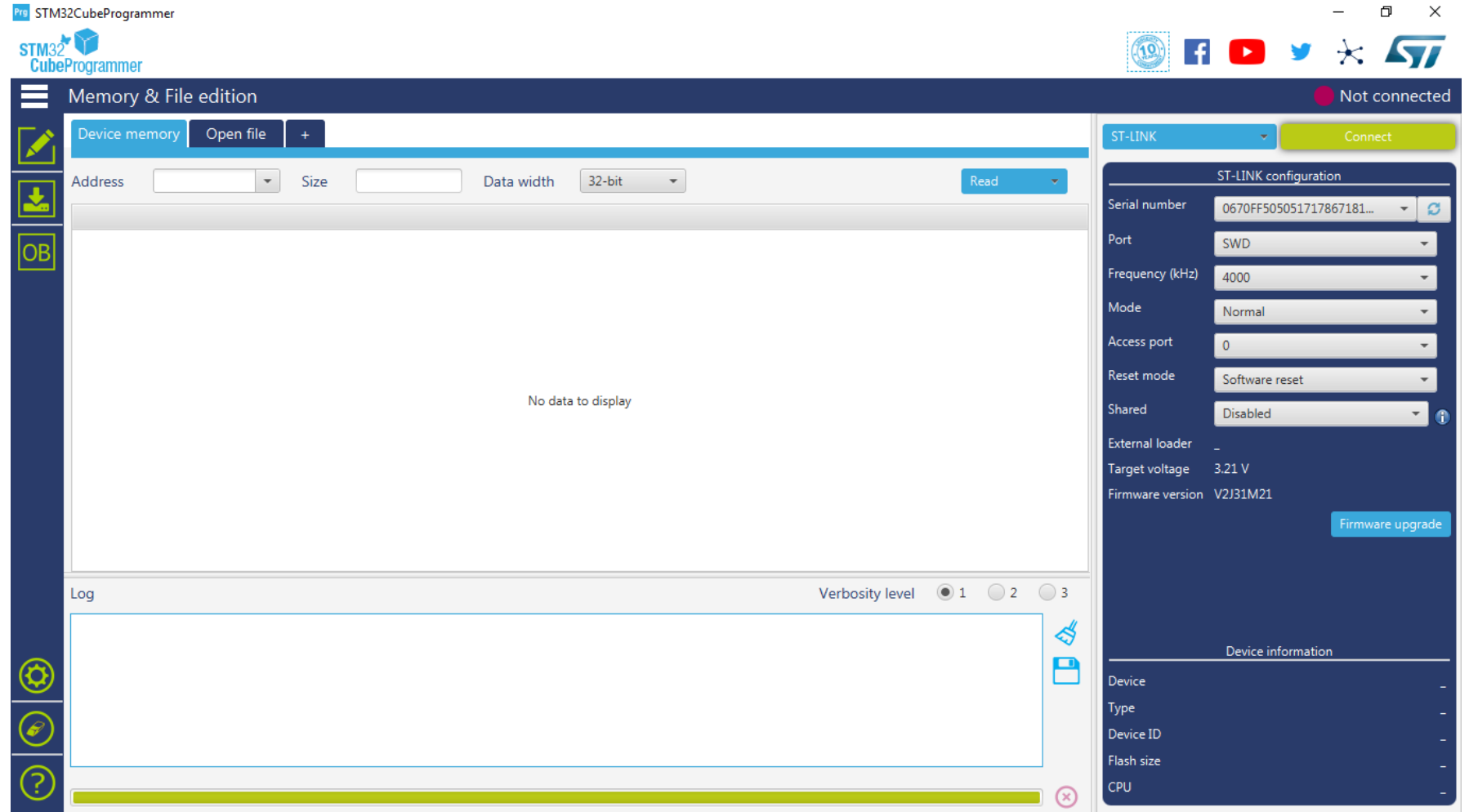
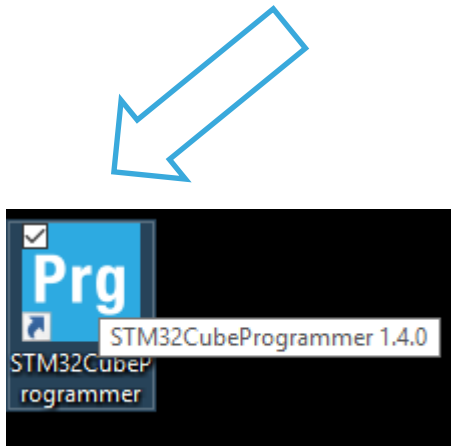
- Debug and bootloader interfaces support: ST-LINK debug probe (JTAG/SWD), Bootloader interfaces (UART, USB DFU, SPI, I²C and CAN)

- Secure programming



Run STM32CubeProgrammer

44



Connect to the ST-LINK

45

STM32CubeProgrammer

Memory & File edition

Device memory Open file +

Address Size Data width 32-bit Read

No data to display

Log Verbosity level 1 2 3

ST-LINK NOT connected

ST-LINK configuration

Serial number 0670FF505051717867181L...

Port SWD

Frequency (kHz) 4000

Mode Normal

Access port 0

Reset mode Software reset

Shared Disabled

External loader -

Target voltage 3.21 V

Firmware version V2J31M21

Firmware upgrade

Device information

Device -

Type -

Device ID -

Flash size -

CPU -

Connected

ST-LINK configuration

Serial number 0670FF505051717867181L...

Port SWD

Frequency (kHz) 4000

Mode Normal

Access port 0

Reset mode Software reset

Shared Disabled

External loader -

Target voltage 3.21 V

Firmware version V2J31M21

Firmware upgrade

Device information

Device STM32G07x/STM32G08x

Type MCU

Device ID 0x460

Flash size 128 KB

CPU Cortex-M0+

Log

17:32:52 : Address : 0x40022020

17:32:52 : Size : 32 Bytes

17:32:52 : Bank : 0x01

17:32:52 : Address : 0x40022080

17:32:52 : Size : 4 Bytes

17:32:53 : UPLOADING ...

17:32:53 : Size : 1024 Bytes

17:32:53 : Address : 0x8000000

17:32:53 : Read progress:

17:32:53 : Data read successfully

17:32:53 : Time elapsed during the read operation is: 00:00:00.007

life.augmented

Save the content of the flash

46

- Change the size to: 0x20000 ①
 - Equivalent to 128K which is the size of the Flash of the STM32G0 on the Nucleo board
- Then click: ②
Read -> Save As...

STM32CubeProgrammer

Memory & File edition

Device memory +

Address: 0x08000000 Size: 0x20000 Data width: 32-bit

Read Save As ...

Address	0	4	8	C	ASCII
0x08000000	20000890	08003BF9	080037B9	080037BB	... ù;...'7...'7..
0x08000010	00000000	00000000	00000000	00000000	
0x08000020	00000000	00000000	00000000	080037BF	¿7..
0x08000030	00000000	00000000	080037C1	080037C3	A7..A7..
0x08000040	08000A13	08001493	08001497	0800278F	'..
0x08000050	080038C7	0800382B	08003837	080037DD	Ç8...+;...7;...Y7..
0x08000060	08003C45	08003C47	08003C49	08003C4B	E<...G<...I<...K<...
0x08000070	08003C4D	08003C4F	08003C51	08003C53	M<...O<...Q<...S<...
0x08000080	08003803	08003C55	08003C57	08003C59	.8...U<...W<...Y<...
0x08000090	08003C5B	08003C5D	08003C5F	08003C61	[<...]<...<...a<...
0x080000A0	08003C63	08003C65	08003C67	08003C69	c<...e<...g<...i<...
0x080000B0	08003C6B	08003C6D	08003C6F	08003C71	k<...m<...o<...q<...
0x080000C0	0004B510	D1012C00	E0372001	280069A0	.µ...,.Ñ. 7à i.(

Log

Verbosity level 1 2 3

```
10:27:00 : Address : 0x40022020
10:27:00 : Size : 32 Bytes
10:27:00 : Bank : 0x01
10:27:00 : Address : 0x40022080
10:27:00 : Size : 4 Bytes
10:27:00 : UPLOADING ...
10:27:00 : Size : 131072 Bytes
10:27:00 : Address : 0x8000000
10:27:00 : Read progress:
10:27:01 : Data read successfully
10:27:01 : Time elapsed during the read operation is: 00:00:00.944
```

ST-LINK configuration

Serial number: 066FFF484951717867111750

Port: SWD

Frequency (kHz): 4000

Mode: Normal

Access port: 0

Reset mode: Software reset

Shared: Disabled

External loader: -

Target voltage: 3.23 V

Firmware version: V2J31M21

Firmware upgrade

Device information

Device: STM32G07x/STM32G08x

Type: MCU

Device ID: 0x460

Flash size: 128 KB

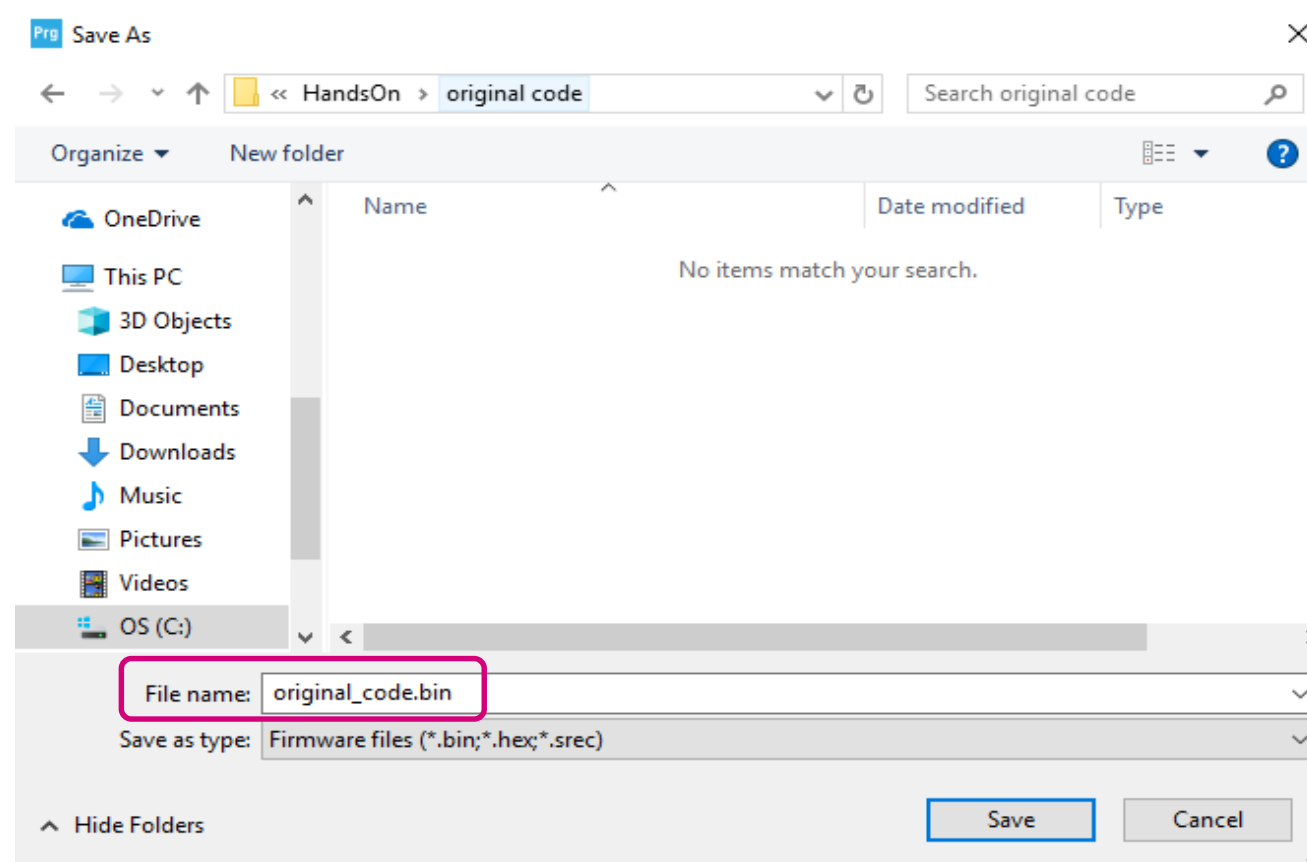
CPU: Cortex-M0+

Save the content of the flash

47

- Save it to a location you will remember because we will restore it at the end of the workshop.
- Save as a binary file (*.bin)

For example: C:\STM32G0WorkShop\HandsOn\original code**original_code.bin**



Disconnect to the ST-LINK

48

STM32CubeProgrammer

Memory & File edition

Device memory

Open file

Address: 0x08000000, Size: 0x400, Data width: 32-bit

Read

Address	0	4	8	C	ASCII
0x08000000	20000410	080000D1	0800097B	08000979	... N...{...y...
0x08000010	00000000	00000000	00000000	00000000	
0x08000020	00000000	00000000	00000000	0800097F	
0x08000030	00000000	00000000	0800097D	08000981	}
0x08000040	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...ä...
0x08000050	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...ä...
0x08000060	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...ä...
0x08000070	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...ä...
0x08000080	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...ä...
0x08000090	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...ä...
0x080000A0	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...ä...
0x080000B0	080000E3	080000E3	080000E3	46854803	ä...ä...ä...H.F
0x080000C0	F83CF000	47004800	08000A49	20000410	.ð<ø.H.GI.....

Log

Verbosity level: 1 2 3

17:32:52 : Address : 0x40022020
17:32:52 : Size : 32 Bytes
17:32:52 : Bank : 0x01
17:32:52 : Address : 0x40022080
17:32:52 : Size : 4 Bytes
17:32:53 : UPLOADING ...
17:32:53 : Size : 1024 Bytes
17:32:53 : Address : 0x80000000
17:32:53 : Read progress:
17:32:53 : Data read successfully
17:32:53 : Time elapsed during the read operation is: 00:00:00.007

ST-LINK configuration

Serial number: 0670FF505051717867181...

Port: SWD

Frequency (kHz): 4000

Mode: Normal

Access port: 0

Reset mode: Software reset

Shared: Disabled

External loader: -

Target voltage: 3.21 V

Firmware version: V2J31M21

Firmware upgrade

Device information

Device: STM32G07x/STM32G08x

Type: MCU

Device ID: 0x460

Flash size: 128 KB

CPU: Cortex-M0+

Click On "Disconnect"

Not connected

Connect

ST-LINK configuration

Serial number: 0670FF505051717867181...

Port: SWD

Frequency (kHz): 4000

Mode: Normal

Access port: 0

Reset mode: Software reset

Shared: Disabled

External loader: -

Target voltage: 3.21 V

Firmware version: V2J31M21

Firmware upgrade

Device information

Device: -

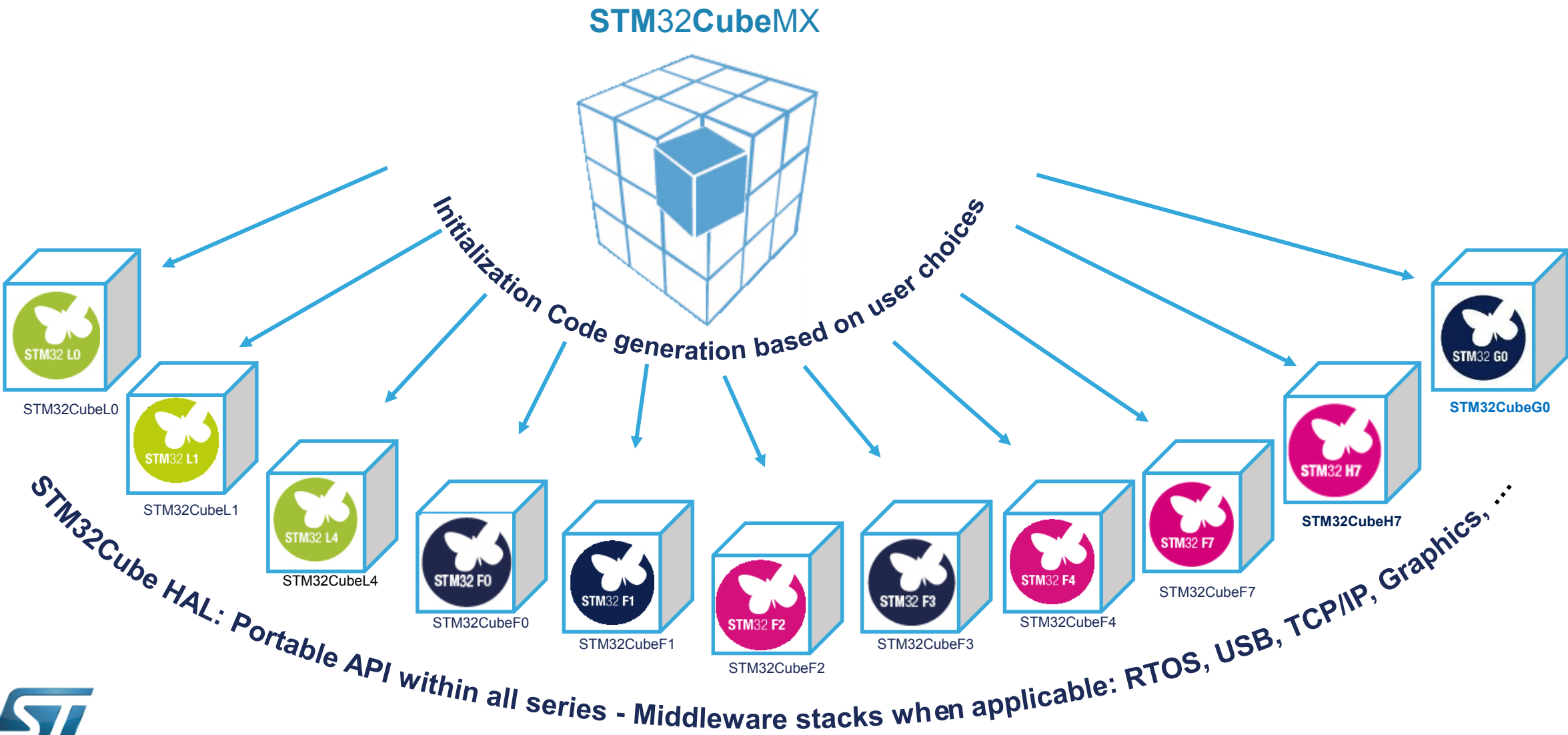
Type: -

Device ID: -

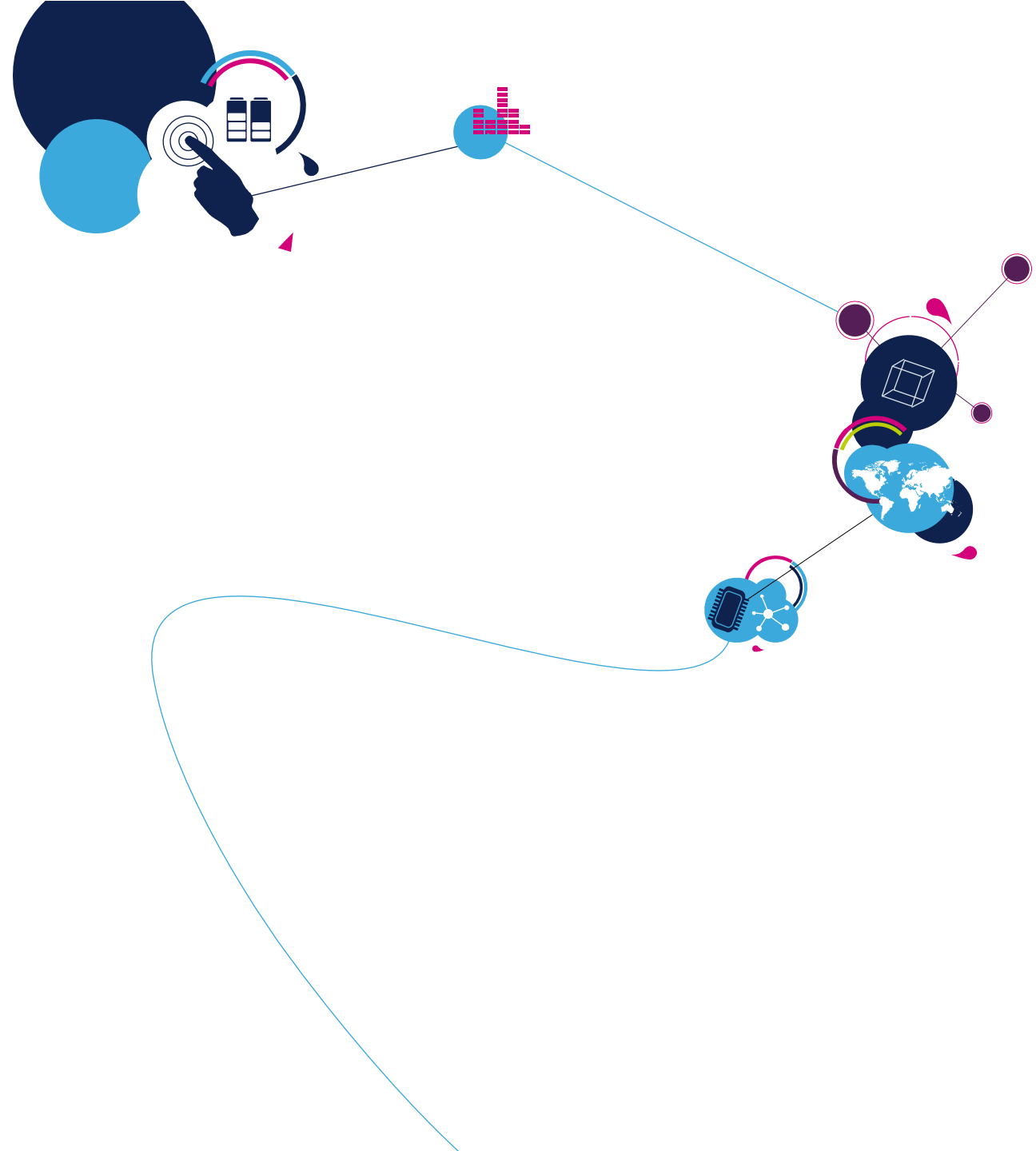
Flash size: -

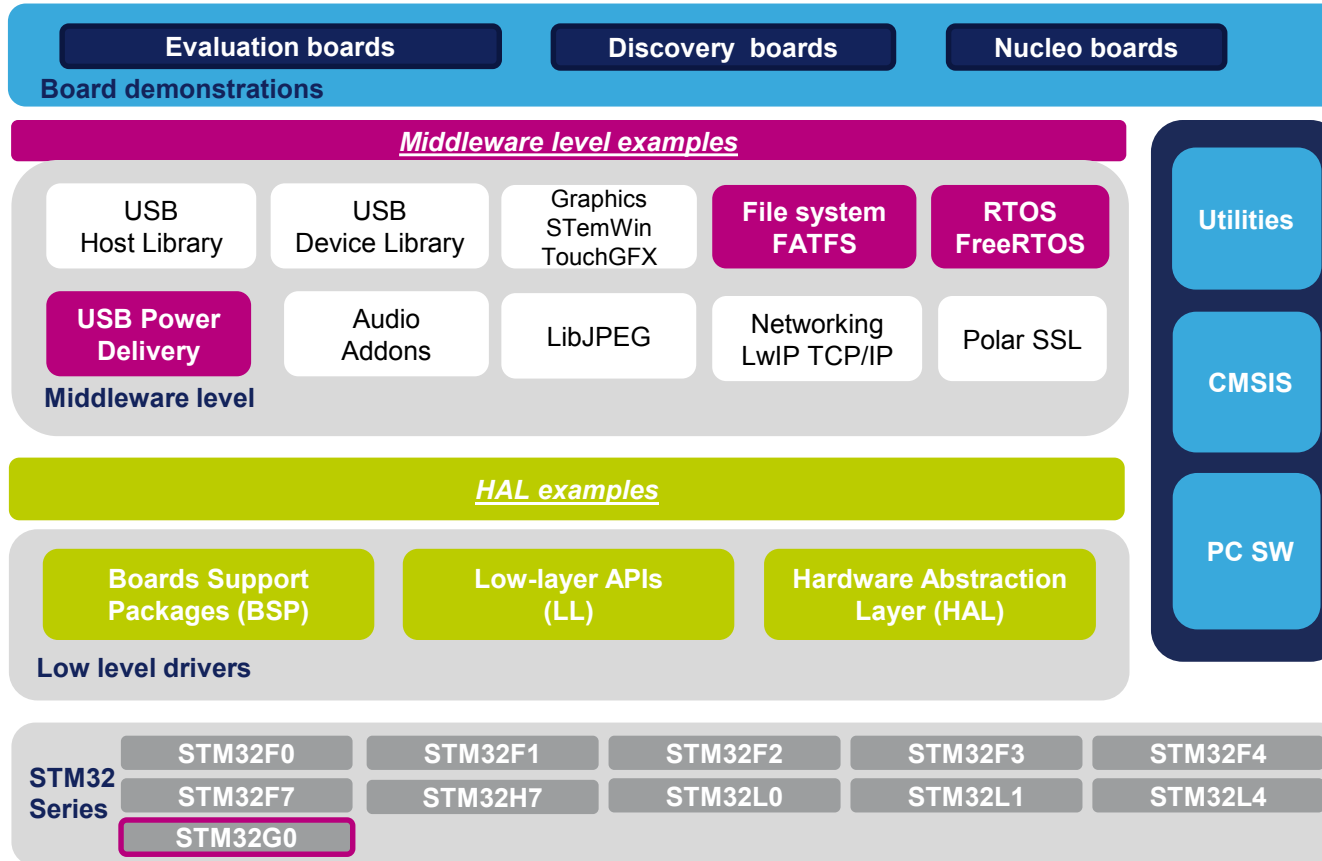
CPU: -

STM32Cube and STM32CubeMX



STM32Cube



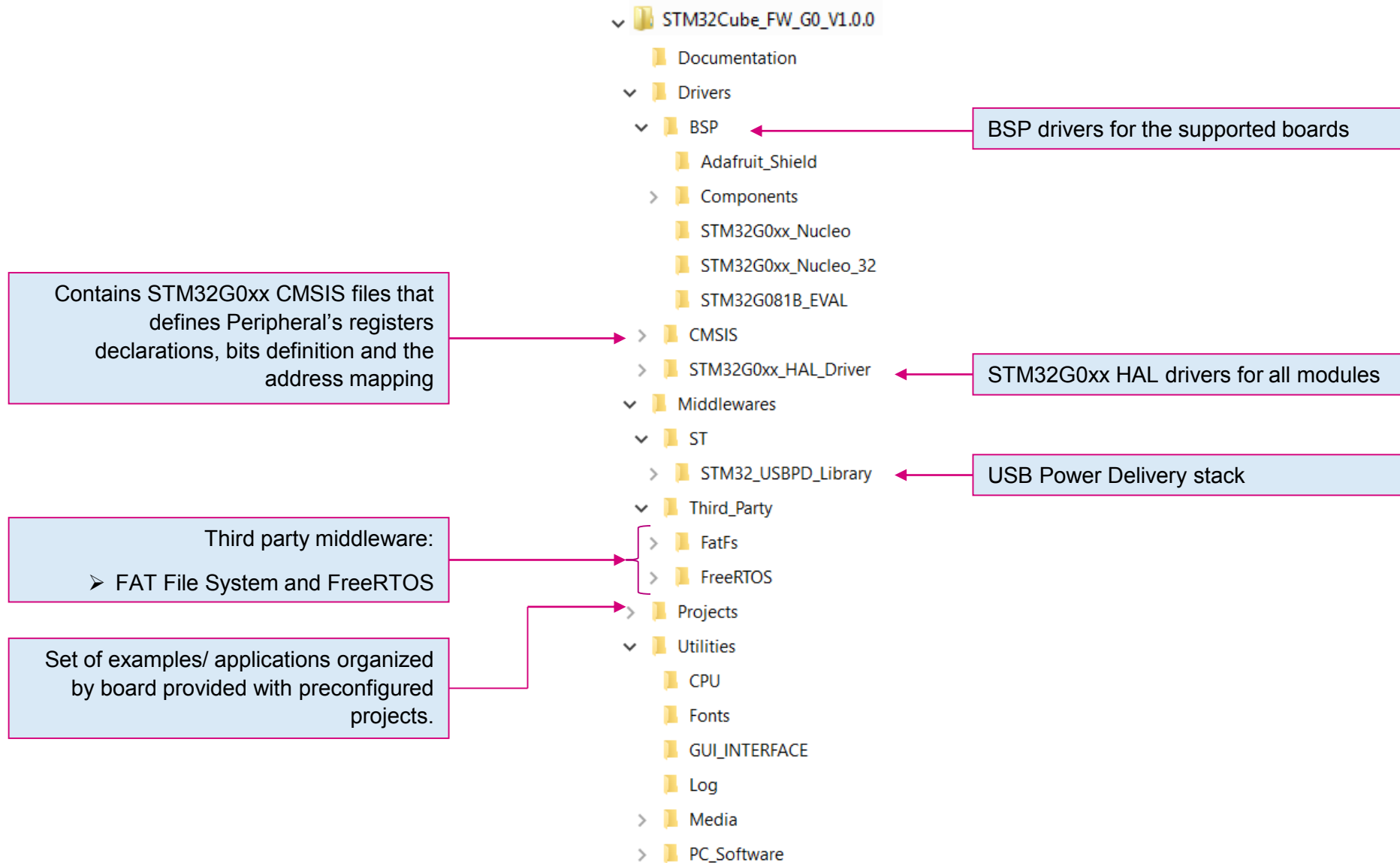


Application benefits

- Single package
- Compatible with all STM32 series
- Source code with open-source BSD license

Package organization

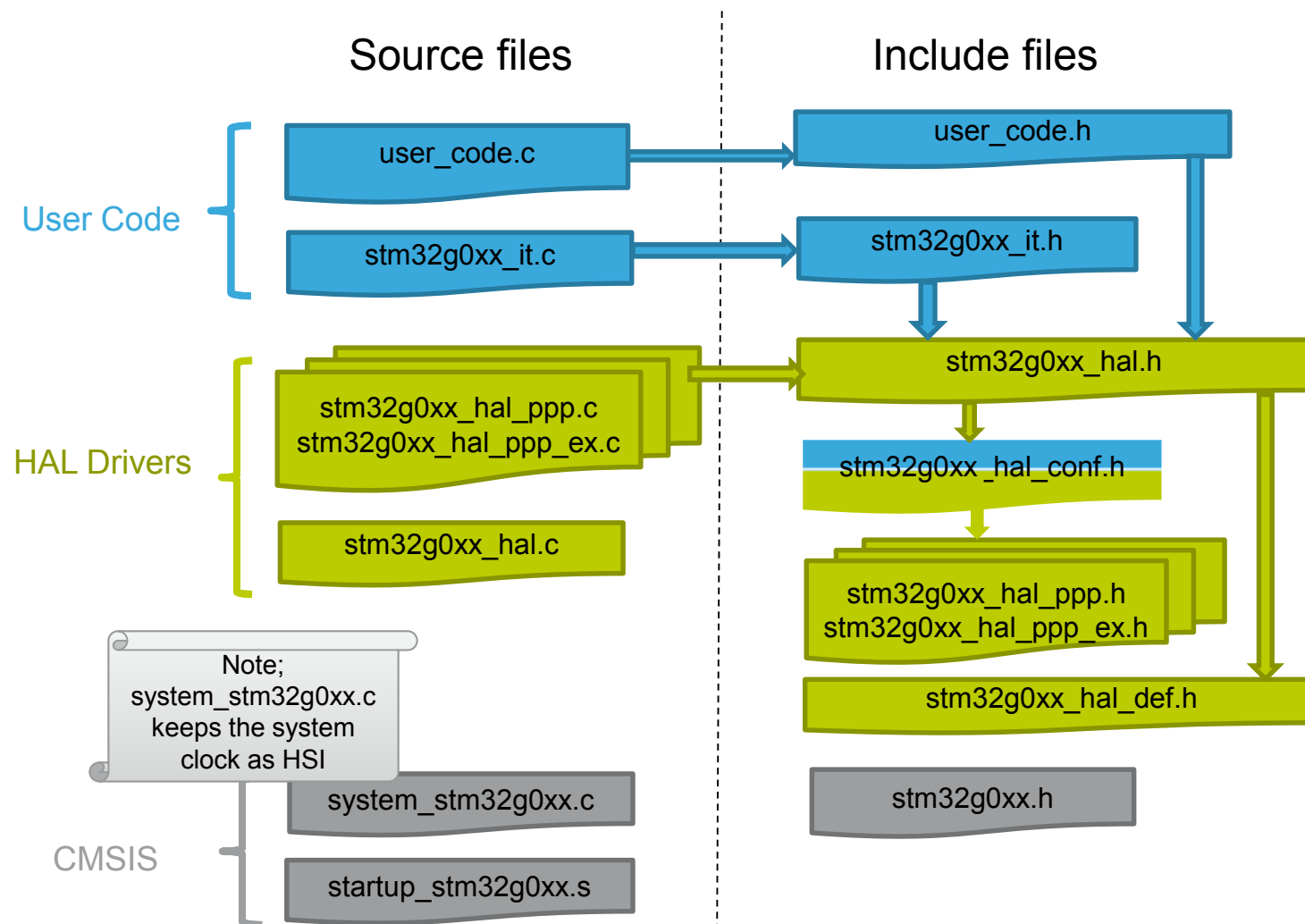
53



HAL general concepts

HAL based project organization

54



HAL general concepts

HAL drivers file

55

File	Description
stm32g0xx_hal_ppp.c/.h	Peripheral driver with cross family portable APIs
stm32g0xx_hal_ppp_ex.c/.h	Extended peripheral features APIs
stm32g0xx_hal.c	HAL global APIs (HAL_Init, HAL_DeInit, HAL_Delay,...)
stm32g0xx_hal.h	HAL header file, it should be included in user code
stm32g0xx_hal_conf.h	Config file for HAL, should be customized by user to select the peripherals to be included
stm32g0xx_hal_def.h	Contains HAL common type definitions and macros

Layer	Category	Provided embedded software	Provided examples
HAL	Analog	Analog/Digital conversion, ...	~66 examples on ST evaluation boards* !
	Timers	Timers, RTC, Watchdogs, ...	
	Connectivity	I2C, USART, SPI, I2S, SDMMC, CEC, ...	
Middleware	RTOS	FreeRTOS open source RTOS, with CMSIS-RTOS wrapper	~10 applications on ST evaluation boards* !
	USB Power Delivery	PD stack, device policy manager, policy engine, protocol layer	
	File System	FatFS open-source file system	
Application	Demonstration	Full demonstrations for ST boards	~1 demonstration project for ST boards!

Examples overview (1/2)

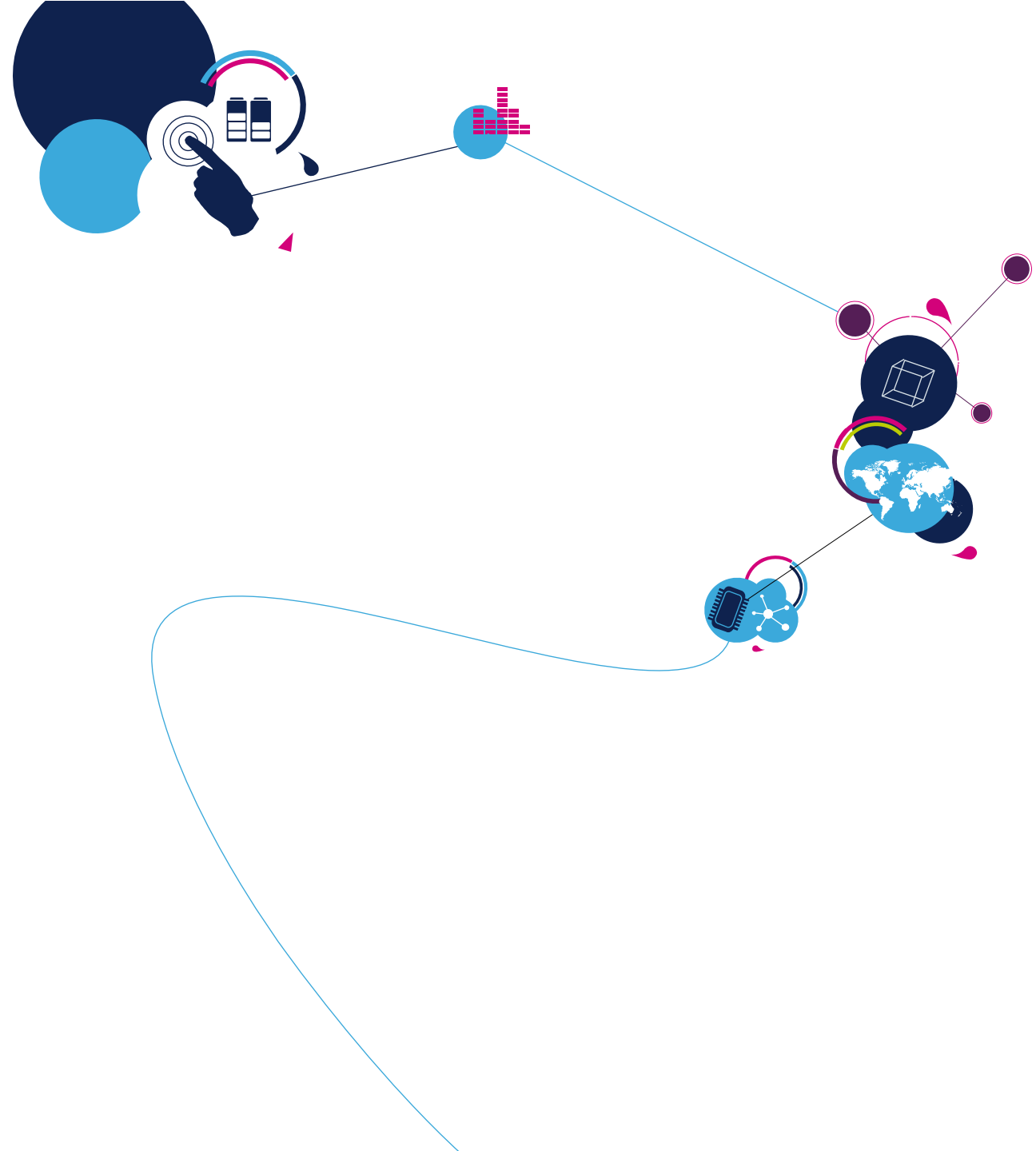
57

- For each board, a set of examples is provided with preconfigured projects for EWARM, MDK-ARM and SW4STM32 toolchains
 - This figure shows the projects structure for the NUCLEO-G071RB board, which is identical for other boards
 - The examples are classified depending on the STM32Cube level they apply to, and are named as follows:
 - Examples in Level 0 are called **Examples**, and use HAL drivers without any middleware component
 - Examples in Level 1 are called **Applications**, and provide typical use cases of each middleware component
 - Examples in Level 2 are called **Demonstration**, and implement all the HAL, BSP and middleware components

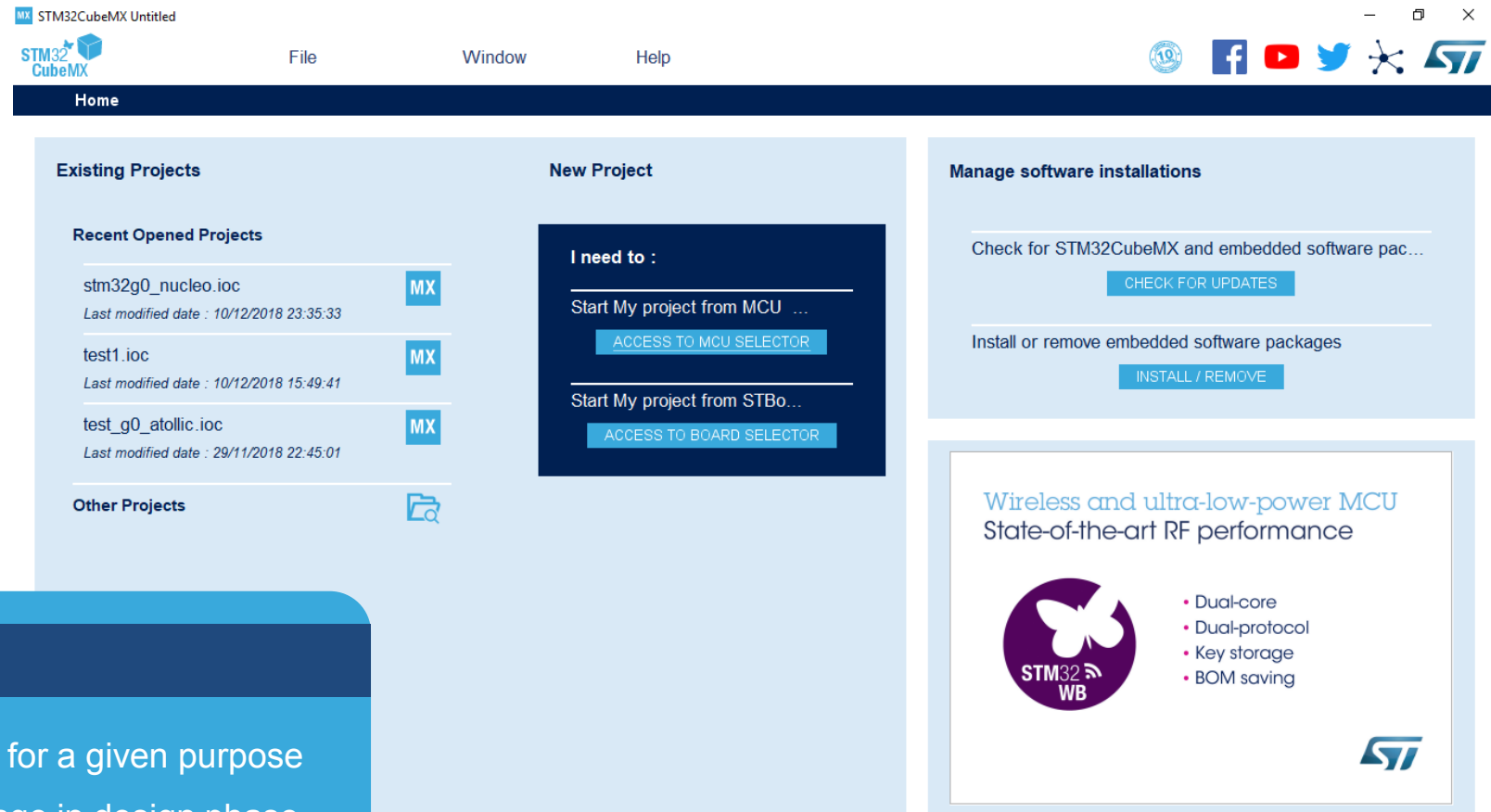
- ▼ Projects
 - > NUCLEO-G031K8
 - > NUCLEO-G070RB
 - ▼ NUCLEO-G071RB
 - ▼ Applications
 - > FatFs
 - > FreeRTOS
 - > Demonstrations
 - ▼ Examples
 - > ADC
 - > COMP
 - > CORTEX
 - > CRC
 - > DAC
 - > DMA
 - > I2C
 - > SPI
 - > TIM
 - > UART
 - > WWDG
 - > Examples_LL
 - > Examples_MIX
 - ▼ Templates
 - EWARM
 - Inc
 - MDK-ARM
 - Src
 - > SW4STM32
 - > Templates_LL

- The Template project is provided to build quickly any firmware application for all supported boards
- All examples have the same structure,
 - \Inc folder contains all header files
 - \Src folder for the source code
 - \EWARM, \MDK-ARM and \SW4STM32 contain the preconfigured project for each toolchain
 - readme.txt describes example behavior and the environment needed to make it work
 - *.ioc file that allows user to open most of FW examples within STM32CubeMX (starting from STM32CubeMX 5.0)

STM32CubeMX



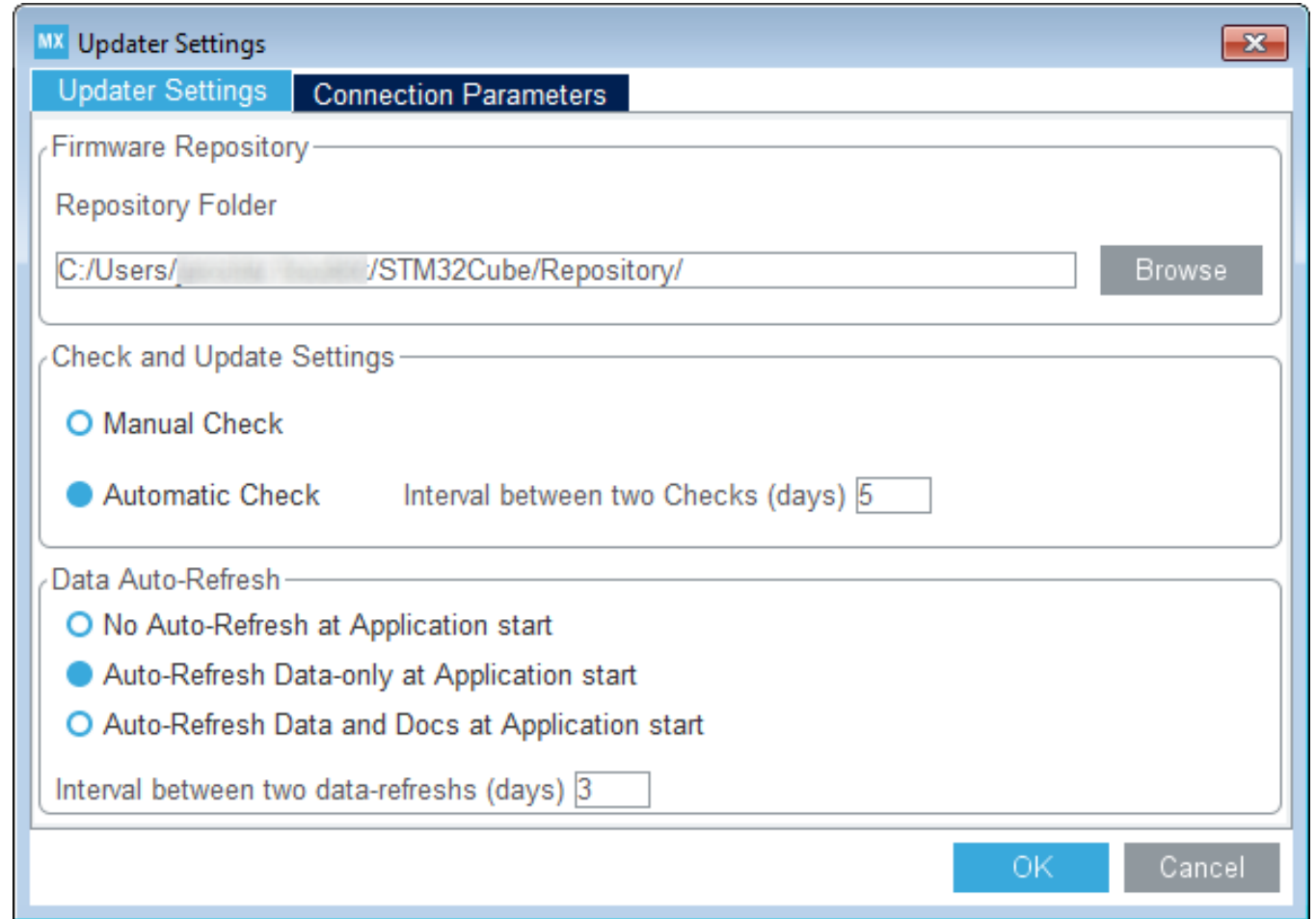
- Choose ideal MCU and simply configure
 - Pinouts
 - Clocks and oscillators
 - Peripherals
 - Low-power modes
 - Middleware



Application benefits

- Helps choose the correct MCU for a given purpose
- Simulation provides an advantage in design phase
- Boosts development speed with a headstart

- STM32CubeMX needs Java RE
 - Check release notes of the particular version for additional requirements
 - Multiplatform tool runs on Windows, Linux and macOS
- After installation, hit Alt+S to configure the updater – not only for the GUI but also for Cube FW libraries
- Select SW library placement.

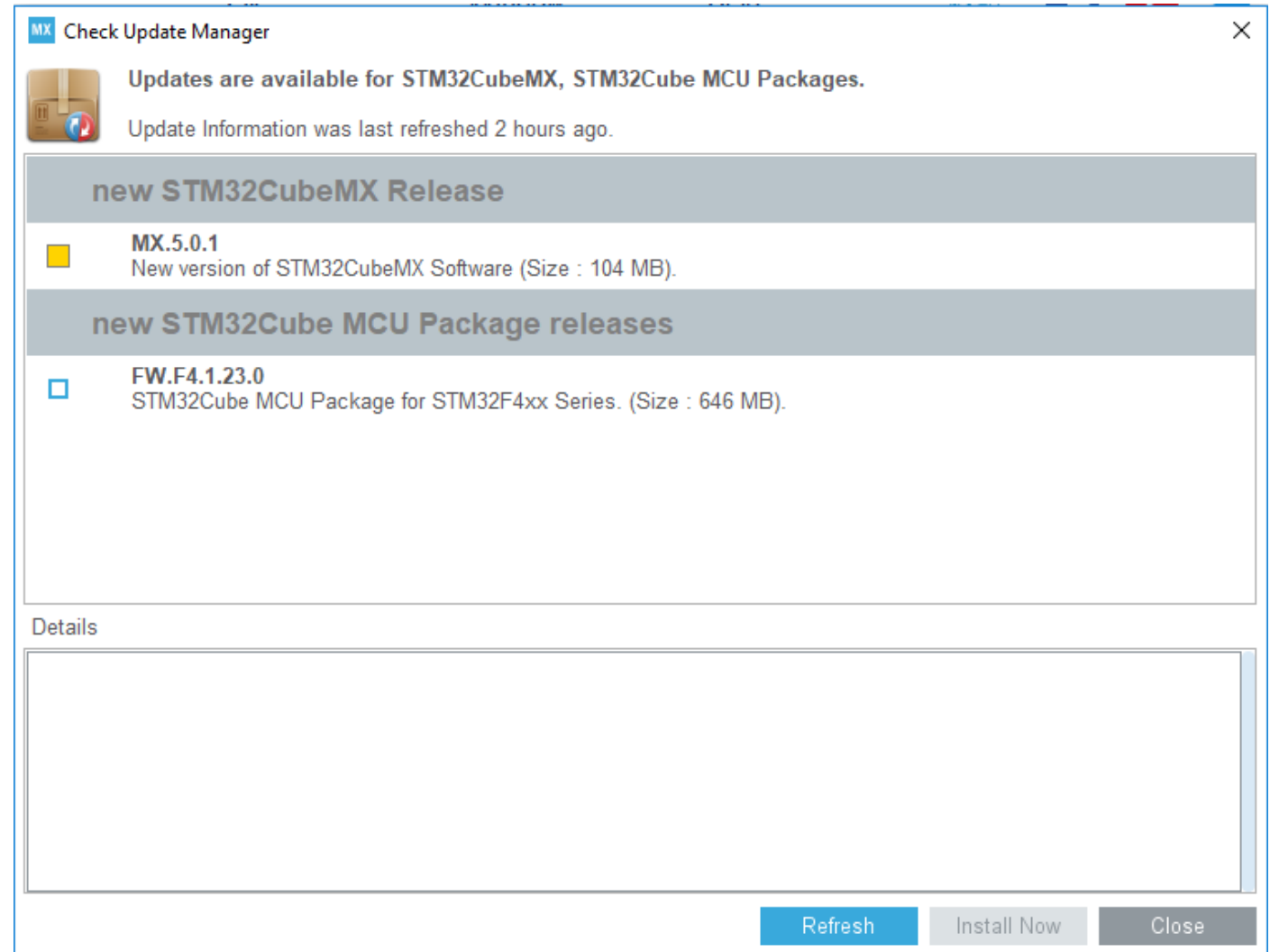


The screenshot shows the 'Updater Settings' dialog box with two tabs: 'Updater Settings' (active) and 'Connection Parameters'. The 'Updater Settings' tab contains three sections:

- Firmware Repository**: A text field for 'Repository Folder' containing 'C:/Users/[username]/STM32Cube/Repository/' and a 'Browse' button.
- Check and Update Settings**: Two radio buttons, 'Manual Check' (unselected) and 'Automatic Check' (selected). Next to 'Automatic Check' is a text field for 'Interval between two Checks (days)' with the value '5'.
- Data Auto-Refresh**: Three radio buttons: 'No Auto-Refresh at Application start' (unselected), 'Auto-Refresh Data-only at Application start' (selected), and 'Auto-Refresh Data and Docs at Application start' (unselected). Below these is a text field for 'Interval between two data-refreshes (days)' with the value '3'.

At the bottom right are 'OK' and 'Cancel' buttons.

- Updates are accessible from the Help menu
- The tool updater can detect new releases of the tool and the associated Cube library
- Use the libraries manager to download new library packages



- Find MCU by name ...
 - Quickly locate by Series and Lines
- ... or application needs
 - Package (pin count)
 - RAM size
 - NV memory requirements
 - Embedded peripherals
 - Number and type of interfaces
 - Core and frequency
 - Price
- Convenient links to documentation
- Export table to Excel file

MCU Filters

Part Number Search

Core

Series

Line

Package

Other

Advanced Graphic

Peripheral

ADC 12-bit 0 16

ADC 16-bit 0 0

AES 0 0

CAN 0 0

COMP 0 2

CRYP 0 2

DAC 12-bit 0 2

DCMI 0 0

DFSDM 0 0

DSIHOST 0 0

Ethernet 0 0

FDCAN 0 0

FMC 0 0

FMPI2C 0 0

FSMC 0 0

GFXMMU 0 0

HASH 0 0

HDMI CEC 0 0

HMAC 0 0

HRTIM 0 0

I2C 0 2

I2S 0 1

IRTIM 0 0

JPEG 0 0

LPTIM 0 2

LPUART 0 0

MCU Selector Board Selector

Features Block Diagram Docs & Resources Datasheet Buy Start Project

STM32G070RB

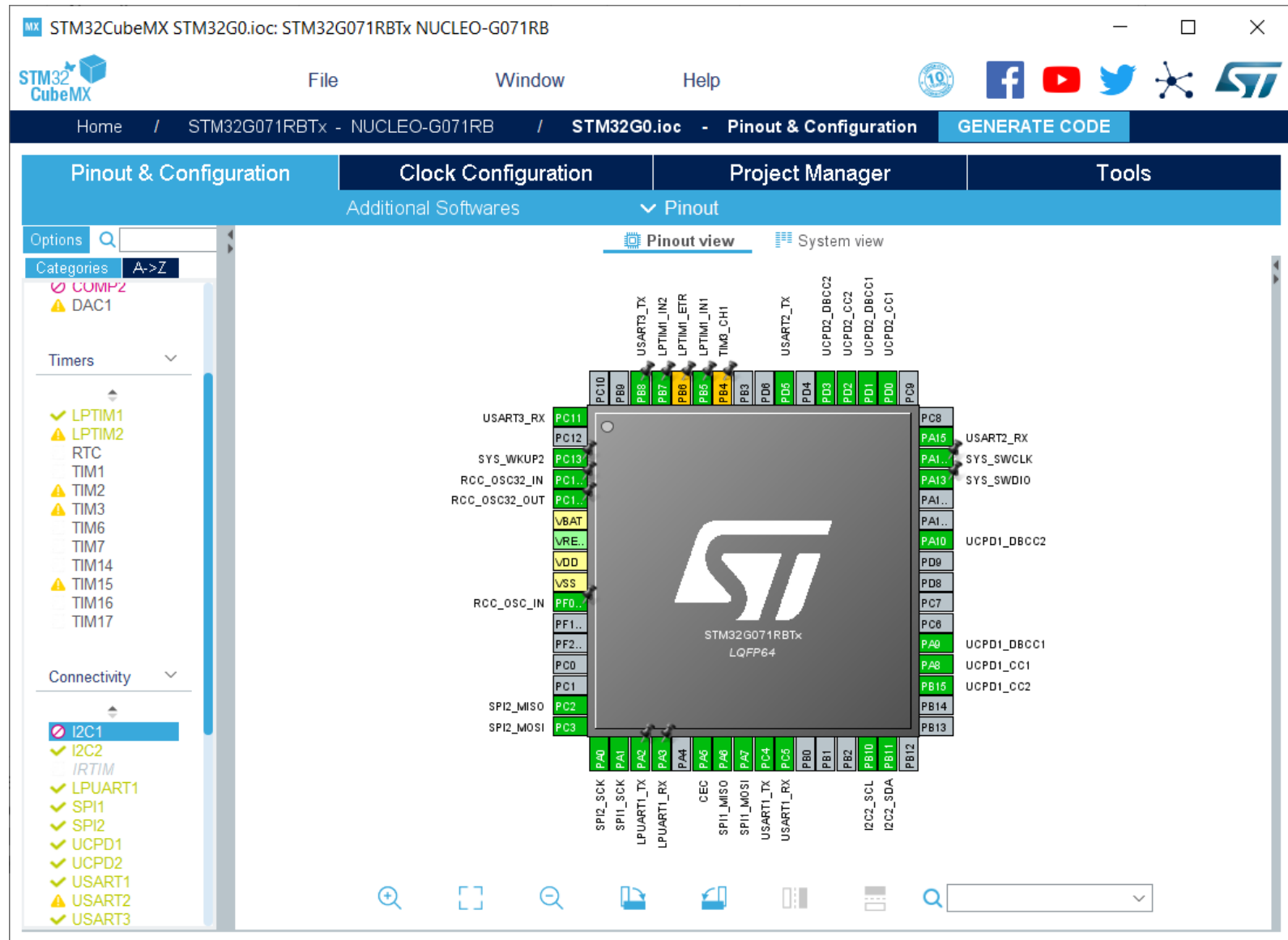
STM32 G0 LQFP64

STM32G070RBTx

MCUs List: 40 items Display similar items

*	Part No	Market	Board	Package	Flash	RAM	IO	Freq	GFX Score
☆	STM32G070KB	...	...	LQFP32	128 kBytes	36 kBytes	30	64 MHz	0.0
☆	STM32G070RB	...	NUCLEO-G070RB	LQFP64	128 kBytes	36 kBytes	60	64 MHz	0.0
☆	STM32G071C6	...	...	LQFP48	32 kBytes	36 kBytes	44	64 MHz	0.0
☆	STM32G071C6	...	...	UFQFPN48	32 kBytes	36 kBytes	44	64 MHz	0.0
☆	STM32G071C8	...	...	LQFP48	64 kBytes	36 kBytes	44	64 MHz	0.0
☆	STM32G071C8	...	...	UFQFPN48	64 kBytes	36 kBytes	44	64 MHz	0.0
☆	STM32G071CB	...	...	LQFP48	128 kBytes	36 kBytes	44	64 MHz	0.0
☆	STM32G071CB	...	...	UFQFPN48	128 kBytes	36 kBytes	44	64 MHz	0.0
☆	STM32G071EB	...	...	WLCSP25	128 kBytes	36 kBytes	23	64 MHz	0.0
☆	STM32G071G6	...	...	UFQFPN28	32 kBytes	36 kBytes	26	64 MHz	0.0
☆	STM32G071G8	...	...	UFQFPN28	64 kBytes	36 kBytes	26	64 MHz	0.0
☆	STM32G071G8	...	...	UFQFPN28	64 kBytes	36 kBytes	26	64 MHz	0.0
☆	STM32G071GB	...	...	UFQFPN28	128 kBytes	36 kBytes	26	64 MHz	0.0
☆	STM32G071GB	...	...	UFQFPN28	128 kBytes	36 kBytes	26	64 MHz	0.0
☆	STM32G071K6	...	...	LQFP32	32 kBytes	36 kBytes	30	64 MHz	0.0
☆	STM32G071K6	...	...	UFQFPN32	32 kBytes	36 kBytes	30	64 MHz	0.0
☆	STM32G071K8	...	...	LQFP32	64 kBytes	36 kBytes	30	64 MHz	0.0
☆	STM32G071K8	...	...	LQFP32	64 kBytes	36 kBytes	30	64 MHz	0.0
☆	STM32G071K8	...	...	UFQFPN32	64 kBytes	36 kBytes	30	64 MHz	0.0
☆	STM32G071K8	...	...	UFQFPN32	64 kBytes	36 kBytes	30	64 MHz	0.0

- Pinout from:
 - Peripheral tree
 - Manually
- Automatic signal remapping
- Management of dependencies between peripherals and/or middleware (FatFS, USB ...)



Pin assignment continued

65

The screenshot shows the STM32CubeMX Pinout & Configuration window for an STM32G071RBTx NUCLEO-G071RB. The interface includes a sidebar with categories like Timers and Connectivity, a central pinout diagram, and a top navigation bar. Callouts provide the following information:

- Peripheral is not available, all its alternate pins are assigned elsewhere:** Points to the I2C1 entry in the Connectivity list, which is marked with a red 'X'.
- Orange means the peripheral is not enabled, only the pin is assigned:** Points to pin PB6 in the pinout diagram, which is highlighted in orange.
- Click on the pin to view alternate functions:** Points to pin PB3 in the pinout diagram, which has a tooltip showing various alternate functions like Reset_State, COMP2_INM, I2S1_CK, etc.
- Freeze the signal placement using the pin icon:** Points to pin PA15 in the pinout diagram, which has a small icon next to it.

Peripheral and Middleware configuration

66

- Global view of used peripherals and middleware
- Highlight of configuration errors
 - + Not configured
 - ✓ OK
 - ⚠ Non-blocking problem
 - ✗ Error

The screenshot shows the STM32CubeMX Pinout & Configuration window for the STM32G071RBx NUCLEO-G071RB. The window is divided into several sections: Pinout & Configuration, Clock Configuration, Project Manager, and Tools. The Pinout & Configuration section is active, showing a System view of the configuration. The configuration is organized into categories: System Core, Analog, Timers, Connectivity, and Multimedia. Each category contains a list of peripherals with their configuration status indicated by icons: a green checkmark for OK, a yellow warning triangle for non-blocking problems, and a red X for errors.

Callouts highlight the following features and errors:

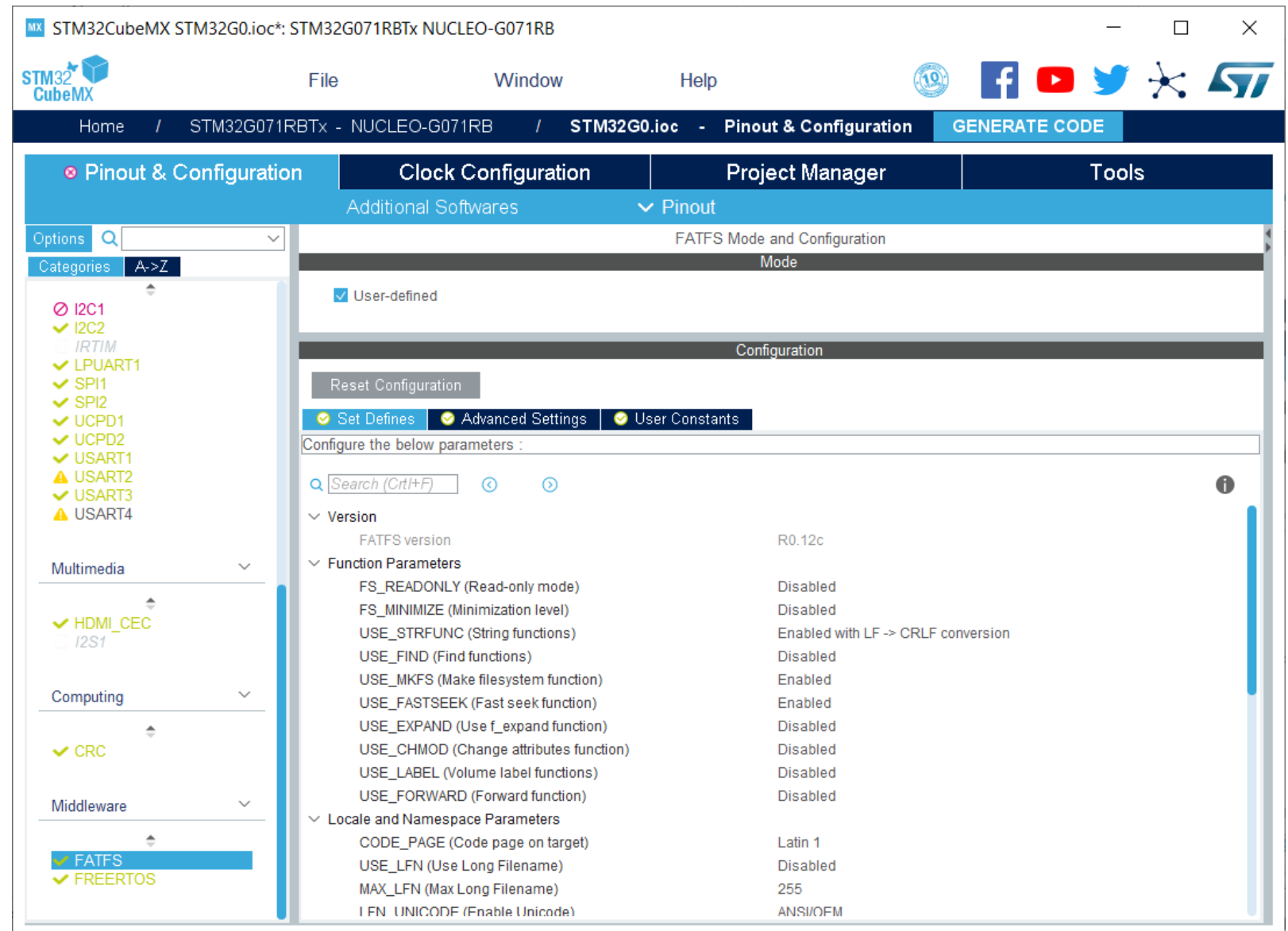
- Click to configure DMA**: Points to the DMA button in the System Core category.
- Quickly switch pinout and system view**: Points to the Pinout view and System view tabs.
- GPIO configuration is considered incorrect, but code may be generated**: Points to the GPIO button in the System Core category.
- Error in configuration, code generator will display a warning message**: Points to the TIM1 button in the Timers category, which has a red X icon.
- Configuration is valid here**: Points to the LPUART1 button in the Connectivity category, which has a green checkmark icon.

Category	Peripheral	Status
System Core	DMA	⚠
	GPIO	⚠
	NVIC	✓
	RCC	✓
Timers	LPTIM1	✓
	LPTIM2	✓
	TIM1	✗
Connectivity	I2C2	✓
	LPUART1	✓
	SPI1	✓
	SPI2	✓
	UCPD1	✓
	UCPD2	✓
Multimedia	HDMI_CEC	✓
	CRC	✓

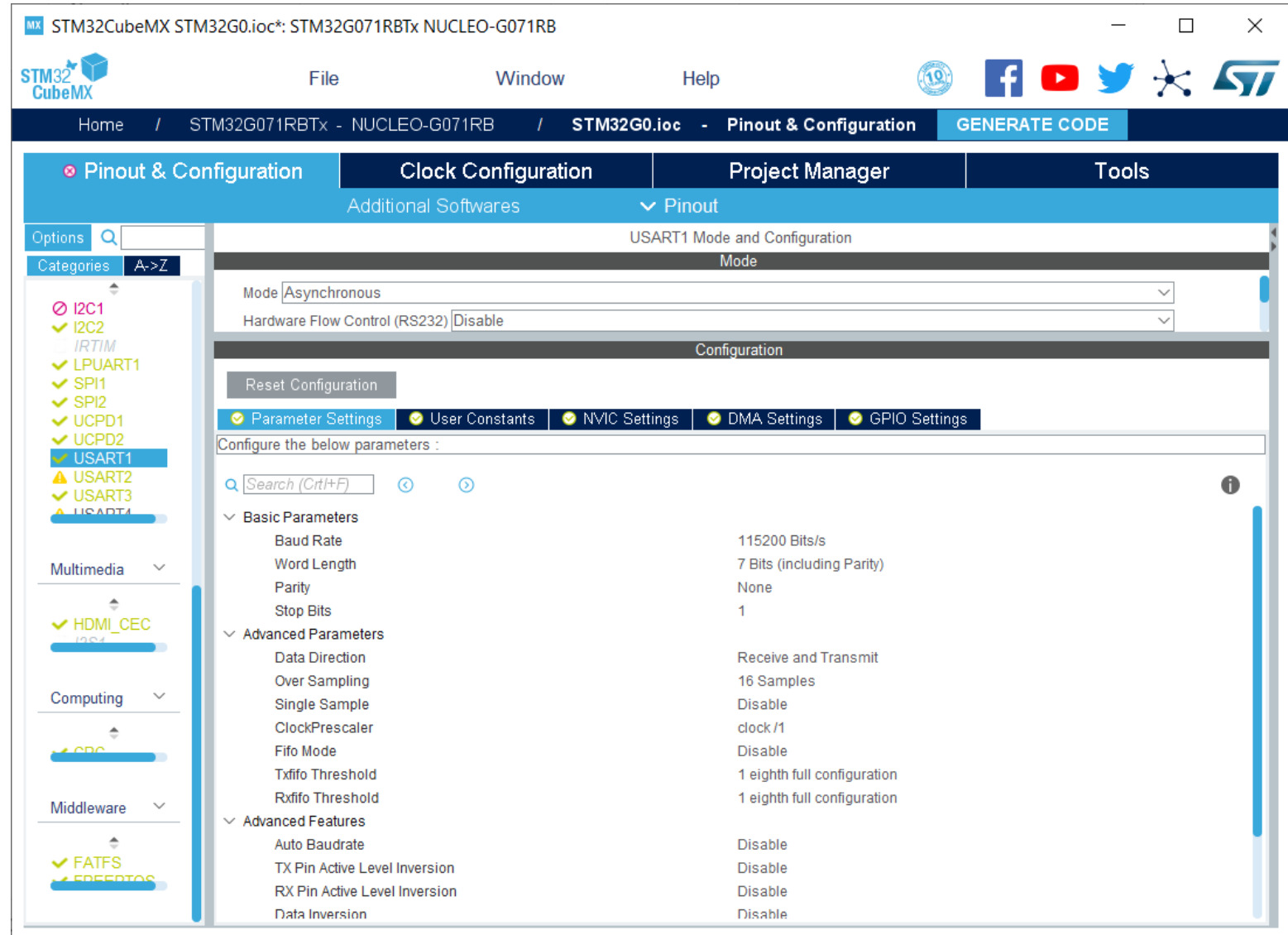
Middleware configuration

67

- Presents options specific to each supported software component
- All settings are organized in logical groups
- Description and constraints are available for quick reference



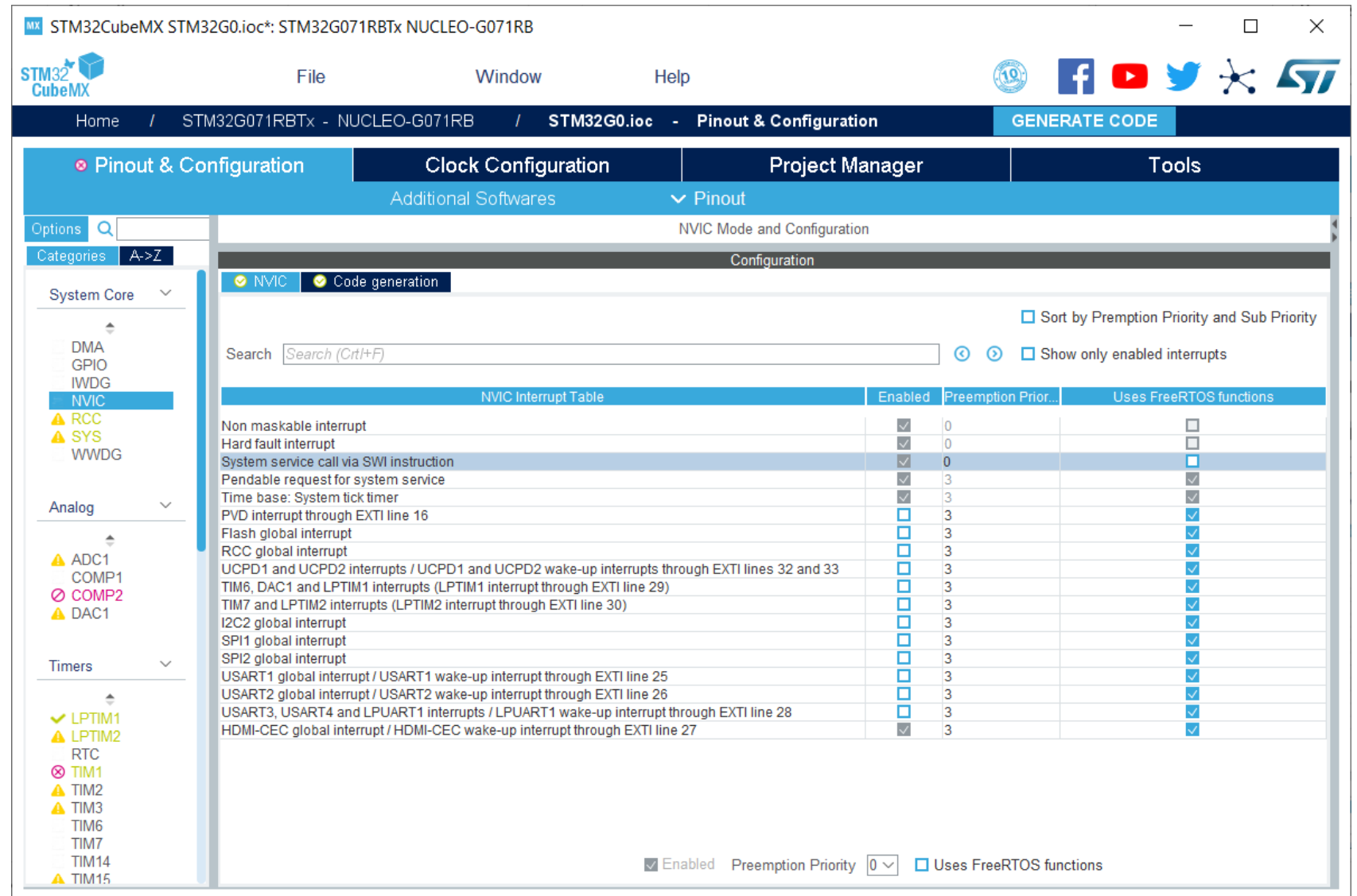
- All available initialization parameters are presented with short description and options
- Interrupt may be assigned to peripherals
- DMA may be associated, where applicable
- GPIO settings for peripherals with input and/or output



NVIC configuration panel

69

- Single control panel for all interrupts
- Manage priorities and sub-priorities
- Searching, filtering and sorting interrupts in the list
- Code generation tab allows to customize interrupt initialization



The screenshot displays the STM32CubeMX software interface for configuring the NVIC (Nested Vectored Interrupt Controller) for the STM32G071RBx NUCLEO-G071RB. The 'Pinout & Configuration' tab is active, and the 'NVIC Mode and Configuration' section is expanded. The 'Configuration' sub-tab is selected, showing the 'NVIC Interrupt Table'.

The table lists various interrupts with their status (Enabled/Disabled), Preemption Priority, and whether they use FreeRTOS functions. The 'System service call via SWI instruction' is highlighted.

NVIC Interrupt Table	Enabled	Preemption Prior...	Uses FreeRTOS functions
Non maskable interrupt	☒	0	☐
Hard fault interrupt	☒	0	☐
System service call via SWI instruction	☒	0	☐
Pendable request for system service	☒	3	☒
Time base: System tick timer	☒	3	☒
PVD interrupt through EXTI line 16	☐	3	☒
Flash global interrupt	☐	3	☒
RCC global interrupt	☐	3	☒
UCPD1 and UCPD2 interrupts / UCPD1 and UCPD2 wake-up interrupts through EXTI lines 32 and 33	☐	3	☒
TIM6, DAC1 and LPTIM1 interrupts (LPTIM1 interrupt through EXTI line 29)	☐	3	☒
TIM7 and LPTIM2 interrupts (LPTIM2 interrupt through EXTI line 30)	☐	3	☒
I2C2 global interrupt	☐	3	☒
SPI1 global interrupt	☐	3	☒
SPI2 global interrupt	☐	3	☒
USART1 global interrupt / USART1 wake-up interrupt through EXTI line 25	☐	3	☒
USART2 global interrupt / USART2 wake-up interrupt through EXTI line 26	☐	3	☒
USART3, USART4 and LPUART1 interrupts / LPUART1 wake-up interrupt through EXTI line 28	☐	3	☒
HDMI-CEC global interrupt / HDMI-CEC wake-up interrupt through EXTI line 27	☒	3	☒

At the bottom, there are checkboxes for 'Enabled', 'Preemption Priority' (set to 0), and 'Uses FreeRTOS functions'.

DMA configuration panel

70

- Manages all DMA requests including memory to memory
- Configure direction, priority and other settings

The screenshot shows the STM32CubeMX software interface for configuring the DMA controller on an STM32G071RB NUCLEO-G071RB board. The 'Pinout & Configuration' tab is active, and the 'DMA Mode and Configuration' section is expanded.

Configuration Table:

DMA Request	Channel	Direction	Priority
LPUART1_RX	DMA1 Channel 1	Peripheral To Memory	Low

DMA Request Settings:

- Mode: Normal
- Increment Address: ☐ (Peripheral), ☒ (Memory)
- Data Width: Byte (Peripheral), Word (Memory)

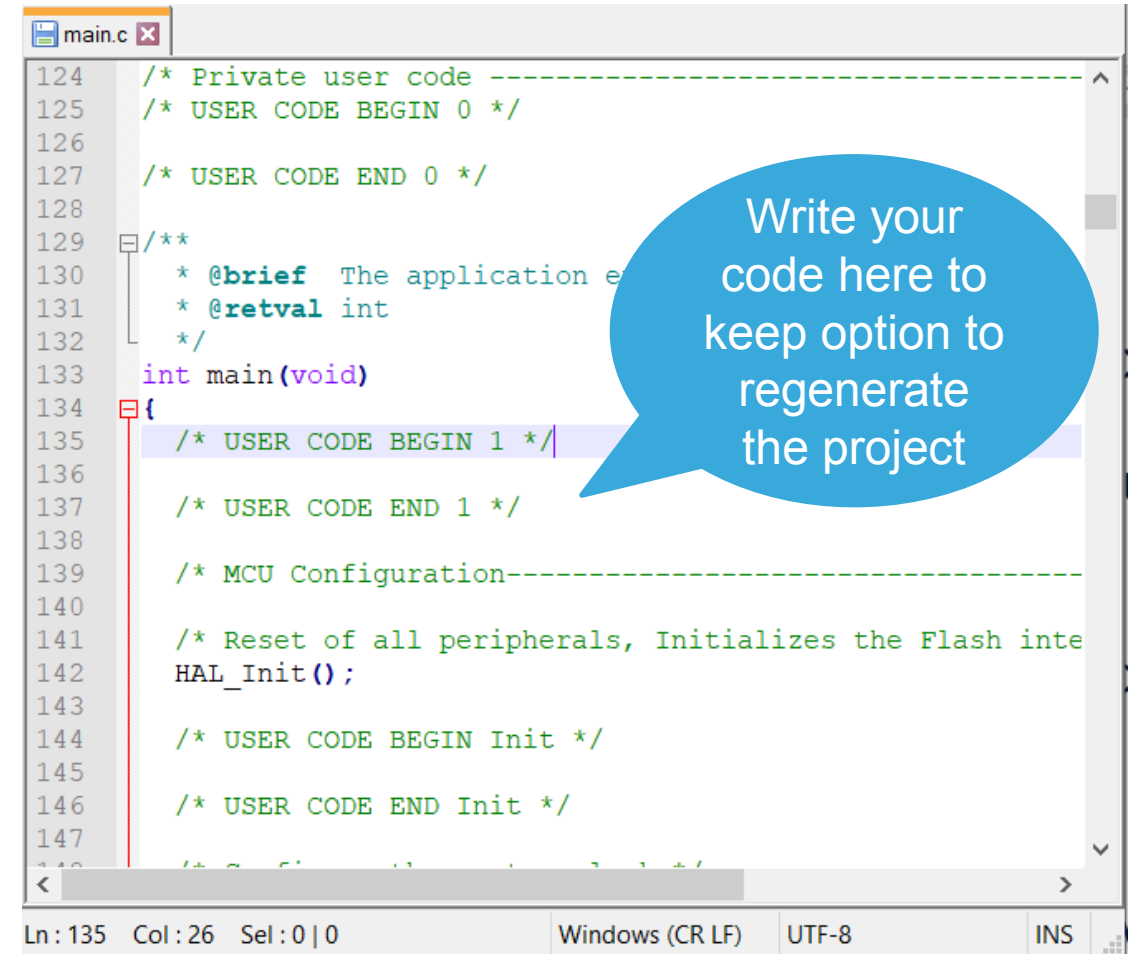
DMA Request Generator Settings:

- Request generation Signal: [Dropdown]
- Signal polarity: [Dropdown]
- Request number: [Input]

DMA Request Synchronization Settings:

- Enable synchronization: ☐
- Synchronization signal: [Dropdown]
- Signal polarity: [Dropdown]
- Enable event: ☐
- Request number: [Input]

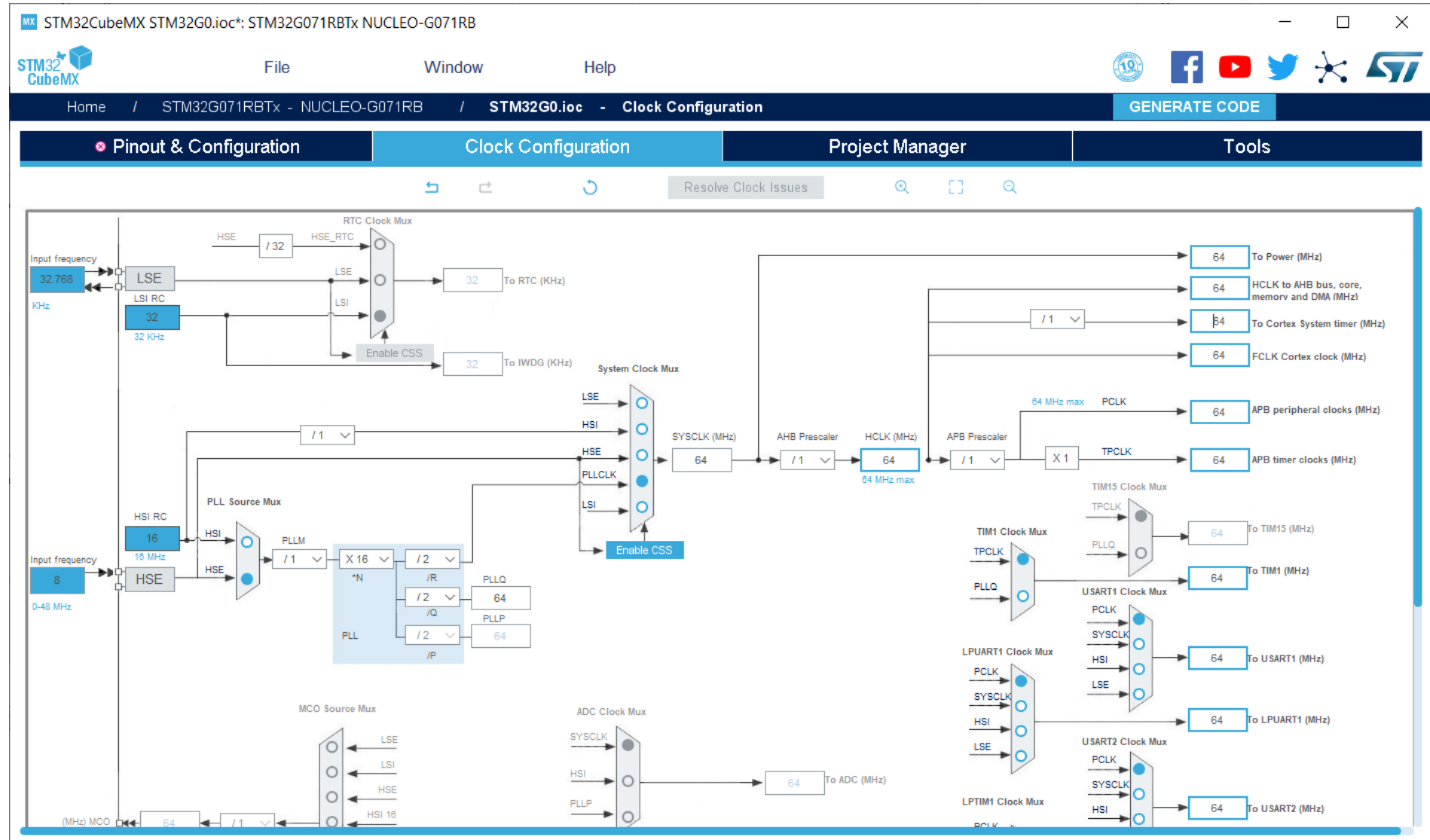
- Generate all the initialization code in C
- Generates project file for any supported development toolchain
- User code can be added in dedicated sections and will be kept upon regeneration
- Option to use the latest library version or keep the same even if regenerating



```
124  /* Private user code -----
125  /* USER CODE BEGIN 0 */
126
127  /* USER CODE END 0 */
128
129  /**
130   * @brief The application e
131   * @retval int
132   */
133  int main(void)
134  {
135    /* USER CODE BEGIN 1 */
136
137    /* USER CODE END 1 */
138
139    /* MCU Configuration-----
140
141    /* Reset of all peripherals, Initializes the Flash inte
142    HAL_Init();
143
144    /* USER CODE BEGIN Init */
145
146    /* USER CODE END Init */
147
148    /* ...
149    ...
150    ...
151    ...
152    ...
153    ...
154    ...
155    ...
156    ...
157    ...
158    ...
159    ...
160    ...
161    ...
162    ...
163    ...
164    ...
165    ...
166    ...
167    ...
168    ...
169    ...
170    ...
171    ...
172    ...
173    ...
174    ...
175    ...
176    ...
177    ...
178    ...
179    ...
180    ...
181    ...
182    ...
183    ...
184    ...
185    ...
186    ...
187    ...
188    ...
189    ...
190    ...
191    ...
192    ...
193    ...
194    ...
195    ...
196    ...
197    ...
198    ...
199    ...
200    ...
201    ...
202    ...
203    ...
204    ...
205    ...
206    ...
207    ...
208    ...
209    ...
210    ...
211    ...
212    ...
213    ...
214    ...
215    ...
216    ...
217    ...
218    ...
219    ...
220    ...
221    ...
222    ...
223    ...
224    ...
225    ...
226    ...
227    ...
228    ...
229    ...
230    ...
231    ...
232    ...
233    ...
234    ...
235    ...
236    ...
237    ...
238    ...
239    ...
240    ...
241    ...
242    ...
243    ...
244    ...
245    ...
246    ...
247    ...
248    ...
249    ...
250    ...
251    ...
252    ...
253    ...
254    ...
255    ...
256    ...
257    ...
258    ...
259    ...
260    ...
261    ...
262    ...
263    ...
264    ...
265    ...
266    ...
267    ...
268    ...
269    ...
270    ...
271    ...
272    ...
273    ...
274    ...
275    ...
276    ...
277    ...
278    ...
279    ...
280    ...
281    ...
282    ...
283    ...
284    ...
285    ...
286    ...
287    ...
288    ...
289    ...
290    ...
291    ...
292    ...
293    ...
294    ...
295    ...
296    ...
297    ...
298    ...
299    ...
300    ...
301    ...
302    ...
303    ...
304    ...
305    ...
306    ...
307    ...
308    ...
309    ...
310    ...
311    ...
312    ...
313    ...
314    ...
315    ...
316    ...
317    ...
318    ...
319    ...
320    ...
321    ...
322    ...
323    ...
324    ...
325    ...
326    ...
327    ...
328    ...
329    ...
330    ...
331    ...
332    ...
333    ...
334    ...
335    ...
336    ...
337    ...
338    ...
339    ...
340    ...
341    ...
342    ...
343    ...
344    ...
345    ...
346    ...
347    ...
348    ...
349    ...
350    ...
351    ...
352    ...
353    ...
354    ...
355    ...
356    ...
357    ...
358    ...
359    ...
360    ...
361    ...
362    ...
363    ...
364    ...
365    ...
366    ...
367    ...
368    ...
369    ...
370    ...
371    ...
372    ...
373    ...
374    ...
375    ...
376    ...
377    ...
378    ...
379    ...
380    ...
381    ...
382    ...
383    ...
384    ...
385    ...
386    ...
387    ...
388    ...
389    ...
390    ...
391    ...
392    ...
393    ...
394    ...
395    ...
396    ...
397    ...
398    ...
399    ...
400    ...
401    ...
402    ...
403    ...
404    ...
405    ...
406    ...
407    ...
408    ...
409    ...
410    ...
411    ...
412    ...
413    ...
414    ...
415    ...
416    ...
417    ...
418    ...
419    ...
420    ...
421    ...
422    ...
423    ...
424    ...
425    ...
426    ...
427    ...
428    ...
429    ...
430    ...
431    ...
432    ...
433    ...
434    ...
435    ...
436    ...
437    ...
438    ...
439    ...
440    ...
441    ...
442    ...
443    ...
444    ...
445    ...
446    ...
447    ...
448    ...
449    ...
450    ...
451    ...
452    ...
453    ...
454    ...
455    ...
456    ...
457    ...
458    ...
459    ...
460    ...
461    ...
462    ...
463    ...
464    ...
465    ...
466    ...
467    ...
468    ...
469    ...
470    ...
471    ...
472    ...
473    ...
474    ...
475    ...
476    ...
477    ...
478    ...
479    ...
480    ...
481    ...
482    ...
483    ...
484    ...
485    ...
486    ...
487    ...
488    ...
489    ...
490    ...
491    ...
492    ...
493    ...
494    ...
495    ...
496    ...
497    ...
498    ...
499    ...
500    ...
501    ...
502    ...
503    ...
504    ...
505    ...
506    ...
507    ...
508    ...
509    ...
510    ...
511    ...
512    ...
513    ...
514    ...
515    ...
516    ...
517    ...
518    ...
519    ...
520    ...
521    ...
522    ...
523    ...
524    ...
525    ...
526    ...
527    ...
528    ...
529    ...
530    ...
531    ...
532    ...
533    ...
534    ...
535    ...
536    ...
537    ...
538    ...
539    ...
540    ...
541    ...
542    ...
543    ...
544    ...
545    ...
546    ...
547    ...
548    ...
549    ...
550    ...
551    ...
552    ...
553    ...
554    ...
555    ...
556    ...
557    ...
558    ...
559    ...
560    ...
561    ...
562    ...
563    ...
564    ...
565    ...
566    ...
567    ...
568    ...
569    ...
570    ...
571    ...
572    ...
573    ...
574    ...
575    ...
576    ...
577    ...
578    ...
579    ...
580    ...
581    ...
582    ...
583    ...
584    ...
585    ...
586    ...
587    ...
588    ...
589    ...
590    ...
591    ...
592    ...
593    ...
594    ...
595    ...
596    ...
597    ...
598    ...
599    ...
600    ...
601    ...
602    ...
603    ...
604    ...
605    ...
606    ...
607    ...
608    ...
609    ...
610    ...
611    ...
612    ...
613    ...
614    ...
615    ...
616    ...
617    ...
618    ...
619    ...
620    ...
621    ...
622    ...
623    ...
624    ...
625    ...
626    ...
627    ...
628    ...
629    ...
630    ...
631    ...
632    ...
633    ...
634    ...
635    ...
636    ...
637    ...
638    ...
639    ...
640    ...
641    ...
642    ...
643    ...
644    ...
645    ...
646    ...
647    ...
648    ...
649    ...
650    ...
651    ...
652    ...
653    ...
654    ...
655    ...
656    ...
657    ...
658    ...
659    ...
660    ...
661    ...
662    ...
663    ...
664    ...
665    ...
666    ...
667    ...
668    ...
669    ...
670    ...
671    ...
672    ...
673    ...
674    ...
675    ...
676    ...
677    ...
678    ...
679    ...
680    ...
681    ...
682    ...
683    ...
684    ...
685    ...
686    ...
687    ...
688    ...
689    ...
690    ...
691    ...
692    ...
693    ...
694    ...
695    ...
696    ...
697    ...
698    ...
699    ...
700    ...
701    ...
702    ...
703    ...
704    ...
705    ...
706    ...
707    ...
708    ...
709    ...
710    ...
711    ...
712    ...
713    ...
714    ...
715    ...
716    ...
717    ...
718    ...
719    ...
720    ...
721    ...
722    ...
723    ...
724    ...
725    ...
726    ...
727    ...
728    ...
729    ...
730    ...
731    ...
732    ...
733    ...
734    ...
735    ...
736    ...
737    ...
738    ...
739    ...
740    ...
741    ...
742    ...
743    ...
744    ...
745    ...
746    ...
747    ...
748    ...
749    ...
750    ...
751    ...
752    ...
753    ...
754    ...
755    ...
756    ...
757    ...
758    ...
759    ...
760    ...
761    ...
762    ...
763    ...
764    ...
765    ...
766    ...
767    ...
768    ...
769    ...
770    ...
771    ...
772    ...
773    ...
774    ...
775    ...
776    ...
777    ...
778    ...
779    ...
780    ...
781    ...
782    ...
783    ...
784    ...
785    ...
786    ...
787    ...
788    ...
789    ...
790    ...
791    ...
792    ...
793    ...
794    ...
795    ...
796    ...
797    ...
798    ...
799    ...
800    ...
801    ...
802    ...
803    ...
804    ...
805    ...
806    ...
807    ...
808    ...
809    ...
810    ...
811    ...
812    ...
813    ...
814    ...
815    ...
816    ...
817    ...
818    ...
819    ...
820    ...
821    ...
822    ...
823    ...
824    ...
825    ...
826    ...
827    ...
828    ...
829    ...
830    ...
831    ...
832    ...
833    ...
834    ...
835    ...
836    ...
837    ...
838    ...
839    ...
840    ...
841    ...
842    ...
843    ...
844    ...
845    ...
846    ...
847    ...
848    ...
849    ...
850    ...
851    ...
852    ...
853    ...
854    ...
855    ...
856    ...
857    ...
858    ...
859    ...
860    ...
861    ...
862    ...
863    ...
864    ...
865    ...
866    ...
867    ...
868    ...
869    ...
870    ...
871    ...
872    ...
873    ...
874    ...
875    ...
876    ...
877    ...
878    ...
879    ...
880    ...
881    ...
882    ...
883    ...
884    ...
885    ...
886    ...
887    ...
888    ...
889    ...
890    ...
891    ...
892    ...
893    ...
894    ...
895    ...
896    ...
897    ...
898    ...
899    ...
900    ...
901    ...
902    ...
903    ...
904    ...
905    ...
906    ...
907    ...
908    ...
909    ...
910    ...
911    ...
912    ...
913    ...
914    ...
915    ...
916    ...
917    ...
918    ...
919    ...
920    ...
921    ...
922    ...
923    ...
924    ...
925    ...
926    ...
927    ...
928    ...
929    ...
930    ...
931    ...
932    ...
933    ...
934    ...
935    ...
936    ...
937    ...
938    ...
939    ...
940    ...
941    ...
942    ...
943    ...
944    ...
945    ...
946    ...
947    ...
948    ...
949    ...
950    ...
951    ...
952    ...
953    ...
954    ...
955    ...
956    ...
957    ...
958    ...
959    ...
960    ...
961    ...
962    ...
963    ...
964    ...
965    ...
966    ...
967    ...
968    ...
969    ...
970    ...
971    ...
972    ...
973    ...
974    ...
975    ...
976    ...
977    ...
978    ...
979    ...
980    ...
981    ...
982    ...
983    ...
984    ...
985    ...
986    ...
987    ...
988    ...
989    ...
990    ...
991    ...
992    ...
993    ...
994    ...
995    ...
996    ...
997    ...
998    ...
999    ...
1000   ...
```

Ln: 135 Col: 26 Sel: 0 | 0 Windows (CR LF) UTF-8 INS

- Immediate display of all clock values
- Active and inactive clock paths are differentiated
- Management of clock constraints and features



Code generation project settings

73

- Name your project when saving
- Browse for project location
- Pick the preferred toolchain
- Review the exact MCU type and library version

The screenshot shows the STM32CubeMX Project Manager window. The title bar reads "STM32CubeMX STM32G0.ioc*: STM32G071RBTx NUCLEO-G071RB". The menu bar includes "File", "Window", and "Help". The breadcrumb navigation shows "Home / STM32G071RBTx - NUCLEO-G071RB / STM32G0.ioc - Project Manager". A "GENERATE CODE" button is visible in the top right. The left sidebar has four tabs: "Pinout & Configuration", "Clock Configuration", "Project Manager" (selected), and "Tools". The "Project Manager" tab is active, displaying the "Project Settings" section. The settings are organized into four main categories on the left: "Project", "Code Generator", "Advanced Settings", and "Mcu and Firmware Package".

Project Settings

- Project Name:** STM32G0
- Project Location:** F:\Trainings\Labs
- Application Structure:** Basic (dropdown menu). ☐ Do not generate the main()
- Toolchain Folder Location:** F:\Trainings\Labs\STM32G0\
- Toolchain / IDE:** EWARM V8 (dropdown menu). ☐ Generate Under Root

Linker Settings

- Minimum Heap Size:** 0x200
- Minimum Stack Size:** 0x400

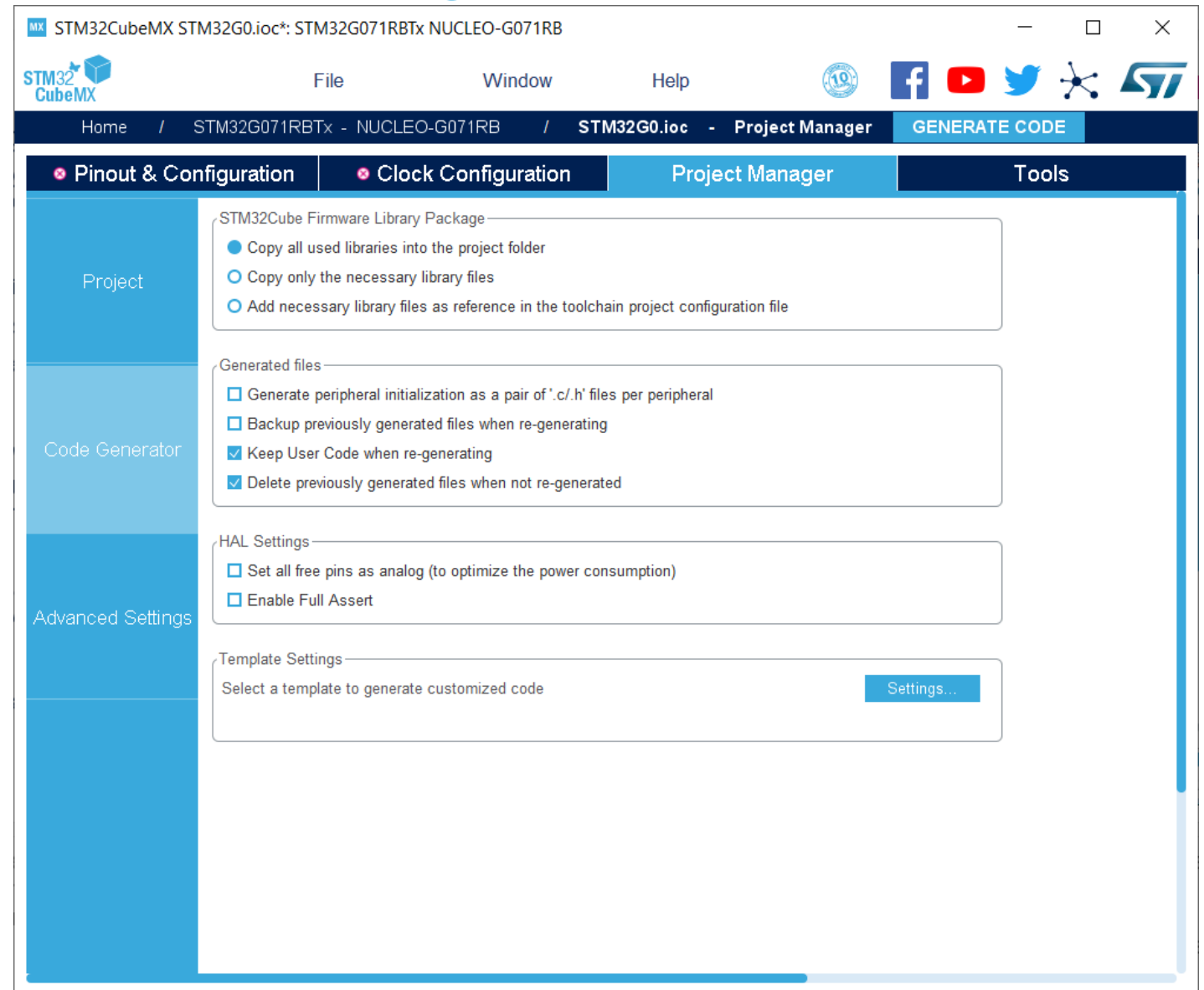
Mcu and Firmware Package

- Mcu Reference:** STM32G071RBTx
- Firmware Package Name and Version:** STM32Cube FW_G0 V1.0.0
- ☒ Use Default Firmware Location
- Firmware Path:** C:/Users/Trainer/STM32Cube/Repository/STM32Cube_FW_G0_V1.0.0 (with a "Browse" button)

Code generation options

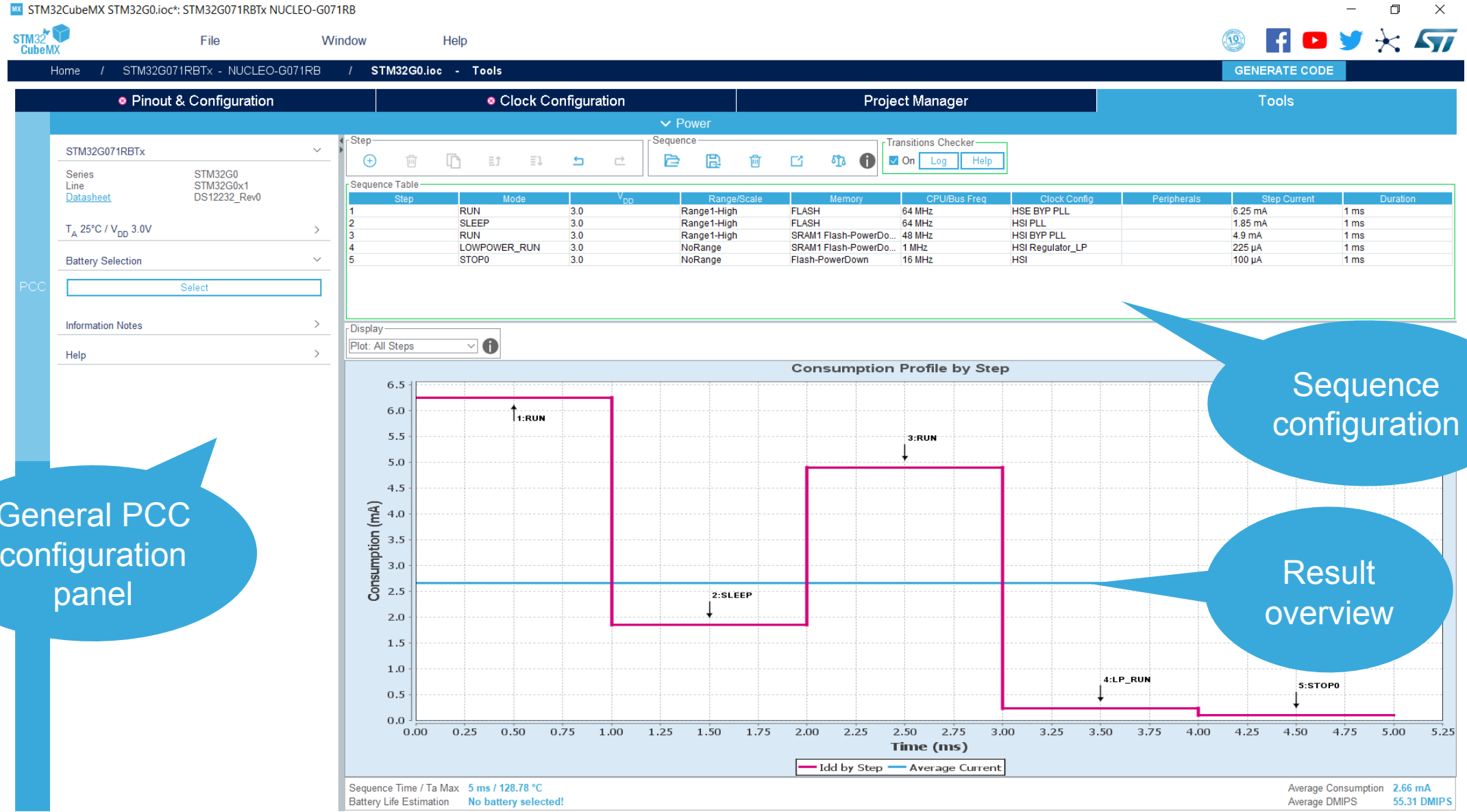
74

- Library package
 - Whole library or the necessary part may be copied to the generated project folder
 - Or keep the library in original place and refer to it from all projects
- Generated files
 - Each peripheral initialized in separate file or in common source file
 - Options for working with old files
 - **The option to keep user code intact is here**
- HAL settings
 - Setting available pins to analog reduces power consumption, but be careful to **explicitly select SWD/JTAG in pinout**
 - Full assert is useful for debugging



PCC (Power Consumption Calculator)

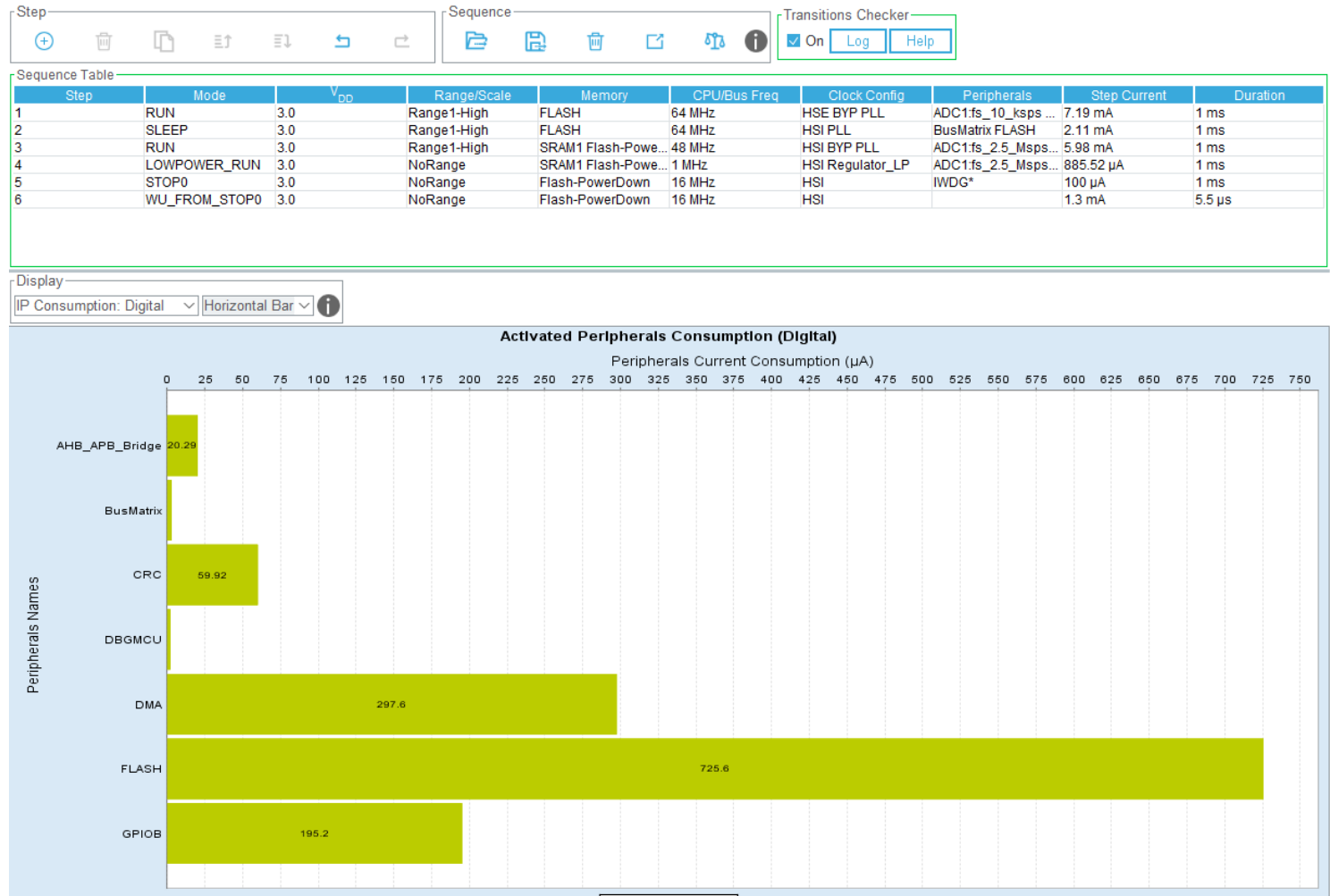
75



PCC - Sequence consumption profile display

76

- It's possible to detach the charts to external display for presentation purposes
- Several different views selectable
 - Plot current vs time
 - Pie chart
 - Consumption of peripherals



- An optional step is to generate a PDF report
- The PDF report is also available without PCC
- Complete saved project work includes:
 - Project.ioc
 - Project.pcs
 - Project.pdf
 - Project.txt
 - Project.jpg
 - ... and the generated project for a supported development environment

6. Power Consumption Calculator report

6.1. Microcontroller Selection

Series	STM32G0
Line	STM32G0x1
MCU	STM32G071RBTx
Datasheet	DS12232 Rev0

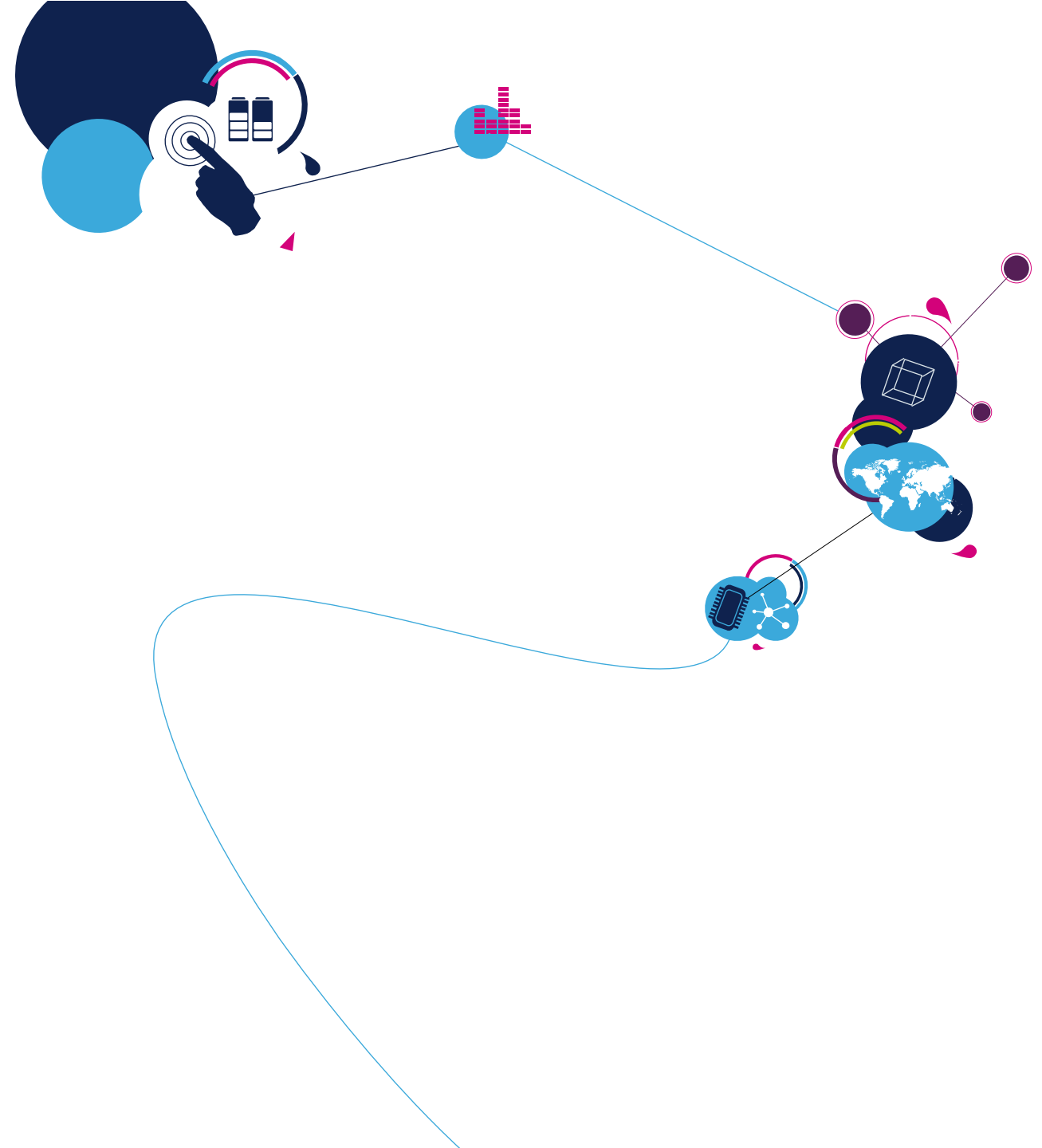
6.2. Parameter Selection

Temperature	25
Vdd	3.0

6.3. Battery Selection

Battery	Li-MnO2(CR1225)
Capacity	48.0 mAh
Self Discharge	0.12 %/month
Nominal Voltage	3.0 V
Max Cont Current	1.0 mA
Max Pulse Current	5.0 mA
Cells in series	1
Cells in parallel	1

Lab: Blinky



Objective:

- The objective of this lab is to generate a very simple project using STM32CubeMX Software.
- In this example we are going to blink one of the LEDs present on the STM32G0 Nucleo board, connected to PA5 of the STM32G0 MCU.

Run STM32CubeMX

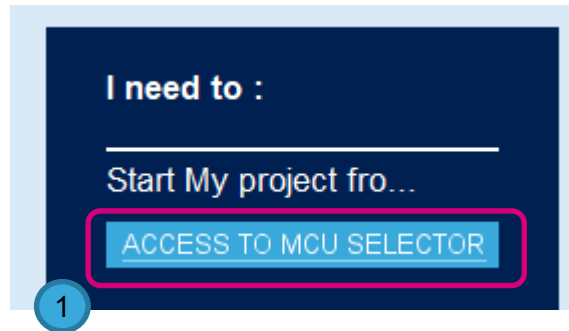
80



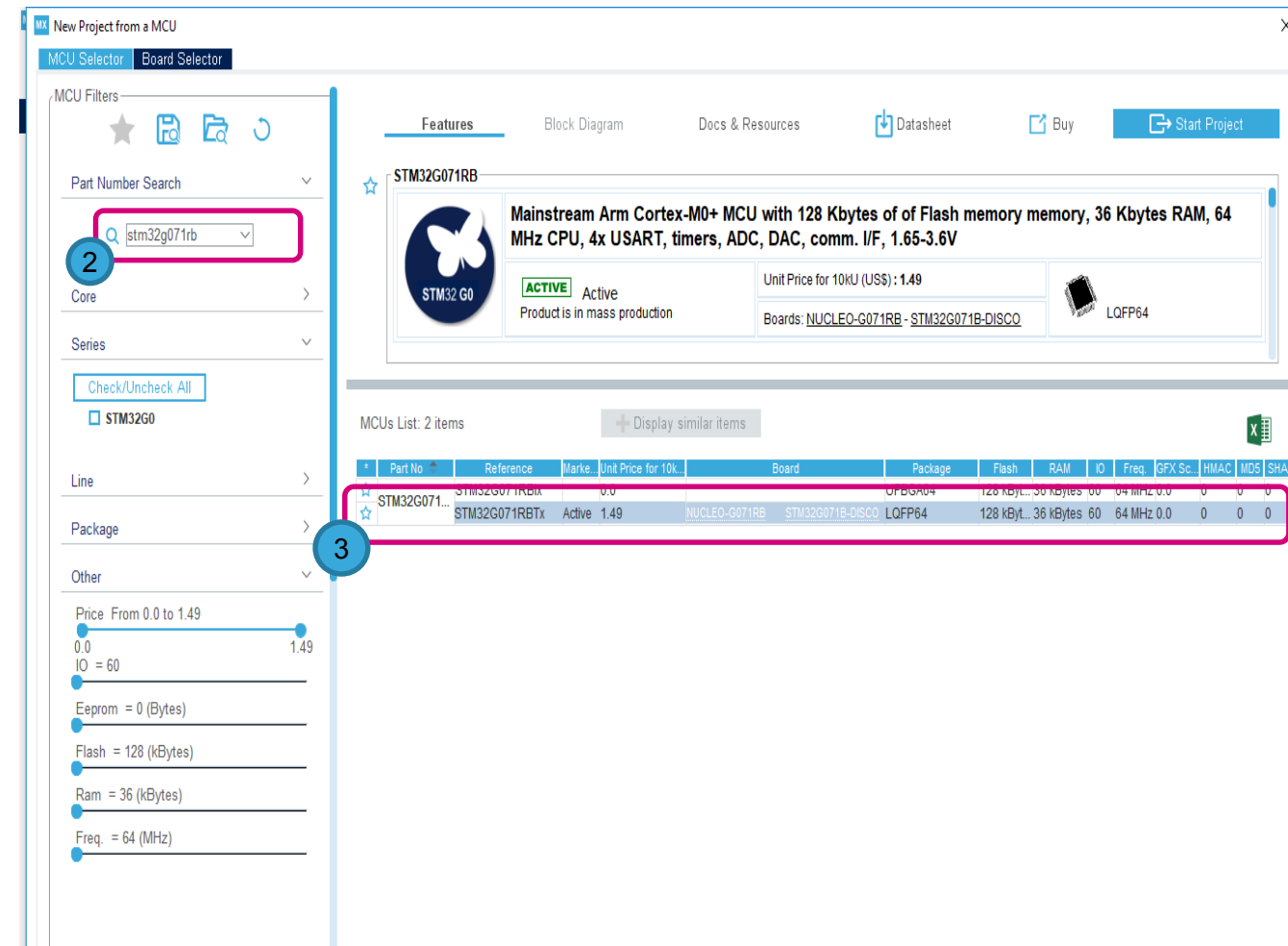
The screenshot shows the STM32CubeMX software interface. At the top, there is a title bar with 'MX STM32CubeMX Untitled' and a menu bar with 'File', 'Window', and 'Help'. Below the menu bar is a dark blue header with the 'Home' button. The main area is divided into three panels. The left panel, titled 'Existing Projects', contains a section 'Recent Opened Projects' with a list of three projects: 'stm32g0_nucleo.ioc', 'test1.ioc', and 'test_g0_atollc.ioc', each with a 'Last modified date' and a small 'MX' icon. Below this is a section 'Other Projects' with a folder icon. The middle panel, titled 'New Project', contains a section 'I need to :' with two options: 'Start My project from MCU ...' and 'Start My project from STBo...', each with a corresponding button ('ACCESS TO MCU SELECTOR' and 'ACCESS TO BOARD SELECTOR'). The right panel, titled 'Manage software installations', contains two sections: 'Check for STM32CubeMX and embedded software pac...' with a 'CHECK FOR UPDATES' button, and 'Install or remove embedded software packages' with an 'INSTALL / REMOVE' button. At the bottom right, there is a promotional banner for 'Wireless and ultra-low-power MCU' with the text 'State-of-the-art RF performance' and a list of features: 'Dual-core', 'Dual-protocol', 'Key storage', and 'BOM saving'. The banner also features the STM32WB logo and the ST logo.

Step 1: Create New Project

81



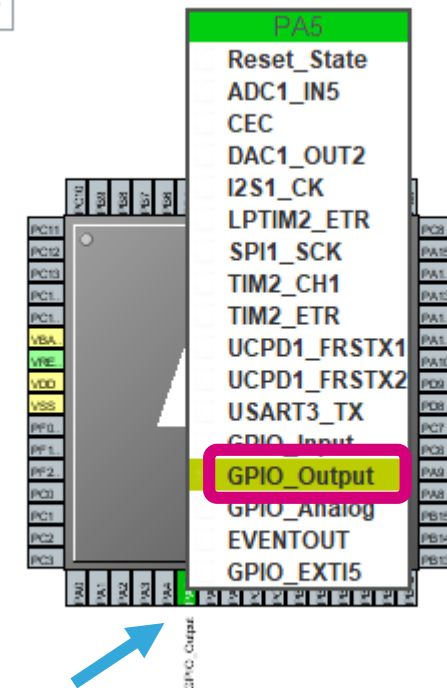
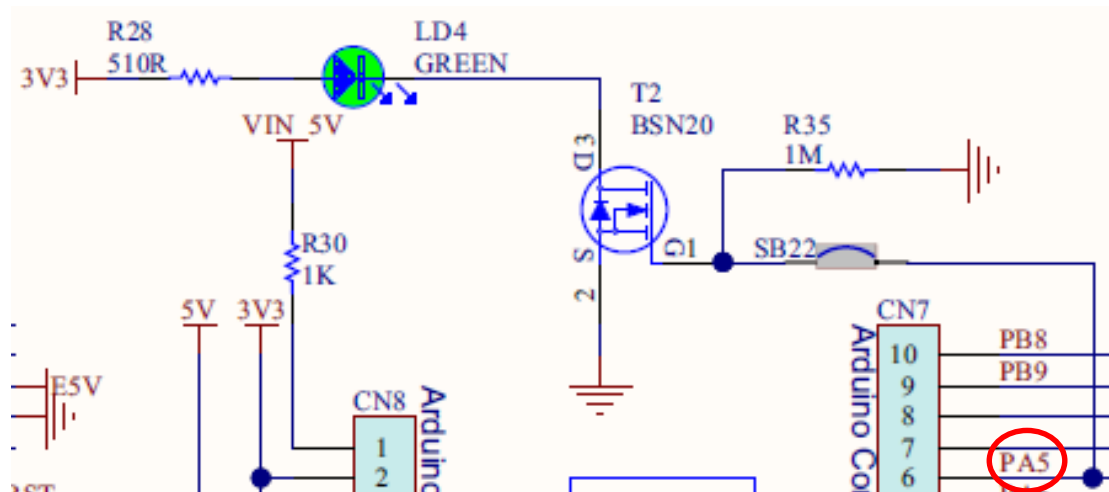
- Click **Access To MCU Selector** 1
- Type: “stm32g071rb” in the Part Number Search 2
- Select **STM32G071RBTx** 3
 - LQFP64, 128kB Flash
- Double Click



Step 2: Pin Configuration

82

- In this example we are going to use one of the LEDs present on the STM32G0 Nucleo board (connected to PA5 as seen in the schematic below)
- Search for **PA5** in the search window at the bottom
- Left-click **PA5** and set it to **GPIO_Output** mode



Step 3: Generate Source Code

83

- Open **Project Manager**



- Set the project name (**blinky**) and the project location (**C:\STM32G0Workshop\HandsOn**)

1
2

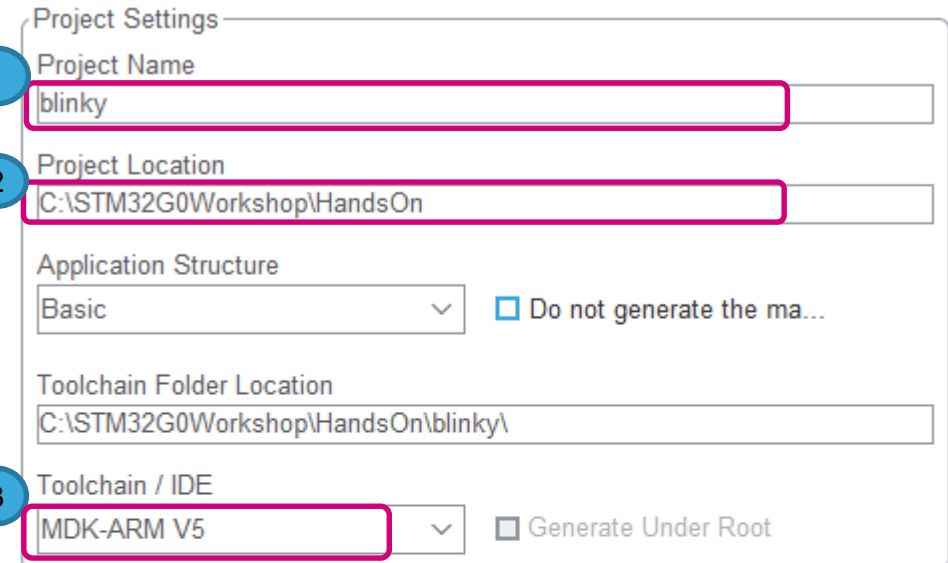
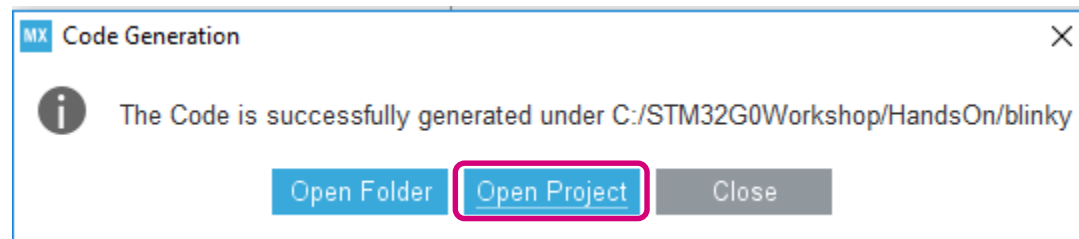
- Set the IDE Toolchain to **MDK-ARM V5**

3

- **Generate Code**



- Click **Open Project**

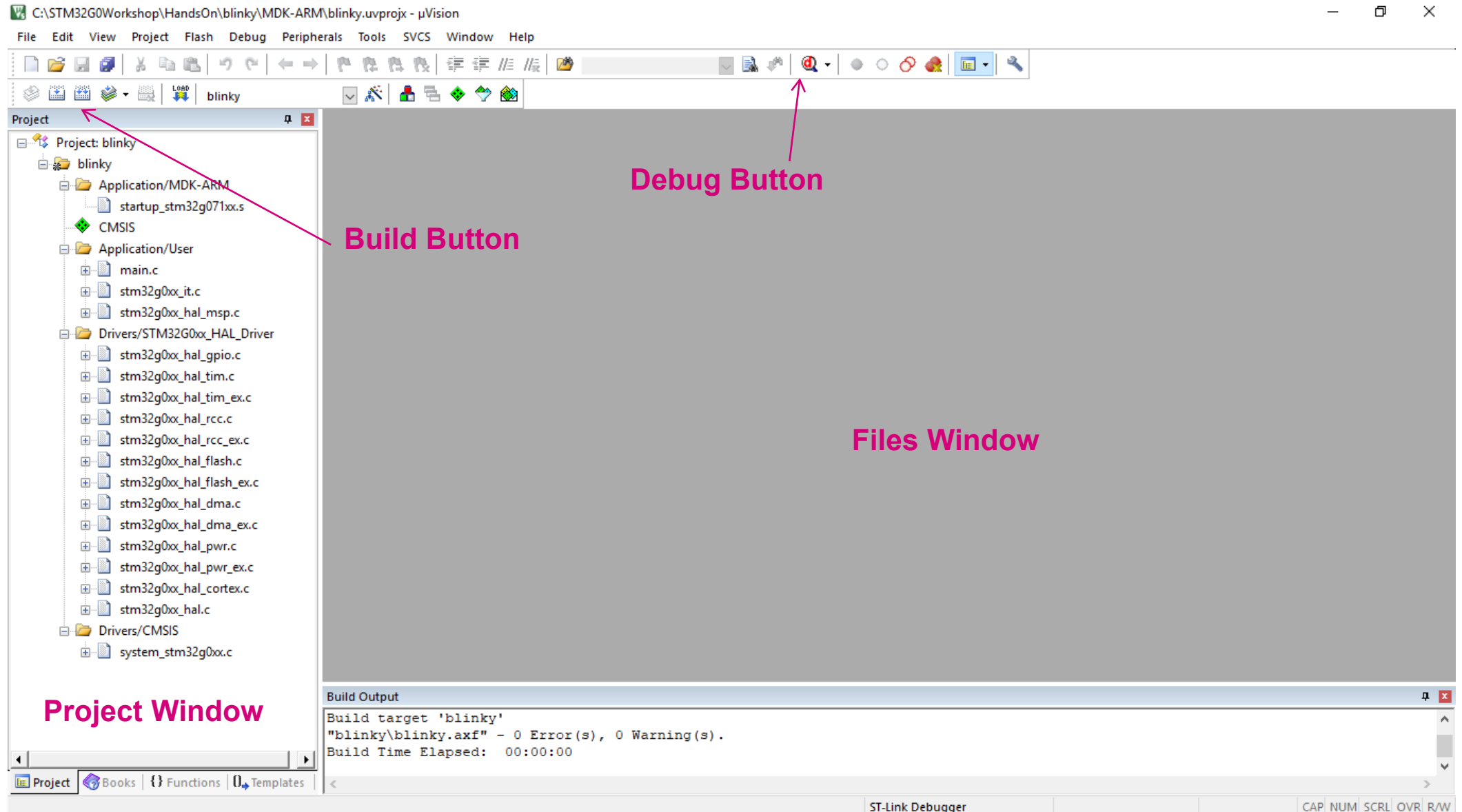
A screenshot of the Project Settings dialog box. The title bar is light gray and contains the text 'Project Settings'. The main area is white and contains several fields: 'Project Name' (blinky), 'Project Location' (C:\STM32G0Workshop\HandsOn), 'Application Structure' (Basic), 'Toolchain Folder Location' (C:\STM32G0Workshop\HandsOn\blinky\), 'Toolchain / IDE' (MDK-ARM V5), and 'Linker Settings' (Minimum Heap Size: 0x200, Minimum Stack Size: 0x400). The 'Open Project' button is highlighted with a red border.

Note: STM32CubeMX projects have an *ioc* file extension

- Free licenses for STM32 devices based on Cortex-M0/M0+ cores :
 - Applicable immediately to all **STM32G0**, STM32F0 and STM32L0 MCUs.
 - PC-locked multi-year licenses.
 - No code size limit.
 - Multiple language support.
 - Technical support included.
- Direct download from Keil website :
 - No limit of number of downloads by customer.
 - Direct access to configuration files for STM32 and associated boards.
 - Free access to MDK-ARM periodic updates.

- To get a free MDK-ARM license for STM32F0, STM32L0 and **STM32G0**:

- Go to Keil website at : www.keil.com/mdk-st
- Download MDK-ARM tool chain.
- Activate the free license using this Product Serial Number (PSN) :
4PPFW-QBEHZ-M0D5M



The screenshot shows the KEIL MDK-ARM IDE Debugger interface. The top menu bar includes File, Edit, View, Project, Flash, Debug, Peripherals, Tools, SVCS, Window, and Help. Below the menu is a toolbar with various icons for file operations, debugging, and viewing. The main workspace is divided into several windows:

- Registers Window:** Located on the left, it displays a list of registers (R0 to R15, xPSR) and their current values. A pink arrow points to the 'Run' button (a play icon) in the toolbar, with the text 'Run Button' next to it.
- Disassembly Window:** Located at the top right, it shows the disassembled code for the selected address (87:). It includes comments like '/* USER CODE BEGIN 1 */' and '/* USER CODE END 1 */'.
- Files Window:** Located in the center, it shows the source code files (startup_stm32g071xx.s and main.c) and the current line of code (line 87: int main(void)). A pink arrow points to the line number '87', with the text 'Program counter position' next to it.
- Command Window:** Located at the bottom left, it shows the command prompt with the text 'Running with Code Size Limit: 256K' and 'Load "blinky\blinky.axf"'. Below this, it lists various debugger commands: ASSIGN, BreakDisable, BreakEnable, BreakKill, BreakList, BreakSet, BreakAccess, and COVERAGE.
- Memory Windows:** Located at the bottom right, it shows the memory address (Address:) and the memory content.

The status bar at the bottom indicates the current state: ST-Link Debugger, t1: 0.00000000 sec, L:87 C:1, CAP, NUM, SCRL, OVR, R/W.

Step 4: Toggle The LED

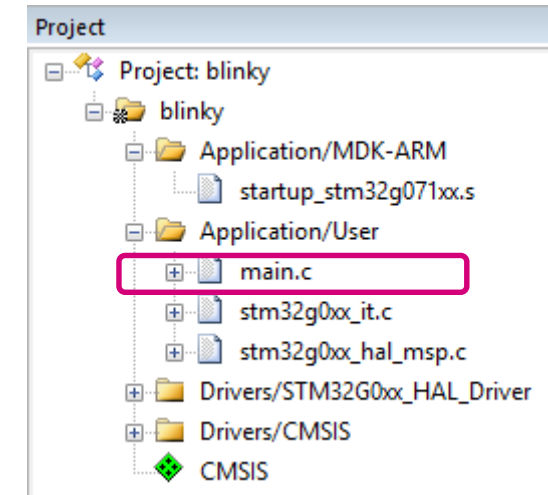
87

In the Keil uVision5 IDE:

- Expand the file tree and open the **main.c** file
- Add the following code **inside the while(1) loop (~Line 118(*))** in “main.c”

```
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);  
HAL_Delay(100);
```

```
114  /* Infinite loop */  
115  /* USER CODE BEGIN WHILE */  
116  while (1)  
117  {  
118      HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);  
119      HAL_Delay(100);  
120  /* USER CODE END WHILE */
```



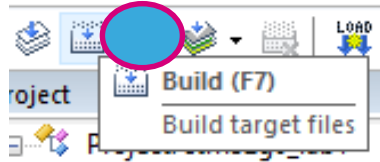
Note: Code within the “USER CODE BEGIN WHILE” / “USER CODE END WHILE” section will be preserved after regeneration by STM32CubeMX.

TIP: The code to be added for the labs are located in a text called “code_to_add_vx.x.txt”
(*) Line numbers throughout the presentation are given for STM32CubeMX v5.0.1 and STM32CubeG0 Library v1.0.0 and may vary with other versions.

Step 5: Build the Project and Debug

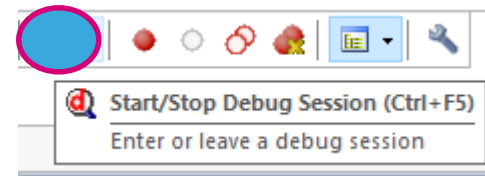
88

- Click the “Build” button (F7)



```
Build target 'blinky'  
"blinky\blinky.axf" - 0 Error(s), 0 Warning(s).  
Build Time Elapsed: 00:00:00
```

- Click the “Start/Stop Debug Session” button (Ctrl + F5)



Step 5: Build the Project

89

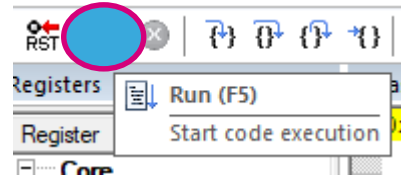
- If you see this warning message due to a minor Syntax error in the startup file, just press OK to continue.



Note: To correct the Syntax error in the startup_stm32g071x.s:

- Remove "<h2><center>©;" from line 17.
- Remove "</center></h2>" from line 18.

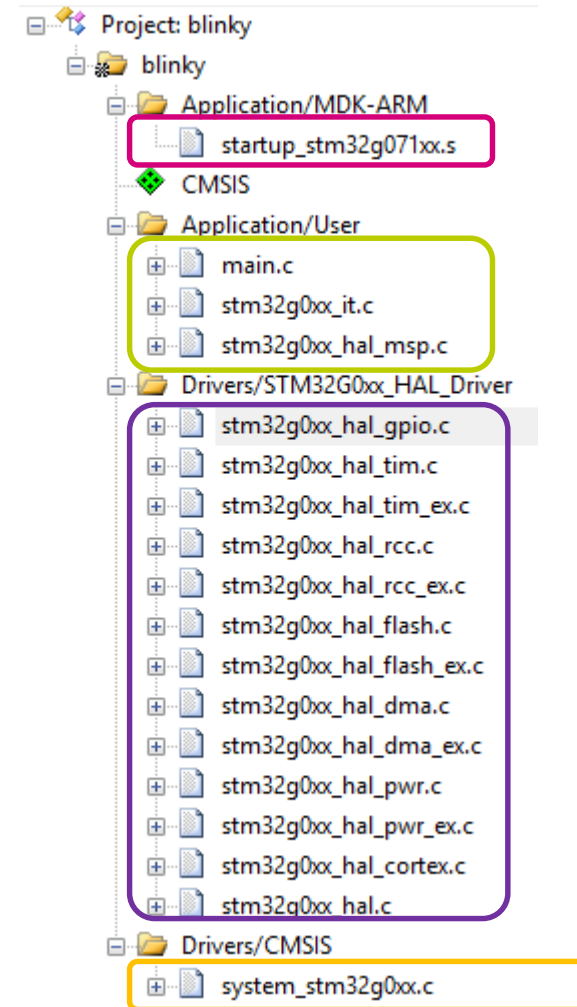
- Click the "Run" button (F5)



- Enjoy the flashing Green LED (LD4)!

Firmware Project Overview

- **startup_stm32g071x.s**
 - System initialization, vector table, reset and jump to main()
- **User files**
 - main.c (program entry point)
 - stm32g0xx_hal_msp.c (micro specific peripheral initialization and de-initialization functions)
 - stm32g0xx_it.c (Peripheral interrupt handlers)
- **HAL driver files**
 - Peripheral source files
- **system_stm32g0xx.c**
 - Contains SystemInit() function called at startup before branch to main()
 - Contains core clock variable (HCLK)
 - SystemCoreClockUpdate() function



- Main Characteristics

- Initializes stack pointer

- Contains the device vector table

- Contains Reset handler

- Called after RESET
- Calls SystemInit()
- Branch to main()

```

;*****
;* File Name      : startup_stm32g071xx.s
;* Author         : MCD Application Team
;* Description    : STM32G071xx devices vector table for MDK-ARM toolchain.
;*               : This module performs:
;*               : - Set the initial SP
;*               : - Set the initial PC == Reset_Handler
;*               : - Set the vector table entries with the exceptions ISR address
;*               : - Branches to __main in the C library (which eventually
;*               :   calls main()).
;*               : After Reset the CortexM0 processor is in Thread mode,
;*               : priority is Privileged, and the Stack is set to Main.
;* <<< Use Configuration Wizard in Context Menu >>>
;*****

```

```

; Vector Table Mapped to Address 0 at Reset
      AREA RESET, DATA, READONLY
      EXPORT __Vectors
      EXPORT __Vectors_End
      EXPORT __Vectors_Size

__Vectors      DCD      __initial_sp          ; Top of Stack
               DCD      Reset_Handler        ; Reset Handler
               DCD      NMI_Handler          ; NMI Handler
               DCD      HardFault_Handler    ; Hard Fault Handler

```

```

; Reset handler routine
Reset_Handler  PROC
               EXPORT Reset_Handler             [WEAK]
               IMPORT __main
               IMPORT SystemInit
               LDR     R0, =SystemInit
               BLX     R0
               LDR     R0, =__main
               BX      R0
               ENDP

```

- **SystemInit():**

- This function is called at startup after reset and before branch to main.

SystemInit()

```
164  /**
165   * @brief  Setup the microcontroller system.
166   * @param  None
167   * @retval None
168   */
169  void SystemInit(void)
170  {
171      /* Configure the Vector Table location add offset address -----*/
172      #ifdef VECT_TAB_SRAM
173          SCB->VTOR = SRAM_BASE | VECT_TAB_OFFSET; /* Vector Table Relocation in Internal SRAM */
174      #else
175          SCB->VTOR = FLASH_BASE | VECT_TAB_OFFSET; /* Vector Table Relocation in Internal FLASH */
176      #endif
177  }
178
```

- Header files

- Main.h

```
/* Includes -----*/
#include "main.h"
```

- main() function

- Configures system clock
- Call peripheral config. functions
- Infinite loop

```
int main(void)
{
    /* USER CODE BEGIN 1 */
    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */
    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */
    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
}
```

- Configuration functions

- SystemClock_Config()
- MX_GPIO_Init()

```
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOA_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);

    /*Configure GPIO pin : PA5 */
    GPIO_InitStruct.Pin = GPIO_PIN_5;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}
```

```
/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /**Configure the main internal regulator output voltage
    */
    HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1);
    /**Initializes the CPU, AHB and APB busses clocks
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSISpeed = RCC_HSI_ON;
    RCC_OscInitStruct.HSIClock = RCC_HSI_DIV1;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }
}
```

- stm32g0xx_hal_gpio.c
 - GPIO Initialize and De-Initialize API functions
 - GPIO Read and Write API function
 - GPIO EXTI IRQ handler
 - GPIO Callback function

```
void HAL_GPIO_Init(GPIO_TypeDef *GPIOx, GPIO_InitTypeDef *GPIO_Init)
{
    uint32_t position = 0x00u;
```

```
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
{
    GPIO_PinState bitstatus;
```

```
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
    /* EXTI line interrupt detected */
    if (__HAL_GPIO_EXTI_GET_RISING_IT(GPIO_Pin) != 0x00u)
    {
        __HAL_GPIO_EXTI_CLEAR_RISING_IT(GPIO_Pin);
        HAL_GPIO_EXTI_Rising_Callback(GPIO_Pin);
    }

    if (__HAL_GPIO_EXTI_GET_FALLING_IT(GPIO_Pin) != 0x00u)
    {
        __HAL_GPIO_EXTI_CLEAR_FALLING_IT(GPIO_Pin);
        HAL_GPIO_EXTI_Falling_Callback(GPIO_Pin);
    }
}
```

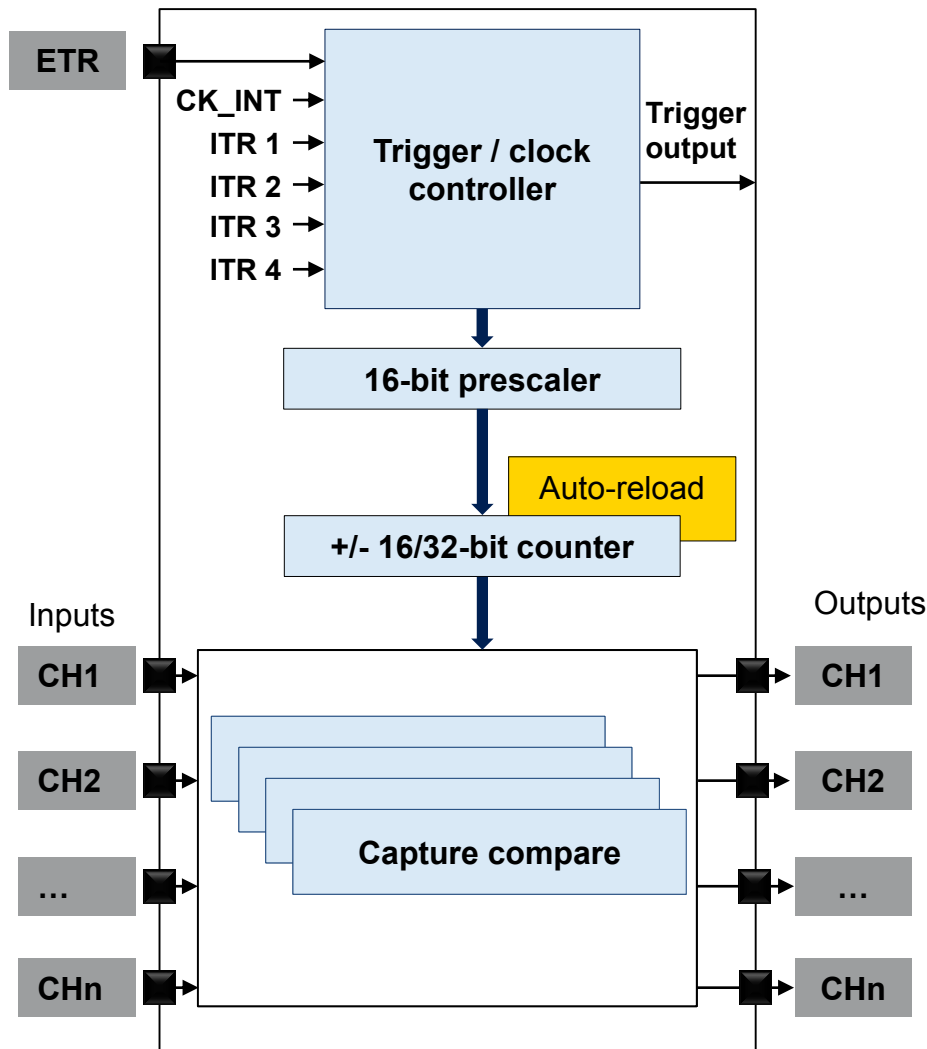
```
__weak void HAL_GPIO_EXTI_Rising_Callback(uint16_t GPIO_Pin)
{
    /* Prevent unused argument(s) compilation warning */
    UNUSED(GPIO_Pin);

    /* NOTE: This function should not be modified, when the callback is needed,
       the HAL_GPIO_EXTI_Callback could be implemented in the user file */
}
```

Lab: PWM (Pulse Width Modulation) Timer

Objective:

- Now let's use a more advanced peripheral like the Timer.
- In this lab we are going to configure a Timer in a PWM mode to blink the LED that we previously controlled with a GPIO.
- PA5 has an alternate Timer channel alternate function which is Timer 2 Channel 1: TIM2_CH1 that we will be using.



- Multiple timer units providing timing resources

- Internally (triggers and time-bases)
- Externally, for outputs or inputs:
 - For waveform generation (PWM)
 - For signal monitoring or measurement (frequency or timing)

Application benefits

- Versatile operating modes reducing CPU burden and minimizing interfacing circuitry needs
- A single architecture for all timer instances offers scalability and ease-of-use
- Also fully featured for motor control and digital power conversion applications

STM32G0 timer instance features

99

Feature		TIM1 (Advanced Control)	TIM2	TIM3	TIM6	TIM7	TIM14	TIM15	TIM16	TIM17
			(General-Purpose)		(Basic)		(General-Purpose)			
Clock source		CK_INT External input pin External trigger input ETR	CK_INT External input pin External trigger input ETR Internal trigger inputs		CK_INT		CK_INT	CK_INT External input pin Internal trigger inputs		CK_INT External input pin
Resolution		16-bit	32-bit	16-bit	16-bit		16-bit	16-bit		
Prescaler		16-bit								
Counter direction		Up, Down, Up&Down	Up, Down, Up&Down		Up		Up	Up		
Repetition counter		✓	-		-		-	✓		
Synchronization	Master	✓	✓		✓		-	✓		
	Slave	✓	✓		-		-	✓	-	
Number of channels		6: ➤ CH1/CH1N ➤ CH2/CH2N ➤ CH3/CH3N ➤ CH4 ➤ CH5 and CH6 output only, not available externally	4: ➤ CH1 ➤ CH2 ➤ CH3 ➤ CH4		0		1: ➤ CH1	2: ➤ CH1/CH1N ➤ CH2		1: ➤ CH1/CH1N
Trigger input		✓	✓							

STM32G0 timer instance features

100

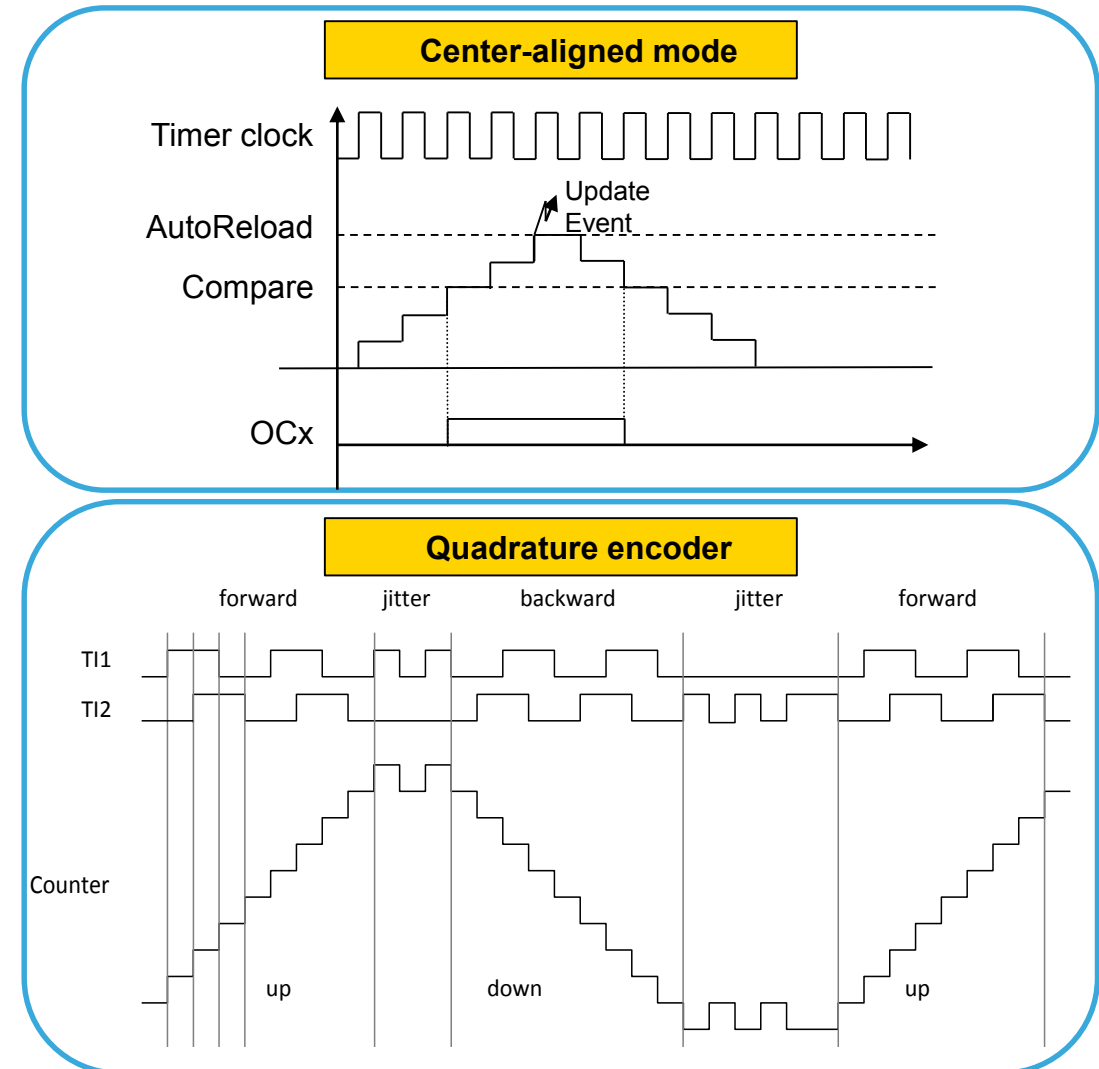
Feature	TIM1 (Advanced Control)	TIM2	TIM3	TIM6	TIM7	TIM14	TIM15	TIM16	TIM17
Input capture mode	✓	✓		-		✓		✓	
PWM input mode	✓	✓		-			✓		-
Forced output mode	✓	✓		-		✓		✓	
Output compare mode	✓	✓		-		✓		✓	
PWM	Standard Asymmetric Combined Combined 3-phase 6-step PWM	Standard Asymmetric Combined		-		Standard	Standard Asymmetric Combined		Standard
Programmable dead-time	✓ (CH1-3)	-		-		-	✓ (CH1)		-
Break inputs	2 bidirectional	0		0		0		1 bidirectional	
One-Pulse Mode	✓	✓		-		✓		✓	
Retriggerable one pulse mode	✓	✓		-		-	✓		-
Encoder interface mode	✓	✓		-		-		-	
Timer input XOR function	✓	-		-		-	✓		-
DMA	✓	✓		✓		-		✓	

Multiple internal or external clocking options

- Timers 1 and 15 are clocked up to 128 MHz to bring additional resolution below 8 ns
 - Finer resolution for buck converters (10-bit accuracy @ 100 kHz PWM)
 - Lower frequency steps for variable frequency resonant converters (e.g. LLC), e.g. 0.4kHz max. frequency step at 200 kHz switching (0.2%)
- Uses cases
 - Timer 1 has 3 complementary pairs: LLC primary and secondary sides (synchronous rectification), boundary conduction mode PFC, buck
 - Timer 15 has one pair only (buck, LLC primary side)

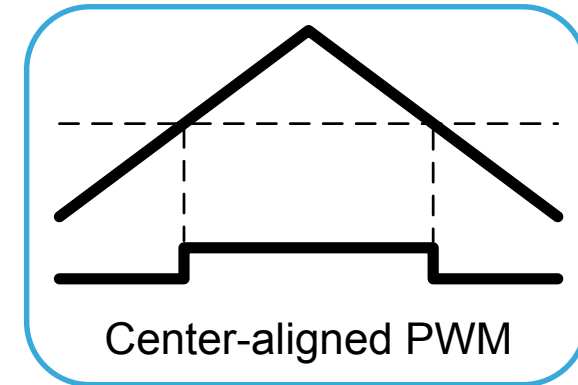
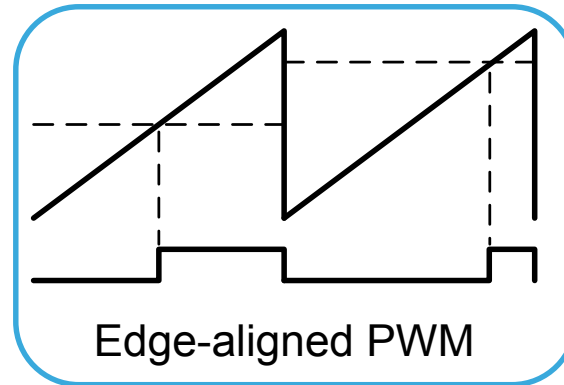
- Support of incremental / quadrature encoders and motor drive applications

- Up- and down-counting modes supported
 - On TIM1, TIM2 and TIM3
- Center-aligned PWM generation
 - Direction changes on overflow and underflow
 - Reduces acoustic noise in electric motors
- Built-in support of quadrature encoders
 - Rotary encoder / digital potentiometer
 - Position sensor
 - Allows direct angle reading in timer

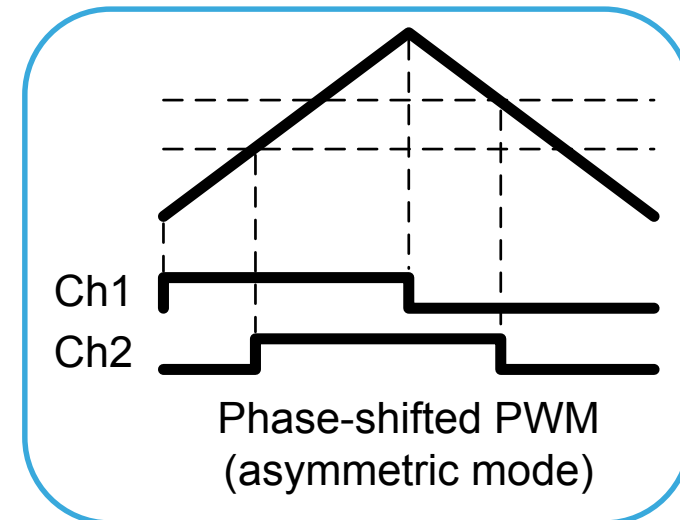
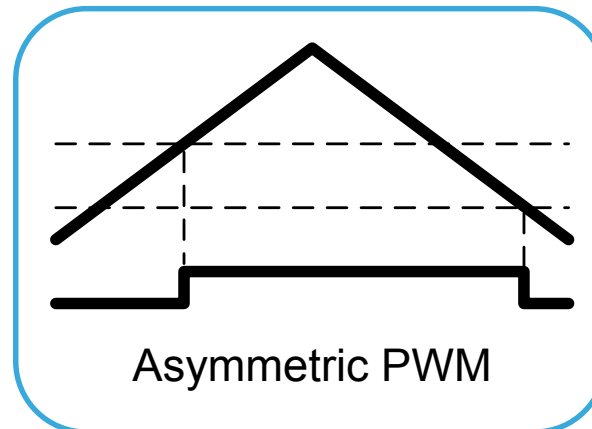


A variety of PWM modes to address multiple applications

- Basic PWM, edge- or center-aligned

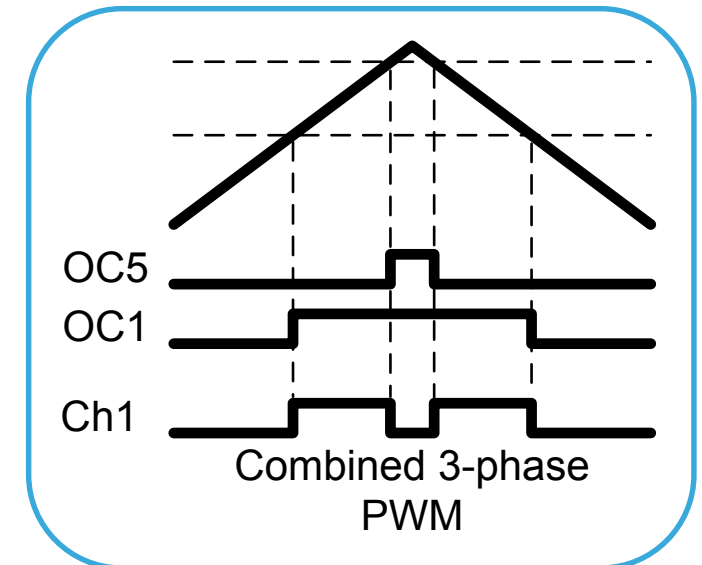
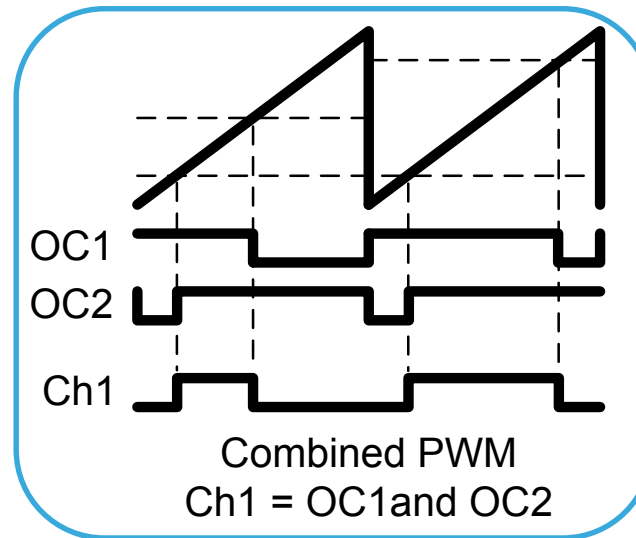
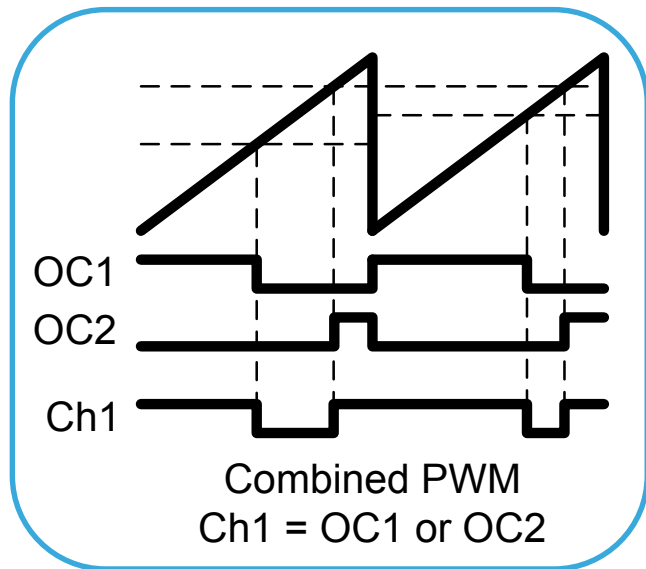


- Asymmetric center-aligned PWM



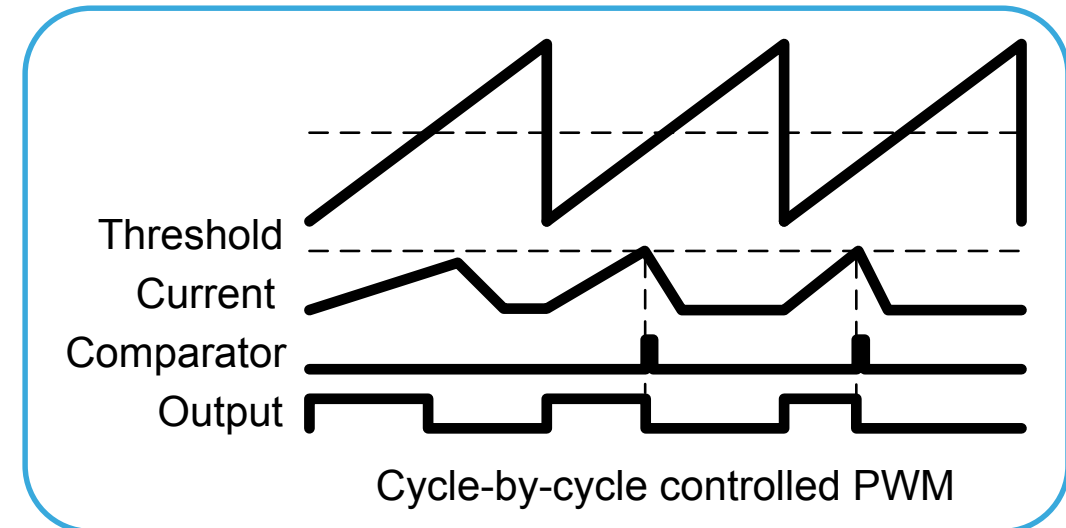
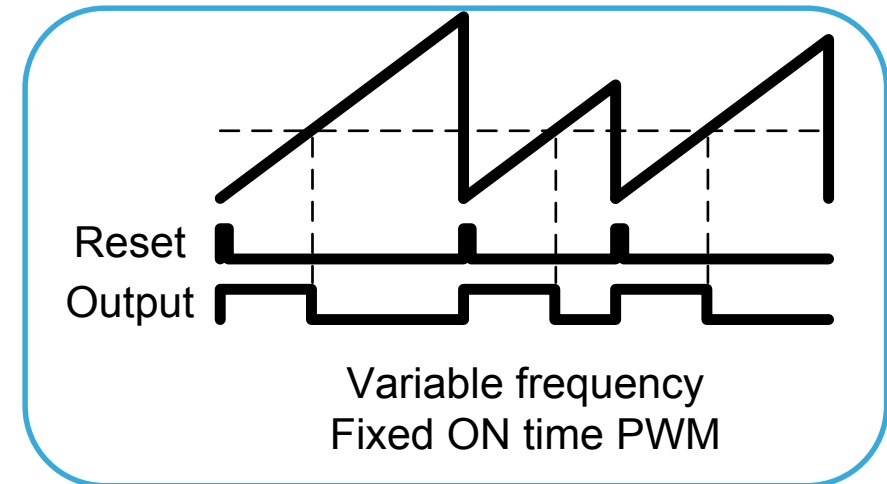
Extends the PWM capabilities and avoids external glue logic

- Combined PWM mode
 - Combines two channels with OR or AND function for more complex waveforms
- Combined 3-phase mode
 - Allows a 4th PWM to be combined with a regular 3-phase PWM for zero vector insertion



For PWM signals requiring external control

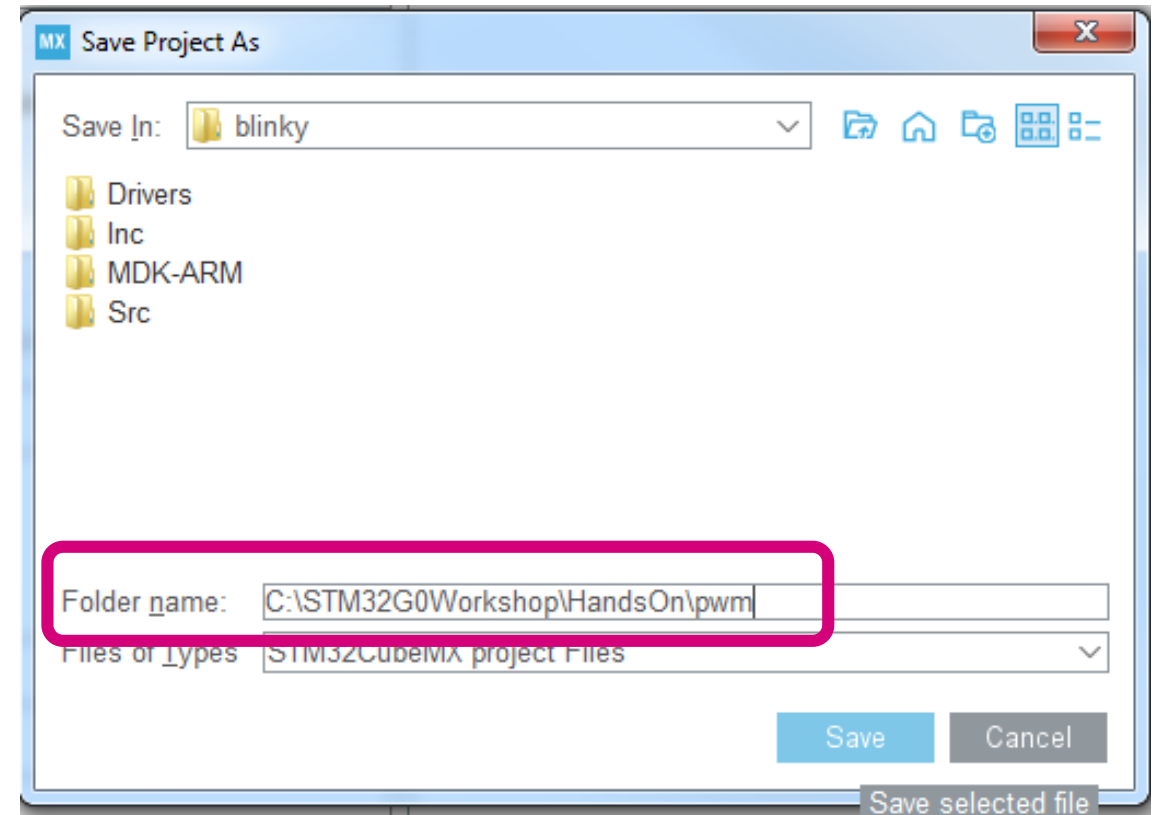
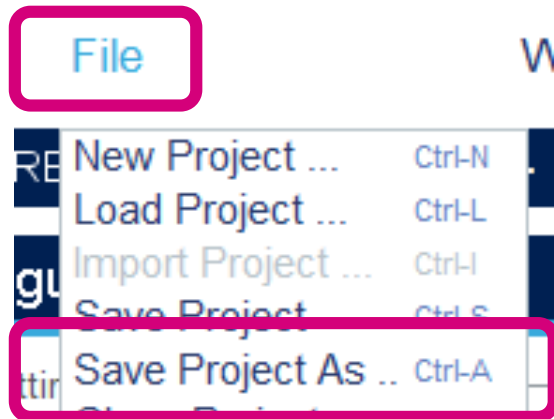
- Variable-frequency PWM
 - Driven by an external signal
- Cycle-by-cycle controlled duty cycle
 - For current loops, driven by comparator or external pin



Lab: Rename the project

106

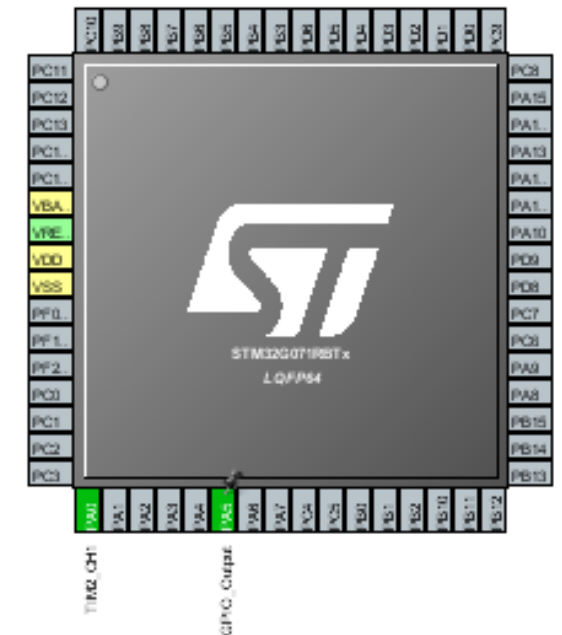
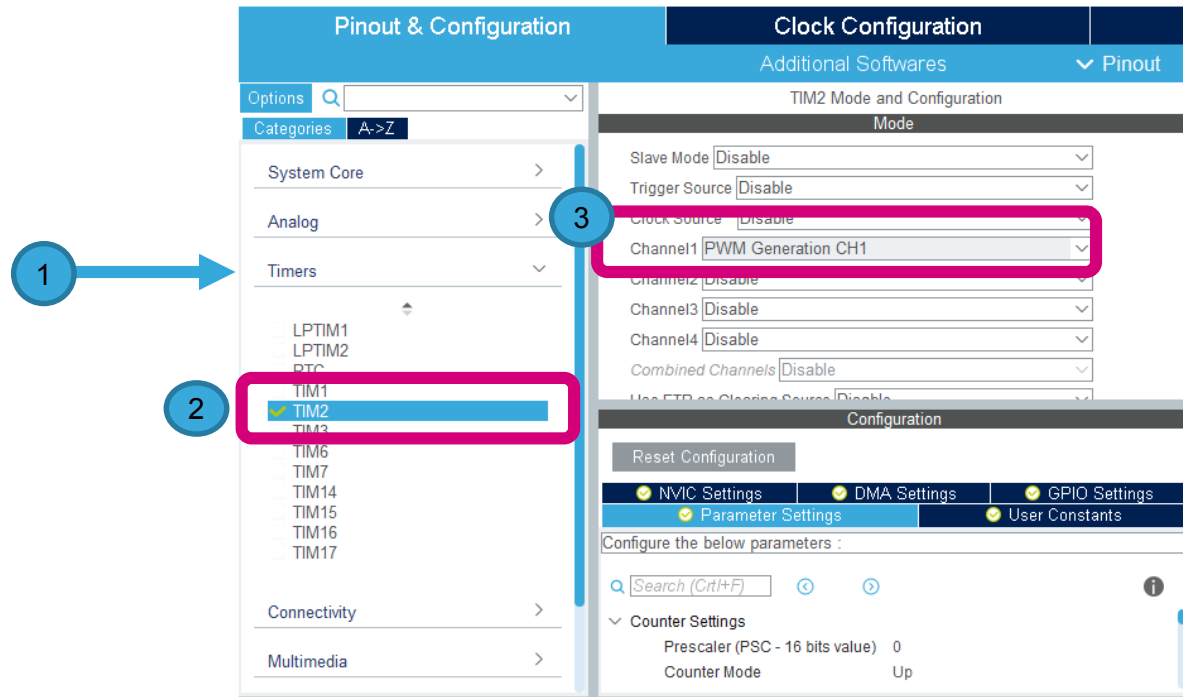
- Close Keil uVision5 IDE if it is open
- Open the last STM32CubeMX project (“**blinky**”) (using File->Recent Projects) and save it as a new project name “**pwm**” (using File -> Save Project As)



Lab: Timer 2 CH1 Configuration

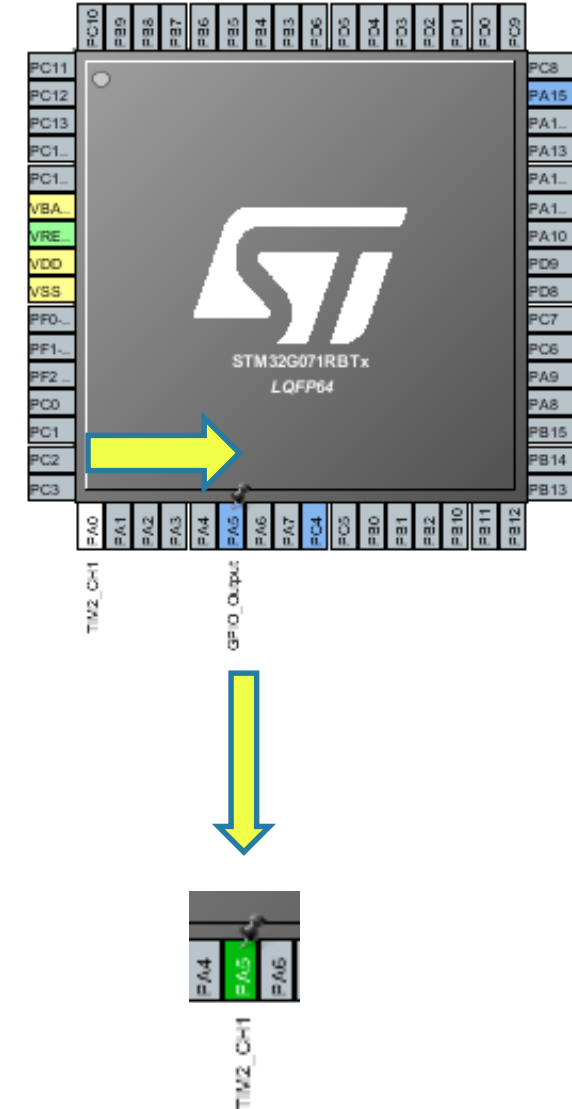
107

- In this STM32CubeMX project we are going to add Timer 2 Channel 1 to blink LD4 (PA5) on the Nucleo board.
- In the **Pinout & Configuration** tab, Expand **Timers** Categories, then click on **TIM2** peripheral and set Channel1 to “**PWM Generation CH1**”.



Remapping Timer 2 CH1 output to PA5

- By default the tool will configure Timer 2 CH1 to **PA0**
- We want to remap it to **PA5**
 - NOTE: PA5 is connected to LD4
- Hold “**Ctrl**” button and left mouse click on **PA0**
- Then drag the mouse pointer to **PA5** and then release



- We can calculate Timer parameters for 1 Hz period @ 50% duty cycle
- If we choose Timer input clock to be 8 MHz
- Let us say the prescaler is div-by-128; $8\text{MHz}/128 = 62500 \text{ Hz}$
 - So, prescaler register should be $128-1 = 127$ (actual divide is $\text{PSC}+1$)
 - If we set Period = 62500; Pulse = 31250, we get 1 Hz Period @ 50% duty cycle

110



Help



GENERATE CODE

Tools

E



TIM2 Configuration – 4 steps

111

- Select the **Pinout & Configuration**
- In **Parameters Settings** of the TIM2 1
- Configure 1 Hz timer
 - Prescaler = 127 2
 - Period = 62500 3
- Set CH1 PWM
 - Pulse = 31250 4

TIM2 Mode and Configuration

Configuration

Reset Configuration

✓ NVIC Settings | ✓ DMA Settings | ✓ GPIO Settings | ✓ Parameter Settings 1 | ✓ User Constants

Configure the below parameters :

Search (Ctrl+F)

Counter Settings

Prescaler (PSC - 16 bits value) 127 2

Counter Mode Up

Counter Period (AutoReload R... 62500 3

Internal Clock Division (CKD) No Division

auto-reload preload Disable

Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit) Disable (Trigger input effect not delayed)

Trigger Event Selection TRGO Reset (UG bit from TIMx_EGR)

Clear Input

Clear Input Source Disable

PWM Generation Channel 1

Mode PWM mode 1

Pulse (32 bits value) 31250 4

Fast Mode Disable

CH Polarity High

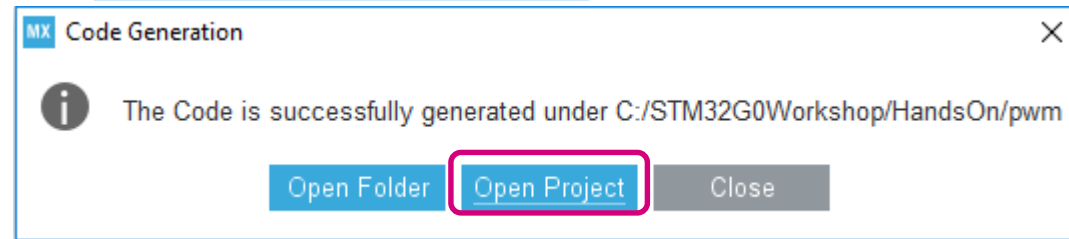
Generate Source Code

112

- **Generate Code**



- **Click Open Project**



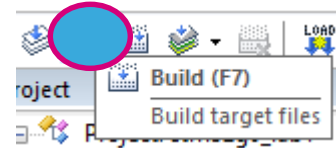
- Open the **main.c**, Add the following code before the **while(1)** loop in order to start the PWM Timer:

Note : within "USER CODE BEGIN 2" / "USER CODE END 2" section (~Line 114)

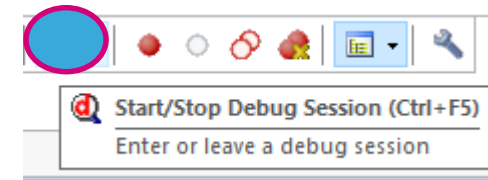
```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
```

```
113  /* USER CODE BEGIN 2 */
114  HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
115  /* USER CODE END 2 */
...
```

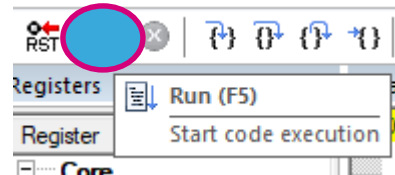
- Click the “Build” button



- Click the “Start/Stop Debug Session” button



- Click “Run” button



- Enjoy the flashing LED (LD4)!
 - LD4 is flashing using the PWM Timer

Lab: NVIC + External Interrupts

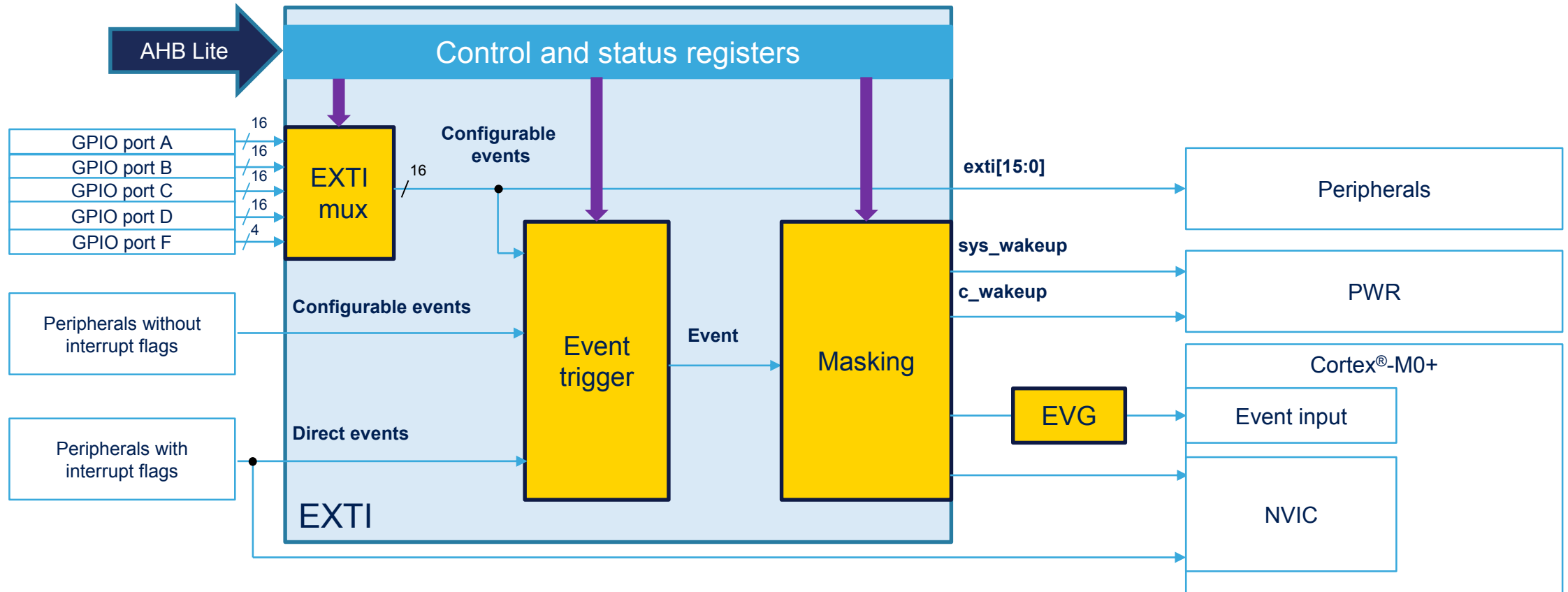
Objective:

- In this project we are going to configure the GPIO that is connected to the user button as External Interrupt (EXTI) with rising edge trigger.
- We will also configure the Interrupt Controller: the NVIC.

- Wake-up from Stop mode, interrupts and events generation
 - Independent interrupt and event masks
- Configurable events
 - Active edge selection
 - Dedicated pending flag
 - Trigger-able by software
 - Linked to:
 - GPIO, PVD, and COMPx
- Direct events
 - Status flag provided by related peripheral
 - Linked to:
 - RTC, TAMP, I2C1, USARTx, CEC, LPUART1, LPTIMx, LSE_CSS and UCPDx

EXTI - block diagram

117



EVG: Event Generator

EXTI - lines mapping

118

EXTI line	Line source	Line type
0-15	GPIO	Configurable
16	PVD output	Configurable
17	COMP1 output	Configurable
18	COMP2 output	Configurable
19	RTC	Direct
20	Reserved	Direct
21	TAMP	Direct
22	Reserved	Direct
23	I2C1 wakeup	Direct
24	Reserved	Direct
25	USART1 wakeup	Direct
26	USART2 wakeup	Direct
27	CEC wakeup	Direct
28	LPUART1 wakeup	Direct
29	LPTIM1	Direct
30	LPTIM2	Direct
31	LSE_CSS	Direct
32	UCPD1 wakeup	Direct
33	UCPD2 wakeup	Direct

- The NVIC (Nested vector Interrupt Controller) is integrated in the Cortex®-M0+ CPU:
 - 32 maskable interrupt channels
 - 4 programmable priority levels
 - Low-latency exception and interrupt handling
 - Power management control

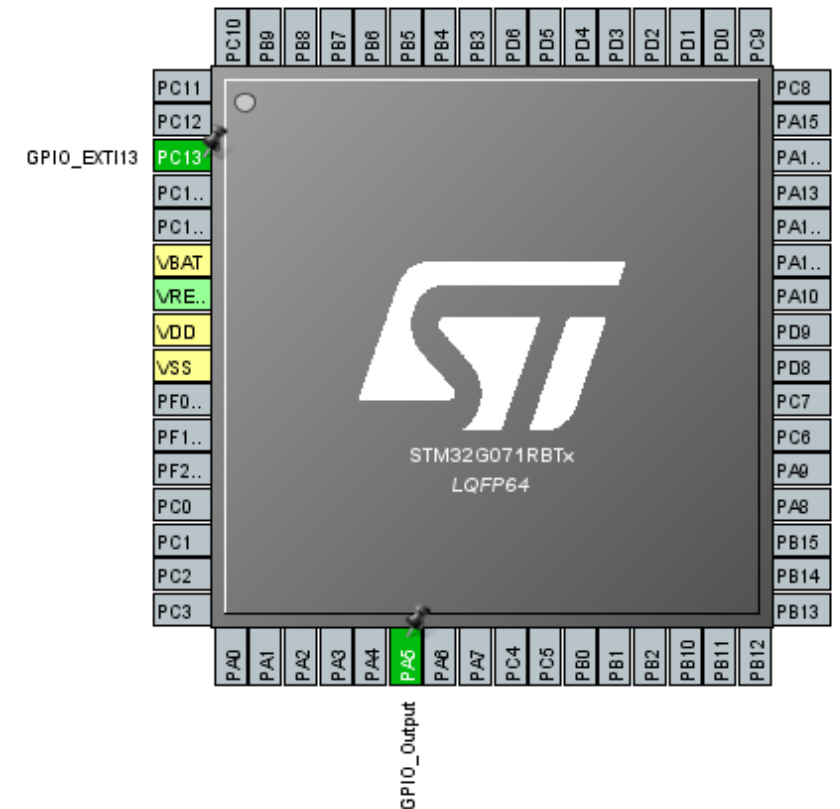
Application benefits

- Supports prioritization levels with dynamic control
- Fast response to interrupt requests
- Relocatable vector table

Lab: Pinout Configuration

120

- Close Keil uVision5 IDE if it is open; Open the “**blinky**” STM32CubeMX project (using File->Recent Projects) and save it as a new project named “**exti**”.
- Add configuration of the IO that is connected to the User Button (connected to **PC13**) to toggle the LED LD4 (connected to **PA5**) on the STM32G0 Nucleo board.
- **PA5** is already configured as **GPIO** output push-pull.
- Left-click on **PC13** and set it to **GPIO_EXTI13** mode.



GPIO Configuration

121

- Select **GPIO** under **System View** ①
- Click on Pin Name **PC13** ②
- Make sure GPIO mode is “External **Interrupt Mode** with Rising edge trigger detection” ③

The screenshot shows the STM32CubeMX software interface. In the top left, the 'System view' tab is selected, and the 'GPIO' button is highlighted with a red box and a circled '1'. Below this, the 'Configuration' window is open. The 'Group By Peripherals' checkbox is unchecked. The 'GPIO' and 'NVIC' tabs are both selected. A table lists the configured pins, with the row for 'PC13' highlighted by a red box and a circled '2'. The table has columns: Pin, Signal, GPIO P..., GPIO ..., GPIO P..., Maxim..., Fast M..., User L..., and Modified. The 'PC13' row shows 'n/a' for Signal, 'n/a' for GPIO P..., 'Extern...' for GPIO ..., 'No pull...' for GPIO P..., 'n/a' for Maxim..., 'n/a' for Fast M..., and an empty 'User L...' field. Below the table, the 'PC13 Configuration' section is shown. The 'GPIO mode' dropdown is set to 'External Interrupt Mode with Rising edge trigger detec...' and is highlighted with a red box and a circled '3'. The 'GPIO Pull-u...' dropdown is set to 'No pull-up and no pull-down'. The 'User Label' field is empty.

Pin ...	Signal ...	GPIO P...	GPIO ...	GPIO P...	Maxim...	Fast M...	User L...	Modified
PA5	n/a	Low	Output ...	No pull...	Low	n/a		☐
PC13	n/a	n/a	Extern...	No pull...	n/a	n/a		☐

PC13 Configuration :

GPIO mode: External Interrupt Mode with Rising edge trigger detec... ③

GPIO Pull-u...: No pull-up and no pull-down

User Label:

- Select **NVIC** under **System View**



- Enable “**EXTI line 4 to 15 interrupts**” (by checking the box) 1

NVIC Mode and Configuration

Configuration

☒ NVIC ☒ Code generation

☐ Sort by Preemption Priority and Sub Priority

Search ☐ Show only enabled interrupts

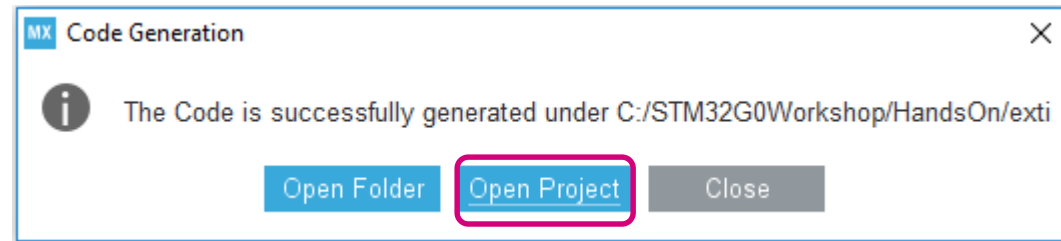
NVIC Interrupt Table	Enabled	Preemption Priority
Non maskable interrupt	☒	0
Hard fault interrupt	☒	0
System service call via SWI instruction	☒	0
Pendable request for system service	☒	0
Time base: System tick timer	☒	0
PVD interrupt through EXTI line 16	☐	0
Flash global interrupt	☐	0
RCC global interrupt	☐	0
EXTI line 4 to 15 interrupts	☒	0

1

- **Generate Code**



- Click **Open Project**



- Open **main.c**, add the following code:
 - within “USER CODE BEGIN PV” / “USER CODE END PV” section (~Line 67)

```
uint8_t PC13_flag = 0;
```

```
66  /* USER CODE BEGIN PV */  
67  uint8_t PC13_flag=0;  
68  /* USER CODE END PV */
```

Add EXTI Rising Edge Callback Function

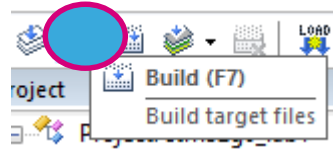
124

- Also in **main.c** add the following code,
 - within “USER CODE BEGIN 4” / “USER CODE END 4” section (~line 198)

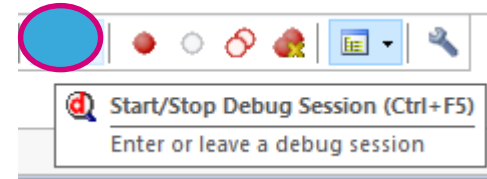
```
void HAL_GPIO_EXTI_Rising_Callback(uint16_t GPIO_Pin)
{
    PC13_flag++;
    if ( ( PC13_flag & 0x01 ) == 0x01 )
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
    }
    else
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
    }
}
```

```
197  /* USER CODE BEGIN 4 */
198  void HAL_GPIO_EXTI_Rising_Callback(uint16_t GPIO_Pin)
199  {
200      PC13_flag++;
201      if ((PC13_flag & 0x01) == 0x01)
202      {
203          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
204      }
205      else
206      {
207          HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
208      }
209  }
210  /* USER CODE END 4 */
```

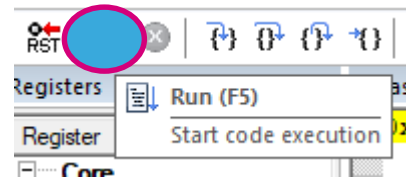
- Click the “Build” button



- Click the “Start/Stop Debug Session” button

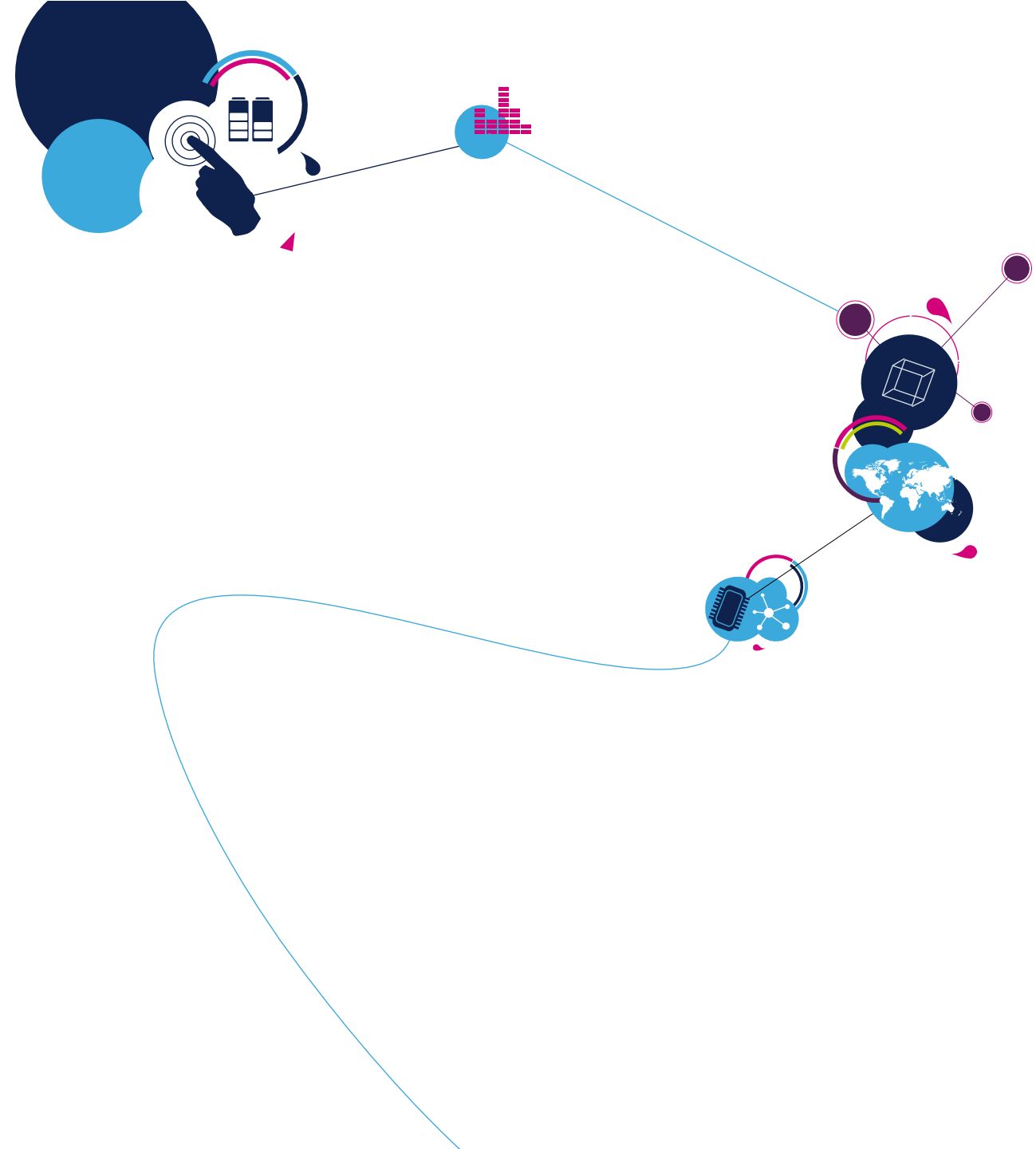


- Click “Run” button



- Push the Blue “USER” button to toggle the LED LD4!

Lab: Low Power



Objective:

- In this lab we are going use the STOP 1 mode and wakeup from RTC which is configured to wakeup the STM32 every 5 seconds.
- When the STM32 wakes up it will turn on the LED (LD4) for one second and then go back to STOP mode.
- The MCU can also wake-up using the user button which is configured as EXTI.

Low Power Modes

128

Wakeup
time

6 cycles

0.7 μ s

4 μ s

5 μ s

14 μ s

14 μ s

256 μ s

RUN (Range1) at 64 MHz

100 μ A / MHz

RUN (Range2) at 16 MHz

93 μ A / MHz

LPRUN at 2 MHz

90 μ A / MHz

SLEEP at 16 MHz

32 μ A / MHz

LPSLEEP at 2 MHz

42 μ A / MHz

STOP 0 (full retention)

100 μ A

STOP 1 (no retention)

4.1 μ A*

STANDBY + SRAM

320 nA/670 nA*

STANDBY

130 nA/480 nA*

SHUTDOWN

40 nA/380 nA

VBAT

340 nA*

Typ @ VDD = 3 V @ 25 °C

* : with RTC

FlexPowerControl

- Efficient running
- 7 low-power modes, several sub-modes
- High flexibility

Application benefits

- High performance
 - ➔ CoreMark score = 142.88
- Outstanding power efficiency

Lowest power modes with full retention, 5 μ s wakeup time to 16 MHz

- SRAM and all peripheral registers retention
 - All high-speed clocks are stopped
 - Flash can be switched OFF
- LSE (32.768 kHz external oscillator) and LSI (32 kHz internal oscillator) can be enabled
- Several peripherals can be active and wake up from Stop modes
- System clock at wakeup is HSI16 (2 μ s wakeup time on RAM, 5.5 μ s on FLASH not powered)
- Stop 1 is equivalent to Stop 0 with Main Regulator off, resulting in a smaller current consumption but longer wake up time

Stop 0 mode

130

Available peripherals

GPIO
DMA
BOR
PVD
USART
LP UART
I2C 1
I2C 2
SPI
ADC
DAC
COMP
Temp Sensor
Timers
LPTIM 1
LPTIM 2
IWDG
WWDG
Systick Timer
UCPD
RNG
AES
CRC
CEC



I/Os kept, and configurable

Zzz

Cortex M0+

Flash
memory

SRAM
(36 Kbytes)

Available clocks

HSI16
HSE
LSI
LSE

97 μ A @ 3.0 V

Wakeup time to 16 MHz:

- In SRAM: 2 μ s
- In Flash ON: 2 μ s
- In Flash OFF: 5.5 μ s

Main regulator (MR)

Range 1 (up to 64 MHz)

Range 2 (up to 16 MHz)

Low Power regulator (LPR) up to 2 MHz

Backup domain

Backup Register (5x32 bits)

RTC & TAMPER

Wake-up event

NRST
BOR
PVD
RTC + Tamper
USART
LP UART
I2C 1
CEC
COMP
LPTIM 1
LPTIM 2
IWDG
GPIOs

Active cell

Clocked-off
cell

Cell in power-
down

Available
Periph and clock

Stop 1 mode

131

Available peripherals

GPIO
DMA
BOR
PVD
USART
LP UART
I2C 1
I2C 2
SPI
ADC
DAC
COMP
Temp Sensor
Timers
LPTIM 1
LPTIM 2
IWDG
WWDG
Systick Timer
UCPD
RNG
AES
CRC
CEC



I/Os kept, and configurable

Zzz

Cortex M0+

Flash
memory

SRAM
(36 Kbytes)

Available clocks

HSI16
HSE
LSI
LSE

Flash memory not powered:

- w/o RTC: 1.3 μA @ 3.0 V
- w/ RTC: 4.1 μA @ 3.0 V

Flash memory powered:

- w/o RTC: 7.0 μA @ 3.0 V

Wakeup time to 16 MHz:

- In SRAM: 5 μs
- In Flash ON: 5 μs
- In Flash OFF: 9 μs

Main regulator (MR)

Low Power regulator (LPR) up to 2 MHz

Backup domain

Backup Register (5x32 bits)

RTC & TAMPER

Wake-up event

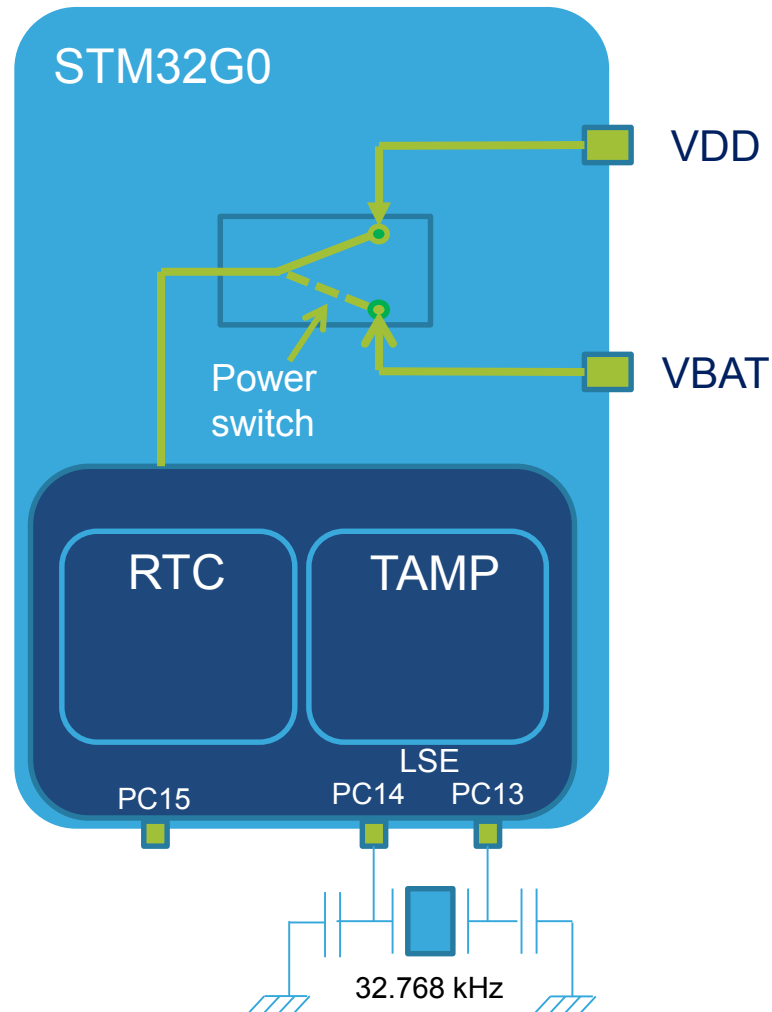
NRST
BOR
PVD
RTC + Tamper
USART
LP UART
I2C 1
CEC
COMP
LPTIM 1
LPTIM 2
IWDG
GPIOs

Active cell

Clocked-off
cell

Cell in power-
down

Available
Periph and clock



- The RTC provides an ultra-low-power hardware calendar with alarms, in all low-power modes
- It belongs to the Battery Backup Domain, so it is kept functional when the main supply is off and VBAT is present
- The TAMP peripheral features the backup registers and tamper detection

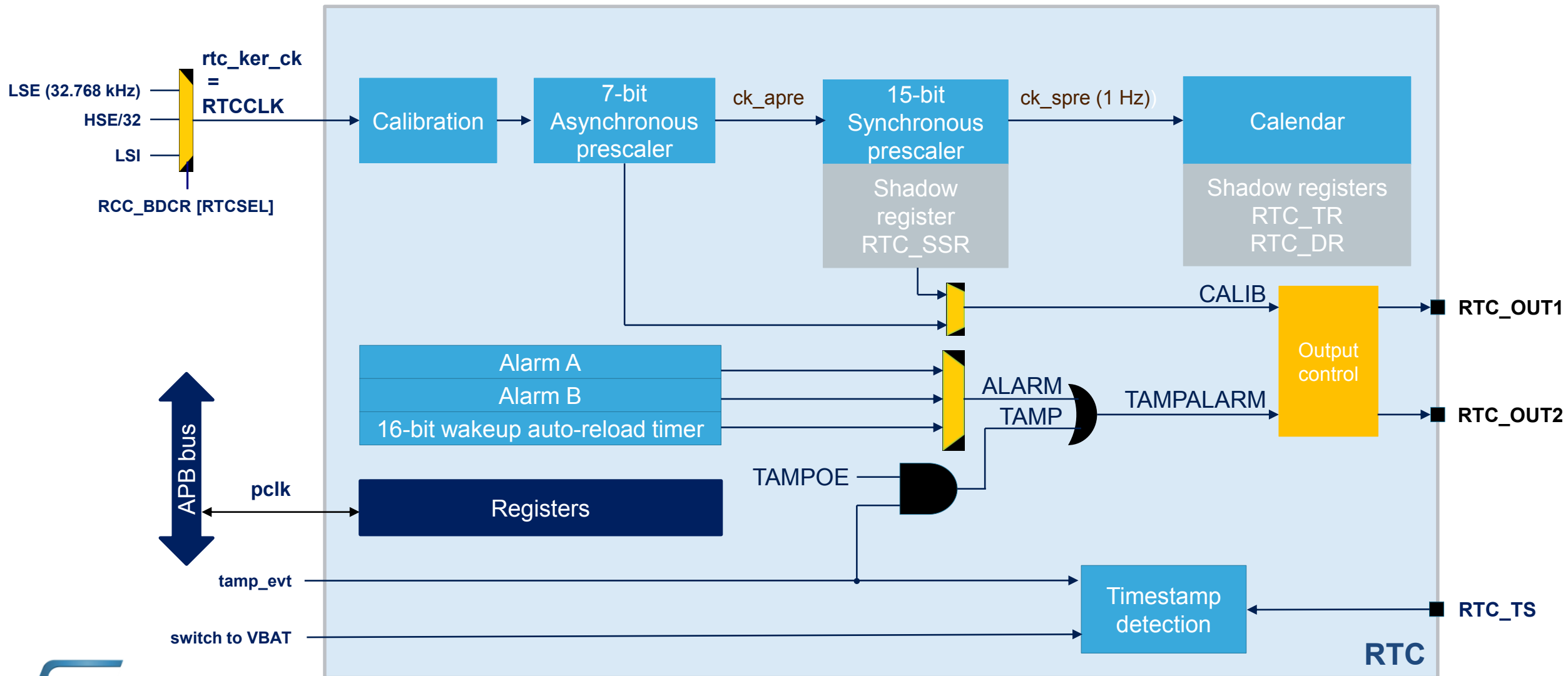
Application benefits

- Ultra-low power: 300 nA at 1.8 V
- Hardware BCD calendar to reduce software load

- Sub-seconds, seconds, minutes, hours, week day, date, month, year in BCD format
- “On the fly” programmable daylight savings compensation
- Two programmable alarms with wakeup interrupt function
- A periodic event with programmable resolution, triggering wakeup interrupt
- A reference clock source (50 or 60 Hz) can be used to enhance the calendar precision
- Digital calibration circuit to achieve 0.95 ppm accuracy
- Timestamp feature which can be used to save the calendar content with sub-second precision (one event)

RTC - Block diagram

134



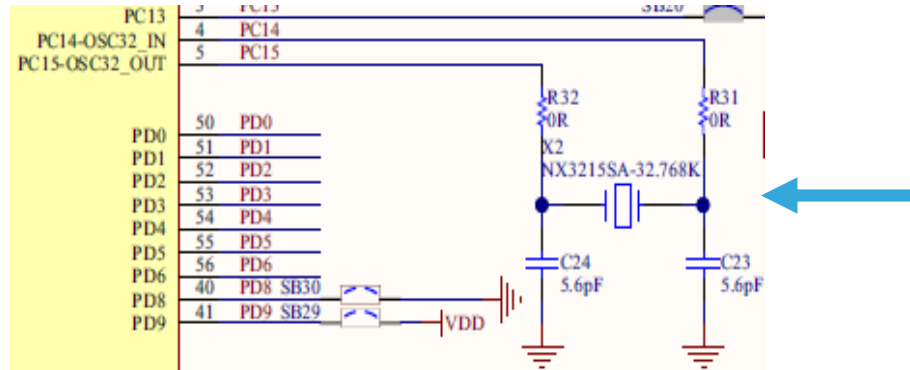
- RTC not affected by system reset when clocked by LSE

- Close Keil uVision5 IDE if it is open; In STM32CubeMX open the “exti” STM32CubeMX project save it as a new project like “lowpower”.

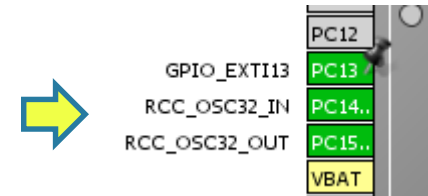
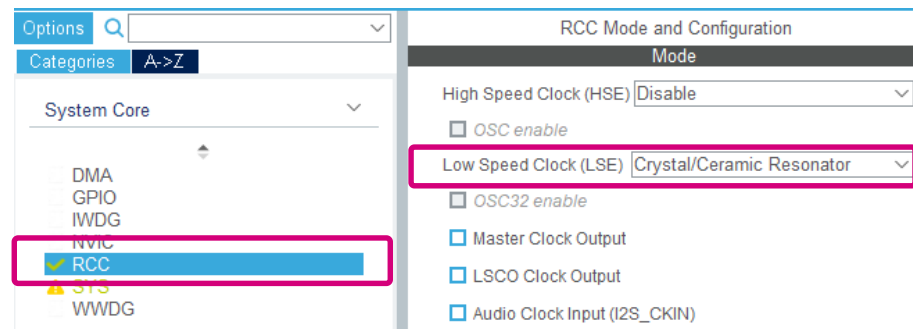
Enable LSE (Low Speed External) Clock

136

- We are going to use the 32 KHz Crystal that is on the Nucleo board (see schematic below) to clock the RTC:



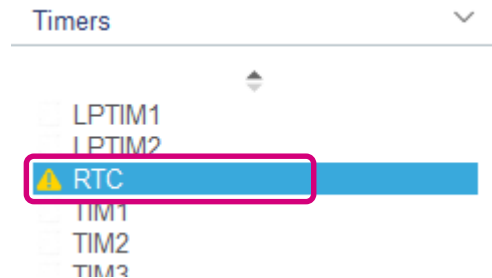
- In the pinout tab, expand RCC and choose Crystal/Ceramic Resonator for Low Speed Clock (LSE) clock:



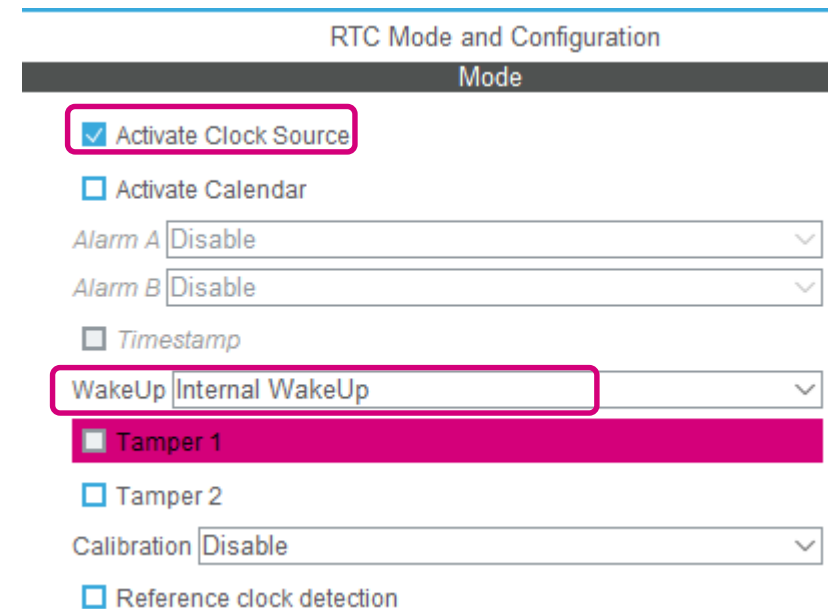
Enable and Configure the RTC

137

- In the **Pinout** tab, under **Timers**, expand **RTC**



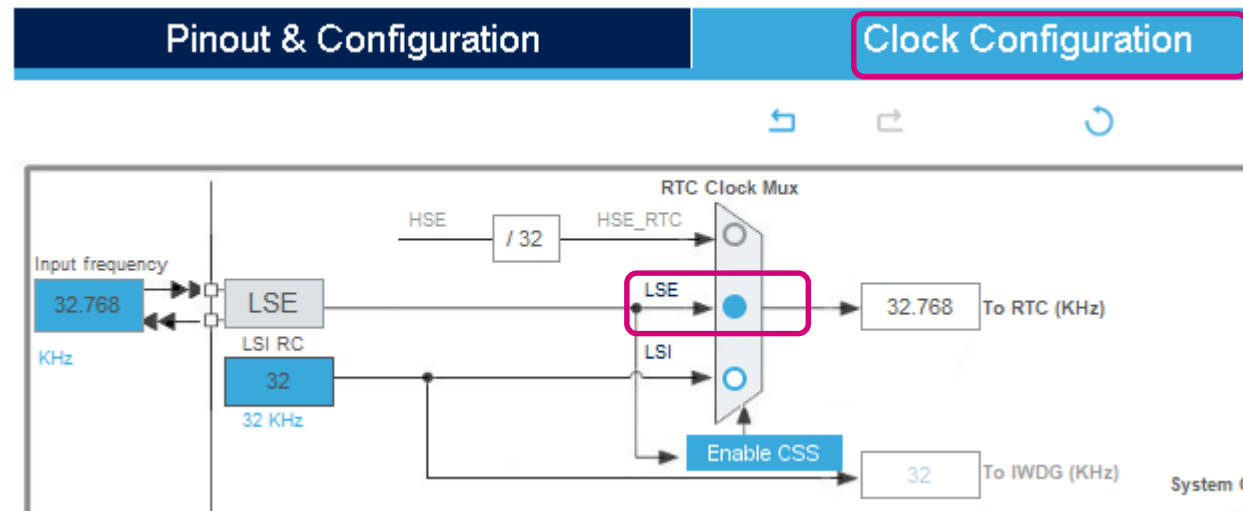
- Check the **Activate Clock Source**
- Select **Internal Wakeup** for the **Wakeup** mode



Choose RTC clock source

138

- In the **Clock Configuration** tab, select LSE as input clock for RTC



Note: For applications that do not require precise RTC timings the LSI (Low Speed Internal RC) can be used to clock the RTC

Wakeup Counter Calculation:

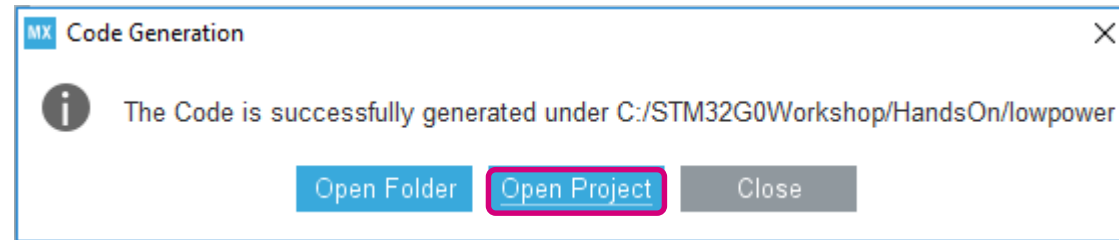
- To configure the wake up timer for 5s, the WakeUpCounter should be set to **10246** as calculated below:
- With RTC Clock set to **RTCCLK /16**
- Wakeup Time Base = $\text{RTC_PRESCALER} / \text{LSE} = 16 / (32.768\text{KHz}) = 0.488 \text{ ms}$
- Wakeup Time = Wakeup Time Base * WakeUpCounter = $0.488\text{ms} * \text{WakeUpCounter}$

$$\Rightarrow \text{WakeUpCounter} = 5\text{s} / 0.488\text{ms} = \mathbf{10246}$$

- **Generate Code**



- **Click Open Project**



Add code – to main function

143

- Open the **main.c**, add the following code in the **while(1)** loop of the main function in the **USER CODE WHILE** section (~ line 121):

```
/* USER CODE BEGIN WHILE */
while (1)
{

    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
    HAL_Delay(1000);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);

    // enter STOP mode
    HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);

    // reconfigure system clock
    SystemClock_Config();
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

```
117  /* Infinite loop */
118  /* USER CODE BEGIN WHILE */
119  while (1)
120  {
121      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
122      HAL_Delay(1000);
123      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
124
125      // enter STOP mode
126      HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);
127
128      // reconfigure system clock
129      SystemClock_Config();
130      /* USER CODE END WHILE */
131
132      /* USER CODE BEGIN 3 */
133  }
134  /* USER CODE END 3 */
135  }
```

Add code – to init function

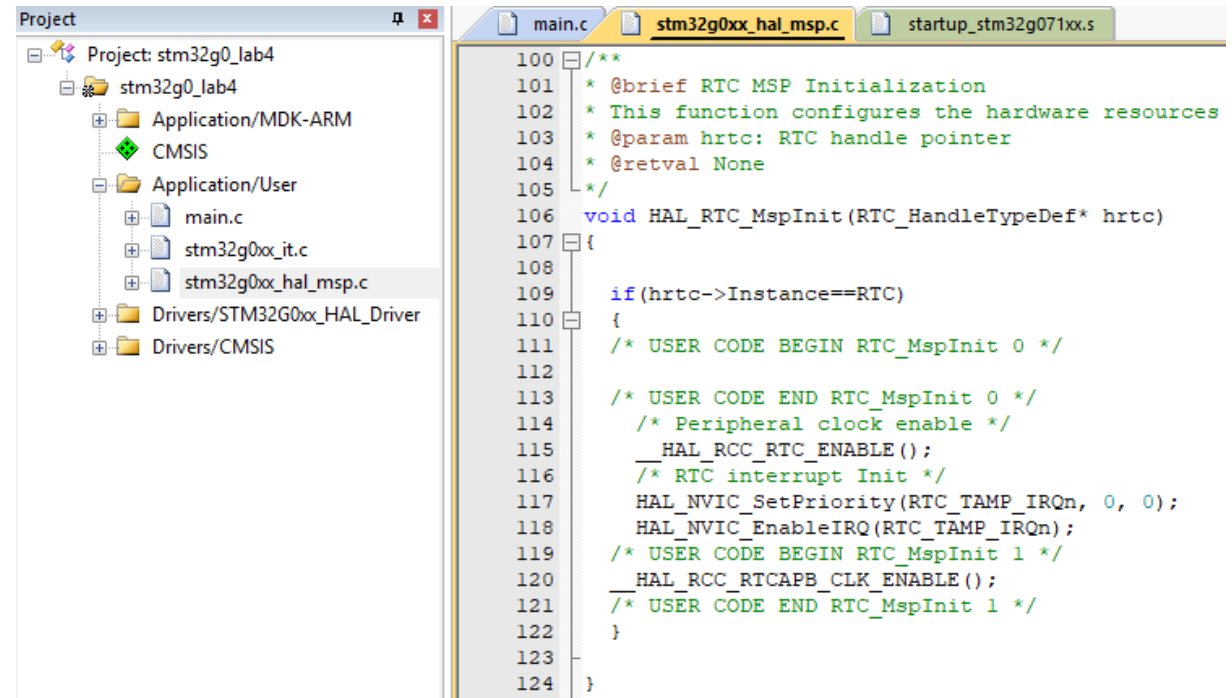
144

- Open the **stm32g0xx_hal_msp.c**, Add the following line of code (marked in red below) to the msp init function HAL_RTC_MspInit():

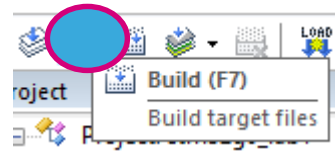
```
void HAL_RTC_MspInit(RTC_HandleTypeDef* hrtc)
{

    if(hrtc->Instance==RTC)
    {
        /* USER CODE BEGIN RTC_MspInit 0 */

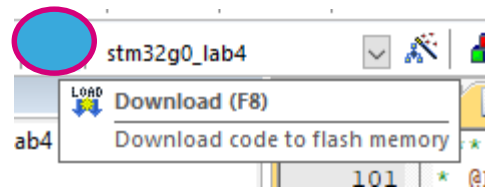
        /* USER CODE END RTC_MspInit 0 */
        /* Peripheral clock enable */
        __HAL_RCC_RTC_ENABLE();
        /* RTC interrupt Init */
        HAL_NVIC_SetPriority(RTC_TAMP_IRQn, 0, 0);
        HAL_NVIC_EnableIRQ(RTC_TAMP_IRQn);
        /* USER CODE BEGIN RTC_MspInit 1 */
        __HAL_RCC_RTCAPB_CLK_ENABLE();
        /* USER CODE END RTC_MspInit 1 */
    }
}
```



- Click the “Build” button; or use menu Project > Build target.



- Click the “**Load**” button (F8) to flash the code into the STM32 (not using the debug session because we are using low power modes)



- Press Reset on your board (black button) once the code is loaded and the application will work as follows:
 - RUN** mode for 1 second (LD4 LED on)
 - STOP** mode for 5 seconds (LD4 LED off) with wakeup by RTC
 - If during the **STOP** mode (LD4 LED off) you press the user button: the interrupt (EXTI) will wakeup from STOP mode

Lab: Estimation of power consumption

Lab: Estimation of power consumption

147

Objective:

- Use the Power tool inside the STM32CubeMX to estimate the average power consumption of the low power lab we just finished.

Power Supply and Power Source Selection

148

- Using the “**lowpower**” project in STM32CubeMX
- Click on the **Tools** tab in STM32CubeMX
- Select **3V** for **VDD**



T_A 25°C / V_{DD} 3.0V

$T_{Ambient}$ 25°C

V_{DD} 3.0

- In the **Battery Selection** section, select **AA Alkaline** batteries (2 in series, 1 in parallel) as the power source for the application

Alkaline(AA LR6) (2x1)

Change Reset

In Series 2 In Parallel 1

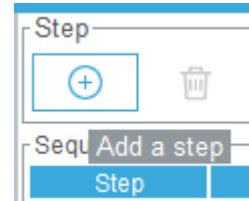
Capacity	2850.0 mAh
Self Discharge	0.3 %/month
Nominal Voltage	3.0 V
Max Cont Current	1000.0 mA

Adding a RUN mode step

149

- **Add a step to our power sequence:**

- Click: Step.. Add



- **Configure a first step: RUN mode**

- 8.0 MHz Range2 RUN mode
- from Flash at 3V from battery
- HSI clock
- Enable IPs from Pinout function
- Duration: 1 second
- Click “Add”

- Resulting step consumption should be 988.25uA

MX Edit Step

Enable All IPs Disable All IPs Enable IPs from Pinout

Power/Memory

Power Mode: RUN

Power Range: Range2-Medium

Memory Fetch Type: FLASH

V_{DD}: 3.0

Voltage Source: Battery

Enables the IPs selected in the Pinout tab

Peripherals

ADC1

- ☐ fs_10_ksp
- ☐ fs_1_Msp
- ☐ fs_2.5_Msp
- ☐ AHB_APB_Bridge
- ☐ BusMatrix
- ☐ CRC

DAC1

- ☐ OUT1+OUT2-Buffer
- ☐ OUT1+OUT2-Buffer
- ☐ OUT1+OUT2-Buffer
- ☐ OUT1-Buffer_OFF-
- ☐ OUT1-Buffer_ON-M
- ☐ OUT1-Buffer_ON-V
- ☐ OUT2-Buffer_OFF-
- ☐ OUT2-Buffer_ON-M
- ☐ OUT2-Buffer_ON-V

DBGMCU

DMA

FLASH

☒ GPIOA

GPIOB

☒ GPIOC

☐ GPIOD

Enabled Peripherals

GPIOA GPIOC RTC

SYS-VREFBUF/COMP1

Clocks

CPU Frequency: 8 MHz

Interpolation Ranges:

User Choice (Hz):

Clock Configuration: HSI

Clock Source Frequency: 8 MHz

Optional Settings

Step Duration: 1 s

Additional Consumption: 0 mA

Results

Step Consumption: 988.25 μ A

Without Peripherals: 935 μ A

Peripherals Part: 53.25 μ A (A: 8 μ A - D: 45.25 μ A)

Ta Max (°C): 129.81

Warnings

OK Cancel

Add a STOP1 mode step

150

- **Add a second step: STOP1 mode**

- STOP1 mode
- Flash is Power Down mode at 3V
- Clocks HSI 16 MHz
- RTC enabled (to wakeup the system)
- Duration: 5 seconds
- Click “Add”
- Resulting step consumption should be 3.4 μ A

MX New Step [X]

Enable All IPs | Disable All IPs | Enable IPs from Pinout

Power/Memory

Power Mode: STOP1

Power Range: NoRange

Memory Fetch Type: Flash-PowerDown

V_{DD}: 3.0

Voltage Source: Battery

Clocks

CPU Frequency: 16 MHz

Interpolation Ranges:

User Choice (Hz):

Clock Configuration: HSI

Clock Source Frequency: 16 MHz

Optional Settings

Step Duration: 5 s

Additional Consumption: 0 mA

Results

Step Consumption: 3.4 μ A

Without Peripherals: 3.4 μ A

Peripherals Part: 0 nA (A: 0 nA - D: 0 nA)

Ta Max (°C): 130

Warnings

Peripherals Selection

Peripherals

- ☐ IWDG*
- ☐ LPTIM1*
- ☐ LPTIM2*
- ☒ RTC*

Enabled Peripherals

RTC*

Add **Cancel**

Add a Wakeup from STOP1 mode step

151

- **Add a last step: Wakeup from STOP1 mode**

- VDD = 3V
- Voltage source: Battery
- Click “Add”
- Resulting step consumption should be 1.21 mA

MX New Step

Enable All IPs Disable All IPs Enable IPs from Pinout

Power/Memory

Power Mode	WU_FROM_STOP1
Power Range	NoRange
Memory Fetch Type	Flash-PowerDown
V _{DD}	3.0
Voltage Source	Battery

Clocks

CPU Frequency	16 MHz
Interpolation Ranges	
User Choice (Hz)	
Clock Configuration	HSI
Clock Source Frequency	16 MHz

Optional Settings

Wakeup time	9.0	μs
Additional Consumption	0	mA

Results

Step Consumption	1.21 mA
Without Peripherals	1.21 mA
Peripherals Part	0 nA (A: 0 nA - D: 0 nA)
Ta Max (°C)	129.76

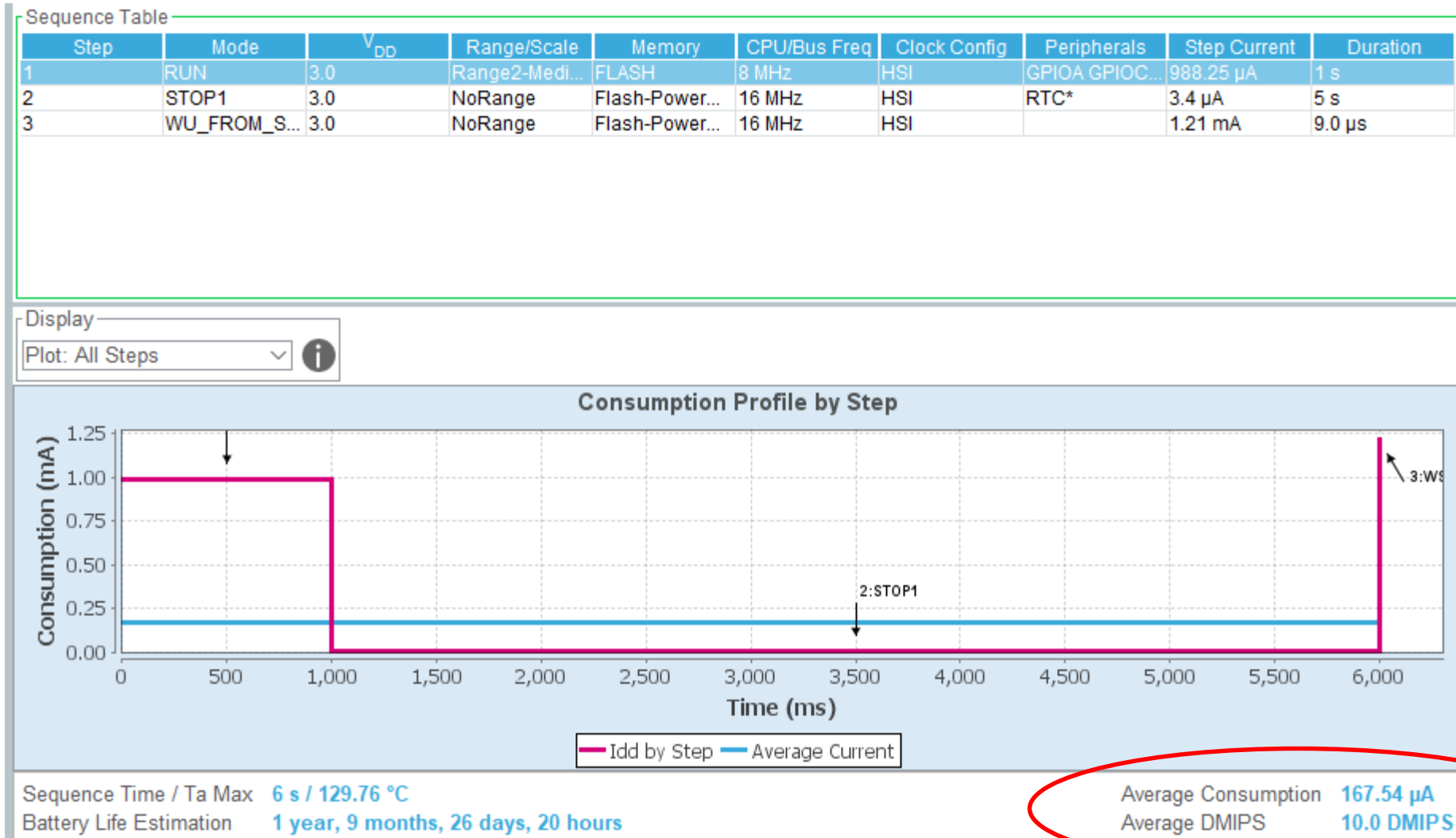
Warnings

Peripherals Selection – Peripherals

Average Current Consumption Result

152

- Note: the Current consumption numbers are for the MCU only.



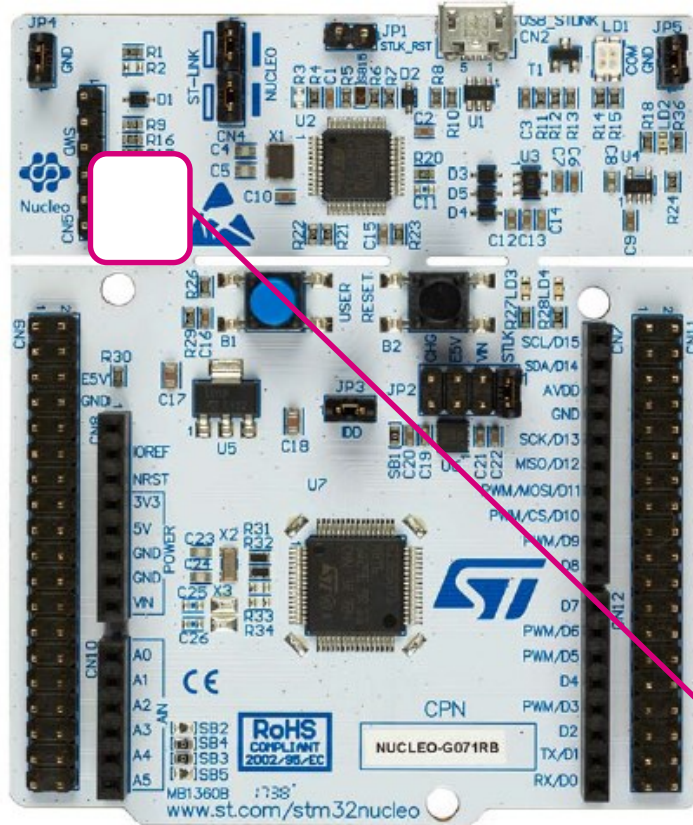
Lab: printf() debugging using UART

Lab: printf() debugging using UART

154

Objective:

- Redirect Printf output to LPUART1 which is connected to the ST-LINK Virtual COM port on the Nucleo board
- Using a Terminal like Teraterm we can view the printf output.



LPUART1 debug will be used via the ST-LINK Virtual-COM port

Set up additional GPIO / Clocks:

PA2 – LPUART1, “LPUART1-TX”

PA3 – LPUART1, “LPUART1-RX”

LPUART1 Clock = PCLK1 (64MHz)

LPUART1 settings:

Asynchronous Mode - 115200 N/8/1, No HW Flow control
Tx/Rx, No advanced features

Teraterm Terminal will be used to display the printf output

LPUART1 is routed to the ST-LINK's USART, and brought via the USB Virtual-COM port class (SB16/18 located on the back on the board have been soldered)

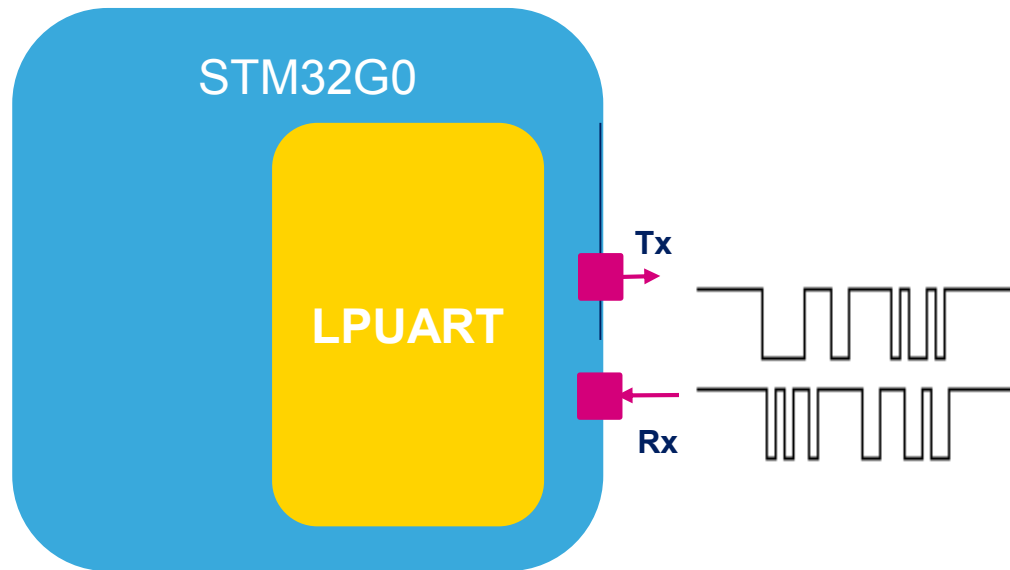
STM32G0 USART/LPUART features

156

USART features	USART1/2	USART3/4	LPUART1
Hardware flow control for modem	X	X	X
Multiprocessor communication	X	X	X
Synchronous mode (Slave/Master)	X	X	-
Smartcard mode	X	-	-
Single wire half duplex communication	X	X	X
IrDA SIR ENDEC	X	-	-
LIN mode	X	-	-
Dual clock domain and wakeup from Stop mode	X	-	X
Receiver timeout	X	-	-
Auto baudrate detection	X	-	-
Driver enable	X	X	X
Data length	7, 8 and 9 bits		
TX/RX FIFO	X	-	X
TX/RX FIFO size (data word)	8	-	8

- Transmit FIFO (TXFIFO) and Receive FIFO (RXFIFO)
- Transmission/Reception even during stop modes
- FIFO mode is enabled/disabled by software
- TXFIFO and RXFIFO are each 8 data words in length
- Adjustable TXFIFO and RXFIFO interrupt request thresholds

- LPUART (Low Power Universal Asynchronous Receiver/Transmitter)
 - Full UART communication at 9600 baud with wakeup from stop modes capability when using the low-speed 32.768 kHz external oscillator (LSE).
- Higher baud rates are available with other clock sources.

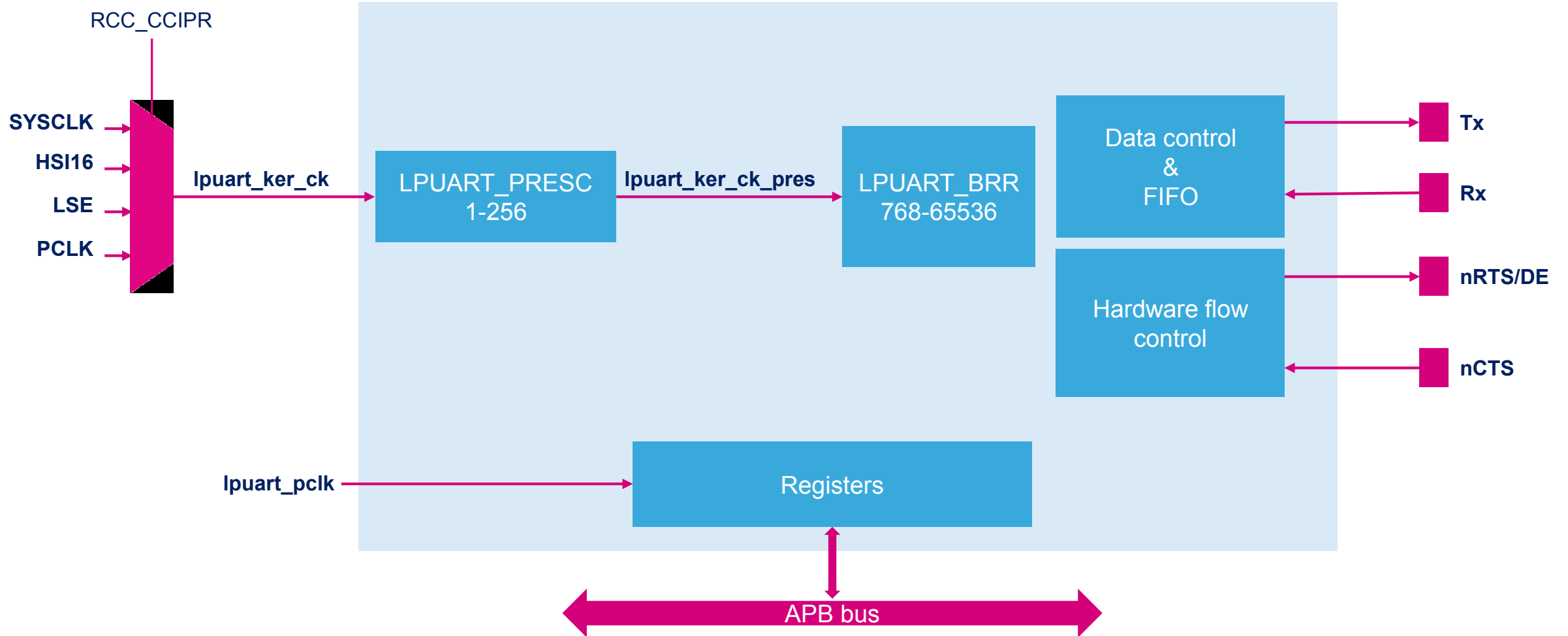


Application benefits

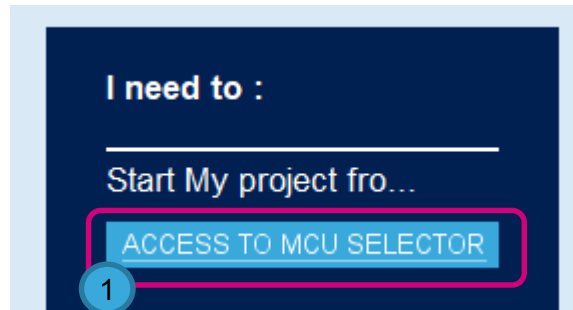
- Inexpensive communication link between devices
- Simple hardware, only a few pins needed
- Wakes from low-power STOP modes
- Transmit and Receive FIFOs, with capability to transmit and receive in stop modes.

LPUART Block diagram

159



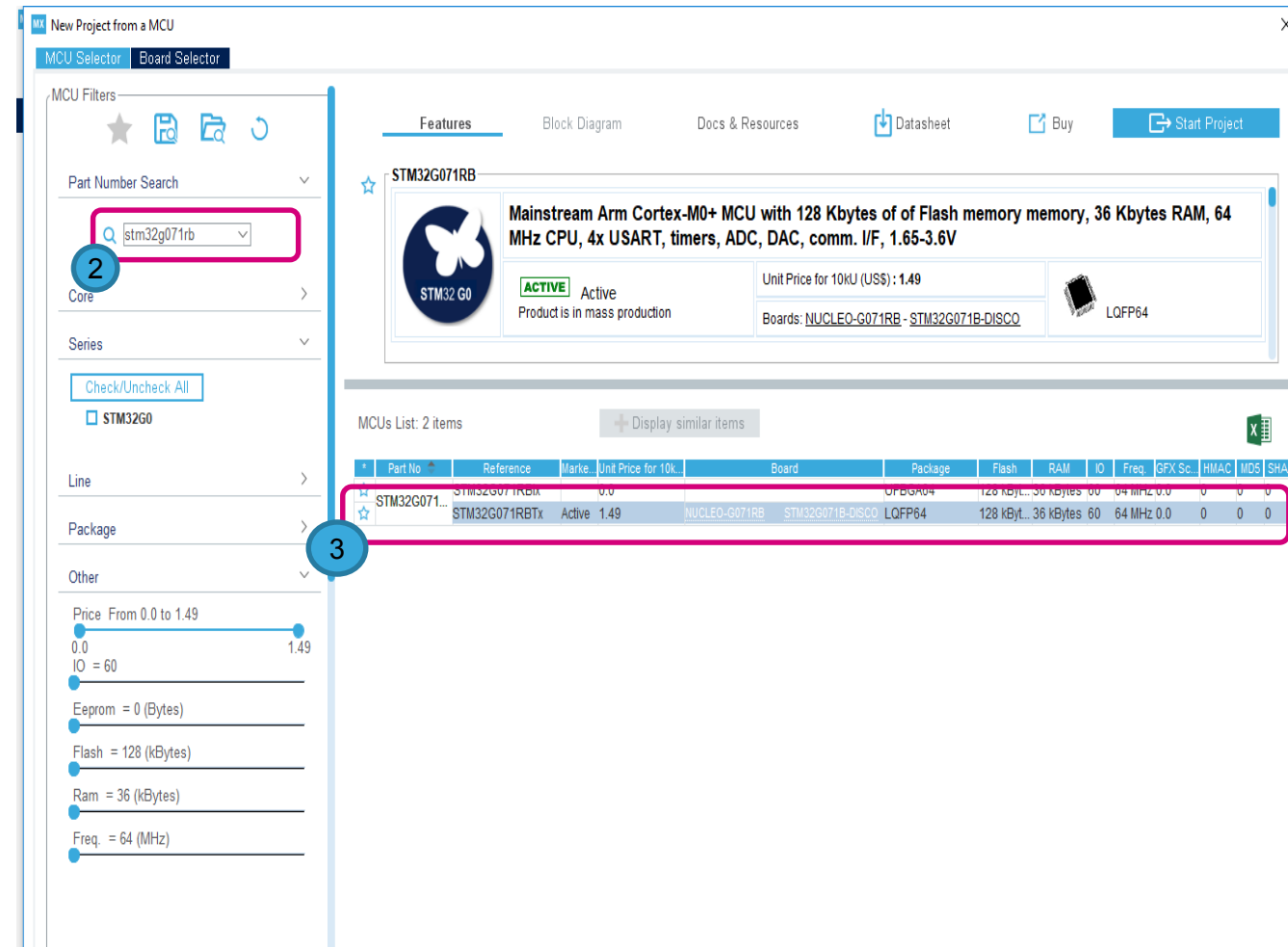
- In STM32CubeMX, click “Home”
- Click **Access To MCU Selector** ①



- Select **STM32G071RBTx** ②
 - LQFP64, 128KB Flash
- Double Click ③



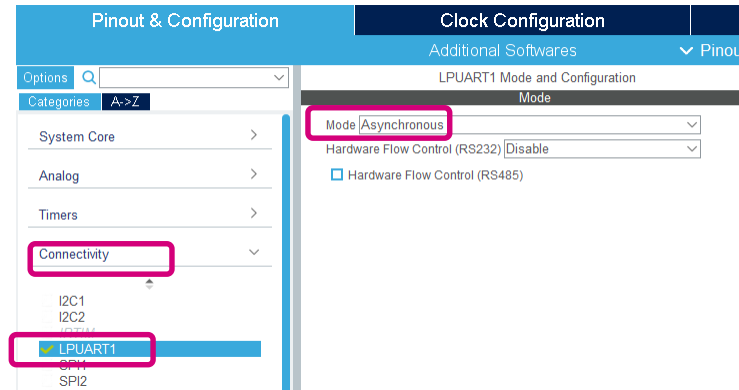
Home /



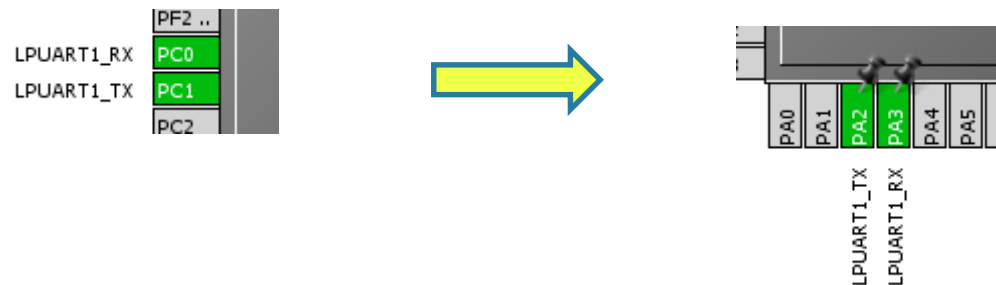
GPIO Configuration additions

161

- Click on **LPUART1** dialog (under Connectivity), and select **Asynchronous** mode:



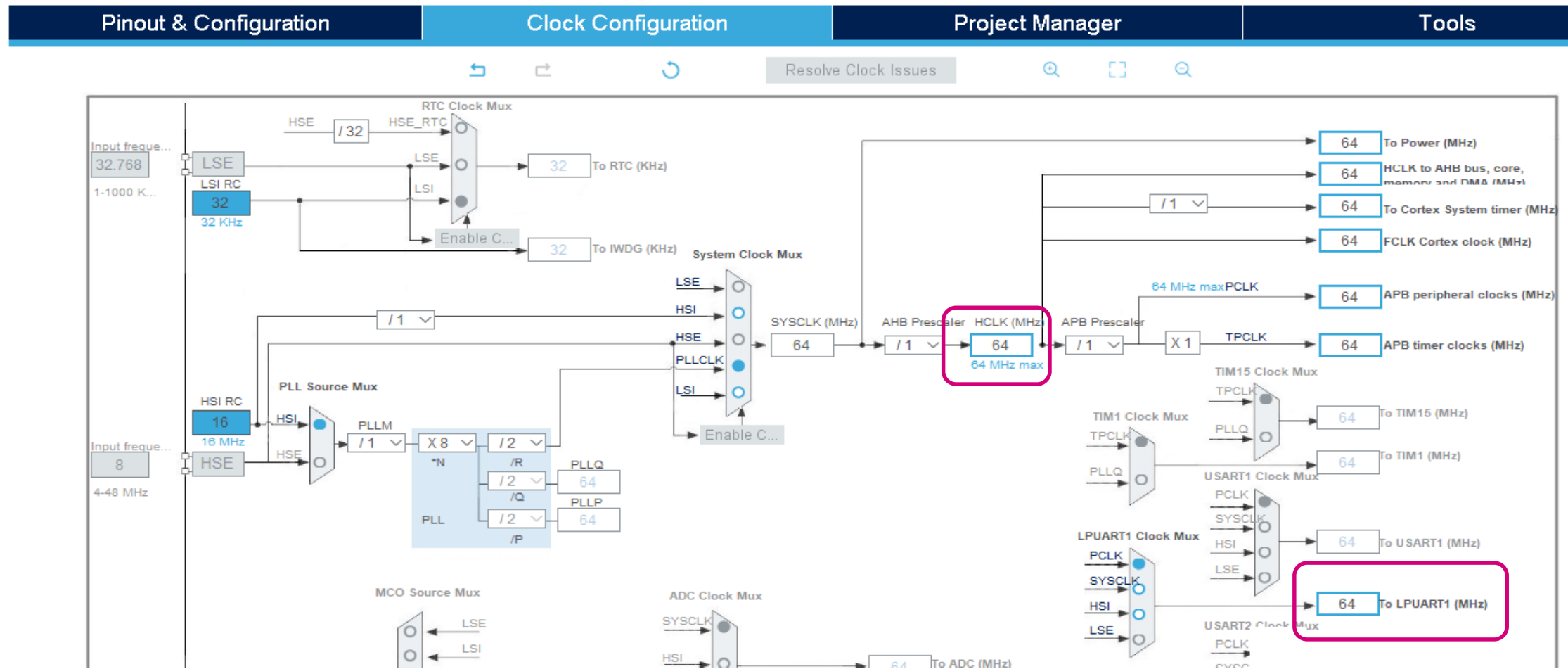
- Use PA2 & PA3 for Tx / Rx pins:
 - These are the alternate mapping pins (PC0/PC1 are default)
 - So need to remap



Clock Configuration

162

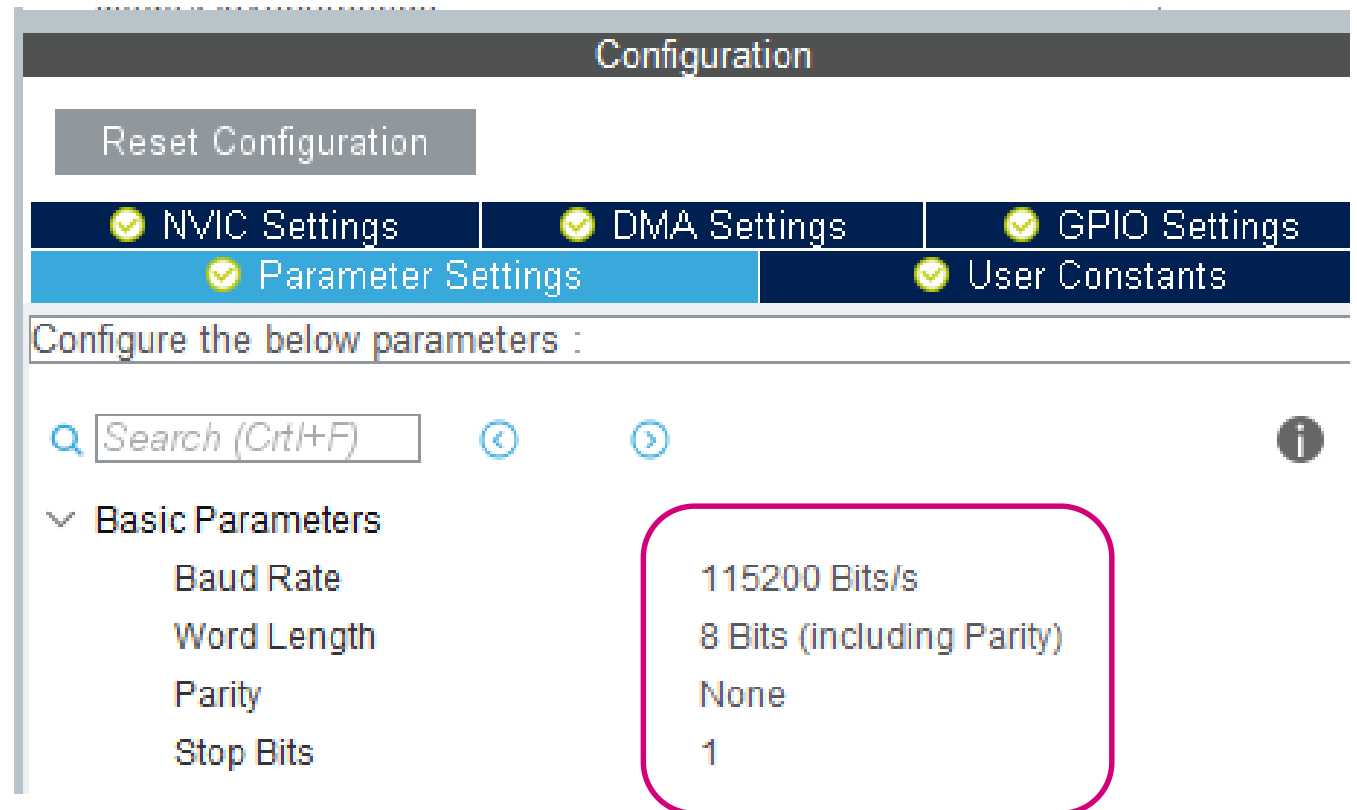
- Run the STM32G0 at 64 MHz for this lab, the LPUART1 clock also at 64 MHz.



- Click on the Configuration tab and select LPUART1

- Parameter Settings tab

- 115200 Bits/s
- 8-bit word length
- No parity bit
- 1 Stop bit
- Keep Default settings for the rest



Generate Source Code

164

- Open **Project Manager**
- Set the project name (**printf**) and the project location (**C:\STM32G0Workshop\STM32G0\HandsOn**)
- Set the IDE Toolchain to **MDK-ARM V5**

Project Manager

1 Project Name
printf

2 Project Location
C:\STM32G0Workshop\HandsOn

Application Structure
Basic ☐ Do not generate the ma...

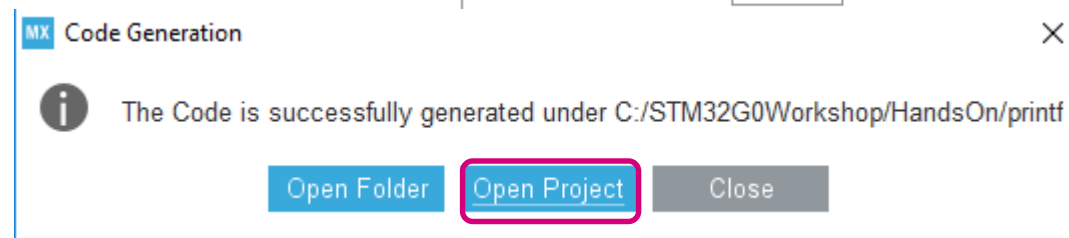
Toolchain Folder Location
C:\STM32G0Workshop\HandsOn\printf\

3 Toolchain / IDE
MDK-ARM V5 ☐ Generate Under Root

Linker Settings
Minimum Heap Size 0x200
Minimum Stack Size 0x400



- **Generate Code**
- Click **Open Project**



Adding printf redirecting code in main.c

165

1- Add the stdio include:

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
/* USER CODE END Includes */
```

```
44  /* Private includes -----
45  /* USER CODE BEGIN Includes */
46  #include <stdio.h>
47  /* USER CODE END Includes */
```

2- Add following code in the section below:

```
/* USER CODE BEGIN PFP */
/* Private function prototypes -----*/
#define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
/* USER CODE END PFP */
```

```
75  /* USER CODE BEGIN PFP */
76  #define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
77  /* USER CODE END PFP */
```

3- Add following function in the section below:

```
/* USER CODE BEGIN 4 */
PUTCHAR_PROTOTYPE
{
    HAL_UART_Transmit(&hlpuart1, (uint8_t *)&ch, 1, 0xFFFF);
    return ch;
}
/* USER CODE END 4 */
```

```
241  /* USER CODE BEGIN 4 */
242  PUTCHAR_PROTOTYPE
243  {
244      HAL_UART_Transmit(&hlpuart1, (uint8_t *)&ch, 1, 0xFFFF);
245      return ch;
246  }
247  /* USER CODE END 4 */
```

Adding application code in main.c

166

Add application code in main loop:

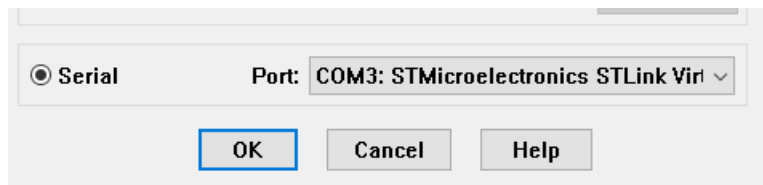
```
/* USER CODE BEGIN WHILE */
while (1)
{
    printf("*** Hello World ** \n\r")
    HAL_Delay(1000);
/* USER CODE END WHILE */
```

```
117  /* Infinite loop */
118  /* USER CODE BEGIN WHILE */
119  while (1)
120  {
121      printf("*** Hello World ** \n\r");
122      HAL_Delay(1000);
123
124      /* USER CODE END WHILE */
125
126      /* USER CODE BEGIN 3 */
127  }
128  /* USER CODE END 3 */
129  }
```

Build the Project and run the application

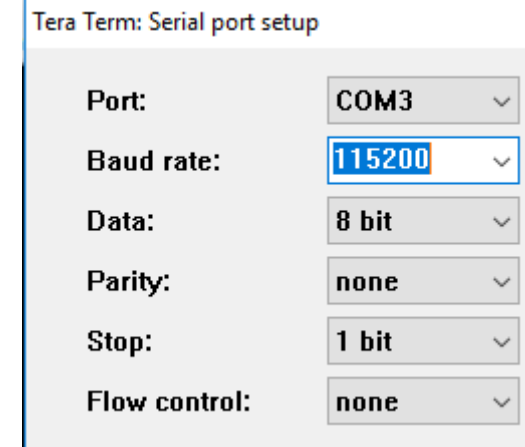
167

- Click the “Build” button; or use menu Project > Build target.
- Click the “Start/Stop Debug Session” button
- Click “Run” button
- Open a Terminal emulator like TeraTerm, using LPUART1 settings, connect ST-LINK Virtual COM port xx

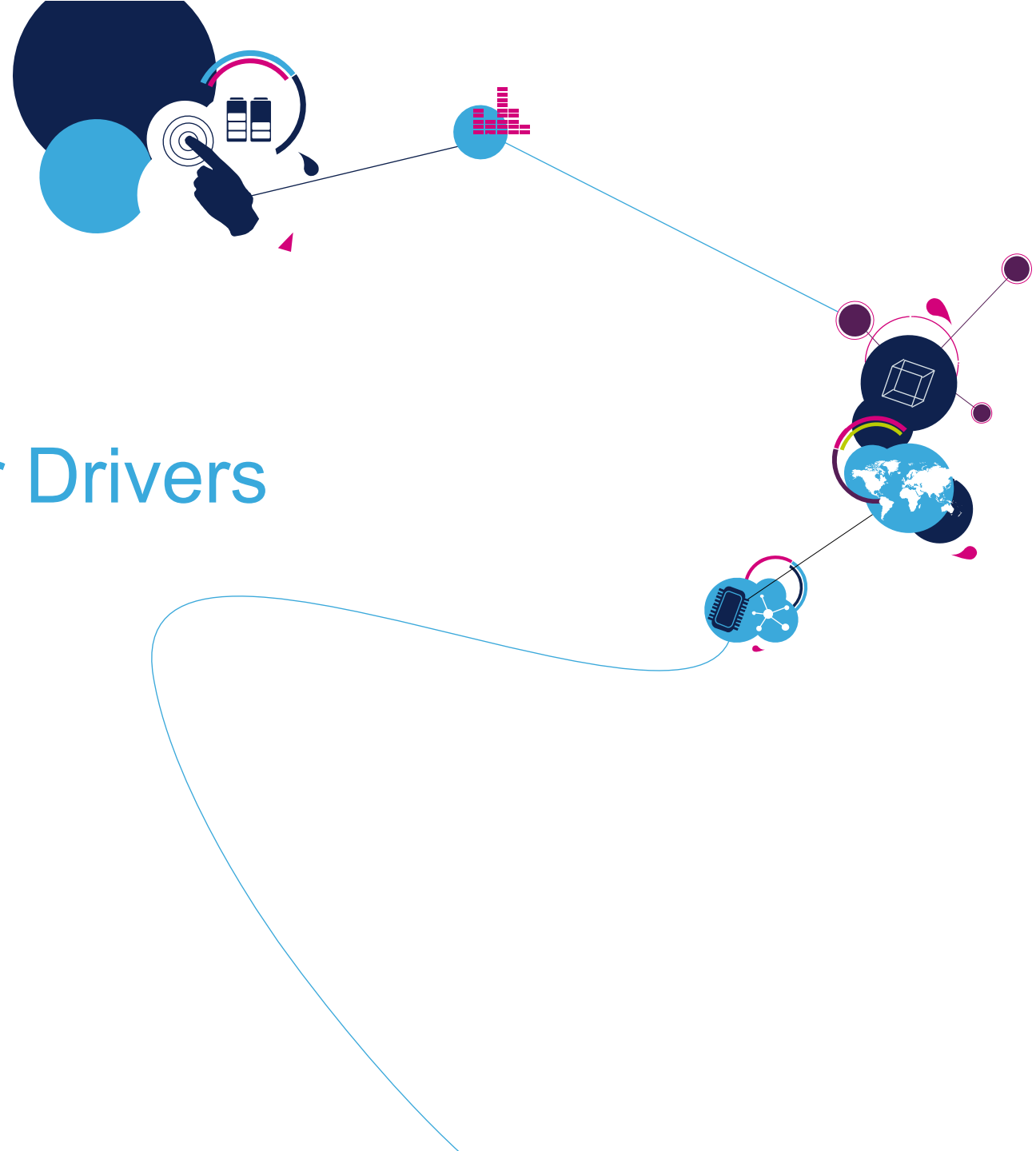


- You should see the printf message being displayed.

```
## Hello World ##
## Hello World ##
## Hello World ##
## Hello World ##
## Hello World ##
## Hello World ##
## Hello World ##
## Hello World ##
## Hello World ##
## Hello World ##
## Hello World ##
## Hello World ##
## Hello World ##
## Hello World ##
## Hello World ##
```



STM32Cube Low Layer Drivers



- STM32Cube HAL & LL are complementary and covers a wide range of applications requirements:
 - HAL offers high level and functionalities oriented APIs, with high portability level and hide product/IPs complexity to end user
 - LL offers low level APIs at registers level, w/ better optimization but less portability and require deep knowledge of the product/IPs specification

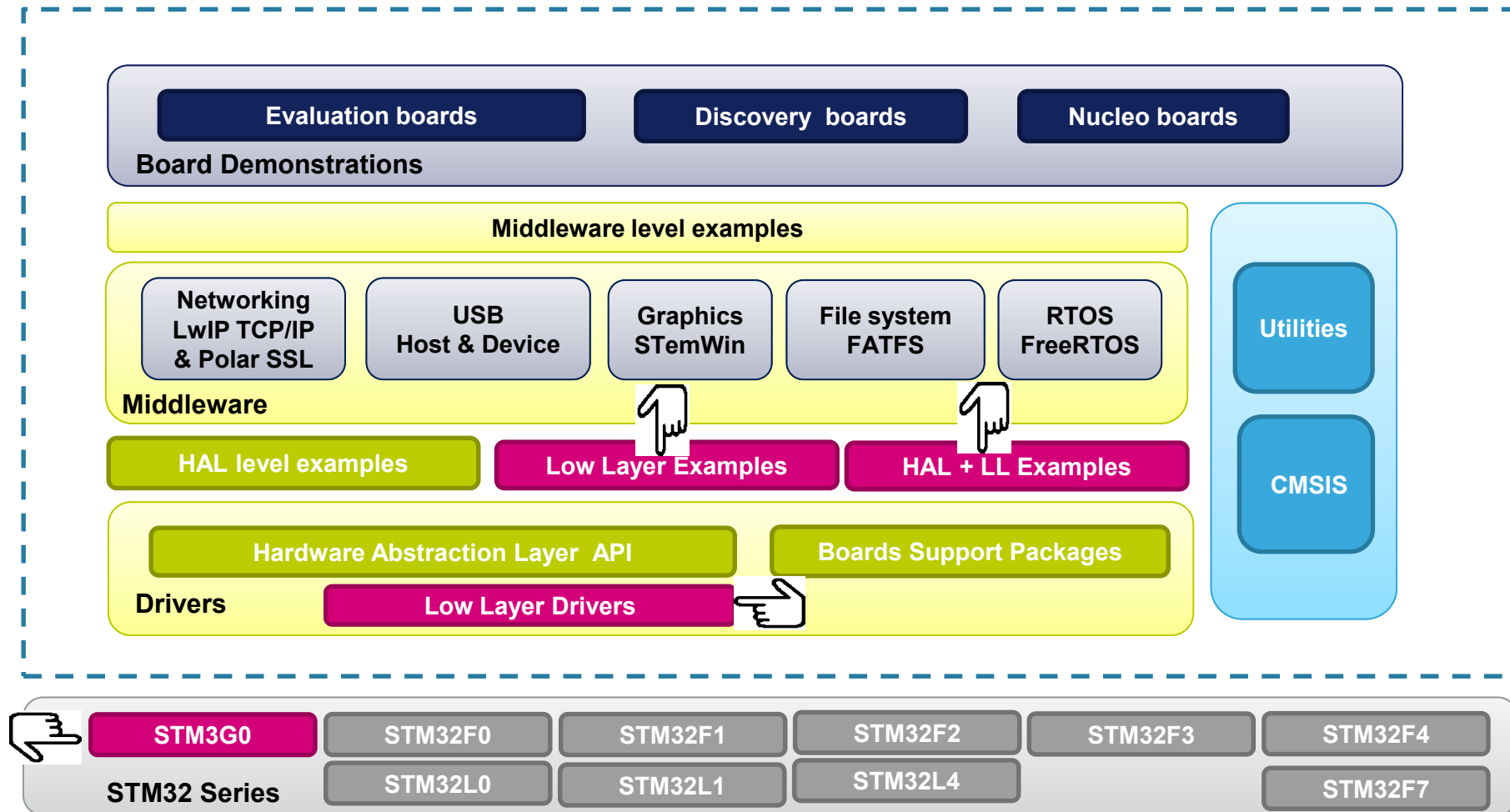
Low Layer (LL) Library Features

170

- The Low Layer (LL) Library offers the following services:
 - Set of static inline function for direct register access (provided in *.h files only)
 - One-shot operations that can be used by the HAL drivers or from application level.
 - Independent from HAL and can be used standalone (without HAL drivers)
 - Full feature coverage of the supported peripherals
- The LL APIs are Not Fully Portable across the STM32 families; the availability of some macros depends on the physical availability of the relative feature on the product
- Most STM32 Peripherals covered
- Same standard compliancy as HAL (MISRA-C, ANSIC...)
- Low Layer (LL) is available in STM32CubeMX i.e. user can choose between HAL and LL by Peripheral

STM32Cube FW package block view

171



Benchmark- USART transmit Example

172

- The below data are based on the “USART Transmitter IT” example:
 - Configure GPIO & USART peripheral for sending characters to HyperTerminal (PC) in Asynchronous mode using IT
 - Using below configuration:
 - Platform: STM32L486xx
 - Compiler : IAR
 - Optimization : High Size
 - Heap Size = 512 Bytes / Stack Size = 512 Bytes

	HAL Drivers	Low layer Drivers
read-only code memory (Bytes)	7206	2154
read-only data memory (Bytes)	204	94
read write data memory (Bytes) (*)	1408	1093

ROM Size divided by ~4

RAM Size reduction

- **LL** offer smaller footprint & high performance but less portability & require expertise
- **HAL** offer high level API (hide complexity) & portability but higher footprint & less performance

Lab: Using the Low Layer (LL) Drivers

Objective:

- Generate a project with STM32CubeMX using the Low Layer Drivers and check how much improvement we get compared to a HAL project in term of Flash and RAM usage

- Close Keil uVision5 IDE if it is open; In STM32CubeMX Open the project (“**blinky**”) and save it as a new project name like “**blinky_II**” through File -> Save Project As

- In Project Manager Tab 1
- In the Advanced Settings Tab 2
- Instead of HAL drivers select LL (Low Level) for both RCC and GPIO 3

The screenshot shows the STM32CubeMX Project Manager interface. The top bar includes the STM32CubeMX logo, a menu (File, Window, Help), and a 'GENERATE CODE' button. The main window has four tabs: 'Pinout & Configuration', 'Clock Configuration', 'Project Manager' (selected and highlighted with a pink box), and 'Tools'. The 'Project Manager' tab displays a 'Driver Selector' section with a search bar and a list of drivers: 'RCC' and 'GPIO'. Both 'RCC' and 'GPIO' are selected with 'LL' (Low Level) drivers, indicated by a pink box and a circled '3'. Below the driver selector is a table titled 'Generated Function Calls'.

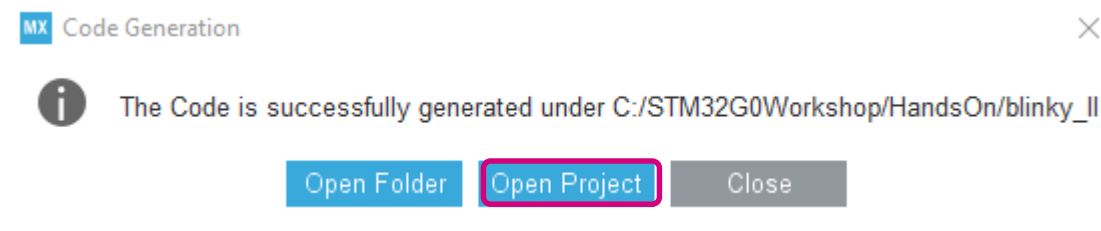
Rank	Function Name	IP Instance Name	☐ Not Generate Function Call	☐ Visibility (Static)
1	MX_GPIO_Init	GPIO	☐	☒
2	SystemClock_Config	RCC	☐	☐

On the left side of the interface, there is a vertical sidebar with three tabs: 'Project', 'Code Generator', and 'Advanced Settings' (highlighted with a pink box and a circled '2').

- **Generate Code**



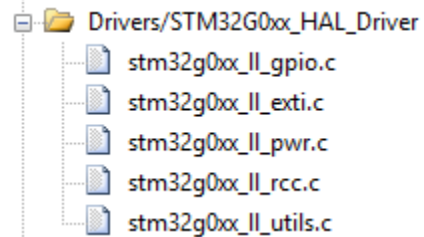
- **Click Open Project**



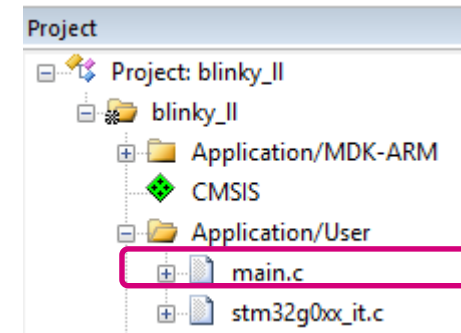
Step 4: Toggle The LED

178

- In Keil uVision5 IDE
- Expand the “Drivers” and notice that now the drivers are low layer (_LL):



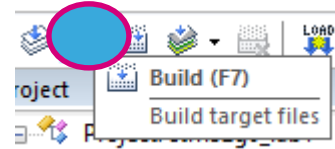
- Expand the file tree and open the **main.c** file
- Add the following code **inside the while(1) loop**
 - Add within “USER CODE BEGIN WHILE” / “USER CODE END WHILE” section (this will preserve your code after code regeneration)
(~Line 123 in “main.c”)



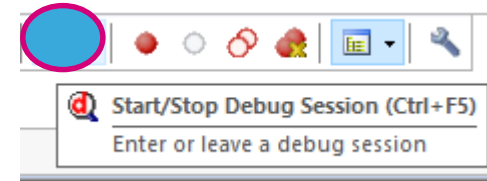
```
LL_GPIO_TogglePin(GPIOA, LL_GPIO_PIN_5);  
// Delay 100 ms  
LL_mDelay(100);
```

```
119  /* Infinite loop */  
120  /* USER CODE BEGIN WHILE */  
121  while (1)  
122  {  
123      LL_GPIO_TogglePin(GPIOA, LL_GPIO_PIN_5);  
124      // Delay 100 ms  
125      LL_mDelay(100);  
126      /* USER CODE END WHILE */
```

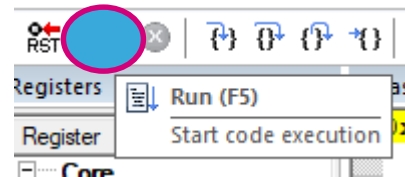
- Click the “Build” button



- Click the “Start/Stop Debug Session” button



- Click “Run” button

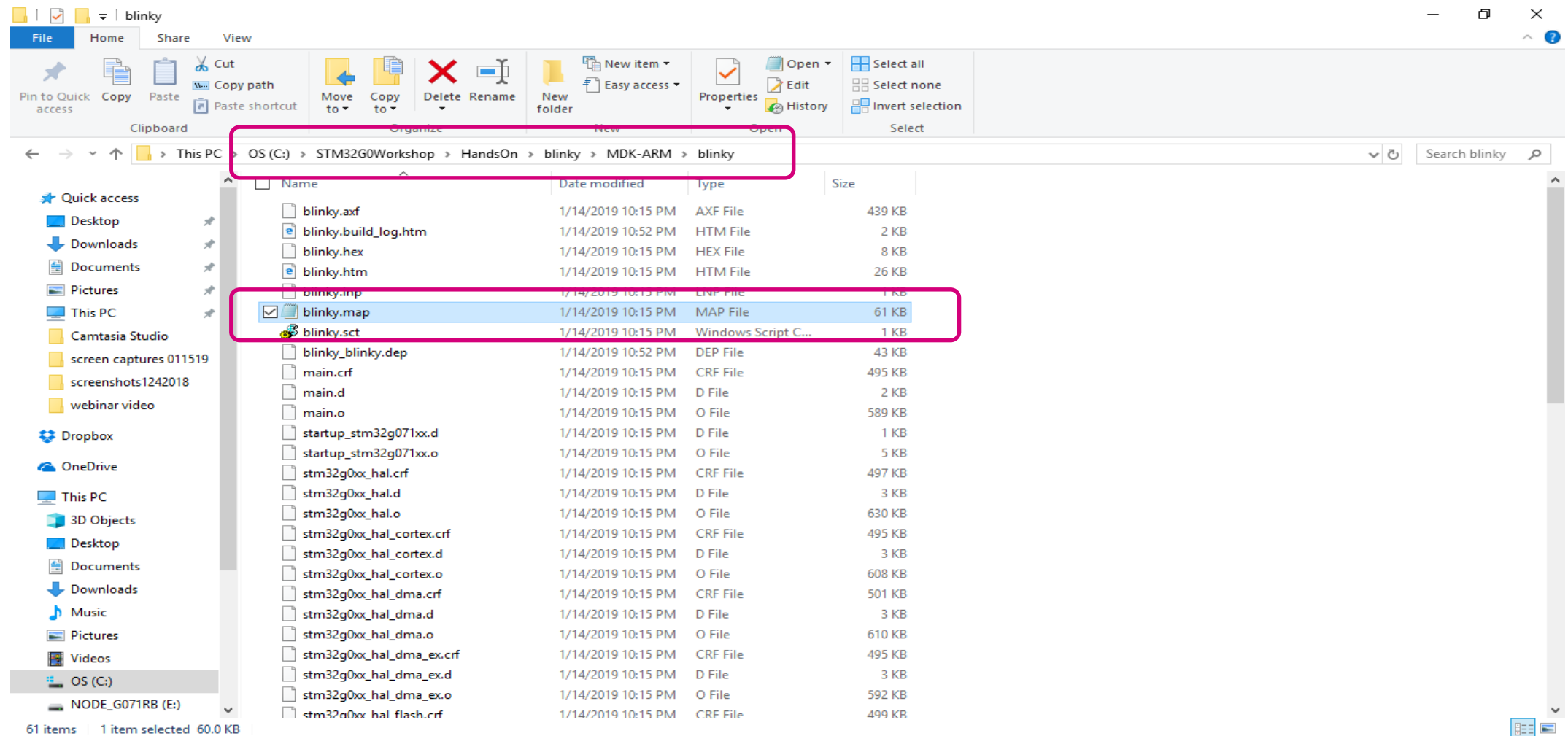


- The Green LED (LD4) should be blinking just like the blinky example with the HAL drivers

Let's compare the map files between HAL and LL projects

180

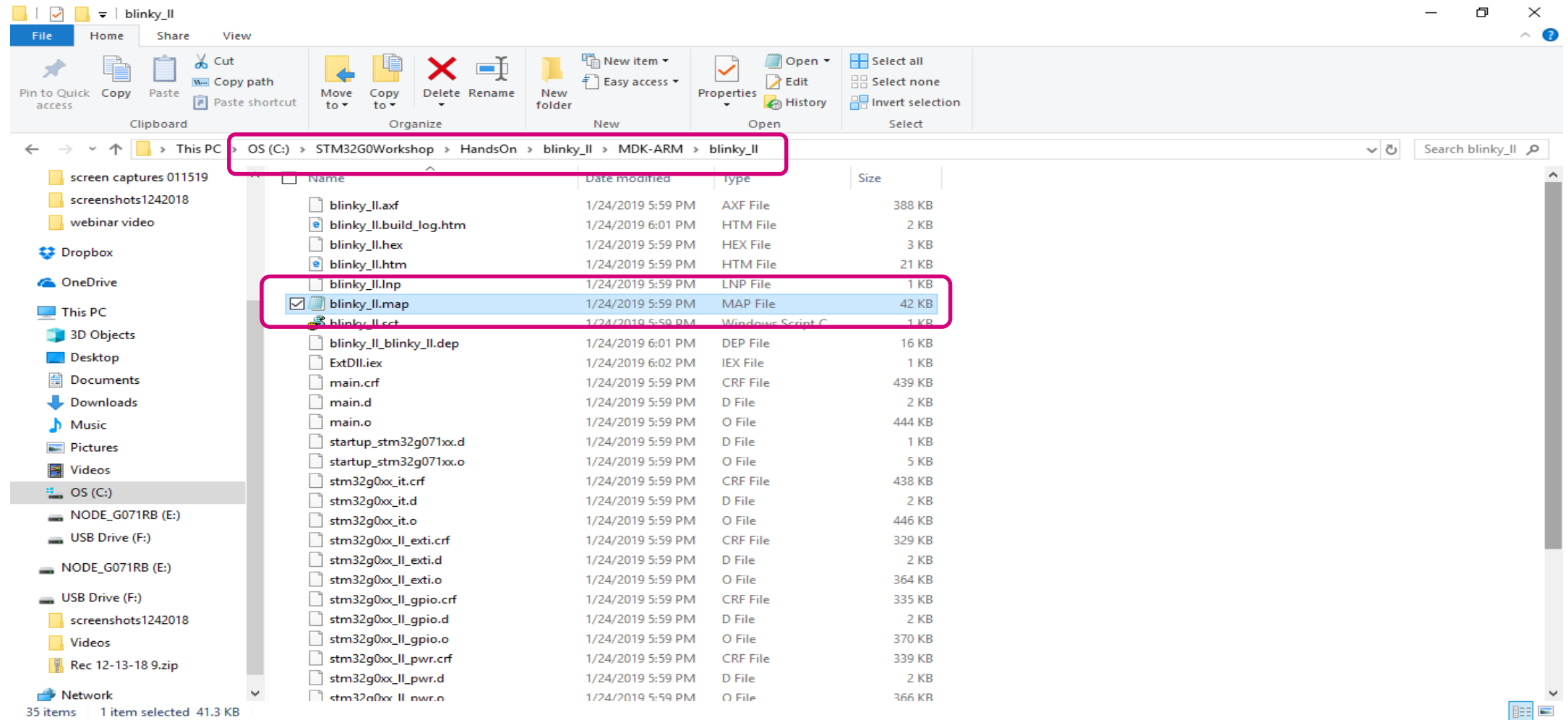
Use a text editor to open the map file for the “blink” project that is using HAL:”blinky.map” located here:



Let's compare the map files between HAL and LL projects

181

Open the map file for the “blinky_ll” project that is using LL: “blinky_ll.map” located here:



Let's compare the Flash and RAM size first

182

Using HAL:

Using LL:

blinky.map - Notepad

File	Edit	Format	View	Help		
44	0	0	0	0	72	uidiv.o

168	16	0	0	0	240	Library
Totals	2	0	0	0	0	(incl. Padding)

Code (inc. data)	RO Data	RW Data	ZI Data	Debug		Library Name
166	16	0	0	240		mc_p.1

168	16	0	0	240		Library
Totals						

Code (inc. data)	RO Data	RW Data	ZI Data	Debug		Library Name
2532	174	284	16	1024	415174	Grand Totals
2532	174	284	16	1024	415174	ELF Image
Totals	2532	174	284	16	0	ROM Totals
=====						
Total RO Size (Code + RO Data)				2816 (	2.75kB)	
Total RW Size (RW Data + ZI Data)				1040 (	1.02kB)	
Total ROM Size (Code + RO Data + RW Data)				2832 (	2.77kB)	
=====						

blinky_ll.map - Notepad

File	Edit	Format	View	Help		
30	0	0	0	0	0	handlers.o
36	8	0	0	0	68	init.o
36	0	0	0	0	100	memset.o
44	0	0	0	0	72	uidiv.o

168	16	0	0	0	240	Library Totals
2	0	0	0	0	0	(incl. Padding)

Code (inc. data)	RO Data	RW Data	ZI Data	Debug		Library Name
166	16	0	0	240		mc_p.1

168	16	0	0	240		Library Totals

Code (inc. data)	RO Data	RW Data	ZI Data	Debug		Library Name
680	66	220	4	1028	376886	Grand Totals
680	66	220	4	1028	376886	ELF Image
Totals	680	66	220	4	0	ROM Totals
=====						
Total RO Size (Code + RO Data)				900 (	0.88kB)	
Total RW Size (RW Data + ZI Data)				1032 (	1.01kB)	
Total ROM Size (Code + RO Data + RW Data)				904 (	0.88kB)	
=====						

Scroll to the bottom of the files



Comparison table between HAL and LL for our “Blinky” code

183

- Using below configuration:
 - Heap Size = 512 Bytes / Stack Size = 512 Bytes
 - STM32Cube G0 1.0.0
 - uVision 5.26
 - CubeMX 5.0.1

	HAL Drivers	Low layer Drivers
read-only code memory (Bytes)	2816	900
read write data memory (Bytes) (*)	$1040 - 1024(**) = 16$	$1032 - 1024(**) = 8$

ROM Size divided by ~3

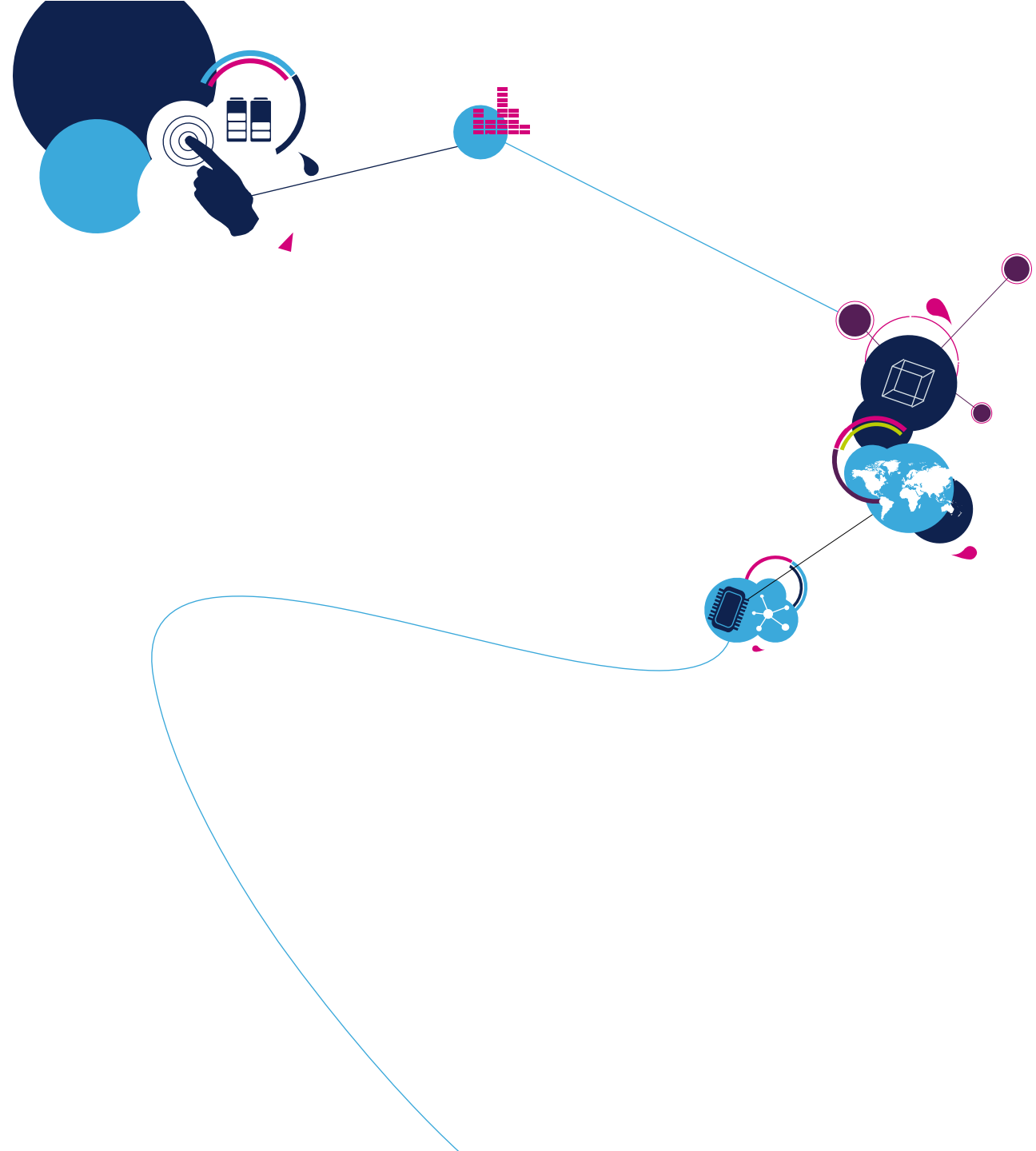
RAM Size reduction divided by 2

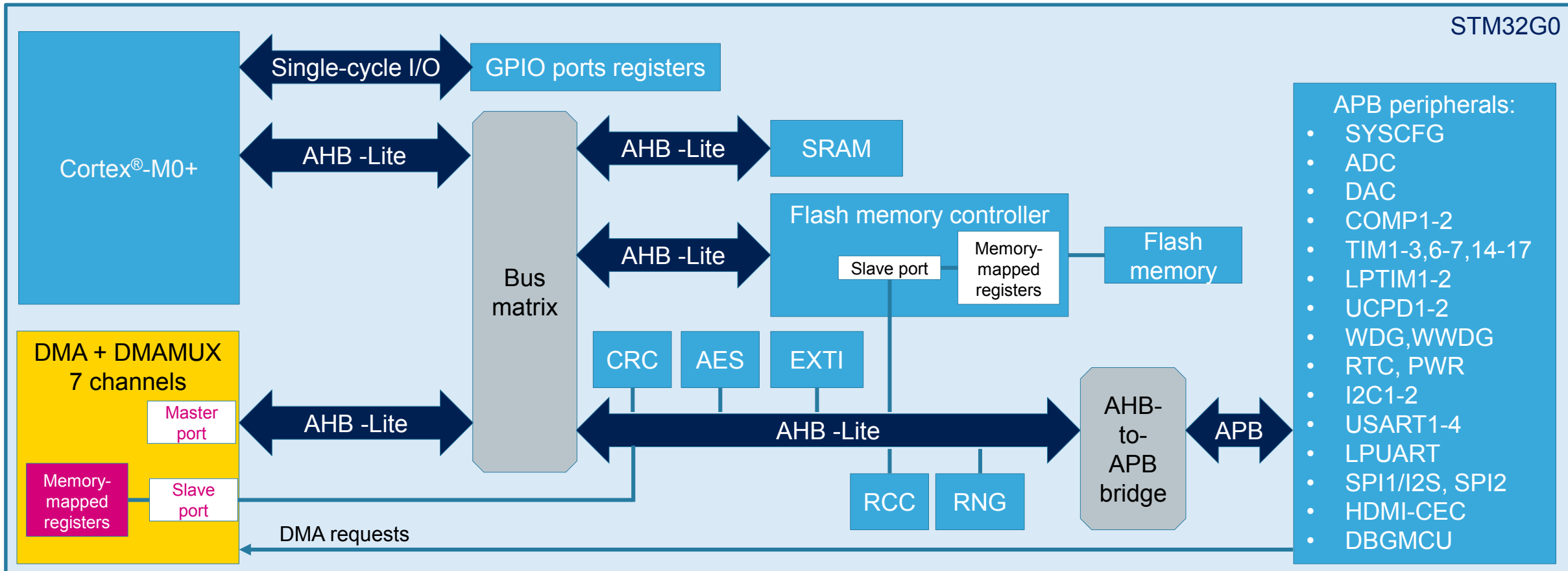
- **LL** offer smaller footprint & high performance but less portability & require expertise
- **HAL** offer high level API (hide complexity) & portability but higher footprint & less performance

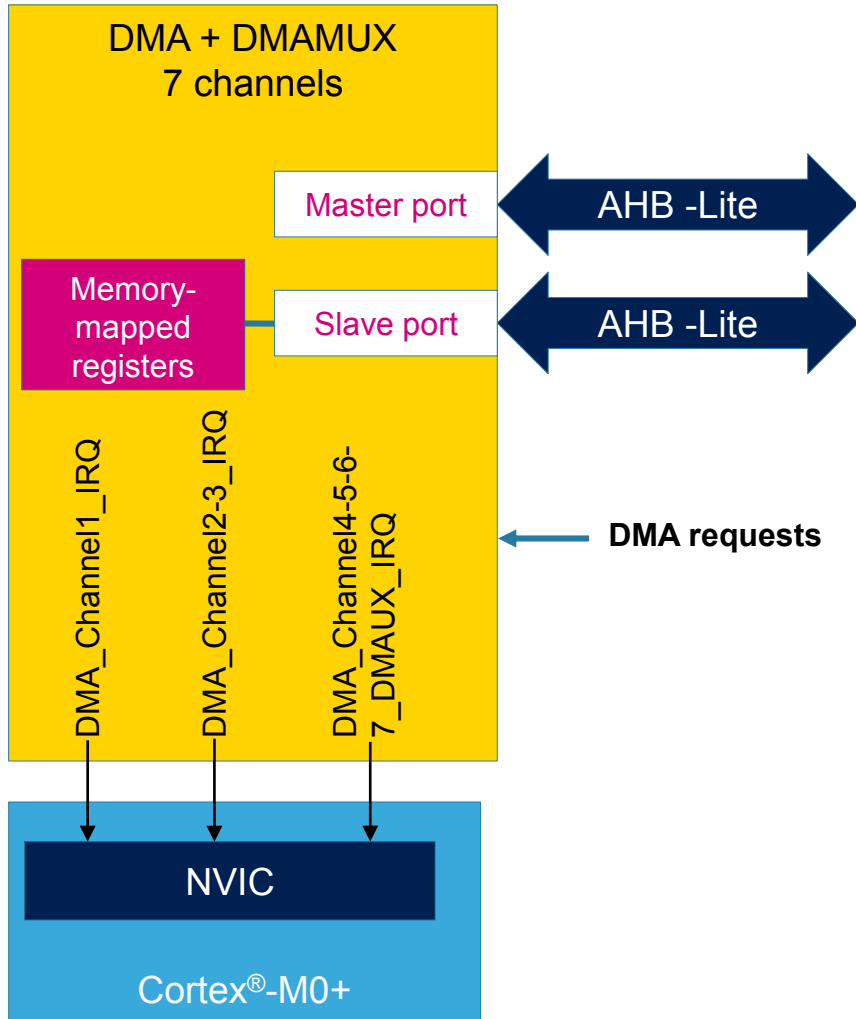
(*) to add Heap and Stack size for total RAM

(**) $1024 = 512 \text{ bytes Heap size} + 512 \text{ bytes Stack size}$

STM32G0 – DMA







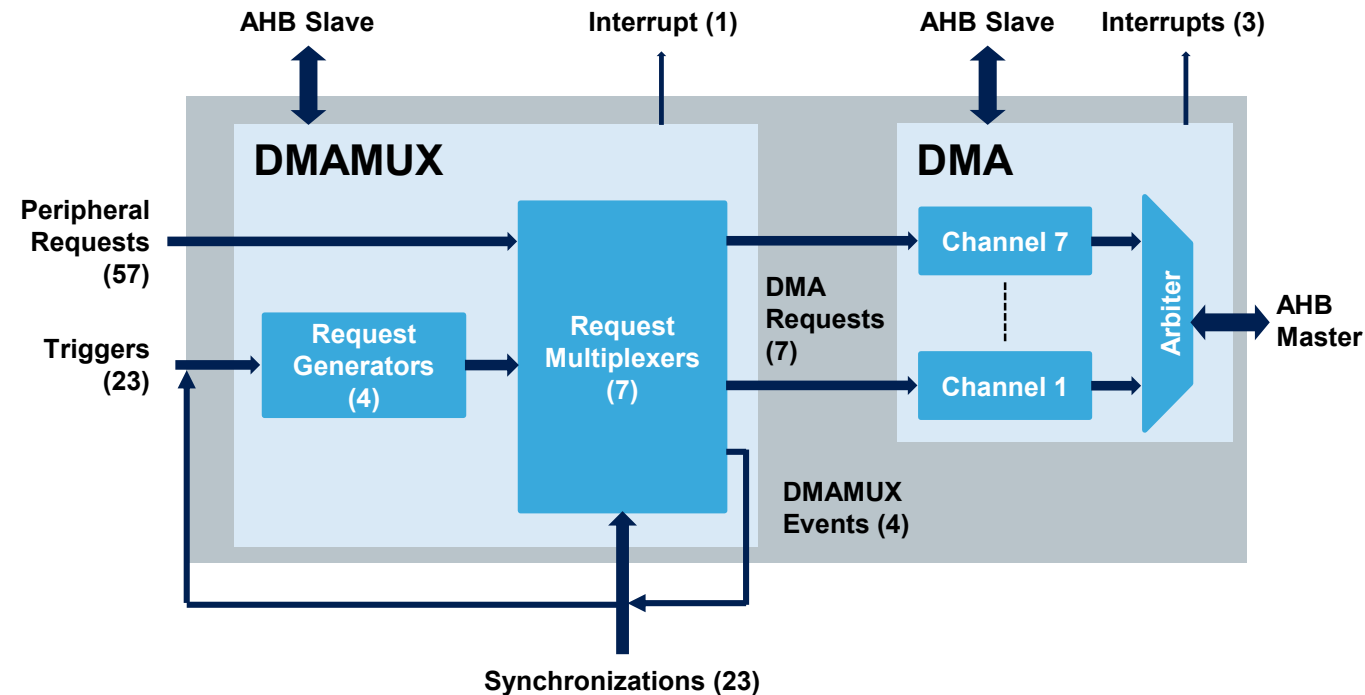
- DMA features

- AHB master bus
- Flexible configuration
- Hardware and software priority management
- Configurable data transfer modes
 - Peripheral-to-Peripheral, Peripheral-to-Memory, Memory-to-Peripheral, and Memory-to-Memory modes

Application benefits

- DMA support for timers, ADC, and communication peripherals
- Offloads CPU from data transfer management
- Simple integration

- STM32G0 DMA features
 - 1x DMA controller
 - Programmable block transfers with 7 concurrent channels, independently configurable
 - Programmable channel-based priority
 - Data transfers via the AHB master port (connected to the bus matrix)
 - 1x new DMA request multiplexer (DMAMUX)
 - Programmable mapping of a DMA request e.g. from any peripheral
 - Event-triggered & synchronized DMA request generation

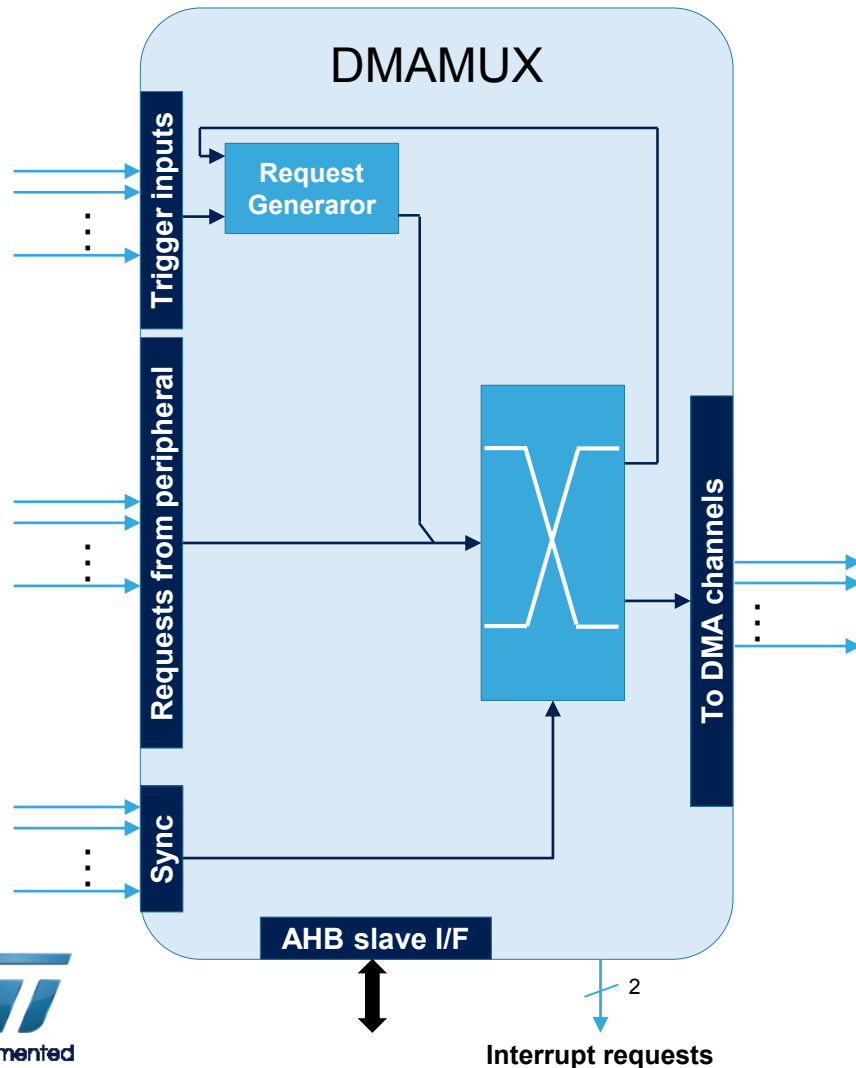


DMA - Main differences with STM32F0

188

- The DMA Controller is similar to the one implemented in STM32F0 microcontrollers, but with the additional DMA request multiplexer (DMAMUX)

	STM32F0	STM32G0
DMA	2 DMAs	1 DMA
DMA Features	Same	
DMAMUX	No	Yes



- The DMA request router (DMAMUX) manages:
 - The assignment of DMA request lines to peripherals
 - The request forwarding synchronization with events on synchronization inputs
 - The request chaining using the DMA request counter and Event generator for DMA

Application benefits

- High flexibility in choice of DMA request mapping
- External and internal DMA request management
- Request synchronization
- Request chaining capability

- DMAMUX is a DMA request multiplexer/router
 - DMAMUX provides a programmable routing of any of the 7 DMA (hardware) requests from any peripheral request
- Additionally, there are 4 request generator channels
 - Software can configure a DMA request to be generated by the DMAMUX itself, upon a trigger input
 - Are programmable:
 - The trigger selection: EXTI0..15, LPTIM1/2OUT, TIM14_OC, or any of the 4 generated DMAMUX events
 - The trigger event: rising edge, falling edge or either edge
 - The number of generated DMA requests upon the trigger event
 - There is a trigger overrun flag & interrupt in order to alert the software when the number of generated DMA requests (as paced by the DMA) have not been completed before a next trigger event

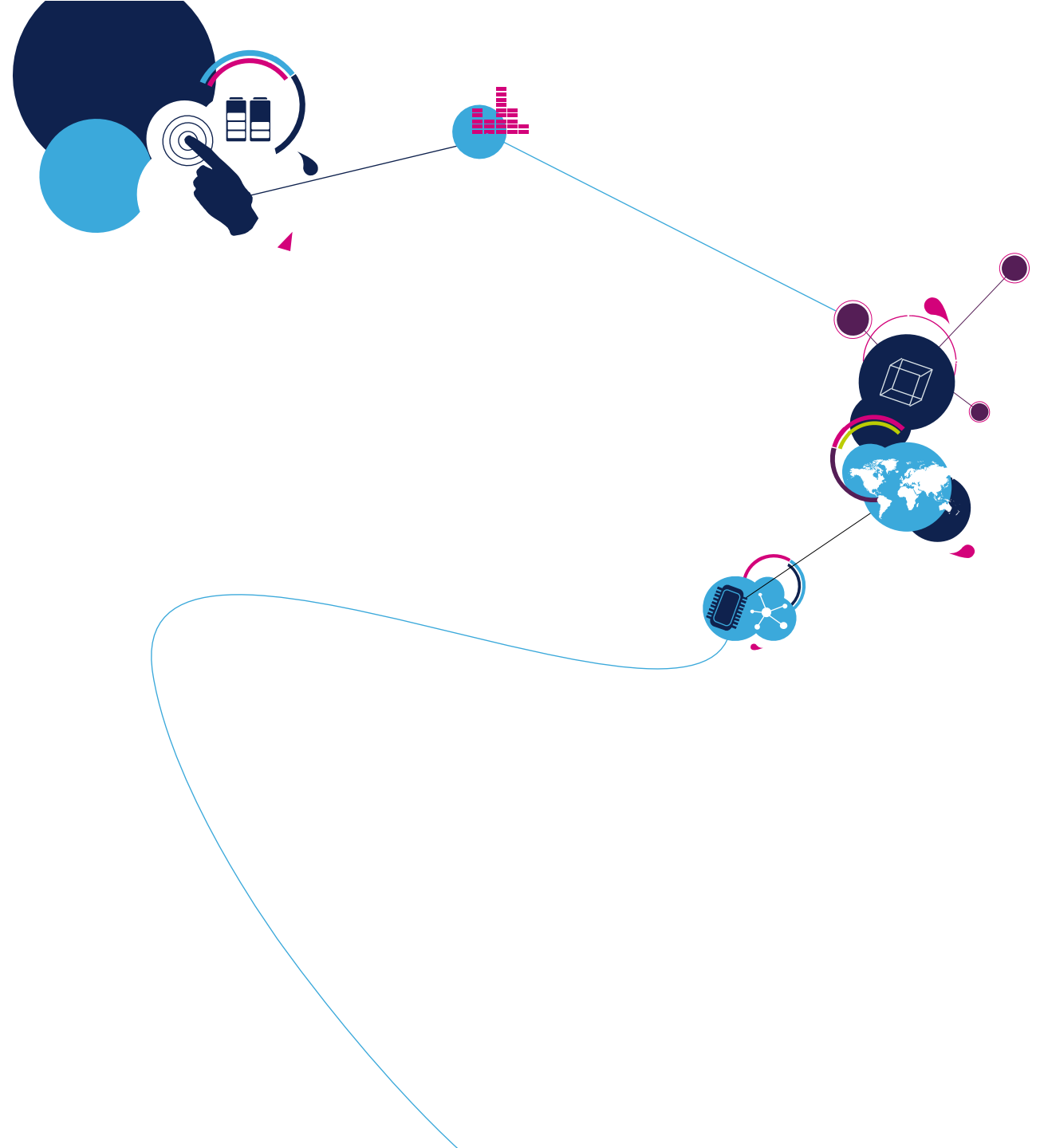
STM32G0 DMA & DMAMUX instance

191

DMAMUX features	DMAMUX
Number of peripheral requests	57
Number of request generator channels	4
Number of trigger inputs	23
Number of synchronization inputs	23
Number of output DMA requests	7

DMA features	DMA
Number of channels	7

Lab: DMA



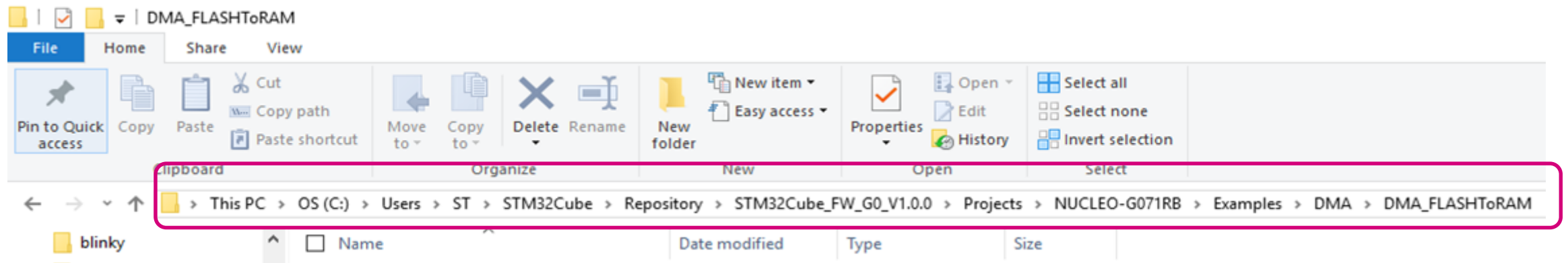
Objective:

- Open a provided STM32CubeMX project (*.ioc) from our STM32CubeG0 Library examples and review the configuration.
- Run the Keil uVision5 (ARM-MDK) project example for the DMA configured as a memory to memory to transfer a buffer from Flash to internal SRAM.

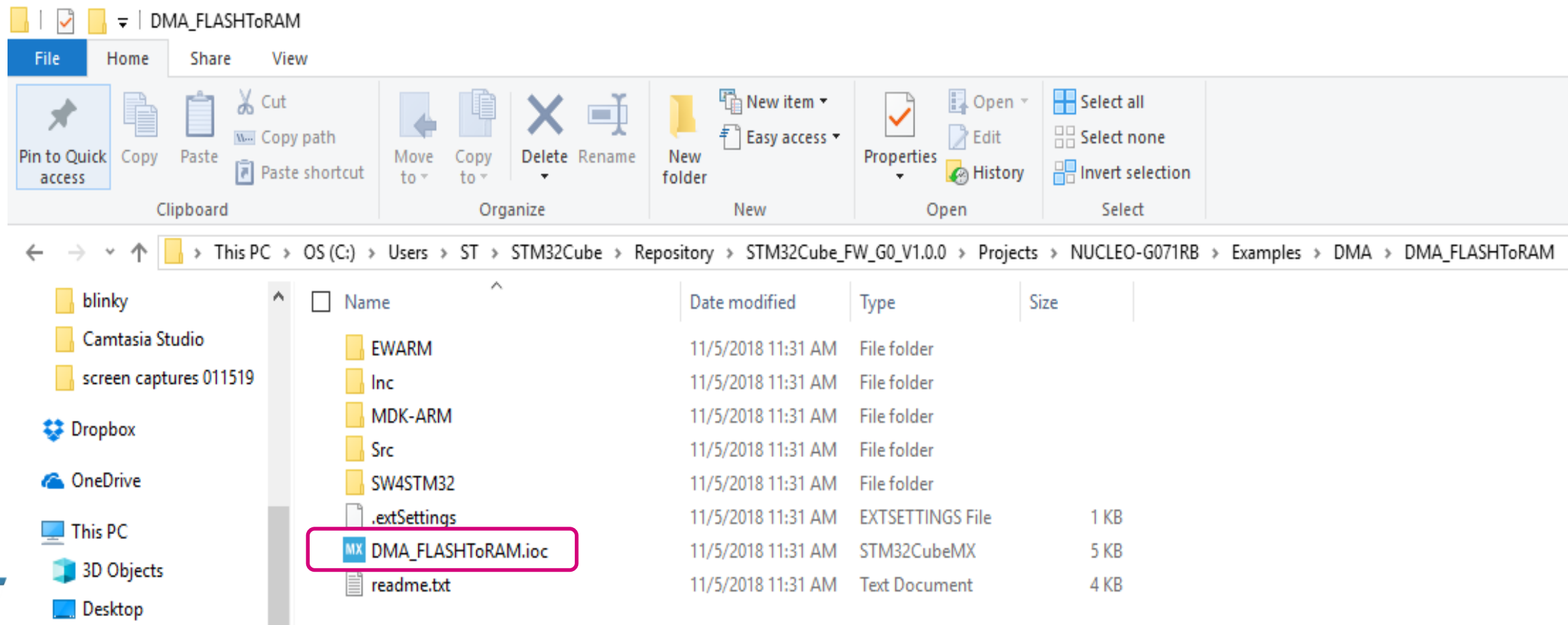
- Close Keil uVision5 IDE if it is open and Close STM32CubeMX if it is open
- In a Windows Explorer window open the following location:

C:\Users\ST\STM32Cube\Repository\STM32Cube_FW_G0_V1.x.0\Projects\NUCLEO-G071RB\Examples\DMA\DMA_FLASHToRAM

Where “ST” is your Windows username and where 1.x .0 is the version of the STM32CubeG) Library that you have installed



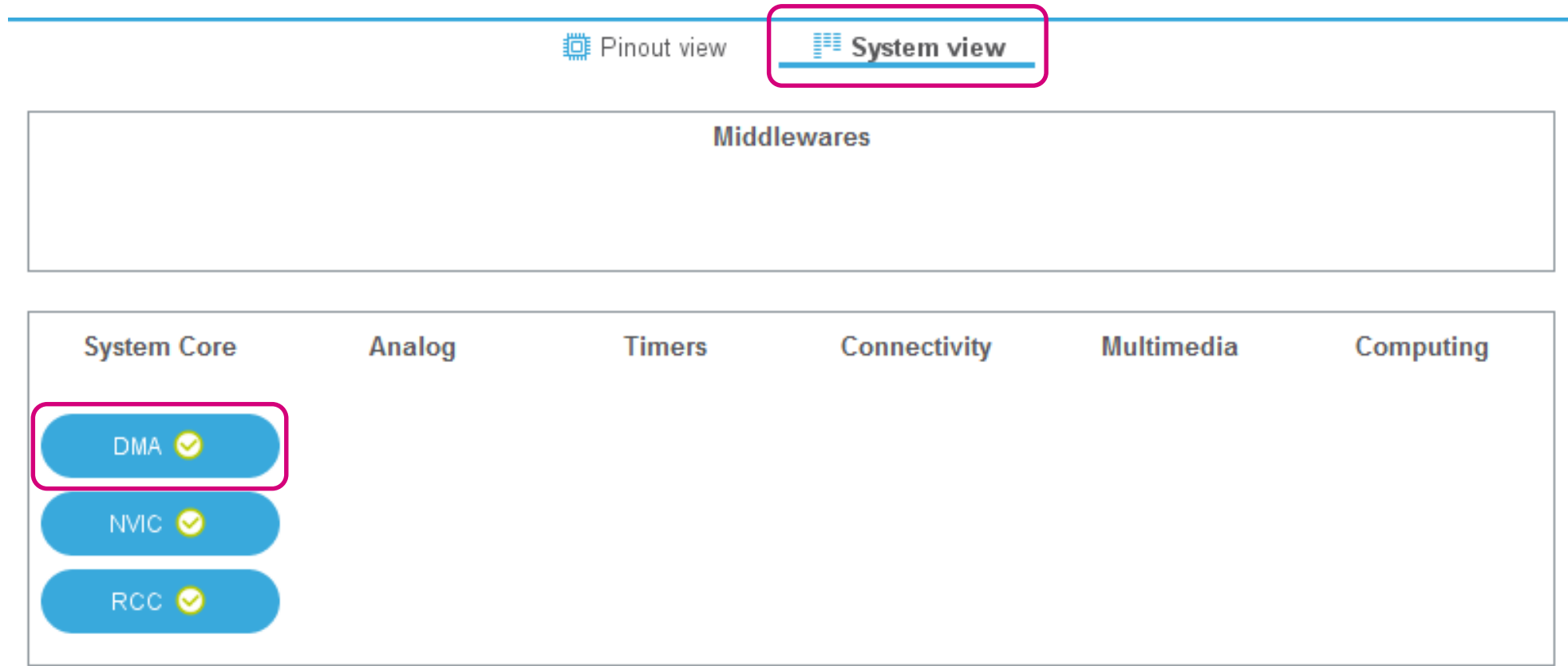
- Double click on the ioc file (DMA_FLASHToRAM.ioc) to open the configuration file with STM32CubeMX:



Check the DMA configuration in STM32CubeMX

196

- In STM32CubeMX click on “**System View**” and then “**DMA**”



Check the DMA configuration in STM32CubeMX

197

- Review the DMA configuration by clicking on the “**MEM2MEM**” under “DMA Request” as shown below:

DMA Mode and Configuration

Configuration

✓ DMA1 ✓ MemToMem

DMA Request	Channel	Direction	Priority
MEMTOMEM	DMA1 Channel 1	Memory To Memory	Low

Add Delete

DMA Request Settings

	Src Memory	Dst Memory
Mode Normal	Increment Address ☒	Increment Address ☒
Data Width	Word	Word

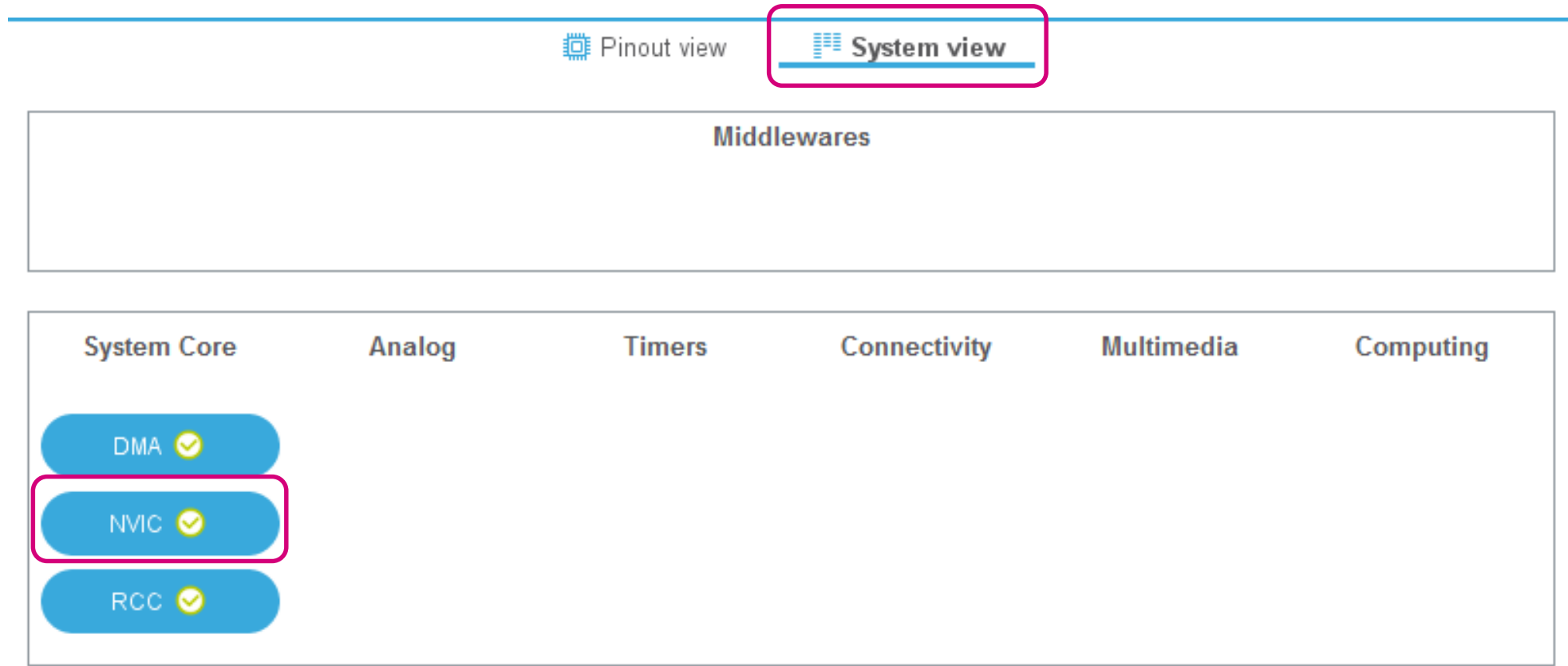
DMA Configuration:

- DMA1 Channel 1
- Memory to Memory
- Priority: Low
- Normal mode
- Incremental Address for source et destination
- Data Width: Word

Check the DMA configuration in STM32CubeMX

198

- In STM32CubeMX click on “**System View**” and then “**NVIC**”



Check the DMA configuration in STM32CubeMX

199

- Notice that the Interrupt is enabled for the DMA1 Channel1:

NVIC Mode and Configuration

Configuration

☒ NVIC ☒ Code generation

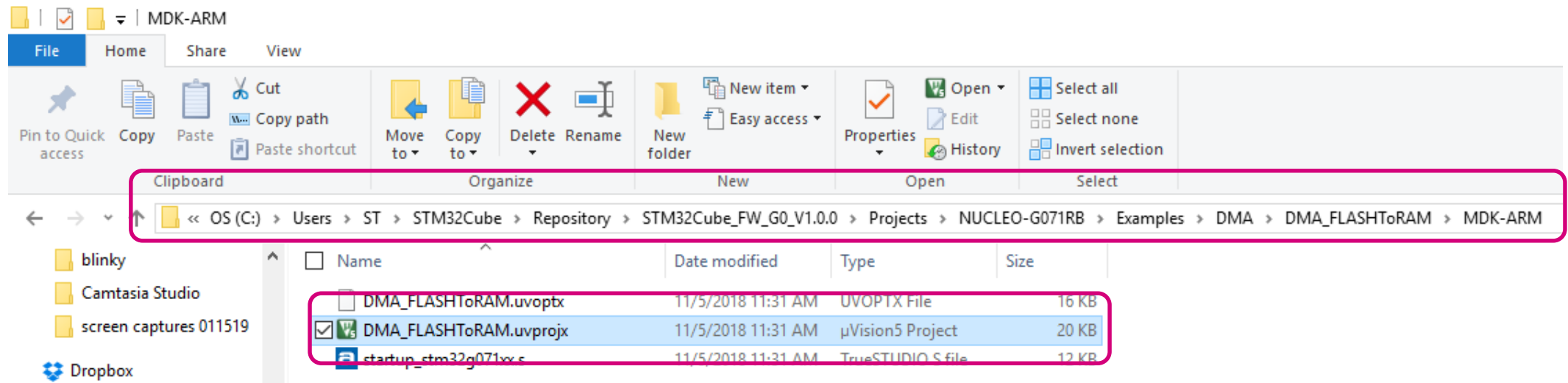
☐ Sort by Preemption Priority and Sub Priority

Search ☐ Show only enabled interrupts

NVIC Interrupt Table	Enabled	Preemption Priority
Non maskable interrupt	☒	0
Hard fault interrupt	☒	0
System service call via SWI instruction	☒	0
Pendable request for system service	☒	0
Time base: System tick timer	☒	0
PVD interrupt through EXTI line 16	☐	0
Flash global interrupt	☐	0
RCC global interrupt	☐	0
DMA1 channel 1 interrupt	☒	0

- In a Windows Explorer window open the following location where the ARM-MDK (Keil uVision5) project is located and double click on the “DMA_FLASHToRam.uvprojx” file:

C:\Users\ST\STM32Cube\Repository\STM32Cube_FW_G0_V1.x.0\Projects\NUCLEO-G071RB\Examples\DMA\DMA_FLASHToRAM\MDK-ARM



Check the DMA configuration in the ARM-MDK project

201

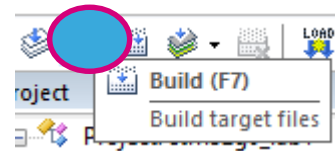
- Open main.c and look for the function MX_DMA_Init() and notice that the configuration is the same as in STM32CubeMX:

```
211 static void MX_DMA_Init(void)
212 {
213     /* DMA controller clock enable */
214     __HAL_RCC_DMA1_CLK_ENABLE();
215
216     /* Configure DMA request hdma_memtmem_dmal_channell on DMA1_Channell */
217     hdma_memtmem_dmal_channell.Instance = DMA1_Channell;
218     hdma_memtmem_dmal_channell.Init.Request = DMA_REQUEST_MEM2MEM;
219     hdma_memtmem_dmal_channell.Init.Direction = DMA_MEMORY_TO_MEMORY;
220     hdma_memtmem_dmal_channell.Init.PeriphInc = DMA_PINC_ENABLE;
221     hdma_memtmem_dmal_channell.Init.MemInc = DMA_MINC_ENABLE;
222     hdma_memtmem_dmal_channell.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;
223     hdma_memtmem_dmal_channell.Init.MemDataAlignment = DMA_MDATAALIGN_WORD;
224     hdma_memtmem_dmal_channell.Init.Mode = DMA_NORMAL;
225     hdma_memtmem_dmal_channell.Init.Priority = DMA_PRIORITY_LOW;
226     if (HAL_DMA_Init(&hdma_memtmem_dmal_channell) != HAL_OK)
227     {
228         Error_Handler();
229     }
230
231     /* DMA interrupt init */
232     /* DMA1_Channell_IRQn interrupt configuration */
233     HAL_NVIC_SetPriority(DMA1_Channell_IRQn, 0, 0);
234     HAL_NVIC_EnableIRQ(DMA1_Channell_IRQn);
235
236     /* USER CODE BEGIN 4 */
```

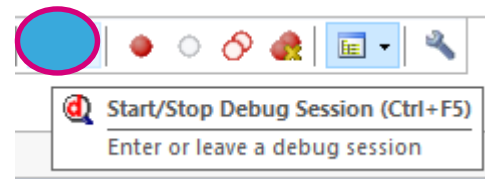
Build the Project

202

- Click the “Build” button



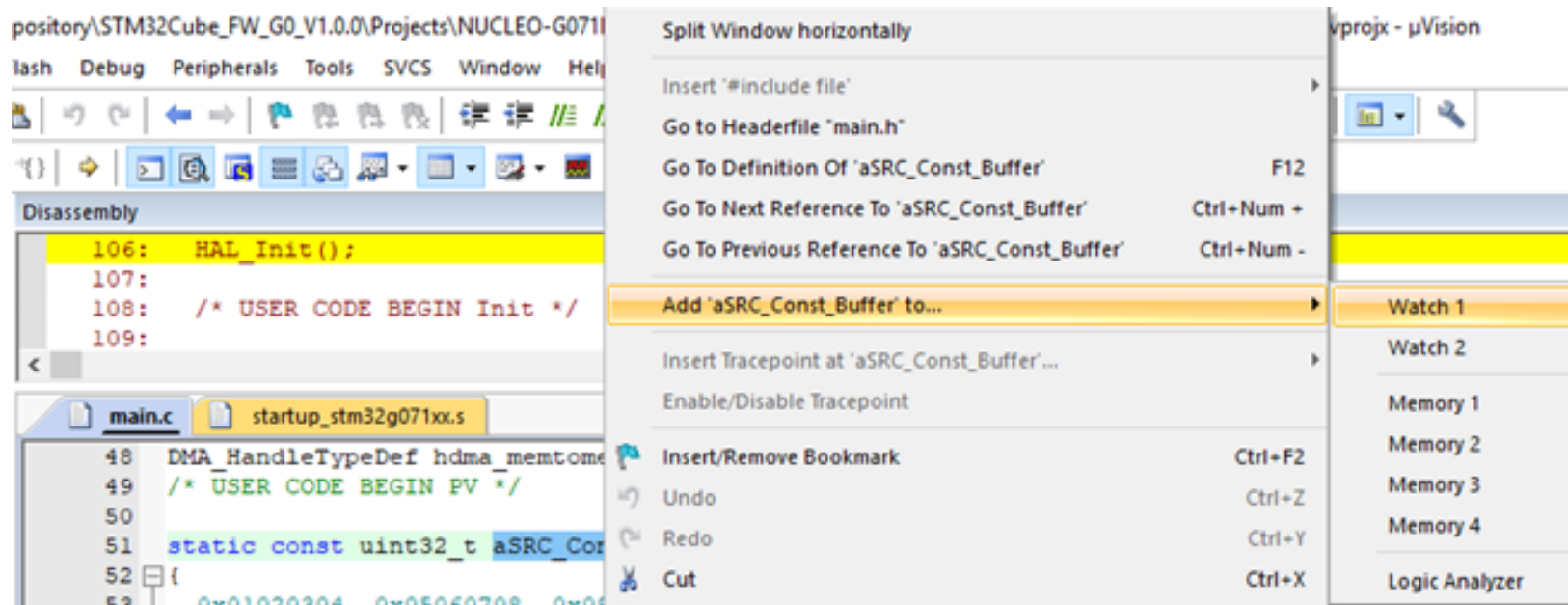
- Click the “Start/Stop Debug Session” button



Add 2 variables to the watch window

203

- Add “aSRC_Const_Buffer” (Source buffer located in Flash) to Watch 1 as shown below:



- Do the same thing for “aDST_Buffer” (Destination Buffer located in internal SRAM)

Add 2 breakpoints in main.c

204

- Add the first breakpoint at this location in main.c just before the execution of the code that will start the DMA transfer:

```
137  /* Configure the source, destination and buffer size DMA fields and Start DMA Channel/Stream transfer */
138  /* Enable All the DMA interrupts */
139  if (HAL_DMA_Start_IT(&hdma_memtomem_dma1_channel1, (uint32_t)&aSRC_Const_Buffer, (uint32_t)&aDST_Buffer, BUFFER_SIZE) != HAL_OK)
140  {
141      /* Transfer Error */
142      Error_Handler();
143  }
```

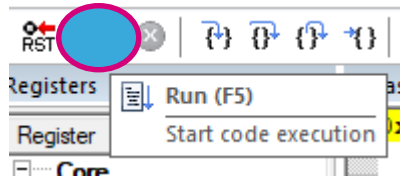
- Add another breakpoint at this code location which will be hit when the DMA transfer has been completed successfully:

```
157
158
159
160
161  if (transferCompleteDetected == 1)
162  {
163      /* Turn LED4 on*/
164      BSP_LED_On(LED4);
165      transferCompleteDetected = 0;
166  }
167  /* USER CODE END 3 */
```

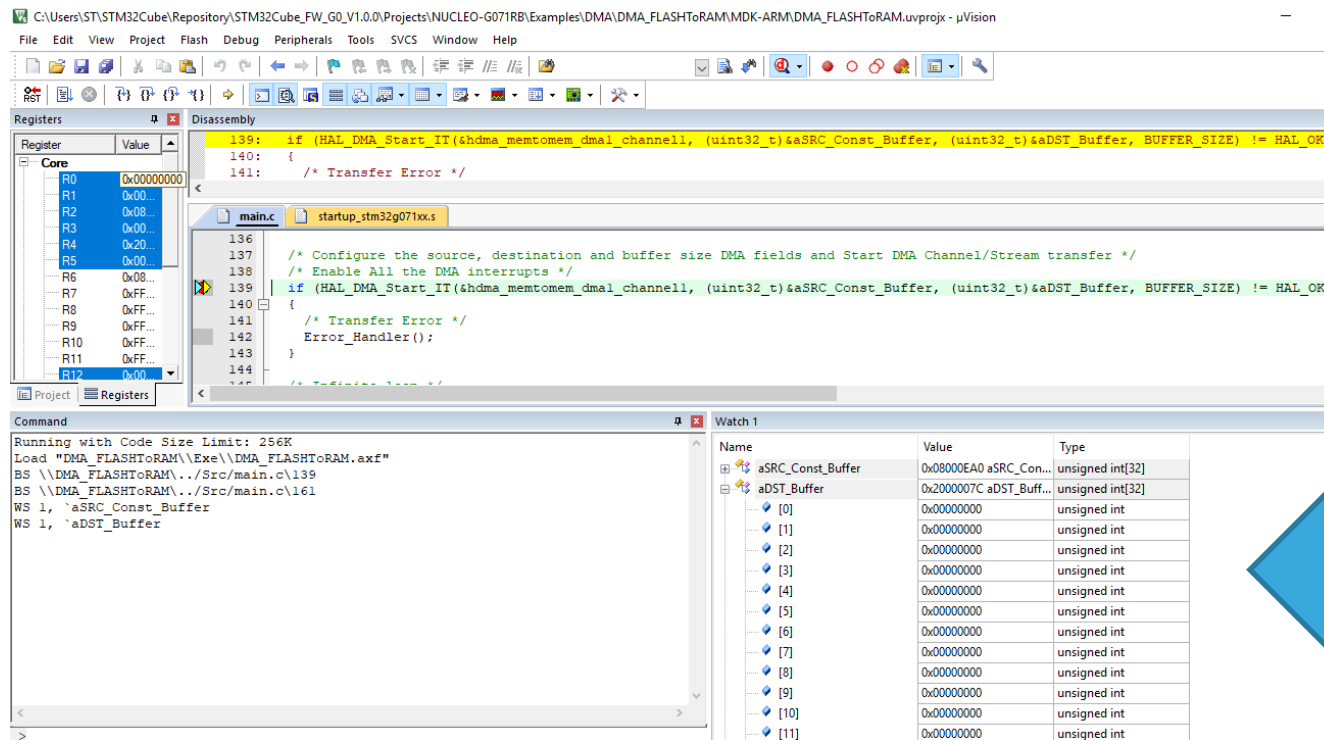
Execute the code until it reaches the first breakpoint

205

- Run the code:



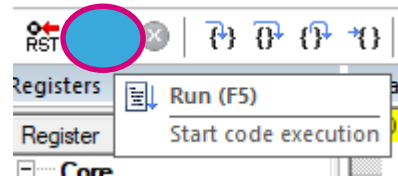
- Once the first breakpoint is hit, notice in the Watch 1 window that the Destination Buffer (aDST_Buffer) is filled with '0'



Execute the code until it reaches the first breakpoint

206

- Now Continue executing the code until it reaches the second breakpoint, press “Run” again:



- Now the destination buffer is filled with values that are the same as the source buffer:

Watch 1		
Name	Value	Type
aDST_Buffer	0x2000007C aDST_Buff...	unsigned int[32]
[0]	0x01020304	unsigned int
[1]	0x05060708	unsigned int
[2]	0x090A0B0C	unsigned int
[3]	0x0D0E0F10	unsigned int
[4]	0x11121314	unsigned int
[5]	0x15161718	unsigned int
[6]	0x191A1B1C	unsigned int
[7]	0x1D1E1F20	unsigned int
[8]	0x21222324	unsigned int
[9]	0x25262728	unsigned int
[10]	0x292A2B2C	unsigned int
[11]	0x2D2E2F30	unsigned int
[12]	0x31323334	unsigned int
[13]	0x35363738	unsigned int
[14]	0x393A3B3C	unsigned int
[15]	0x3D3E3F40	unsigned int
[16]	0x41424344	unsigned int
[17]	0x45464748	unsigned int

Flashing code into the STM32

- Up to 128 Kbytes of single-bank Flash memory
- 2-Kbyte page granularity
- Fast erase (22 ms) and fast programming time (82 μ s for double-words)
- Prefetch & Instruction Cache
- Error Code Correction (ECC): 8 bits for 64-bit double-words
 - Single-bit error detection and correction, notification through a maskable interrupt
 - Double-bit error detection and notification through assertion of the NMI

Flash memory organization

209

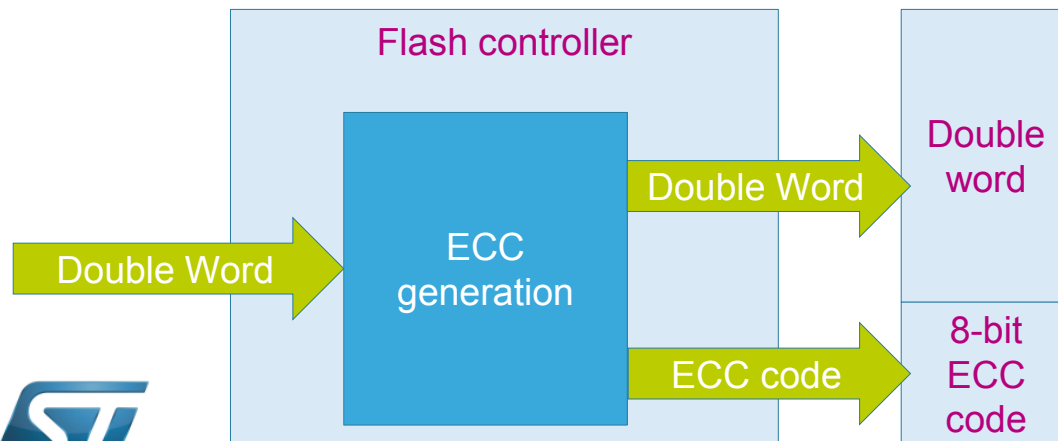
Flash area	Flash memory address	Size	Name
Main memory	0x0800 0000 – 0x0800 07FF	2 Kbytes	Page 0
	...	...	...
	0x0801 F800 – 0x0801 FFFF	2 Kbytes	Page 63
Information block	0x1FFF 0000 – 0x1FFF 6FFF	28 Kbytes	System memory
	0x1FFF 7000 – 0x1FFF 73FF	1 Kbyte	OTP area
	0x1FFF 7800 – 0x1FFF 787F	128 bytes	Option bytes

Operation	Granularity
Programming	8-Byte
Fast-programming	Row of 256 Bytes
Erase	2-Kbyte page
Securable memory	
Write protection	
Read protection	Global
Proprietary Code Readout Protection	512 Bytes

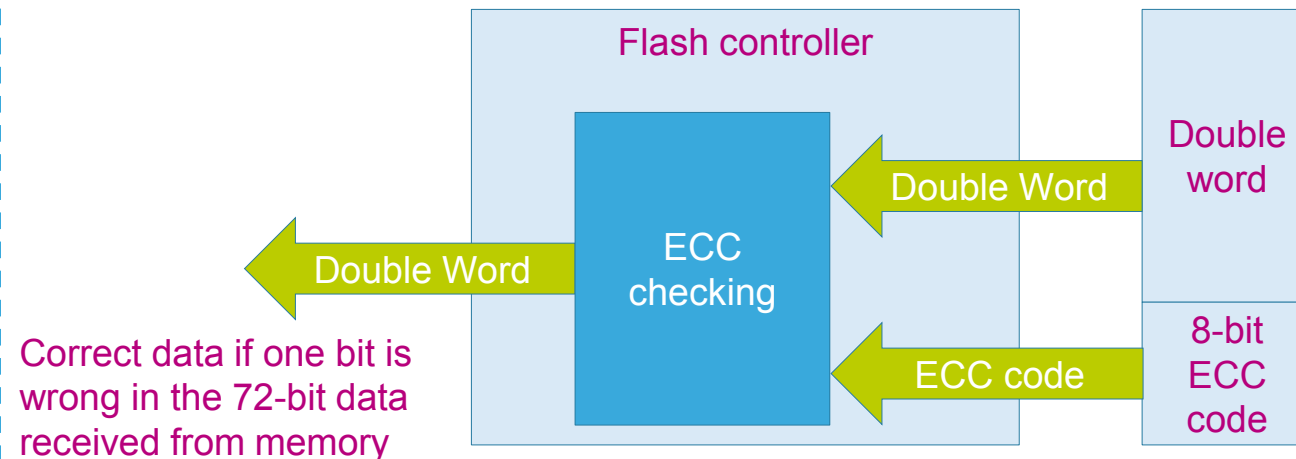
Robust memory integrity and safety

- **ECC (Error Code Correction):** 8 bits long for a 64-bit word
 - Single error correction: ECC bit set in FLASH_ECCR, optional interrupt generation
 - Double error detection: ECCD bit set in FLASH_ECCR => NMI
 - **Failure address saved in FLASH_ECCR register**

Programming



Reading



Robust memory integrity and safety

- Programming granularity is 64 bits (really 72 bits incl. 8-bit ECC)
- 2 programming modes :
 - Standard (for main memory and OTP)
 - Fast (main memory only)
 - Programs 64 double-words without verifying the Flash memory location

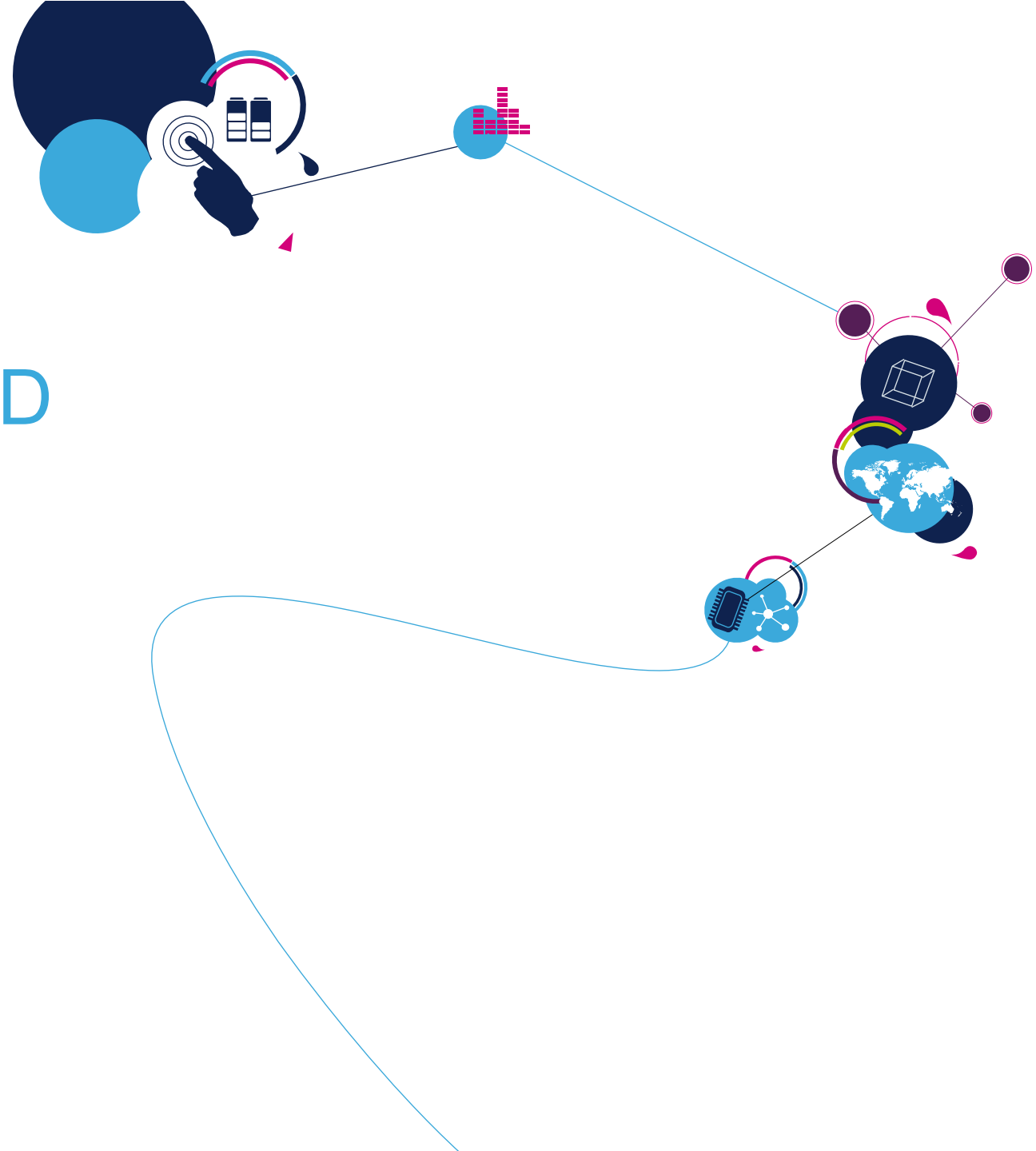
Boot mode configuration					Selected boot area
BOOT_LOCK bit	nBOOT1 bit	BOOT0 pin	nBOOT_SEL bit	nBOOT0 bit	
0	x	0	0	x	Main Flash memory
0	1	1	0	x	System memory
0	0	1	0	x	Embedded SRAM
0	x	x	1	1	Main Flash memory
0	1	x	1	0	System memory
0	0	x	1	0	Embedded SRAM
1	x	x	x	x	Main Flash memory forced

- **BOOT_LOCK Forcing boot from Flash memory**
 - It is possible to force booting from Main Flash memory regardless the other boot options
- The Empty bit is added in Flash memory register to check if programmed

Objective:

- The objective of this lab is to show you how to flash code into the STM32 by restoring the “out-of-the-box” firmware demonstration code that we saved at the beginning of the workshop using 3 different methods:
 - 1st method: using SWD (Serial Wire Debug)
 - 2nd method: using ST-LINK Mass Storage Feature
 - 3rd method: using the STM32G0 System Memory Bootloader

1st method: using SWD (Serial Wire Debug)



In STM32CubeProgrammer, Connect to the ST-LINK

215

STM32CubeProgrammer

Memory & File edition

Device memory Open file +

Address Size Data width 32-bit Read

No data to display

Log Verboosity level 1 2 3

ST-LINK configuration

Serial number 0670FF505051717867181...
Port SWD
Frequency (kHz) 4000
Mode Normal
Access port 0
Reset mode Software reset
Shared Disabled
External loader -
Target voltage 3.21 V
Firmware version V2J31M21
Firmware upgrade

Device information

Device STM32G07x/STM32G08x
Type MCU
Device ID 0x460
Flash size 128 KB
CPU Cortex-M0+

Click On "Connect"

Connected

Disconnect

Log

```
17:32:52 : Address : 0x40022020
17:32:52 : Size : 32 Bytes
17:32:52 : Bank : 0x01
17:32:52 : Address : 0x40022080
17:32:52 : Size : 4 Bytes
17:32:53 : UPLOADING ...
17:32:53 : Size : 1024 Bytes
17:32:53 : Address : 0x8000000
17:32:53 : Read progress:
17:32:53 : Data read successfully
17:32:53 : Time elapsed during the read operation is: 00:00:00.007
```

Program the flash

216

- Click on the icon 1
- Select the path of the original code saved at the beginning of the workshop 2
- Uncheck this box so that the code won't run after programming 3
- Press **Start Programming** 4

The screenshot displays the STM32CubeProgrammer application window. The 'Erasing & Programming' tab is active. In the 'Download' section, the file path is set to '.STM32G0Workshop\HandsOn\original code\original_code.bin'. The 'Programming options' section shows 'Verify programming' checked and 'Run after programming' unchecked. A table of flash memory sectors is visible, with columns for Select, Index, Start Address, and Size. The 'Start Programming' button is highlighted. The log window at the bottom shows the progress of the programming operation, including the upload of the code and the successful read of the data.

STM32CubeProgrammer

Erasing & Programming

Download

File path: .STM32G0Workshop\HandsOn\original code\original_code.bin

Start address: 0x08000000

Programming options

☐ Skip flash erase before programming

☒ Verify programming

☐ Run after programming

Erase flash memory | Erase external memory

Erase selected sectors | Full chip erase

Select	Index	Start Address	Size
☐	0	0x08000000	2K
☐	1	0x08000800	2K
☐	2	0x08001000	2K
☐	3	0x08001800	2K
☐	4	0x08002000	2K
☐	5	0x08002800	2K
☐	6	0x08003000	2K
☐	7	0x08003800	2K
☐	8	0x08004000	2K
☐	9	0x08004800	2K
☐	10	0x08005000	2K

Start Programming

Log

19:53:36 : Size : 32 Bytes

19:53:36 : Bank : 0x01

19:53:36 : Address : 0x40022080

19:53:36 : Size : 4 Bytes

19:53:36 : UPLOADING ...

19:53:36 : Size : 1024 Bytes

19:53:36 : Address : 0x8000000

19:53:36 : Read progress:

19:53:36 : Data read successfully

19:53:36 : Time elapsed during the read operation is: 00:00:00.023

Verbosity level: 1

ST-LINK configuration

Serial number: 066EFF545051717867193442

Port: SWD

Frequency (kHz): 4000

Mode: Normal

Access port: 0

Reset mode: Software reset

Shared: Disabled

External loader: -

Target voltage: 3.23 V

Firmware version: V2J31M21

Firmware upgrade

Device information

Device: STM32G07x/STM32G08x

Type: MCU

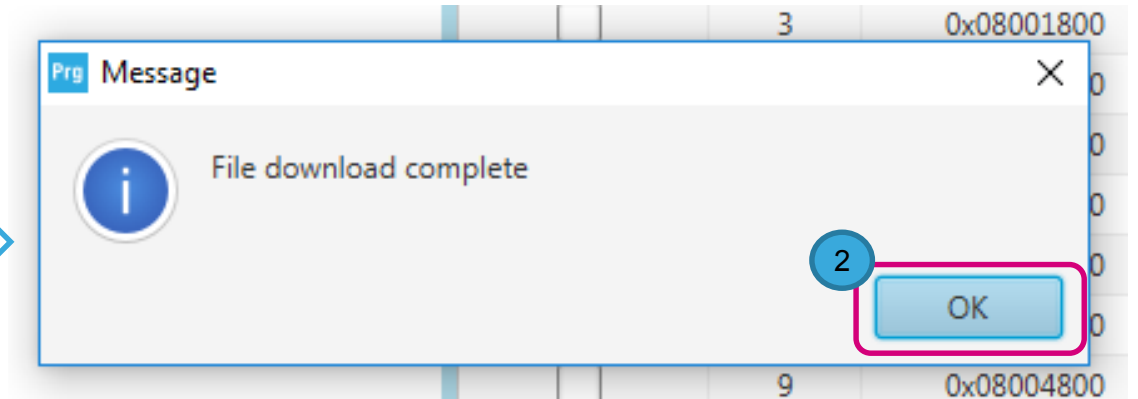
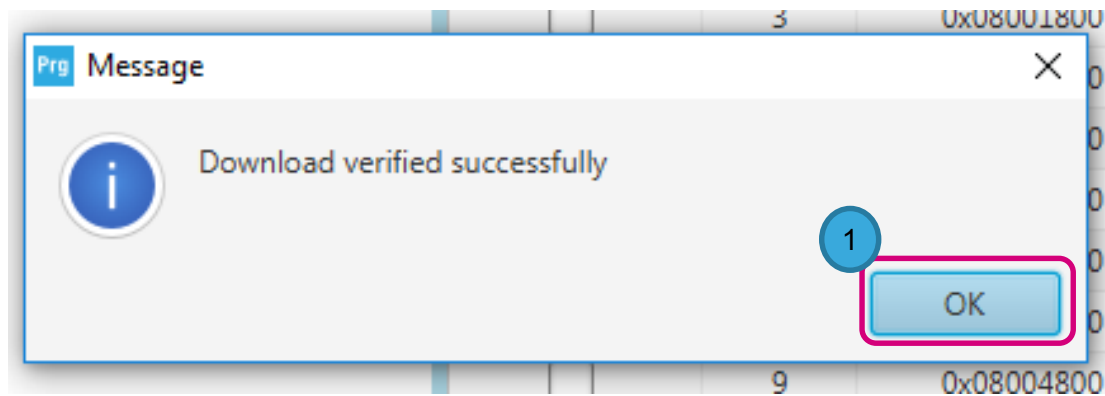
Device ID: 0x460

Flash size: 128 KB

CPU: Cortex-M0+

Flash Programmed

217



Disconnect to the ST-LINK

218

STM32CubeProgrammer

Memory & File edition

Device memory

Address: 0x08000000 Size: 0x400 Data width: 32-bit

Address	0	4	8	C	ASCII
0x08000000	20000410	08000001	08000978	08000979	... N...{...y...
0x08000010	00000000	00000000	00000000	00000000	
0x08000020	00000000	00000000	00000000	0800097F	
0x08000030	00000000	00000000	0800097D	08000981	
0x08000040	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...ä...
0x08000050	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...ä...
0x08000060	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...ä...
0x08000070	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...ä...
0x08000080	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...ä...
0x08000090	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...ä...
0x080000A0	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...ä...
0x080000B0	080000E3	080000E3	080000E3	46854803	ä...ä...ä...H.F
0x080000C0	F83CF000	47004800	08000A49	20000410	.D<ø.H.G.I....

Log

Verbosity level: 1 2 3

17:32:52 : Address : 0x40022020
17:32:52 : Size : 32 Bytes
17:32:52 : Bank : 0x01
17:32:52 : Address : 0x40022080
17:32:52 : Size : 4 Bytes
17:32:53 : UPLOADING ...
17:32:53 : Size : 1024 Bytes
17:32:53 : Address : 0x80000000
17:32:53 : Read progress:
17:32:53 : Data read successfully
17:32:53 : Time elapsed during the read operation is: 00:00:00.007

ST-LINK

Connected

ST-LINK configuration

Serial number: 0670FF505051717867181...

Port: SWD

Frequency (kHz): 4000

Mode: Normal

Access port: 0

Reset mode: Software reset

Shared: Disabled

External loader: -

Target voltage: 3.21 V

Firmware version: V2J31M21

Firmware upgrade

Device information

Device: STM32G07x/STM32G08x

Type: MCU

Device ID: 0x460

Flash size: 128 KB

CPU: Cortex-M0+

Not connected

Connect

ST-LINK configuration

Serial number: 0670FF505051717867181...

Port: SWD

Frequency (kHz): 4000

Mode: Normal

Access port: 0

Reset mode: Software reset

Shared: Disabled

External loader: -

Target voltage: 3.21 V

Firmware version: V2J31M21

Firmware upgrade

Device information

Device: -

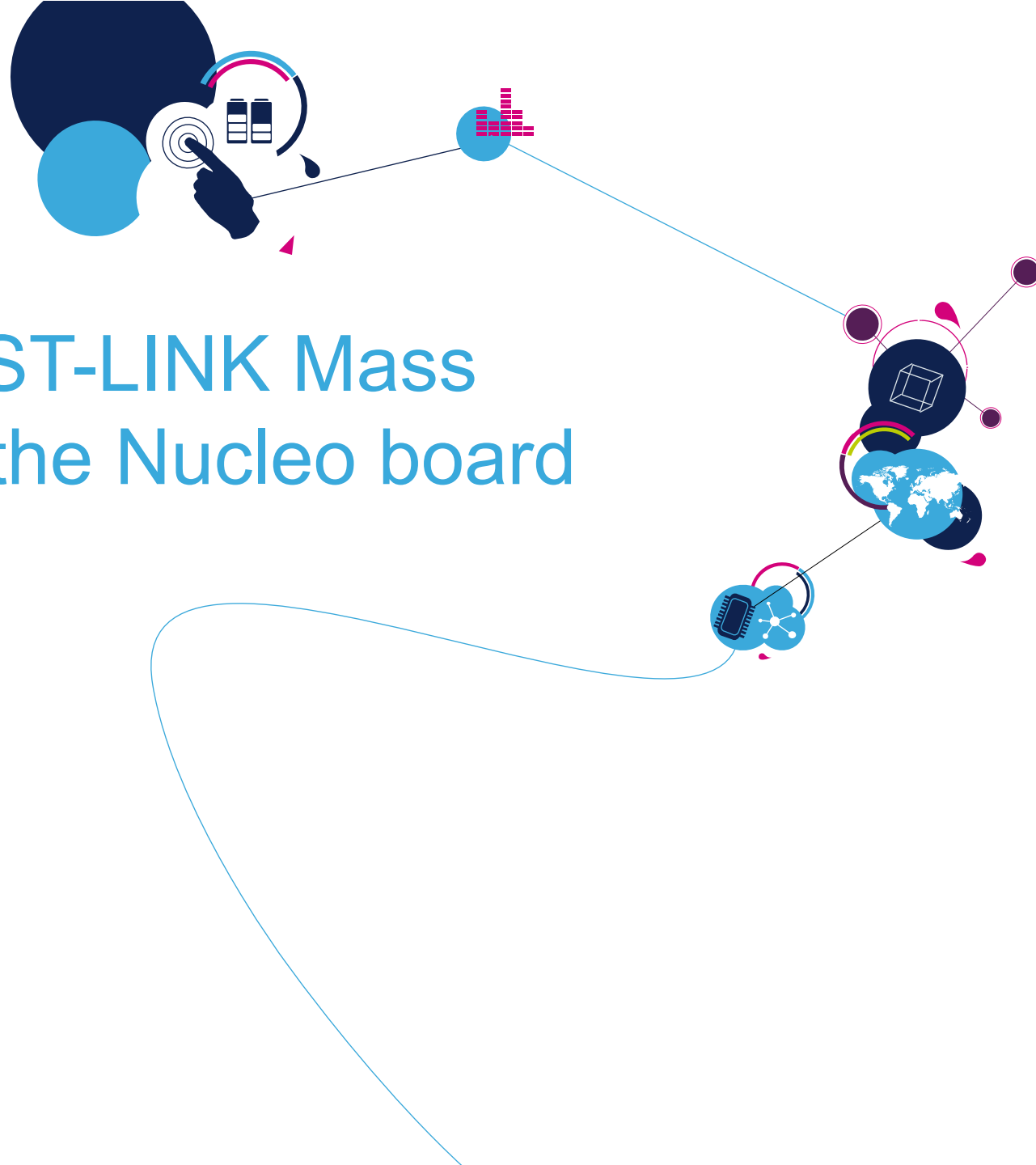
Type: -

Device ID: -

Flash size: -

CPU: -

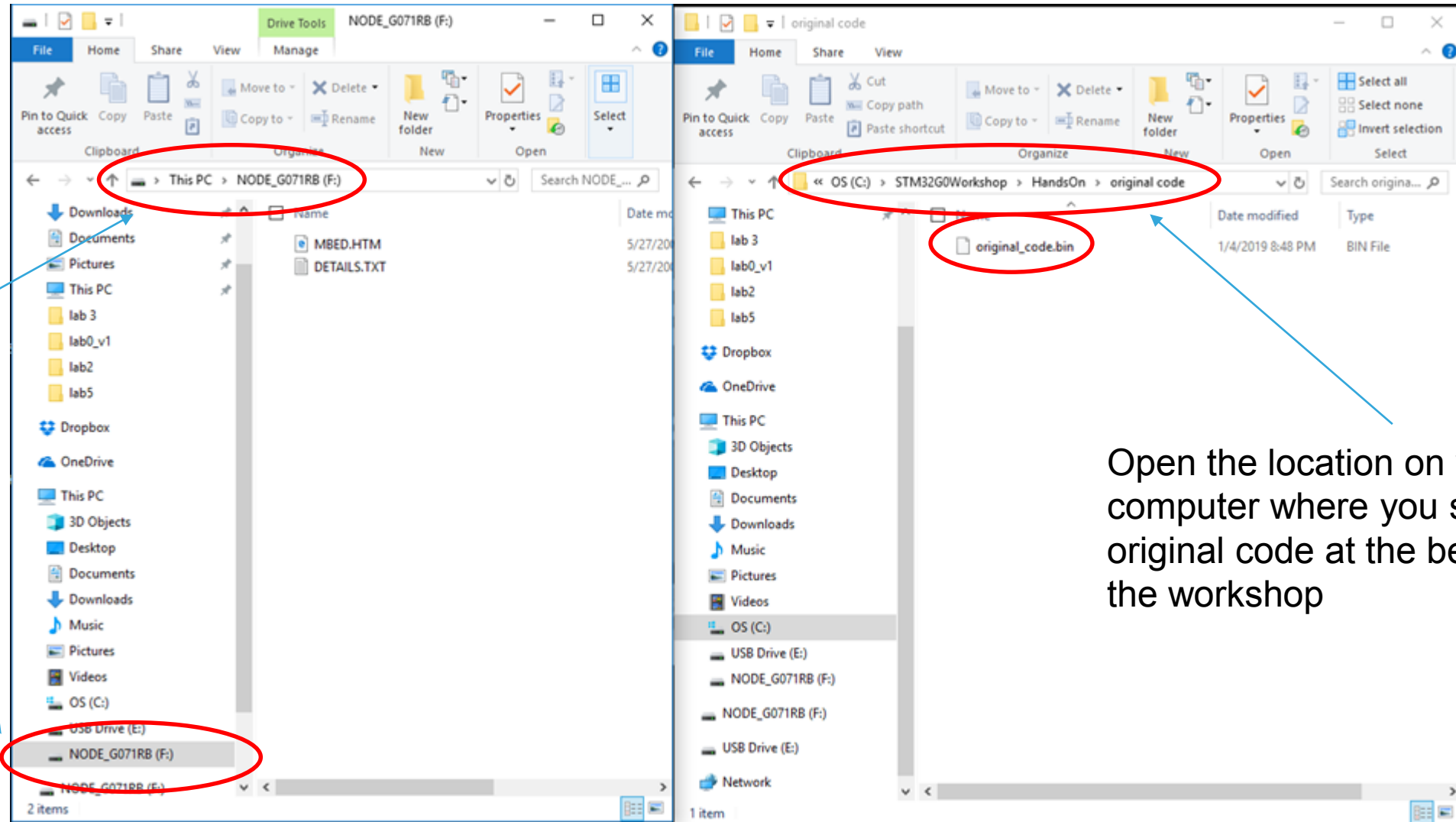
Now press reset on the board to run the original code



2nd method: using ST-LINK Mass
Storage Feature of the Nucleo board

Prepare the copy of binary code to the NUCLEO drive

220

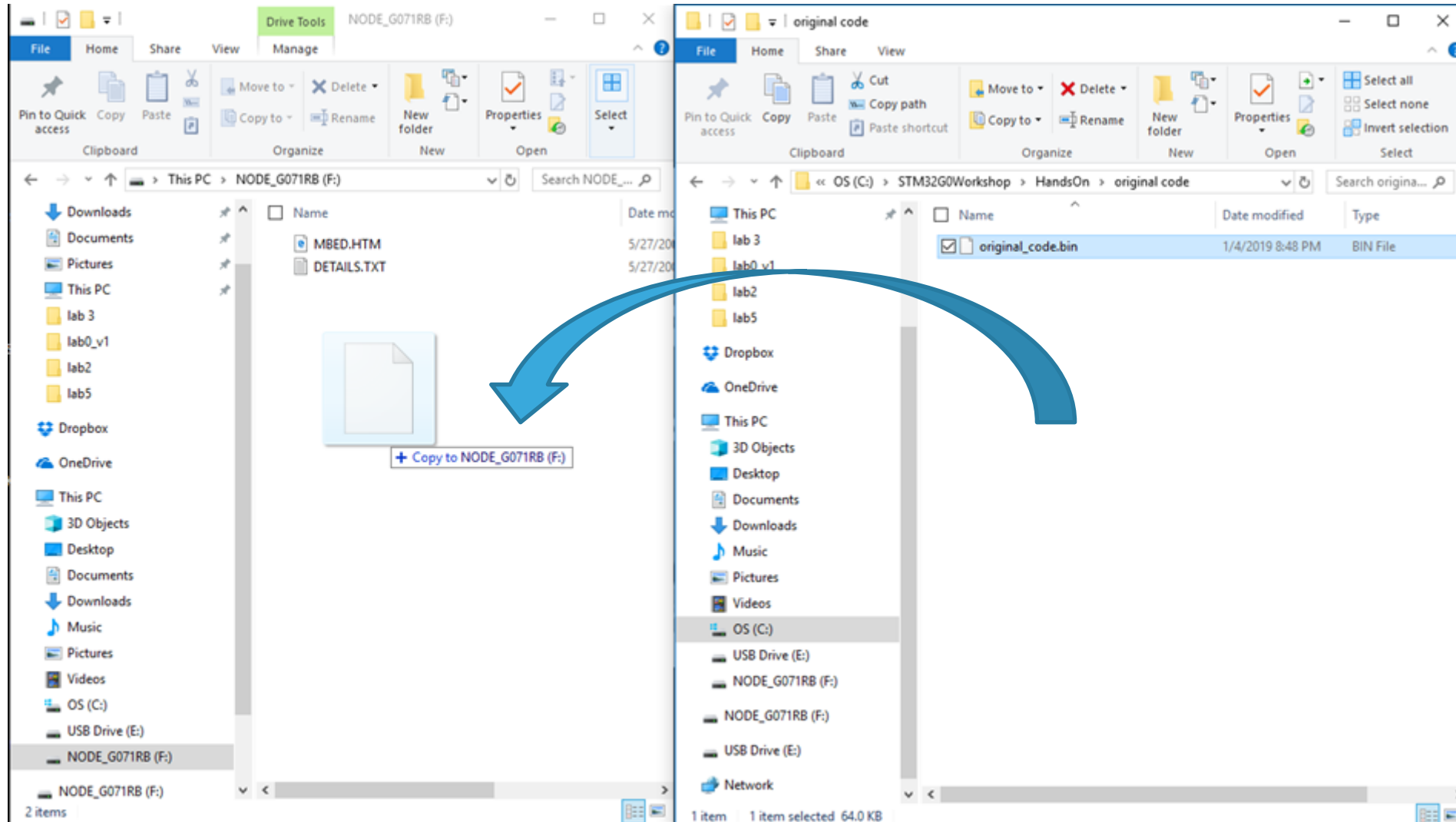


Open the Nucleo Drive on your computer

Open the location on your computer where you saved the original code at the beginning of the workshop

Drag and drop the binary code to the NUCLEO drive

221



The original code will run after the copy has been done

3rd method: using the STM32G0 System Memory Bootloader

Connect to the ST-LINK

223

Click On "Connect"

ST-Microelectronics

STM32CubeProgrammer

Memory & File edition

Device memory Open file +

Address Size Data width 32-bit Read

No data to display

Log Verboosity level 1 2 3

ST-LINK configuration

Serial number 0670FF505051717867181...

Port SWD

Frequency (kHz) 4000

Mode Normal

Access port 0

Reset mode Software reset

Shared Disabled

External loader -

Target voltage 3.21 V

Firmware version V2J31M21

Firmware upgrade

Device information

Device STM32G07x/STM32G08x

Type MCU

Device ID 0x460

Flash size 128 KB

CPU Cortex-M0+

Log

17:32:52 : Address : 0x40022020

17:32:52 : Size : 32 Bytes

17:32:52 : Bank : 0x01

17:32:52 : Address : 0x40022080

17:32:52 : Size : 4 Bytes

17:32:53 : UPLOADING ...

17:32:53 : Size : 1024 Bytes

17:32:53 : Address : 0x8000000

17:32:53 : Read progress:

17:32:53 : Data read successfully

17:32:53 : Time elapsed during the read operation is: 00:00:00.007

Change the option byte to boot from System Memory

224

Option bytes

User Configuration

Name	Value	Description
nRST_STOP	☒	Unchecked : Reset generated when entering Stop mode Checked : No reset generated when entering Stop mode
nRST_STDBY	☒	Unchecked : Reset generated when entering Standby mode Checked : No reset generated when entering Standby mode
nRST_SHDW	☒	Unchecked : Reset generated when entering the Shutdown mode Checked : No reset generated when entering the Shutdown mode
IWDG_SW	☒	Unchecked : Hardware independant watchdog Checked : Software independant watchdog
IWDG_STOP	☒	Unchecked : Freeze IWDG counter in stop mode Checked : IWDG counter active in stop mode
IWDG_STDBY	☒	Unchecked : Freeze IWDG counter in standby mode Checked : IWDG counter active in standby mode
WWDG_SW	☒	Unchecked : Hardware window watchdog Checked : Software window watchdog
RAM_PARITY_CHECK	☒	Unchecked : SRAM2 parity check enable Checked : SRAM2 parity check disable
nBOOT_SEL	☒	Unchecked : BOOT0 signal is defined by BOOT0 pin value (legacy mode) Checked : BOOT0 signal is defined by nBOOT0 option bit
nBOOT1	☒	Unchecked : Boot from Flash if BOOT0 = 0, otherwise Embedded SRAM1 Checked : Boot from Flash if BOOT0 = 0, otherwise system memory
nBOOT0	☐	Unchecked : nBOOT0=0 Checked : nBOOT0=1
NRST_MODE	3	0 : Reserved 1 : Reset Input only: a low level on the NRST pin generates system reset, internal RESET not propagated to the NSRT 2 : GPIO: standard GPIO pad functionality, only internal RESET possible

Apply Read

Uncheck
nBoot0 to
boot from
System
Memory
Bootloader

Disconnect to the ST-LINK and reset the board

225

The screenshot displays the STM32CubeProgrammer interface. The main window shows the 'Memory & File edition' tab with a table of memory addresses and data. The 'ST-LINK' configuration panel on the right shows the 'Connected' status. A blue arrow points to the 'Disconnect' button with the text 'Click On "Disconnect"'. Below the main window, a smaller window shows the 'Not connected' status.

Memory & File edition

Device memory

Address: 0x08000000, Size: 0x400, Data width: 32-bit

Address	0	4	8	C	...
0x08000000	20000410	080000D1	0800097B	08000979	... N...{...y...
0x08000010	00000000	00000000	00000000	00000000	
0x08000020	00000000	00000000	00000000	0800097F	
0x08000030	00000000	00000000	0800097D	08000981	}
0x08000040	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...
0x08000050	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...
0x08000060	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...
0x08000070	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...
0x08000080	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...
0x08000090	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...
0x080000A0	080000E3	080000E3	080000E3	080000E3	ä...ä...ä...
0x080000B0	080000E3	080000E3	080000E3	46854803	ä...ä...ä...H.F
0x080000C0	F83CF000	47004800	08000A49	20000410	.ð<ð.H.GI.....

Log

17:32:52 : Address : 0x40022020
17:32:52 : Size : 32 Bytes
17:32:52 : Bank : 0x01
17:32:52 : Address : 0x40022080
17:32:52 : Size : 4 Bytes
17:32:53 : iPLDING...
17:32:53 : Size : 1024 Bytes
17:32:53 : Address : 0x80000000
17:32:53 : Read progress:
17:32:53 : Data read successfully
17:32:53 : Time elapsed during the read operation is: 00:00:00.007

ST-LINK

Connected

Click On "Disconnect"

ST-LINK configuration

Serial number: 0670FF505051717867181...

Port: SWD

Frequency (kHz): 4000

Mode: Normal

Access port: 0

Reset mode: Software reset

Shared: Disabled

External loader: -

Target voltage: 3.21 V

Firmware version: V2J31M21

Firmware upgrade

Device information

Device: STM32G07x/STM32G08x

Type: MCU

Device ID: 0x460

Flash size: 128 KB

CPU: Cortex-M0+

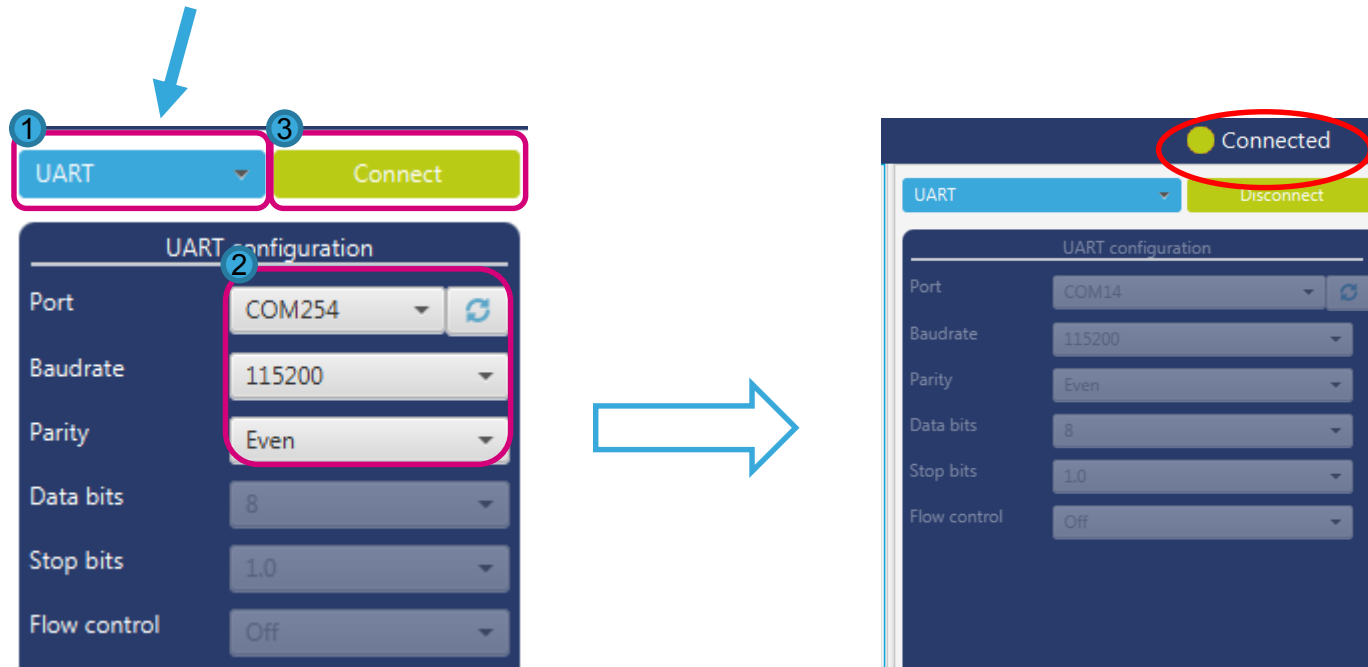
Not connected

Once disconnected is clicked, unplug and replug the Nucleo board's USB cable

With STM32 Cube Programmer connect to the System Memory through UART

226

Instead of “ST-LINK” use “UART” to connect to the System Memory Bootloader through UART mode



Program the binary code

227

The screenshot displays the STM32CubeProgrammer software interface, specifically the 'Erasing & Programming' tab. The interface is annotated with numbered circles (1-7) indicating the steps for programming the binary code.

Download Section:

- 1:** Download icon in the left sidebar.
- 2:** File path field containing 'C:\STM32G0Workshop\HandsOn\original code\original_code.bin'.
- 3:** Start address field containing '0x08000000'.
- 4:** 'Verify programming' checkbox, which is checked.
- 5:** 'Start Programming' button.

Message Dialog:

A 'Message' dialog box is displayed, indicating 'Download verified successfully'. The 'OK' button is highlighted with a pink box and labeled with a blue circle **6**.

Log Section:

The log shows the following messages:

- 20:29:44 : Size : 0x00000000
- 20:29:44 : Address : 0x08000000
- 20:29:44 : Erasing memory corresponding to segment 0:
- 20:29:44 : Erasing internal memory sectors [0 31]
- 20:29:45 : Download in Progress:
- 20:29:53 : File download complete
- 20:29:53 : Time elapsed during download operation: 00:00:09.291
- 20:29:53 : Verifying ...
- 20:29:53 : Read progress:
- 20:30:01 : Download verified successfully

UART Configuration:

The UART configuration section is visible on the right side of the interface. It includes a 'Disconnect' button, which is highlighted with a pink box and labeled with a blue circle **7**.

Device Information:

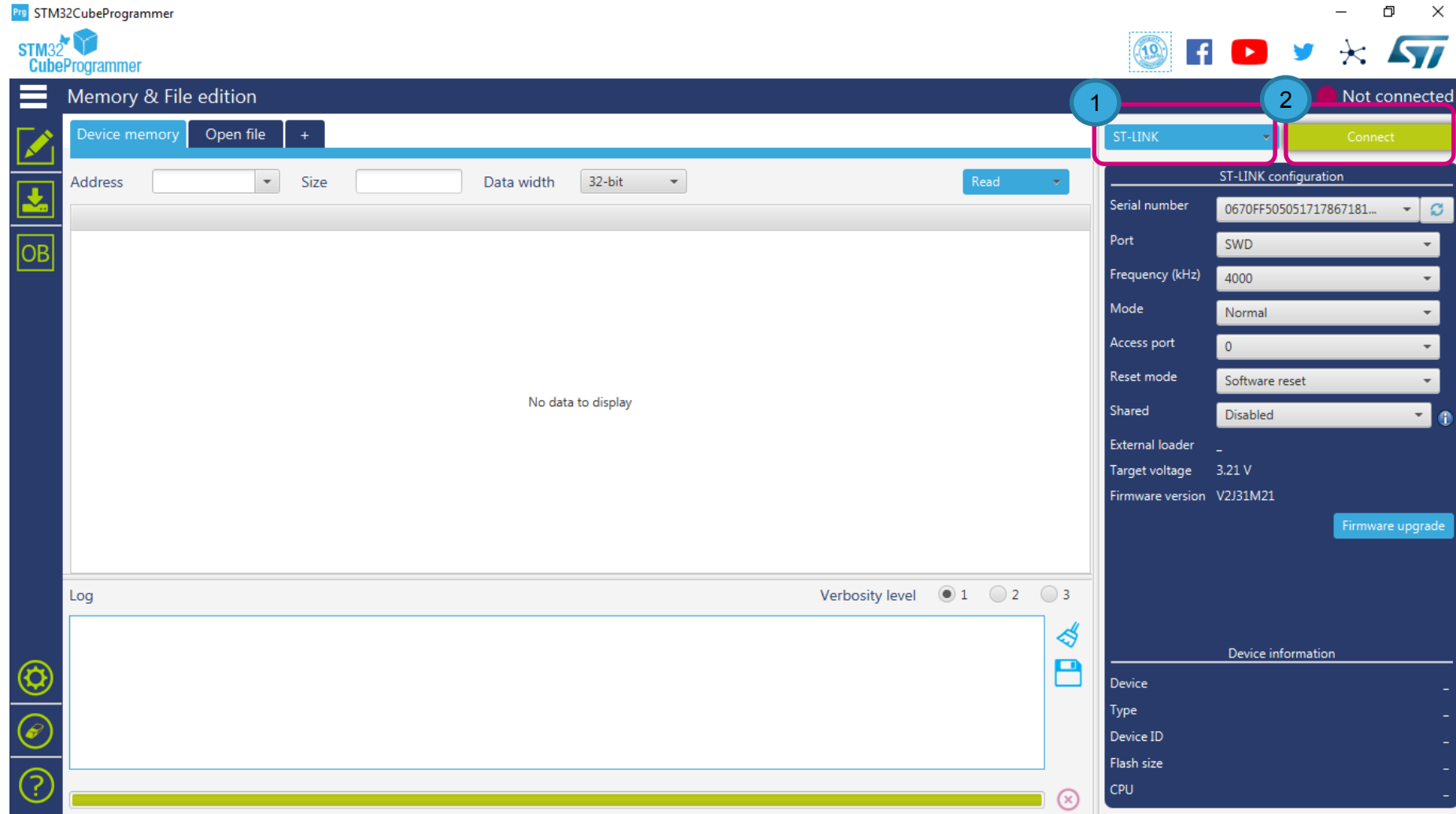
The device information section shows the following details:

- Device: STM32G07x/STM32G08x
- Type: MCU
- Device ID: 0x460
- Flash size: -
- CPU: Cortex-M0+

Restore the option bytes to boot from internal flash

Connect to the ST-LINK through SWD

229



Change the option byte to boot from the Flash Memory

230

STM32CubeProgrammer

Option bytes

User Configuration

Name	Value	Description
nRST_STOP	☒	Unchecked : Reset generated when entering Stop mode Checked : No reset generated when entering Stop mode
nRST_STDBY	☒	Unchecked : Reset generated when entering Standby mode Checked : No reset generated when entering Standby mode
nRST_SHDW	☒	Unchecked : Reset generated when entering the Shutdown mode Checked : No reset generated when entering the Shutdown mode
IWDG_SW	☒	Unchecked : Hardware independant watchdog Checked : Software independant watchdog
IWDG_STOP	☒	Unchecked : Freeze IWDG counter in stop mode Checked : IWDG counter active in stop mode
IWDG_STDBY	☒	Unchecked : Freeze IWDG counter in standby mode Checked : IWDG counter active in standby mode
WWDG_SW	☒	Unchecked : Hardware window watchdog Checked : Software window watchdog
RAM_PARITY_CHECK	☒	Unchecked : SRAM2 parity check enable Checked : SRAM2 parity check disable
nBOOT_SEL	☒	Unchecked : BOOT0 signal is defined by BOOT0 pin value (legacy mode) Checked : BOOT0 signal is defined by nBOOT0 option bit
nBOOT1	☒	Unchecked : Boot from Flash if BOOT0 = 0, otherwise Embedded SRAM1 Checked : Boot from Flash if BOOT0 = 0, otherwise system memory
nBOOT0	☒	Unchecked : nBOOT0=0 Checked : nBOOT0=1 0 : Reserved
NRST_MODE	3	1 : Reset Input only: a low level on the NRST pin generates system reset, internal RESET not propagated to the NSRT 2 : GPIO: standard GPIO pad functionality, only internal RESET possible

Log

22:18:46 : Bank : 0x01
22:18:46 : Address : 0x40022080
22:18:46 : Size : 4 Bytes
22:18:46 : OPTION BYTE PROGRAMMING VERIFICATION:
22:18:46 : Option Bytes successfully programmed

Apply

ST-LINK

Connect

ST-LINK configuration

Serial number: 066FFF484951717867111...

Port: SWD

Frequency (kHz): 4000

Mode: Normal

Access port: 0

Reset mode: Software reset

Shared: Disabled

External loader: -

Target voltage: 3.23 V

Firmware version: V2J31M21

Firmware upgrade

Device information

Device: -

Type: -

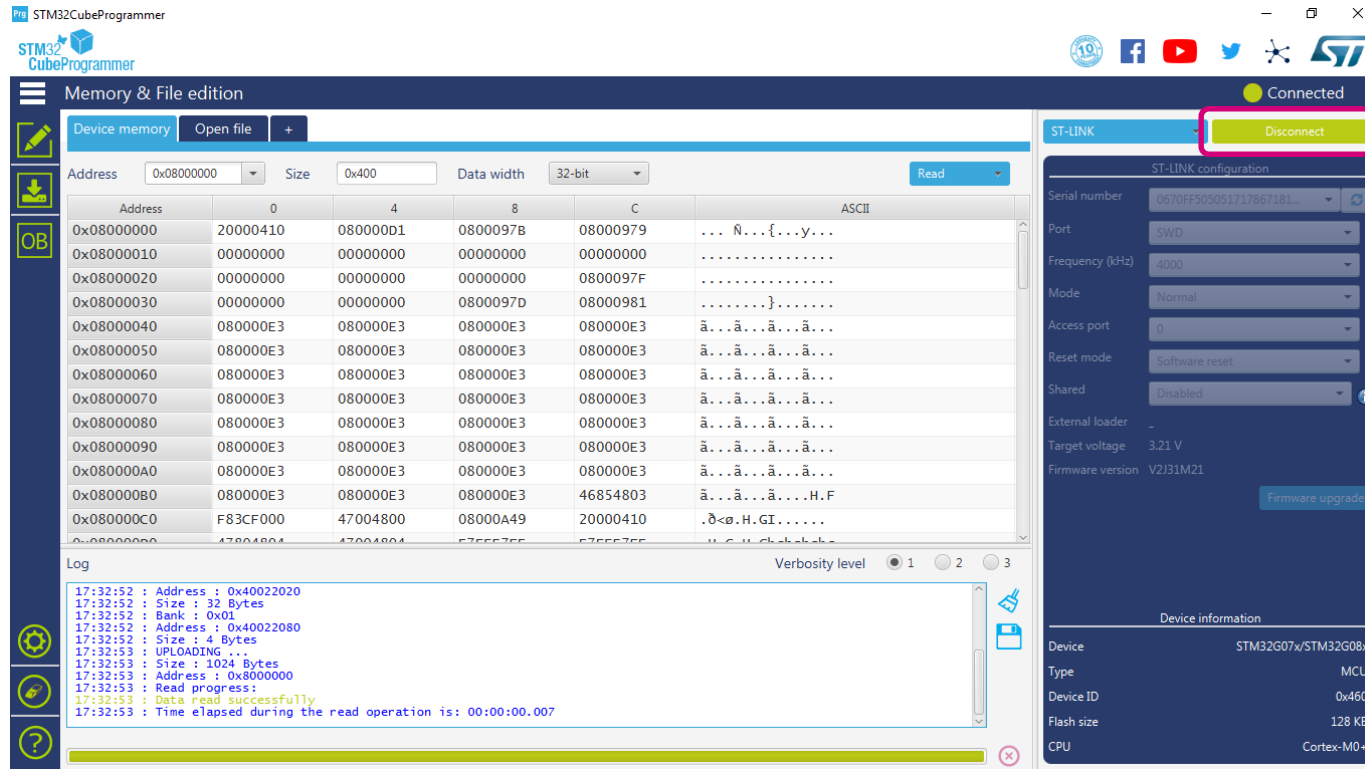
Device ID: -

Flash size: -

CPU: -

Disconnect to the ST-LINK

231



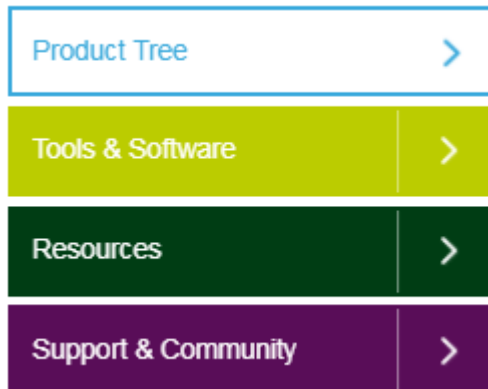
Click On "Disconnect"

Reset the board and the original code will run



- 1 **Efficient** (Power, Performance and Cost)
- 2 **Robust** (EMS, ECC, Clock Monitoring/Watchdogs, Security)
- 3 **Simple** (Easy to configure and develop code)

www.st.com/stm32g0



We Greatly Value Your Feedback

233

Use your phone to scan the QR code or type the link into your browser.



<https://www.surveymonkey.com/r/8WBPJFF>

Thank you!



 [/STM32](https://www.facebook.com/STM32)

 [@ST_World](https://twitter.com/ST_World)

 community.st.com

